

前言

大家好，我是爱学习爱分享的沉默王二。微信搜索「沉默王二」可以关注我的原创公众号，回复关键字「1024」就可以拉取离线版的《JavaGuide 面试突击》下载地址。

不经意间，在 GitHub 上发现了一个 1G 棒的 LeetCode 刷题笔记，重点来了，是纯正的 Java 版。我见过很多牛逼的刷题笔记，有 Go 版的，有 C++ 版的，唯独没有 Java 版的，所以这次，我感觉找到了宝藏！

- 题解预览地址：<https://leetcode.wang>，推荐电脑端打开，手机打开的话将页面滑到最上边，左上角是菜单
- leetcode 题目地址 <https://leetcode.com/problemset/all/>
- github 项目地址：<https://github.com/wind-liang/leetcode>

01、题目描述（简单难度）

Given an array of integers, return **indices** of the two numbers such that they add up to a specific target.

You may assume that each input would have **exactly** one solution, and you may not use the *same* element twice.

Example:

```
Given nums = [2, 7, 11, 15], target = 9,  
  
Because nums[0] + nums[1] = 2 + 7 = 9,  
return [0, 1].
```

给定一个数组和一个目标和，从数组中找两个数字相加等于目标和，输出这两个数字的下标。

解法一

简单粗暴些，两重循环，遍历所有情况看相加是否等于目标和，如果符合直接输出。

```

public int[] twoSum(int[] nums, int target) {
    int []ans=new int[2];
    for(int i=0;i<nums.length;i++){
        for(int j=(i+1);j<nums.length;j++){
            if(nums[i]+nums[j]==target){
                ans[0]=i;
                ans[1]=j;
                return ans;
            }
        }
    }
    return ans;
}

```

时间复杂度：两层 for 循环， $O(n^2)$

空间复杂度： $O(1)$

解法二

在上边的解法中看下第二个 for 循环步骤。

```

for(int j=(i+1);j<nums.length;j++){
    if(nums[i]+nums[j]==target){

```

我们换个理解方式：

```

for(int j=(i+1);j<nums.length;j++){
    sub=target-nums[i]
    if(nums[j]==sub){

```

第二层 for 循环无非是遍历所有的元素，看哪个元素等于 sub，时间复杂度为 $O(n)$ 。

有没有一种方法，不用遍历就可以找到元素里有没有等于 sub 的？

hash table !!!

我们可以把数组的每个元素保存为 hash 的 key，下标保存为 hash 的 value。

这样只需判断 sub 在不在 hash 的 key 里就可以了，而此时的时间复杂度仅为 $O(1)$ ！

需要注意的地方是，还需判断找到的元素不是当前元素，因为题目里讲一个元素只能用一次。

```

public int[] twoSum(int[] nums, int target) {
    Map<Integer,Integer> map=new HashMap<>();
    for(int i=0;i<nums.length;i++){
        map.put(nums[i],i);
    }
    for(int i=0;i<nums.length;i++){
        int sub=target-nums[i];
        if(map.containsKey(sub)&&map.get(sub)!=i){
            return new int[]{i,map.get(sub)};
        }
    }
    throw new IllegalArgumentException("No two sum solution");
}

```

时间复杂度：比解法一少了一个 for 循环，降为 $O(n)$

空间复杂度：所谓的空间换时间，这里就能体现出来，开辟了一个 hash table，空间复杂度变为 $O(n)$

解法三

看解法二中，两个 for 循环，他们长的一样，我们当然可以把它合起来。复杂度上不会带来什么变化，变化仅仅是不需要判断是不是当前元素了，因为当前元素还没有添加进 hash 里。

```

public int[] twoSum(int[] nums, int target) {
    Map<Integer,Integer> map=new HashMap<>();
    for(int i=0;i<nums.length;i++){
        int sub=target-nums[i];
        if(map.containsKey(sub)){
            return new int[]{i,map.get(sub)};
        }
        map.put(nums[i], i);
    }
    throw new IllegalArgumentException("No two sum solution");
}

```

总结

题目比较简单，毕竟暴力的方法也可以解决。唯一闪亮的点就是，时间复杂度从 $O(n^2)$ 降为 $O(n)$ 的时候，对 hash 的应用，有眼前一亮的感觉。

02、题目描述（中等难度）

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order** and each of their nodes contain a single digit. Add the two numbers and return it as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

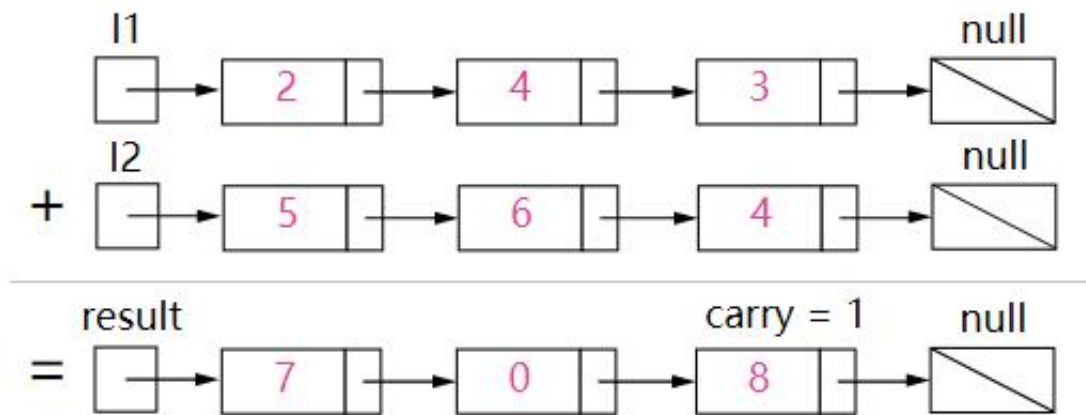
Example

Input: (2 -> 4 -> 3) + (5 -> 6 -> 4)
Output: 7 -> 0 -> 8
Explanation: 342 + 465 = 807.

就是两个链表表示的数相加，这样就可以实现两个很大的数相加了，无需考虑数值 int，float 的限制了。

由于自己实现的很乱，直接按答案的讲解了。

图示



链表最左边表示个位数，代表 $342 + 465 = 807$ 。

思路

首先每一位相加肯定会产生进位，我们用 carry 表示。进位最大会是 1，因为最大的情况无非是 $9 + 9 + 1 = 19$ ，也就是两个最大的数相加，再加进位，这样最大是 19，不会产生进位 2。下边是伪代码。

- 初始化一个节点的头，dummy head，但是这个头不存储数字。并且将 curr 指向它。
- 初始化进位 carry 为 0。
- 初始化 p 和 q 分别为给定的两个链表 l1 和 l2 的头，也就是个位。
- 循环，直到 l1 和 l2 全部到达 null。
 - 设置 x 为 p 节点的值，如果 p 已经到达了 null，设置 x 为 0。
 - 设置 y 为 q 节点的值，如果 q 已经到达了 null，设置 y 为 0。
 - 设置 $sum = x + y + carry$ 。
 - 更新 $carry = sum / 10$ 。
 - 创建一个值为 $sum \bmod 10$ 的节点，并将 curr 的 next 指向它，同时 curr 指向变为当前的新节点。

点。

- 向前移动 p 和 q。
- 判断 carry 是否等于 1，如果等于 1，在链表末尾增加一个为 1 的节点。
- 返回 dummy head 的 next，也就是个位数开始的地方。

初始化的节点 dummy head 没有存储值，最后返回 dummy head 的 next。这样的好处是不用单独对 head 进行判断改变值。也就是如果一开始的 head 就是代表个位数，那么开始初始化的时候并不知道它的值是多少，所以还需要在进入循环前单独对它进行值的更正，不能像现在一样只用一个循环简洁。

代码

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int x) { val = x; }
}

public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummyHead = new ListNode(0);
    ListNode p = l1, q = l2, curr = dummyHead;
    int carry = 0;
    while (p != null || q != null) {
        int x = (p != null) ? p.val : 0;
        int y = (q != null) ? q.val : 0;
        int sum = carry + x + y;
        carry = sum / 10;
        curr.next = new ListNode(sum % 10);
        curr = curr.next;
        if (p != null) p = p.next;
        if (q != null) q = q.next;
    }
    if (carry > 0) {
        curr.next = new ListNode(carry);
    }
    return dummyHead.next;
}
```

时间复杂度：O (max (m, n))，m 和 n 代表 l1 和 l2 的长度。

空间复杂度：O (max (m, n))，m 和 n 代表 l1 和 l2 的长度。而其实新的 List 最大长度是 O (max (m, n)) + 1，因为我们的 head 没有存储值。

扩展

What if the the digits in the linked list are stored in non-reversed order? For example:

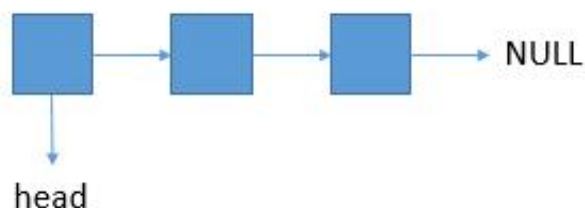
$$(3 \rightarrow 4 \rightarrow 2) + (4 \rightarrow 6 \rightarrow 5) = 8 \rightarrow 0 \rightarrow 7$$

如果链表存储的顺序反过来怎么办？

我首先想到的是链表先逆序计算，然后将结果再逆序呗，这就转换到我们之前的情况了。不知道还有没有其他的解法。下边分析下单链表逆序的思路。

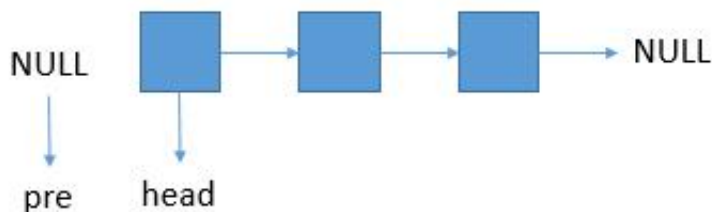
迭代思想

首先看一下原链表。



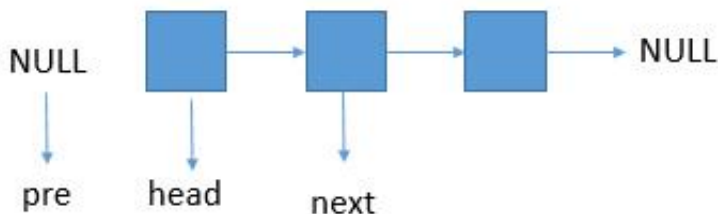
总共需要添加两个指针，pre 和 next。

初始化 pre 指向 NULL。

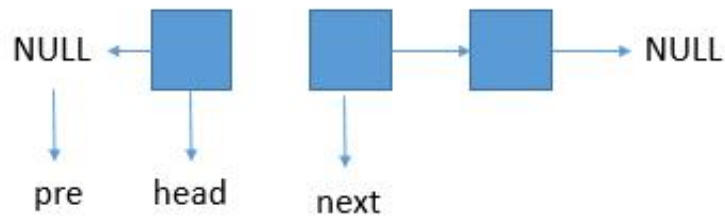


然后就是迭代的步骤，总共四步，顺序一步都不能错。

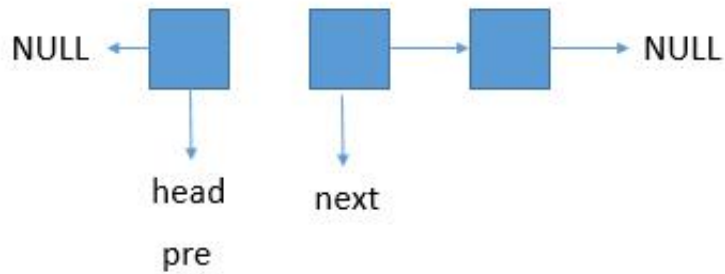
- next 指向 head 的 next，防止原链表丢失



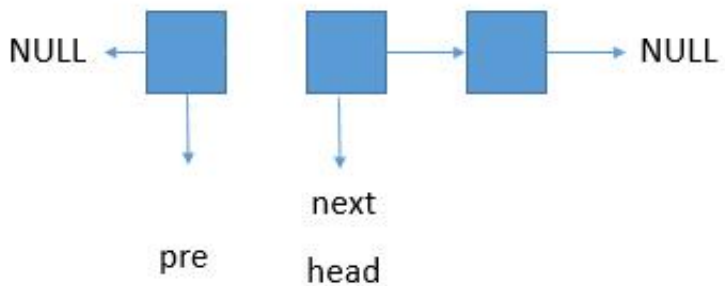
- head 的 next 从原来链表脱离，指向 pre。



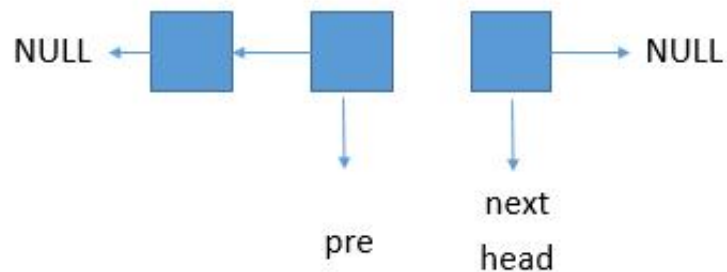
- pre 指向 head



- head 指向 next



一次迭代就完成了，如果再进行一次迭代就变成下边的样子。



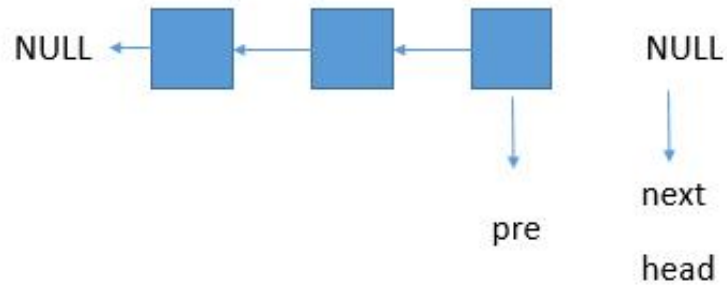
可以看到整个过程无非是把旧链表的 head 取下来，添加的新的链表上。代码怎么写呢？

```

next = head -> next; //保存 head 的 next ， 以防取下 head 后丢失
head -> next = pre; //将 head 从原链表取下来，添加到新链表上
pre = head; // pre 右移
head = next; // head 右移

```

接下来就是停止条件了，我们再进行一次循环。



可以发现当 head 或者 next 指向 null 的时候，我们就可以停止了。此时将 pre 返回，便是逆序了的链表了。

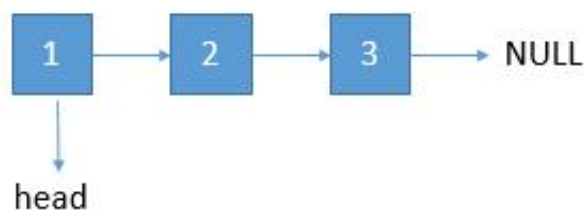
迭代代码

```
public ListNode reverseList(ListNode head){
    if(head==null) return null;
    ListNode pre=null;
    ListNode next;
    while(head!=null){
        next=head.next;
        head.next=pre;
        pre=head;
        head=next;
    }
    return pre;
}
```

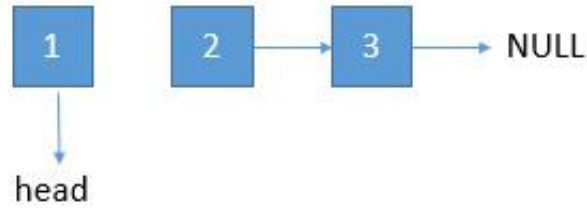
递归思想

- 首先假设我们实现了将单链表逆序的函数，ListNode reverseListRecursion(ListNode head)，传入链表头，返回逆序后的链表头。
- 接着我们确定如何把问题一步一步的化小，我们可以这样想。

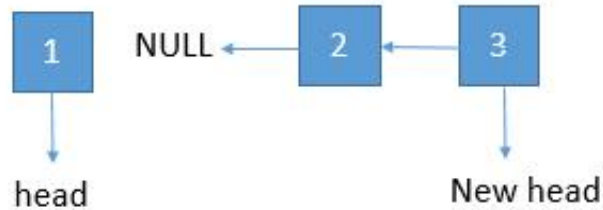
把 head 结点拿出来，剩下的部分我们调用函数 reverseListRecursion，这样剩下的部分就逆序了，接着我们把 head 结点放到新链表的尾部就可以了。这就是整个递归的思想了。



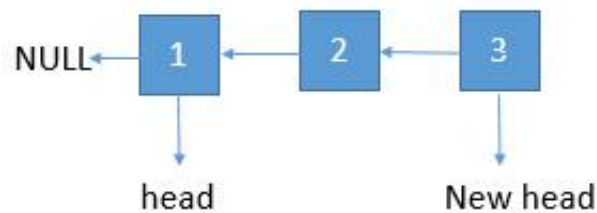
- head 结点拿出来



- 剩余部分调用逆序函数 reverseListRecursion , 并得到了 newhead



- 将 2 指向 1 , 1 指向 null, 将 newhead 返回即可。



- 找到递归出口

当然就是如果结点的个数是一个, 那么逆序的话还是它本身, 直接 return 就够了。怎么判断结点个数是不是一个呢? 它的 next 等于 null 就说明是一个了。但如果传进来的本身就是 null, 那么直接找它的 next 会报错, 所以先判断传进来的是不是 null, 如果是, 也是直接返回就可以了。

代码

```
public ListNode reverseListRecursion(ListNode head){
    ListNode newHead;
    if(head==null || head.next==null ){
        return head;
    }
    newHead=reverseListRecursion(head.next); //head.next 作为剩余部分的头指针
    head.next.next=head; //head.next 代表新链表的尾, 将它的 next 置为 head, 就是将
head 加到最后了。
    head.next=null;
    return newHead;
}
```

03、题目描述（中等难度）

Given a string, find the length of the **longest substring** without repeating characters.

Examples:

Given "abcabcbb", the answer is "abc", which the length is 3.

Given "bbbbbb", the answer is "b", with the length of 1.

Given "pwwkew", the answer is "wke", with the length of 3. Note that the answer must be a **substring**, "pwke" is a *subsequence* and not a substring.

给定一个字符串，找到没有重复字符的最长子串，返回它的长度。

解法一

简单粗暴些，找一个最长子串，那么我们用两个循环穷举所有子串，然后再用一个函数判断该子串中有没有重复的字符。

```
public int lengthOfLongestSubstring(String s) {
    int n = s.length();
    int ans = 0; //保存当前得到满足条件的子串的最大值
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j <= n; j++) //之所以 j <= n, 是因为我们子串是 [i, j), 左闭右开
            if (allUnique(s, i, j)) ans = Math.max(ans, j - i); //更新 ans
    return ans;
}

public boolean allUnique(String s, int start, int end) {
    Set<Character> set = new HashSet<>(); //初始化 hash set
    for (int i = start; i < end; i++) { //遍历每个字符
        Character ch = s.charAt(i);
        if (set.contains(ch)) return false; //判断字符在不在 set 中
        set.add(ch); //不在的话将该字符添加到 set 里边
    }
    return true;
}
```

时间复杂度：两个循环，加上判断子串满足不满足条件的函数中的循环， $O(n^3)$ 。

空间复杂度：使用了一个 set，判断子串中有没有重复的字符。由于 set 中没有重复的字符，所以最长就是整个字符集，假设字符集的大小为 m ，那么 set 最长就是 m 。另一方面，如果字符串的长度小于 m ，是 n 。那么 set 最长也就是 n 了。综上，空间复杂度为 $O(\min(m, n))$ 。

解法二

遗憾的是上边的算法没有通过 leetCode，时间复杂度太大，造成了超时。我们怎么来优化一下呢？

上边的算法中，我们假设当 i 取 0 的时候，

j 取 1，判断字符串 $\text{str}[0,1)$ 中有没有重复的字符。

j 取 2，判断字符串 $\text{str}[0,2)$ 中有没有重复的字符。

j 取 3，判断字符串 $\text{str}[0,3)$ 中有没有重复的字符。

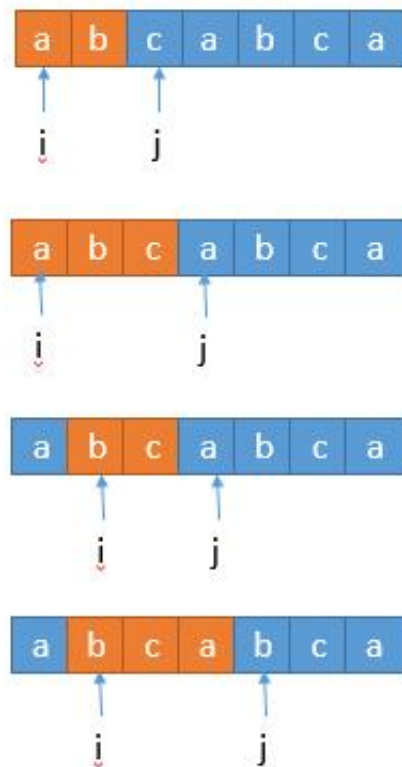
j 取 4，判断字符串 $\text{str}[0,4)$ 中有没有重复的字符。

做了很多重复的工作，因为如果 $\text{str}[0,3)$ 中没有重复的字符，我们不需要再判断整个字符串 $\text{str}[0,4)$ 中有没有重复的字符，而只需要判断 $\text{str}[3]$ 在不在 $\text{str}[0,3)$ 中，不在的话，就表明 $\text{str}[0,4)$ 中没有重复的字符。

如果在的话，那么 $\text{str}[0,5)$ ， $\text{str}[0,6)$ ， $\text{str}[0,7)$ 一定有重复的字符，所以此时后边的 j 也不需要继续增加了。 $i++$ 进入下次的循环就可以了。

此外，我们的 j 也不需要取 $j+1$ ，而只需要从当前的 j 开始就可以了。

综上，其实整个关于 j 的循环我们完全可以去掉了，此时可以理解变成了一个「滑动窗口」。



整体就是橘色窗口在依次向右移动。

判断一个字符在不在字符串中，我们需要可以遍历整个字符串，遍历需要的时间复杂度就是 $O(n)$ ，加上最外层的 i 的循环，总体复杂度就是 $O(n^2)$ 。我们可以继续优化，判断字符在不在一个字符串，我们可以将已有的字符串存到 Hash 里，这样的时间复杂度是 $O(1)$ ，总的时间复杂度就变成了 $O(n)$ 。

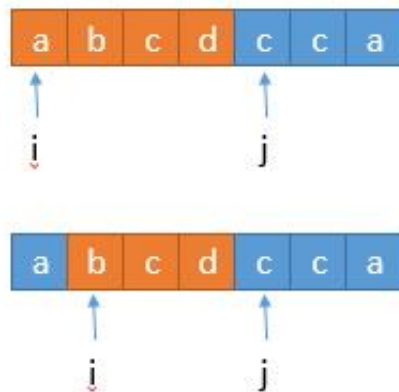
```
public class Solution {
    public int lengthOfLongestSubstring(String s) {
        int n = s.length();
        Set<Character> set = new HashSet<>();
        int ans = 0, i = 0, j = 0;
        while (i < n && j < n) {
            if (!set.contains(s.charAt(j))) {
                set.add(s.charAt(j++));
                ans = Math.max(ans, j - i);
            }
            else {
                set.remove(s.charAt(i++));
            }
        }
        return ans;
    }
}
```

时间复杂度：在最坏的情况下，while 循环中的语句会执行 $2n$ 次，例如 abcdefgg，开始的时候 j 一直后移直到到达第二个 g 的时候固定不变，然后 i 开始一直后移直到 n ，所以总共执行了 $2n$ 次，时间复杂度为 $O(n)$ 。

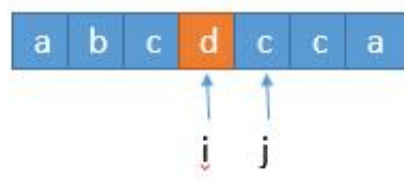
空间复杂度：和上边的类似，需要一个 Hash 保存子串，所以是 $O(\min(m, n))$ 。

解法三

继续优化，我们看上边的算法的一种情况。



当 j 指向的 c 存在于前边的子串 $abcd$ 中，此时 i 向前移到 b ，此时子串中仍然含有 c ，还得继续移动，所以这里其实可以优化。我们可以一步到位，直接移动到子串 c 的位置的下一位！



实现这样的话，我们将 set 改为 map，将字符存为 key，将对应的下标存到 value 里就实现了。

```
public class Solution {
    public int lengthOfLongestSubstring(String s) {
        int n = s.length(), ans = 0;
        Map<Character, Integer> map = new HashMap<>();
        for (int j = 0, i = 0; j < n; j++) {
            if (map.containsKey(s.charAt(j))) {
                i = Math.max(map.get(s.charAt(j)), i);
            }
            ans = Math.max(ans, j - i + 1);
            map.put(s.charAt(j), j + 1); // 下标 + 1 代表 i 要移动的下个位置
        }
        return ans;
    }
}
```

与解法二相比

由于采取了 i 跳跃的形式，所以 map 之前存的字符没有进行 remove，所以 if 语句中进行了 `Math.max(map.get(s.charAt(j)), i)`，要确认得到的下标不是 i 前边的。

还有个不同之处是 j 每次循环都进行了自加 1，因为 i 的跳跃已经保证了 `str[i, j]` 内没有重复的字符串，所以 j 直接可以加 1。而解法二中，要保持 j 的位置不变，因为不知道和 j 重复的字符在哪个位置。

最后个不同之处是，ans 在每次循环中都进行更新，因为 ans 更新前 i 都进行了更新，已经保证了当前的子串符合条件，所以可以更新 ans。而解法二中，只有当当前的子串不包含当前的字符时，才进行更新。

时间复杂度：我们将 $2n$ 优化到了 n ，但最终还是和之前一样， $O(n)$ 。

空间复杂度：也是一样的， $O(\min(m, n))$ 。

解法四

和解法三思路一样，区别的地方在于，我们不用 Hash，而是直接用数组，字符的 ASCII 码值作为数组的下标，数组存储该字符所在字符串的位置。适用于字符集比较小的情况，因为我们会直接开辟和字符集等大的数组。

```

public class Solution {
    public int lengthOfLongestSubstring(String s) {
        int n = s.length(), ans = 0;
        int[] index = new int[128];
        for (int j = 0, i = 0; j < n; j++) {
            i = Math.max(index[s.charAt(j)], i);
            ans = Math.max(ans, j - i + 1);
            index[s.charAt(j)] = j + 1; // (下标 + 1) 代表 i 要移动的下个位置
        }
        return ans;
    }
}

```

和解法 3 不同的地方在于，没有了 if 的判断，因为如果 `index[s.charAt(j)]` 不存在的话，它的值会是 0，对最终结果不会影响。

时间复杂度： $O(n)$ 。

空间复杂度： $O(m)$ ， m 代表字符集的大小。这次不论原字符串多小，都会利用这么大的空间。

总结

综上，我们一步一步的寻求可优化的地方，对算法进行了优化。又加深了 Hash 的应用，以及利用数组巧妙的实现了 Hash 的作用。

04、题目描述（困难难度）

There are two sorted arrays **nums1** and **nums2** of size m and n respectively.

Find the median of the two sorted arrays. The overall run time complexity should be $O(\log(m+n))$.

Example 1:

```

nums1 = [1, 3]
nums2 = [2]

The median is 2.0

```

Example 2:

```

nums1 = [1, 2]
nums2 = [3, 4]

The median is (2 + 3)/2 = 2.5

```

已知两个有序数组，找到两个数组合并后的中位数。

解法一

简单粗暴，先将两个数组合并，两个有序数组的合并也是归并排序中的一部分。然后根据奇数，还是偶数，返回中位数。

代码

```
public double findMedianSortedArrays(int[] nums1, int[] nums2) {
    int[] nums;
    int m = nums1.length;
    int n = nums2.length;
    nums = new int[m + n];
    if (m == 0) {
        if (n % 2 == 0) {
            return (nums2[n / 2 - 1] + nums2[n / 2]) / 2.0;
        } else {
            return nums2[n / 2];
        }
    }
    if (n == 0) {
        if (m % 2 == 0) {
            return (nums1[m / 2 - 1] + nums1[m / 2]) / 2.0;
        } else {
            return nums1[m / 2];
        }
    }

    int count = 0;
    int i = 0, j = 0;
    while (count != (m + n)) {
        if (i == m) {
            while (j != n) {
                nums[count++] = nums2[j++];
            }
            break;
        }
        if (j == n) {
            while (i != m) {
                nums[count++] = nums1[i++];
            }
            break;
        }
    }
}
```

```

        if (nums1[i] < nums2[j]) {
            nums[count++] = nums1[i++];
        } else {
            nums[count++] = nums2[j++];
        }
    }

    if (count % 2 == 0) {
        return (nums[count / 2 - 1] + nums[count / 2]) / 2.0;
    } else {
        return nums[count / 2];
    }
}

```

时间复杂度：遍历全部数组， $O(m + n)$

空间复杂度：开辟了一个数组，保存合并后的两个数组， $O(m + n)$

解法二

其实，我们不需要将两个数组真的合并，我们只需要找到中位数在哪里就可以了。

开始的思路是写一个循环，然后里边判断是否到了中位数的位置，到了就返回结果，但这里对偶数和奇数的分类会很麻烦。当其中一个数组遍历完后，出了 for 循环对边界的判断也会分几种情况。总体来说，虽然复杂度不影响，但代码会看起来很乱。然后在 [这里](#) 找到了另一种思路。

首先是怎么将奇数和偶数的情况合并一下。

用 len 表示合并后数组的长度，如果是奇数，我们需要知道第 $(len + 1) / 2$ 个数就可以了，如果遍历的话需要遍历 $\text{int}(len / 2) + 1$ 次。如果是偶数，我们需要知道第 $len / 2$ 和 $len / 2 + 1$ 个数，也是需要遍历 $len / 2 + 1$ 次。所以遍历的话，奇数和偶数都是 $len / 2 + 1$ 次。

返回中位数的话，奇数需要最后一次遍历的结果就可以了，偶数需要最后一次和上一次遍历的结果。所以我们用两个变量 left 和 right，right 保存当前循环的结果，在每次循环前将 right 的值赋给 left。这样在最后一次循环的时候，left 将得到 right 的值，也就是上一次循环的结果，接下来 right 更新为最后一次的结果。

循环中该怎么写，什么时候 A 数组后移，什么时候 B 数组后移。用 aStart 和 bStart 分别表示当前指向 A 数组和 B 数组的位置。如果 aStart 还没有到最后并且此时 A 位置的数字小于 B 位置的数组，那么就可以后移了。也就是 $aStart < m \ \&\& \ A[aStart] < B[bStart]$ 。

但如果 B 数组此刻已经没有数字了，继续取数字 B [bStart]，则会越界，所以判断下 bStart 是否大于数组长度了，这样 || 后边的就不会执行了，也就不会导致错误了，所以增加为 aStart < m && (bStart >= n || A [aStart] < B [bStart])。

代码

```
public double findMedianSortedArrays(int[] A, int[] B) {
    int m = A.length;
    int n = B.length;
    int len = m + n;
    int left = -1, right = -1;
    int aStart = 0, bStart = 0;
    for (int i = 0; i <= len / 2; i++) {
        left = right;
        if (aStart < m && (bStart >= n || A[aStart] < B[bStart])) {
            right = A[aStart++];
        } else {
            right = B[bStart++];
        }
    }
    if ((len & 1) == 0)
        return (left + right) / 2.0;
    else
        return right;
}
```

时间复杂度：遍历 len/2 + 1 次，len = m + n，所以时间复杂度依旧是 O (m + n)。

空间复杂度：我们申请了常数个变量，也就是 m, n, len, left, right, aStart, bStart 以及 i。

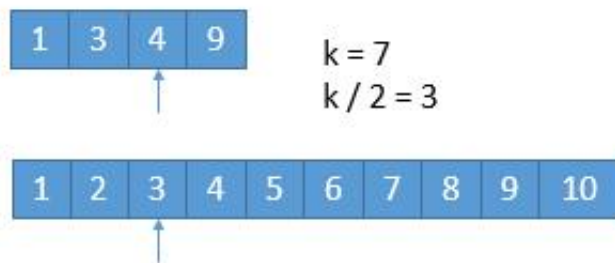
总共 8 个变量，所以空间复杂度是 O (1)。

解法三

上边的两种思路，时间复杂度都达不到题目的要求 O (log (m + n))。看到 log，很明显，我们只有用到二分的方法才能达到。我们不妨用另一种思路，题目是求中位数，其实就是求第 k 小数的一种特殊情况，而求第 k 小数有一种算法。

解法二中，我们一次遍历就相当于去掉不可能是中位数的一个值，也就是一个一个排除。由于数列是有序的，其实我们完全可以一半儿一半儿的排除。假设我们要找第 k 小数，我们可以每次循环排除掉 k / 2 个数。看下边一个例子。

假设我们要找第 7 小的数字。

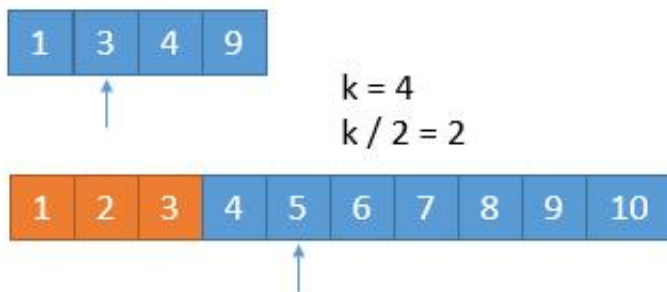


我们比较两个数组的第 $k/2$ 个数字，如果 k 是奇数，向下取整。也就是比较第 3 个数字，上边数组中的 4 和下边数组中的 3，如果哪个小，就表明该数组的前 $k/2$ 个数字都不是第 k 小数字，所以可以排除。也就是 1, 2, 3 这三个数字不可能是第 7 小的数字，我们可以把它排除掉。将 1349 和 45678910 两个数组作为新的数组进行比较。

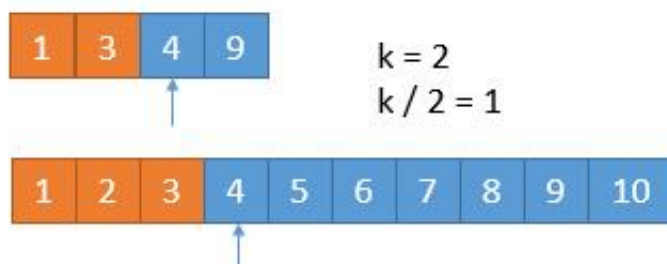
更一般的情况 $A[1], A[2], A[3], A[k/2] \dots, B[1], B[2], B[3], B[k/2] \dots$ ，如果 $A[k/2] < B[k/2]$ ，那么 $A[1], A[2], A[3], A[k/2]$ 都不可能是第 k 小的数字。

A 数组中比 $A[k/2]$ 小的数有 $k/2 - 1$ 个，B 数组中， $B[k/2]$ 比 $A[k/2]$ 大，假设 $B[k/2]$ 前边的数字都比 $A[k/2]$ 小，也只有 $k/2 - 1$ 个，所以比 $A[k/2]$ 小的数字最多有 $k/2 - 1 + k/2 - 1 = k - 2$ 个，所以 $A[k/2]$ 最多是第 $k - 1$ 小的数。而比 $A[k/2]$ 小的数更不可能是第 k 小的数了，所以可以把它们排除。

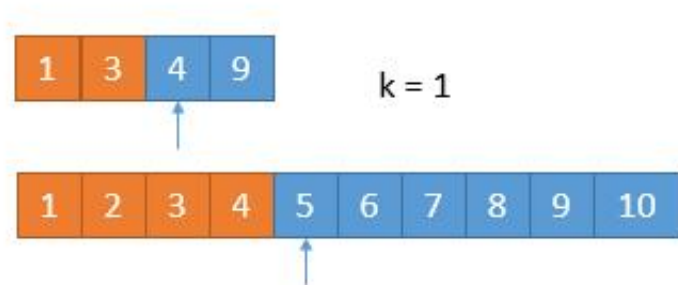
橙色的部分表示已经去掉的数字。



由于我们已经排除掉了 3 个数字，就是这 3 个数字一定在最前边，所以在两个新数组中，我们只需要找第 $7 - 3 = 4$ 小的数字就可以了，也就是 $k = 4$ 。此时两个数组，比较第 2 个数字， $3 < 5$ ，所以我们可以把小的那个数组中的 1, 3 排除掉了。



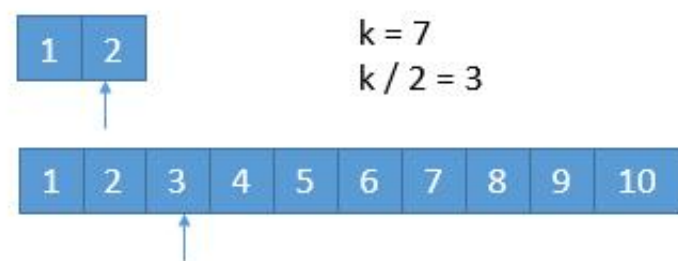
我们又排除掉 2 个数字，所以现在找第 $4 - 2 = 2$ 小的数字就可以了。此时比较两个数组中的第 $k / 2 = 1$ 个数， $4 == 4$ ，怎么办呢？由于两个数相等，所以我们无论去掉哪个数组中的都行，因为去掉 1 个总会保留 1 个的，所以没有影响。为了统一，我们就假设 $4 > 4$ 吧，所以此时将下边的 4 去掉。



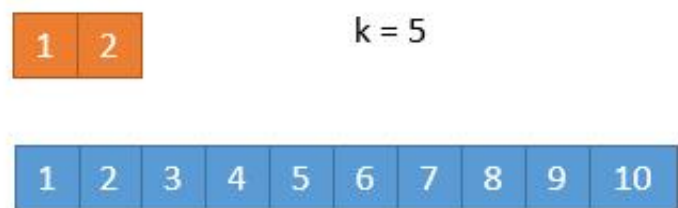
由于又去掉 1 个数字，此时我们要找第 1 小的数字，所以只需判断两个数组中第一个数字哪个小就可以了，也就是 4。

所以第 7 小的数字是 4。

我们每次都是取 $k / 2$ 的数进行比较，有时候可能会遇到数组长度小于 $k / 2$ 的时候。



此时 $k / 2$ 等于 3，而上边的数组长度是 2，我们此时将箭头指向它的末尾就可以了。这样的话，由于 $2 < 3$ ，所以就会导致上边的数组 1, 2 都被排除。造成下边的情况。



由于 2 个元素被排除，所以此时 $k = 5$ ，又由于上边的数组已经空了，我们只需要返回下边的数组的第 5 个数字就可以了。

从上边可以看到，无论是找第奇数个还是第偶数个数字，对我们的算法并没有影响，而且在算法进行中， k 的值都有可能从奇数变为偶数，最终都会变为 1 或者由于一个数组空了，直接返回结果。

所以我们采用递归的思路，为了防止数组长度小于 $k/2$ ，所以每次比较 $\min(k/2, \text{len}(\text{数组}))$ 对应的数字，把小的那个对应的数组的数字排除，将两个新数组进入递归，并且 k 要减去排除的数字的个数。递归出口就是当 $k = 1$ 或者其中一个数字长度是 0 了。

代码

```
public double findMedianSortedArrays(int[] nums1, int[] nums2) {
    int n = nums1.length;
    int m = nums2.length;
    int left = (n + m + 1) / 2;
    int right = (n + m + 2) / 2;
    //将偶数和奇数的情况合并，如果是奇数，会求两次同样的 k 。
    return (getKth(nums1, 0, n - 1, nums2, 0, m - 1, left) + getKth(nums1, 0, n - 1, nums2, 0, m - 1, right)) * 0.5;
}

private int getKth(int[] nums1, int start1, int end1, int[] nums2, int start2, int end2, int k) {
    int len1 = end1 - start1 + 1;
    int len2 = end2 - start2 + 1;
    //让 len1 的长度小于 len2，这样就能保证如果有数组空了，一定是 len1
    if (len1 > len2) return getKth(nums2, start2, end2, nums1, start1, end1, k);
    if (len1 == 0) return nums2[start2 + k - 1];

    if (k == 1) return Math.min(nums1[start1], nums2[start2]);

    int i = start1 + Math.min(len1, k / 2) - 1;
    int j = start2 + Math.min(len2, k / 2) - 1;

    if (nums1[i] > nums2[j]) {
        return getKth(nums1, start1, end1, nums2, j + 1, end2, k - (j - start2 + 1));
    }
    else {
        return getKth(nums1, i + 1, end1, nums2, start2, end2, k - (i - start1 + 1));
    }
}
```

时间复杂度：每进行一次循环，我们就减少 $k/2$ 个元素，所以时间复杂度是 $O(\log(k))$ ，而 $k = (m + n) / 2$ ，所以最终的复杂也就是 $O(\log(m + n))$ 。

空间复杂度：虽然我们用到了递归，但是可以看到这个递归属于尾递归，所以编译器不需要不停地堆栈，所以空间复杂度为 $O(1)$ 。

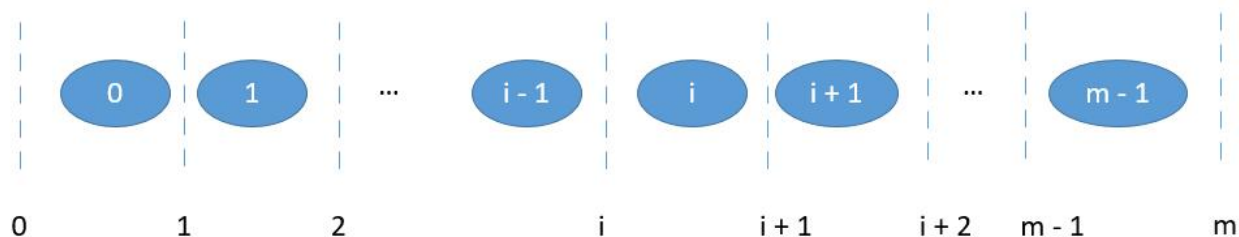
解法四

我们首先理一下中位数的定义是什么

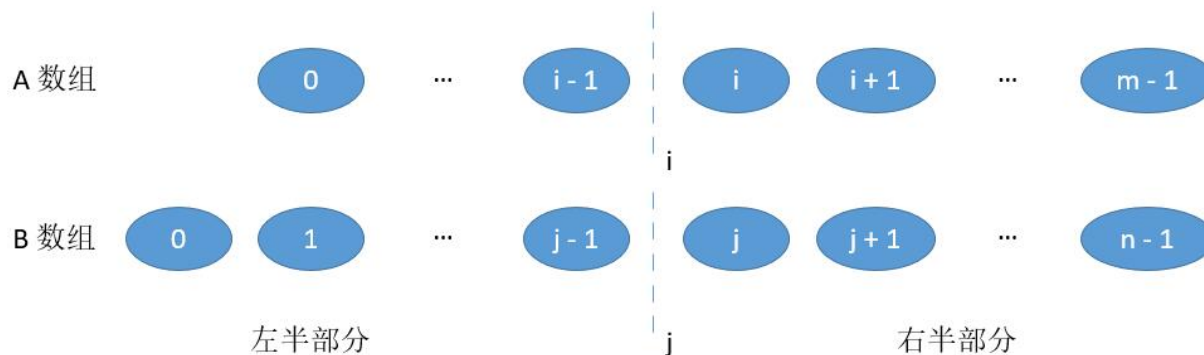
中位数（又称中值，英语：Median），[统计学](#)中的专有名词，代表一个样本、种群或[概率分布](#)中的一个数值，其可将数值集合划分为相等的上下两部分。

所以我们只需要将数组进行切。

一个长度为 m 的数组，有 0 到 m 总共 $m + 1$ 个位置可以切。



我们把数组 A 和数组 B 分别在 i 和 j 进行切割。



将 i 的左边和 j 的左边组合成「左半部分」，将 i 的右边和 j 的右边组合成「右半部分」。

- 当 A 数组和 B 数组的总长度是偶数时，如果我们能够保证
 - 左半部分的长度等于右半部分
$$i + j = m - i + n - j, \text{ 也就是 } j = (m + n) / 2 - i$$
 - 左半部分最大的值小于等于右半部分最小的值 $\max(A[i - 1], B[j - 1]) \leq \min(A[i], B[j])$那么，中位数就可以表示如下
$$(\text{左半部分最大值} + \text{右半部分最小值}) / 2。$$
$$(\max(A[i - 1], B[j - 1]) + \min(A[i], B[j])) / 2$$
- 当 A 数组和 B 数组的总长度是奇数时，如果我们能够保证
 - 左半部分的长度比右半部分大 1
$$i + j = m - i + n - j + 1 \text{ 也就是 } j = (m + n + 1) / 2 - i$$

- 左半部分最大的值小于等于右半部分最小的值 $\max(A[i-1], B[j-1]) \leq \min(A[i], B[j])$

那么，中位数就是

左半部分最大值，也就是左半部比右半部分多出的那一个数。

$\max(A[i-1], B[j-1])$

上边的第一个条件我们其实可以合并为 $j = (m + n + 1) / 2 - i$ ，因为如果 $m + n$ 是偶数，由于我们取的是 int 值，所以加 1 也不会影响结果。当然，由于 $0 \leq i \leq m$ ，为了保证 $0 \leq j \leq n$ ，我们必须保证 $m \leq n$ 。

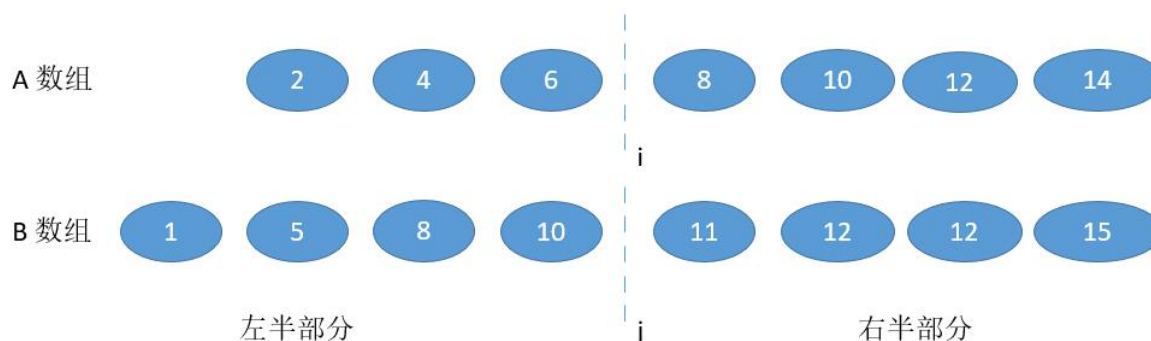
$$m \leq n, i < m, j = (m+n+1)/2 - i \geq (m+m+1)/2 - i > (m+m+1)/2 - m = 0$$

$$m \leq n, i > 0, j = (m+n+1)/2 - i \leq (n+n+1)/2 - i < (n+n+1)/2 = n$$

最后一步由于是 int 间的运算，所以 $1 / 2 = 0$ 。

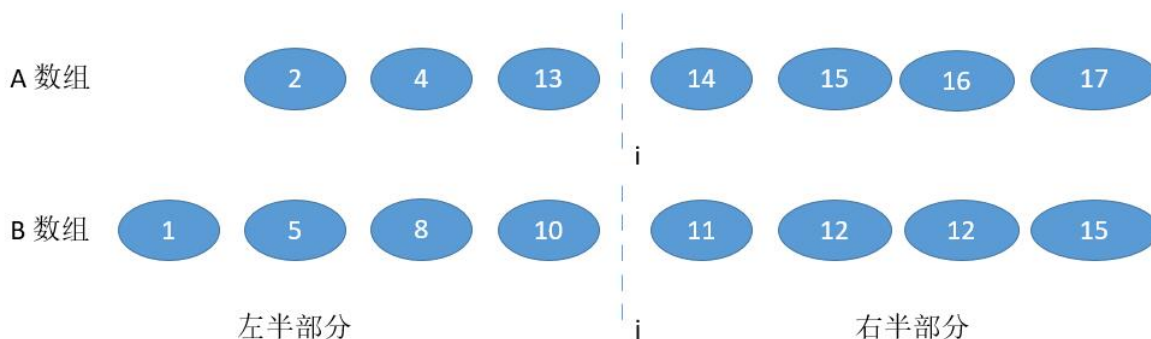
而对于第二个条件，奇数和偶数的情况是一样的，我们进一步分析。为了保证 $\max(A[i-1], B[j-1]) \leq \min(A[i], B[j])$ ，因为 A 数组和 B 数组是有序的，所以 $A[i-1] \leq A[i]$ ， $B[i-1] \leq B[i]$ 这是天然的，所以我们只需要保证 $B[j-1] \leq A[i]$ 和 $A[i-1] \leq B[j]$ 所以我们分两种情况讨论：

- $B[j-1] > A[i]$ ，并且为了不越界，要保证 $j \neq 0$ ， $i \neq m$



此时很明显，我们需要增加 i ，为了数量的平衡还要减少 j ，幸运的是 $j = (m + n + 1) / 2 - i$ ， i 增大， j 自然会减少。

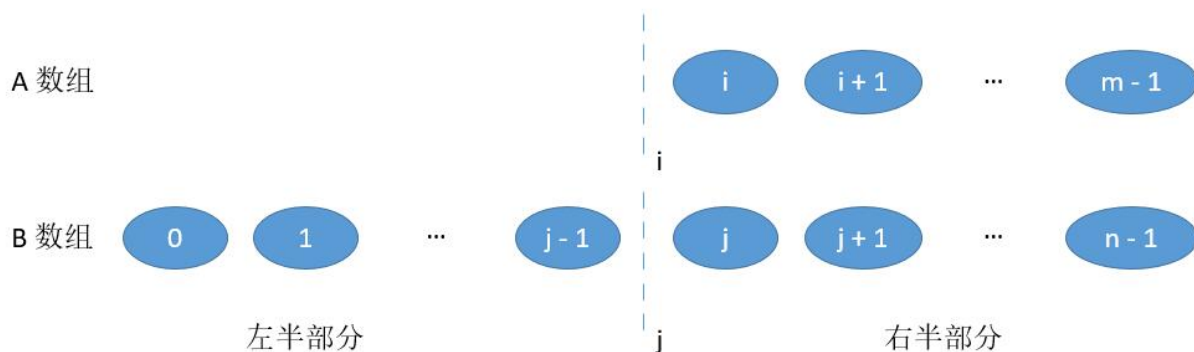
- $A[i-1] > B[j]$ ，并且为了不越界，要保证 $i \neq 0$ ， $j \neq n$



此时和上边的情况相反，我们要减少 i ，增大 j 。

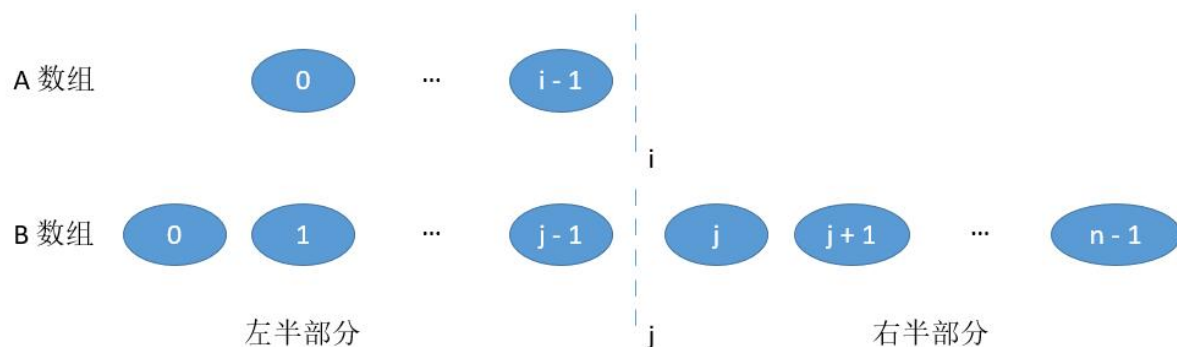
上边两种情况，我们把边界都排除了，需要单独讨论。

- 当 $i = 0$ ，或者 $j = 0$ ，也就是切在了最前边。



此时左半部分当 $j = 0$ 时，最大的值就是 $A[i - 1]$ ；当 $i = 0$ 时最大的值就是 $B[j - 1]$ 。右半部分最小值和之前一样。

- 当 $i = m$ 或者 $j = n$ ，也就是切在了最后边。



此时左半部分最大值和之前一样。右半部分当 $j = n$ 时，最小值就是 $A[i]$ ；当 $i = m$ 时，最小值就是 $B[j]$ 。

所有的思路都理清了，最后一个问题，增加 i 的方式。当然用二分了。初始化 i 为中间的值，然后减半找中间的，减半找中间的，减半找中间的直到答案。

```
class Solution {
    public double findMedianSortedArrays(int[] A, int[] B) {
        int m = A.length;
        int n = B.length;
        if (m > n) {
            return findMedianSortedArrays(B, A); // 保证 m <= n
        }
        int iMin = 0, iMax = m;
        while (iMin <= iMax) {
            int i = (iMin + iMax) / 2;
            int j = (m + n + 1) / 2 - i;
            if (j != 0 && i != m && B[j - 1] > A[i]) { // i 需要增大
                iMin = i + 1;
            }
        }
    }
}
```

```

    }
    else if (i != 0 && j != n && A[i-1] > B[j]) { // i 需要减小
        iMax = i - 1;
    }
    else { // 达到要求，并且将边界条件列出来单独考虑
        int maxLeft = 0;
        if (i == 0) { maxLeft = B[j-1]; }
        else if (j == 0) { maxLeft = A[i-1]; }
        else { maxLeft = Math.max(A[i-1], B[j-1]); }
        if ( (m + n) % 2 == 1 ) { return maxLeft; } // 奇数的话不需要考虑右半
    }

    int minRight = 0;
    if (i == m) { minRight = B[j]; }
    else if (j == n) { minRight = A[i]; }
    else { minRight = Math.min(B[j], A[i]); }

    return (maxLeft + minRight) / 2.0; //如果是偶数的话返回结果
}
}
return 0.0;
}
}

```

部分

时间复杂度：我们对较短的数组进行了二分查找，所以时间复杂度是 $O(\log(\min(m, n)))$ 。

空间复杂度：只有一些固定的变量，和数组长度无关，所以空间复杂度是 $O(1)$ 。

总结

解法二中体会到了对情况的转换，有时候即使有了思路，代码也不一定写的优雅，需要多锻炼才可以。解法三和解法四充分发挥了二分查找的优势，将时间复杂度降为 $\log$ 级别。

05、题目描述（中等难度）

Given a string **s**, find the longest palindromic substring in **s**. You may assume that the maximum length of **s** is 1000.

Example 1:

Input: "babad"
Output: "bab"
Note: "aba" is also a valid answer.

Example 2:

Input: "cbbd"
Output: "bb"

给定一个字符串，输出最长的回文子串。回文串指的是正的读和反的读是一样的字符串，例如 "aba", "cbbcc"。

解法一 暴力破解

暴力求解，列举所有的子串，判断是否为回文串，保存最长的回文串。

```
public boolean isPalindromic(String s) {
    int len = s.length();
    for (int i = 0; i < len / 2; i++) {
        if (s.charAt(i) != s.charAt(len - i - 1)) {
            return false;
        }
    }
    return true;
}

// 暴力解法
public String longestPalindrome(String s) {
    String ans = "";
    int max = 0;
    int len = s.length();
    for (int i = 0; i < len; i++)
        for (int j = i + 1; j <= len; j++) {
            String test = s.substring(i, j);
            if (isPalindromic(test) && test.length() > max) {
                ans = s.substring(i, j);
                max = Math.max(max, ans.length());
            }
        }
    return ans;
}
```

时间复杂度：两层 for 循环 $O(n^2)$ ，for 循环里边判断是否为回文， $O(n)$ ，所以时间复杂度为 $O(n^3)$ 。

空间复杂度： $O(1)$ ，常数个变量。

解法二 最长公共子串

根据回文串的定义，正着和反着读一样，那我们是不是把原来的字符串倒置了，然后找最长的公共子串就可以了。例如， $S = \text{"caba"}$ ， $S' = \text{"abac"}$ ，最长公共子串是 "aba"，所以原字符串的最长回文串就是 "aba"。

关于求最长公共子串（不是公共子序列），有很多方法，这里用动态规划的方法，可以先阅读下边的链接。

<https://blog.csdn.net/u010397369/article/details/38979077>

<https://www.kancloud.cn/digest/pieces-algorithm/163624>

整体思想就是，申请一个二维的数组初始化为 0，然后判断对应的字符是否相等，相等的话

$arr[i][j] = arr[i-1][j-1] + 1$ 。

当 $i = 0$ 或者 $j = 0$ 的时候单独分析，字符相等的话 $arr[i][j]$ 就赋为 1。

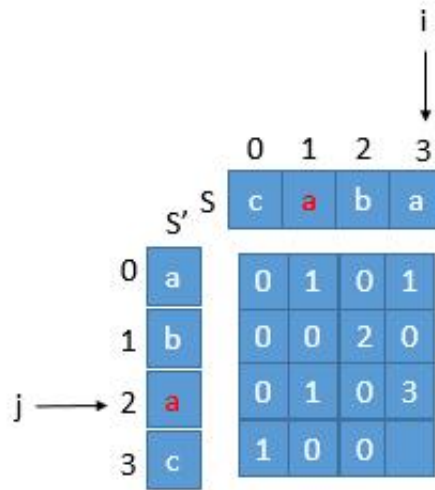
$arr[i][j]$ 保存的就是公共子串的长度。

```
public String longestPalindrome(String s) {
    if (s.equals(""))
        return "";
    String origin = s;
    String reverse = new StringBuffer(s).reverse().toString(); //字符串倒置
    int length = s.length();
    int[][] arr = new int[length][length];
    int maxLen = 0;
    int maxEnd = 0;
    for (int i = 0; i < length; i++)
        for (int j = 0; j < length; j++) {
            if (origin.charAt(i) == reverse.charAt(j)) {
                if (i == 0 || j == 0) {
                    arr[i][j] = 1;
                } else {
                    arr[i][j] = arr[i - 1][j - 1] + 1;
                }
            }
            if (arr[i][j] > maxLen) {
                maxLen = arr[i][j];
                maxEnd = i; //以 i 位置结尾的字符
            }
        }
    return s.substring(maxEnd - maxLen + 1, maxEnd + 1);
}
```

再看一个例子， $S = "abc435cba"$ ， $S' = "abc534cba"$ ，最长公共子串是 "abc" 和 "cba"，但很明显这两个字符串都不是回文串。

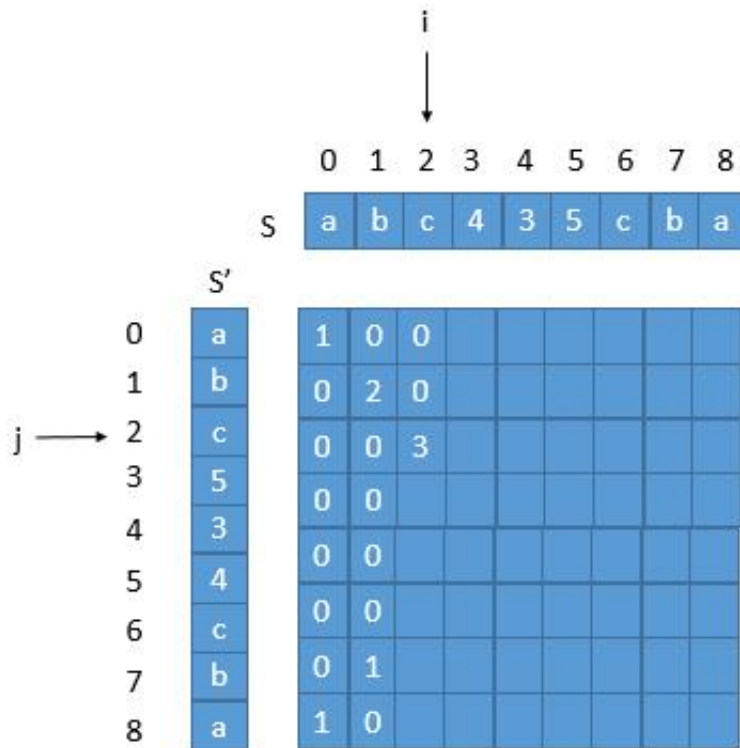
所以我们求出最长公共子串后，并不一定是回文串，我们还需要判断该字符串倒置前的下标和当前的字符串下标是不是匹配。

比如 $S = "caba"$ ， $S' = "abac"$ ， S' 中 aba 的下标是 0 1 2，倒置前是 3 2 1，和 S 中 aba 的下标符合，所以 aba 就是我们需要找的。当然我们不需要每个字符都判断，我们只需要判断末尾字符就可以。



首先 i, j 始终指向子串的末尾字符。所以 j 指向的红色的 a 倒置前的下标是 $\text{beforeRev} = \text{length} - 1 - j = 4 - 1 - 2 = 1$ ，对应的是字符串首位的下标，我们还需要加上字符串的长度才是末尾字符的下标，也就是 $\text{beforeRev} + \text{arr}[i][j] - 1 = 1 + 3 - 1 = 3$ ，因为 $\text{arr}[i][j]$ 保存的就是当前子串的长度，也就是图中的数字 3。此时再和它与 i 比较，如果相等，则说明它是我们要找的回文串。

之前的 $S = \text{"abc435cba"}$ ， $S' = \text{"abc534cba"}$ ，可以看一下图示，为什么不符合。



当前 j 指向的 c ，倒置前的下标是 $\text{beforeRev} = \text{length} - 1 - j = 9 - 1 - 2 = 6$ ，对应的末尾下标是 $\text{beforeRev} + \text{arr}[i][j] - 1 = 6 + 3 - 1 = 8$ ，而此时 $i = 2$ ，所以当前的子串不是回文串。

代码的话，在上边的基础上，保存 maxLen 前判断一下下标匹不匹配就可以了。

```

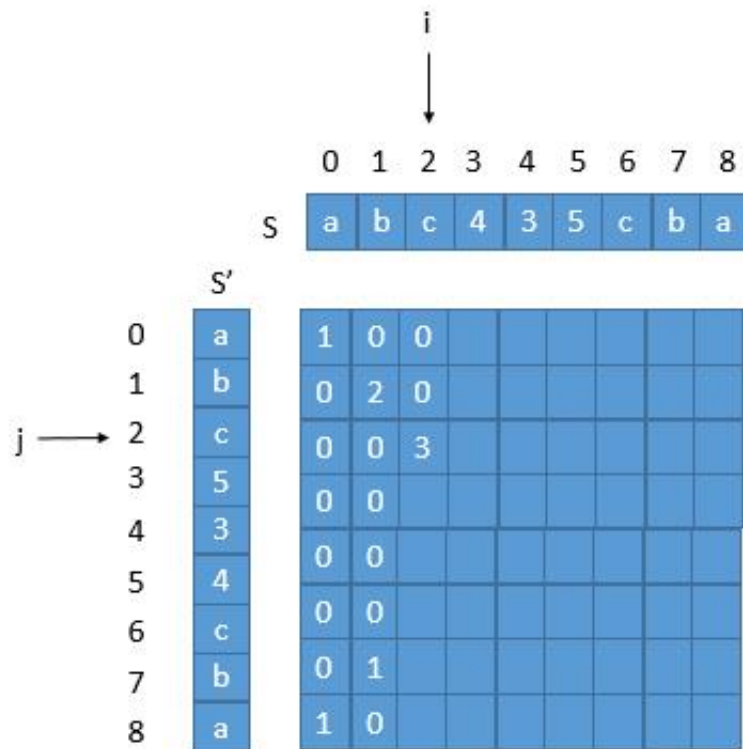
public String longestPalindrome(String s) {
    if (s.equals(""))
        return "";
    String origin = s;
    String reverse = new StringBuffer(s).reverse().toString();
    int length = s.length();
    int[][] arr = new int[length][length];
    int maxLen = 0;
    int maxEnd = 0;
    for (int i = 0; i < length; i++)
        for (int j = 0; j < length; j++) {
            if (origin.charAt(i) == reverse.charAt(j)) {
                if (i == 0 || j == 0) {
                    arr[i][j] = 1;
                } else {
                    arr[i][j] = arr[i - 1][j - 1] + 1;
                }
            }
            /*****修改的地方*****/
            if (arr[i][j] > maxLen) {
                int beforeRev = length - 1 - j;
                if (beforeRev + arr[i][j] - 1 == i) { //判断下标是否对应
                    maxLen = arr[i][j];
                    maxEnd = i;
                }
            }
            /*****/
        }
    return s.substring(maxEnd - maxLen + 1, maxEnd + 1);
}

```

时间复杂度：两层循环， $O(n^2)$ 。

空间复杂度：一个二维数组， $O(n^2)$ 。

空间复杂度其实可以再优化一下。



我们分析一下循环， $i = 0, j = 0, 1, 2 \dots 8$ 更新一列，然后 $i = 1$ ，再更新一列，而更新的时候我们其实只需要上一列的信息，更新第 3 列的时候，第 1 列的信息是没有用的。所以我们只需要一个一维数组就可以了。但是更新 `arr[i]` 的时候我们需要 `arr[i - 1]` 的信息，假设 `a[3] = a[2] + 1`，更新 `a[4]` 的时候，我们需要 `a[3]` 的信息，但是 `a[3]` 在之前已经被更新了，所以 `j` 不能从 0 到 8，应该倒过来，`a[8] = a[7] + 1`，`a[7] = a[6] + 1`，这样更新 `a[8]` 的时候用 `a[7]`，用完后才去更新 `a[7]`，保证了不会出错。

```
public String longestPalindrome(String s) {
    if (s.equals(""))
        return "";
    String origin = s;
    String reverse = new StringBuffer(s).reverse().toString();
    int length = s.length();
    int[] arr = new int[length];
    int maxLen = 0;
    int maxEnd = 0;
    for (int i = 0; i < length; i++)
        /*****修改的地方*****/
        for (int j = length - 1; j >= 0; j--) {
            /*****/
            if (origin.charAt(i) == reverse.charAt(j)) {
                if (i == 0 || j == 0) {
                    arr[j] = 1;
                } else {
                    arr[j] = arr[j - 1] + 1;
                }
            }
        }
    }
```

```

    /*****修改的地方*****/
    //之前二维数组，每次用的是不同的列，所以不用置 0 。
    } else {
        arr[j] = 0;
    }
    /*****/
    if (arr[j] > maxLen) {
        int beforeRev = length - 1 - j;
        if (beforeRev + arr[j] - 1 == i) {
            maxLen = arr[j];
            maxEnd = i;
        }
    }
}
return s.substring(maxEnd - maxLen + 1, maxEnd + 1);
}

```

时间复杂度：O (n²) 。

空间复杂度：降为 O (n) 。

解法三 暴力破解优化

解法一的暴力解法时间复杂度太高，在 leetCode 上并不能 AC 。我们可以考虑，去掉一些暴力解法中重复的判断。我们可以基于下边的发现，进行改进。

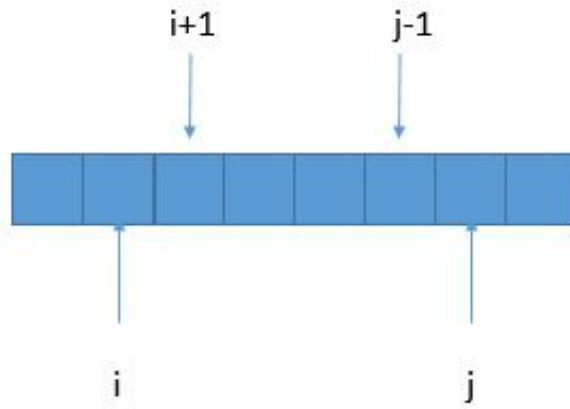
首先定义 P (i, j) 。

$$P(i,j) = \begin{cases} \text{true} & \text{s[i,j]是回文串} \\ \text{false} & \text{s[i,j]不是回文串} \end{cases}$$

接下来

$$P(i,j) = (P(i+1,j-1) \& \& S[i] == S[j])$$

所以如果我们想知道 P (i, j) 的情况，不需要调用判断回文串的函数了，只需要知道 P (i + 1, j - 1) 的情况就可以了，这样时间复杂度就少了 O (n) 。因此我们可以用动态规划的方法，空间换时间，把已经求出的 P (i, j) 存储起来。



如果 $S[i+1, j-1]$ 是回文串，那么只要 $S[i] == S[j]$ ，就可以确定 $S[i, j]$ 也是回文串了。

求长度为 1 和长度为 2 的 $P(i, j)$ 时不能用上边的公式，因为我们代入公式后会遇到 $P[i][j]$ 中 $i > j$ 的情况，比如求 $P[1][2]$ 的话，我们需要知道 $P[1+1][2-1] = P[2][1]$ ，而 $P[2][1]$ 代表着 $S[2, 1]$ 是不是回文串，显然是不对的，所以我们需要单独判断。

所以我们先初始化长度是 1 的回文串的 $P[i, j]$ ，这样利用上边提出的公式 $P(i, j) = (P(i+1, j-1) \&\& S[i] == S[j])$ ，然后两边向外各扩充一个字符，长度为 3 的，为 5 的，所有奇数长度的就都求出来了。

同理，初始化长度是 2 的回文串 $P[i, i+1]$ ，利用公式，长度为 4 的，6 的所有偶数长度的就都求出来了。

```
public String longestPalindrome(String s) {
    int length = s.length();
    boolean[][] P = new boolean[length][length];
    int maxLen = 0;
    String maxPal = "";
    for (int len = 1; len <= length; len++) //遍历所有的长度
        for (int start = 0; start < length; start++) {
            int end = start + len - 1;
            if (end >= length) //下标已经越界，结束本次循环
                break;
            P[start][end] = (len == 1 || len == 2 || P[start + 1][end - 1]) &&
s.charAt(start) == s.charAt(end); //长度为 1 和 2 的单独判断下
            if (P[start][end] && len > maxLen) {
                maxPal = s.substring(start, end + 1);
            }
        }
    return maxPal;
}
```

时间复杂度：两层循环， $O(n^2)$ 。

空间复杂度：用二维数组 P 保存每个子串的情况， $O(n^2)$ 。

我们分析下每次循环用到的 P (i, j) ，看一看能不能向解法二一样优化一下空间复杂度。

P 的存储

	0	1	2	3	4	5	
0	T	F	T	T	T	F	
1		T	T	T	T	F	长度为 6
2			T	T	T	F	长度为 5
3				T	T	F	长度为 4
4					T	F	长度为 3
5						T	长度为 2
							长度为 1

当我们求长度为 6 和 5 的子串的情况时，其实只用到了 4，3 长度的情况，而长度为 1 和 2 的子串情况其实已经不需要了。但是由于我们并不是用 P 数组的下标进行的循环，暂时没有想到优化的方法。

之后看到了另一种动态规划的思路

<https://leetcode.com/problems/longest-palindromic-substring/discuss/2921/Share-my-Java-solution-using-dynamic-programming>。

公式还是这个不变

首先定义 P (i, j) 。

$P(i,j) = \begin{cases} \text{true} & \text{s[i,j] 是回文串} \\ \text{false} & \text{s[i,j] 不是回文串} \end{cases}$

接下来

$P(i,j) = (P(i+1,j-1) \& \& S[i] == S[j])$

递推公式中我们可以看到，我们首先知道了 i+1 才会知道 i，所以我们只需要倒着遍历就行了。

```
public String longestPalindrome(String s) {  
    int n = s.length();  
    String res = "";
```



```

boolean[][] dp = new boolean[n][n];
for (int i = n - 1; i >= 0; i--) {
    for (int j = i; j < n; j++) {
        dp[i][j] = s.charAt(i) == s.charAt(j) && (j - i < 2 || dp[i + 1][j - 1]); //j - i 代表长度减去 1
        if (dp[i][j] && j - i + 1 > res.length()) {
            res = s.substring(i, j + 1);
        }
    }
}
return res;
}

```

时间复杂度和空间复杂和之前都没有变化，我们来看看可不可以优化空间复杂度。

	j	0	1	2	3	4	5
i	0	T	F	T	T	T	F
	1		T	T	T	T	F
	2			T	T	T	F
	3				T	T	F
	4					T	F
	5						T

当求第 i 行的时候我们只需要第 $i + 1$ 行的信息，并且 j 的话需要 $j - 1$ 的信息，所以和之前一样 j 也需要倒叙。

```

public String longestPalindrome7(String s) {
    int n = s.length();
    String res = "";
    boolean[] P = new boolean[n];
    for (int i = n - 1; i >= 0; i--) {
        for (int j = n - 1; j >= i; j--) {
            P[j] = s.charAt(i) == s.charAt(j) && (j - i < 3 || P[j - 1]);
            if (P[j] && j - i + 1 > res.length()) {
                res = s.substring(i, j + 1);
            }
        }
    }
}

```

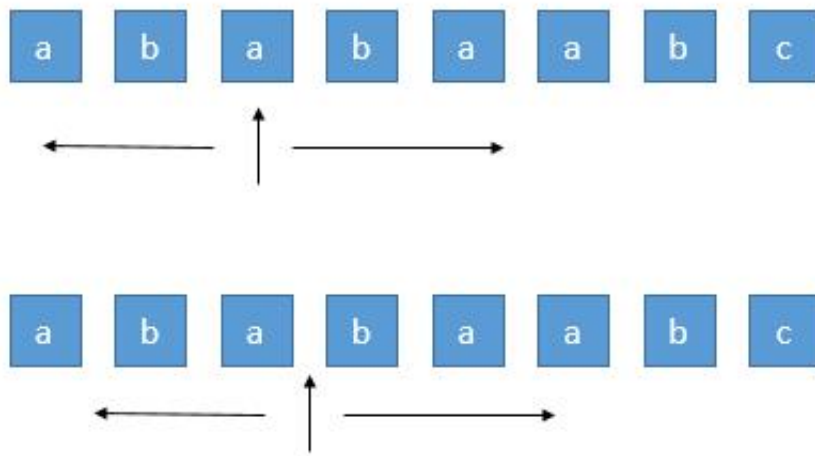
```
}  
return res;  
}
```

时间复杂度：不变， $O(n^2)$ 。

空间复杂度：降为 $O(n)$ 。

解法四 扩展中心

我们知道回文串一定是对称的，所以我们可以每次循环选择一个中心，进行左右扩展，判断左右字符是否相等即可。



由于存在奇数的字符串和偶数的字符串，所以我们需要从一个字符开始扩展，或者从两个字符之间开始扩展，所以总共有 $n + n - 1$ 个中心。

```
public String longestPalindrome(String s) {  
    if (s == null || s.length() < 1) return "";  
    int start = 0, end = 0;  
    for (int i = 0; i < s.length(); i++) {  
        int len1 = expandAroundCenter(s, i, i);  
        int len2 = expandAroundCenter(s, i, i + 1);  
        int len = Math.max(len1, len2);  
        if (len > end - start) {  
            start = i - (len - 1) / 2;  
            end = i + len / 2;  
        }  
    }  
    return s.substring(start, end + 1);  
}  
  
private int expandAroundCenter(String s, int left, int right) {
```

```

int L = left, R = right;
while (L >= 0 && R < s.length() && s.charAt(L) == s.charAt(R)) {
    L--;
    R++;
}
return R - L - 1;
}

```

时间复杂度： $O(n^2)$ 。

空间复杂度： $O(1)$ 。

解法五 Manacher's Algorithm 马拉车算法。

马拉车算法 Manacher's Algorithm 是用来查找一个字符串的最长回文子串的线性方法，由一个叫 Manacher 的人在 1975 年发明的，这个方法的最大贡献是在于将时间复杂度提升到了线性。

主要参考了下边链接进行讲解。

<https://segmentfault.com/a/1190000008484167>

<https://blog.crimx.com/2017/07/06/manachers-algorithm/>

<http://ju.outofmemory.cn/entry/130005>

<https://articles.leetcode.com/longest-palindromic-substring-part-ii/>

首先我们解决下奇数和偶数的问题，在每个字符间插入"#", 并且为了使得扩展的过程中，到边界后自动结束，在两端分别插入 "^" 和 "\$", 两个不可能在字符串中出现的字符，这样中心扩展的时候，判断两端字符是否相等的时候，如果到了边界就一定会不相等，从而出了循环。经过处理，字符串的长度永远都是奇数了。

^ a # b # c # b # a # a # d # e \$

首先我们用一个数组 P 保存从中心扩展的最大个数，而它刚好也是去掉 "#" 的原字符串的总长度。例如下图中下标是 6 的地方。可以看到 $P[6]$ 等于 5，所以它是从左边扩展 5 个字符，相应的右边也是扩展 5 个字符，也就是 "#c#b#c#b#c#"。而去掉 # 恢复到原来的字符串，变成 "cbcbc"，它的长度刚好也就是 5。

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T =	^	#	c	#	b	#	c	#	b	#	c	#	c	#	d	#	e	#	\$
P =	0	0	1	0	3	0	5	0	3	0	1	2	1	0	1	0	1	0	0

T: 处理后的数组 P: 从中心扩展的长度

求原字符串下标

用 P 的下标 i 减去 $P[i]$ ，再除以 2，就是原字符串的开头下标了。

例如我们找到 $P[i]$ 的最大值为 5，也就是回文串的最大长度是 5，对应的下标是 6，所以原字符串的开头下标是 $(6 - 5) / 2 = 0$ 。所以我们只需要返回原字符串的第 0 到第 $(5 - 1)$ 位就可以了。

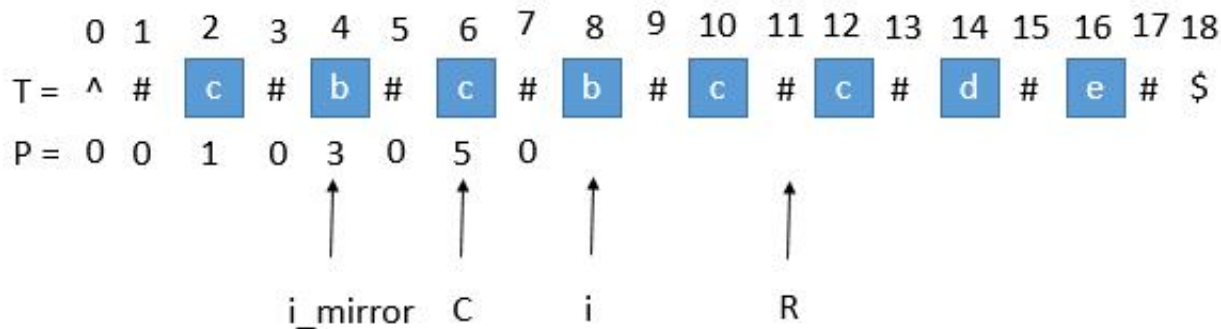
求每个 $P[i]$

接下来是算法的关键了，它充分利用了回文串的对称性。

我们用 C 表示回文串的中心，用 R 表示回文串的右边半径坐标，所以 $R = C + P[C]$ 。 C 和 R 所对应的回文串是当前循环中 R 最靠右的回文串。

让我们考虑求 $P[i]$ 的时候，如下图。

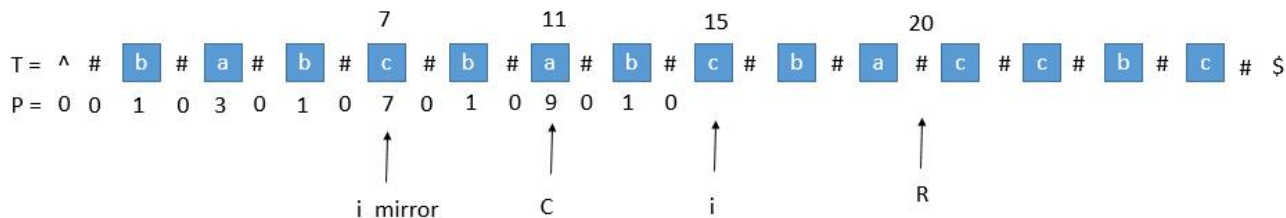
用 i_mirror 表示当前需要求的第 i 个字符关于 C 对应的下标。



我们现在要求 $P[i]$ ，如果是用中心扩展法，那就向两边扩展比对就行了。但是我们其实可以利用回文串 C 的对称性。 i 关于 C 的对称点是 i_mirror ， $P[i_mirror] = 3$ ，所以 $P[i]$ 也等于 3。

但是有三种情况将会造成直接赋值为 $P[i_mirror]$ 是不正确的，下边一一讨论。

1. 超出了 R



当我们要求 $P[i]$ 的时候， $P[mirror] = 7$ ，而此时 $P[i]$ 并不等于 7，为什么呢，因为我们从 i 开始往后数 7 个，等于 22，已经超过了最右的 R ，此时不能利用对称性了，但我们一定可以扩展到 R 的，所以 $P[i]$ 至少等于 $R - i = 20 - 15 = 5$ ，会不会更大呢，我们只需要比较 $T[R+1]$ 和 $T[i+1]$ 关于 i 的对称点就行了，就像中心扩展法一样一个个扩展。

2. P [i_mirror] 遇到了原字符串的左边界

i =	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T =	^	#	c	#	b	#	c	#	b	#	c	#	b	#	d	#	e	#	\$
P =	0	0	1	0	3	0	5	0	3	0									
			↑				↑				↑	↑							
			i_mirror				C				i	R							

此时 $P[i_mirror] = 1$ ，但是 $P[i]$ 赋值成 1 是不正确的，出现这种情况的原因是 $P[i_mirror]$ 在扩展的时候首先是 $\# == \#$ ，之后遇到了 $\wedge$ 和另一个字符比较，也就是到了边界，才终止循环的。而 $P[i]$ 并没有遇到边界，所以我们可以继续通过中心扩展法一步一步向两边扩展就行了。

3. i 等于了 R

此时我们先把 $P[i]$ 赋值为 0，然后通过中心扩展法一步一步扩展就行了。

考虑 C 和 R 的更新

就这样一步一步的求出每个 $P[i]$ ，当求出的 $P[i]$ 的右边界大于当前的 R 时，我们就需要更新 C 和 R 为当前的回文串了。因为我们必须保证 i 在 R 里面，所以一旦有更右边的 R 就要更新 R 。

i =	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T =	^	#	c	#	b	#	c	#	b	#	c	#	b	#	d	#	e	#	\$
P =	0	0	1	0	3	0	5	0	3	0									
			↑				↑				↑	↑							
			i_mirror				C				i	R							

此时的 $P[i]$ 求出来将会是 3， $P[i]$ 对应的右边界将是 $10 + 3 = 13$ ，所以大于当前的 R ，我们需要把 C 更新成 i 的值，也就是 10， R 更新成 13。继续下边的循环。

```
public String preProcess(String s) {  
    int n = s.length();  
    if (n == 0) {  
        return "^$";  
    }  
}
```

```

String ret = "^";
for (int i = 0; i < n; i++)
    ret += "#" + s.charAt(i);
ret += "#$";
return ret;
}

// 马拉车算法
public String longestPalindrome(String s) {
    String T = preprocess(s);
    int n = T.length();
    int[] P = new int[n];
    int C = 0, R = 0;
    for (int i = 1; i < n - 1; i++) {
        int i_mirror = 2 * C - i;
        if (R > i) {
            P[i] = Math.min(R - i, P[i_mirror]); // 防止超出 R
        } else {
            P[i] = 0; // 等于 R 的情况
        }

        // 碰到之前讲的三种情况时候，需要利用中心扩展法
        while (T.charAt(i + 1 + P[i]) == T.charAt(i - 1 - P[i])) {
            P[i]++;
        }

        // 判断是否需要更新 R
        if (i + P[i] > R) {
            C = i;
            R = i + P[i];
        }
    }

    // 找出 P 的最大值
    int maxLen = 0;
    int centerIndex = 0;
    for (int i = 1; i < n - 1; i++) {
        if (P[i] > maxLen) {
            maxLen = P[i];
            centerIndex = i;
        }
    }
    int start = (centerIndex - maxLen) / 2; // 最开始讲的求原字符串下标
    return s.substring(start, start + maxLen);
}

```

时间复杂度：for 循环里边套了一层 while 循环，难道不是 $O(n^2)$ ？不！其实是 $O(n)$ 。不严谨的想一下，因为 while 循环访问 R 右边的数字用来扩展，也就是那些还未求出的节点，然后不断扩展，而期间访问的节点下次就不会再进入 while 了，可以利用对称得到自己的解，所以每个节点访问都是常数次，所以是 $O(n)$ 。

空间复杂度： $O(n)$ 。

总结

时间复杂度从三次方降到了一次，美妙！这里两次用到了动态规划去求解，初步认识了动态规划，就是将之前求的值保存起来，方便后边的计算，使得一些多余的计算消失了。并且在动态规划中，通过观察数组的利用情况，从而降低了空间复杂度。而 Manacher 算法对回文串对称性的充分利用，不得不让人叹服，自己加油啦！

06、题目描述（中等难度）

The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows like this: (you may want to display this pattern in a fixed font for better legibility)

```
P   A   H   N
A P L S I I G
Y   I   R
```

And then read line by line: "PAHNAPLSIIGYIR"

Write the code that will take a string and make this conversion given a number of rows:

```
string convert(string s, int numRows);
```

Example 1:

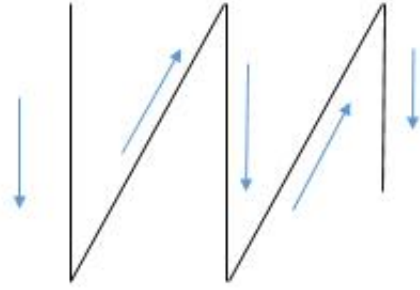
```
Input: s = "PAYPALISHIRING", numRows = 3
Output: "PAHNAPLSIIGYIR"
```

Example 2:

```
Input: s = "PAYPALISHIRING", numRows = 4
Output: "PINALSIGYAHRPI"
Explanation:

P     I     N
A   L S   I G
Y A   H R
P     I
```

就是给定一个字符串，然后按写竖着的「Z」的方式排列字符，就是下边的样子。



然后按行的方式输出每个字符，第 0 行，第 1 行，第 2 行

解法一

按照写 Z 的过程，遍历每个字符，然后将字符存到对应的行中。用 goingDown 保存当前的遍历方向，如果遍历到两端，就改变方向。

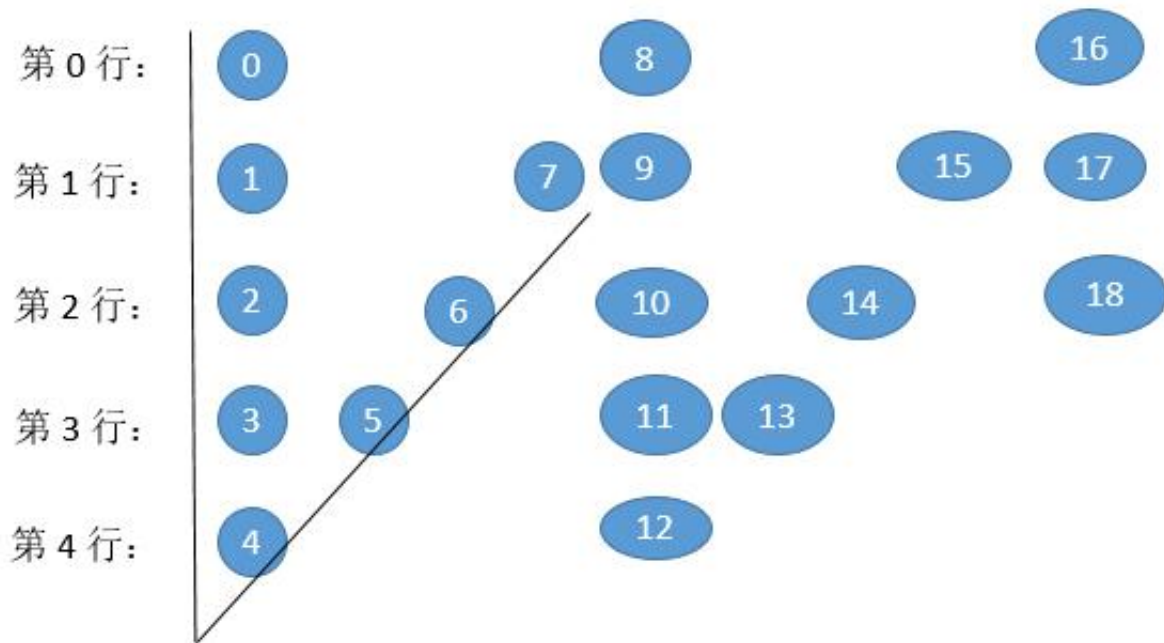
```
public String convert(String s, int numRows) {  
  
    if (numRows == 1) return s;  
  
    List<StringBuilder> rows = new ArrayList<>();  
    for (int i = 0; i < Math.min(numRows, s.length()); i++)  
        rows.add(new StringBuilder());  
  
    int curRow = 0;  
    boolean goingDown = false;  
  
    for (char c : s.toCharArray()) {  
        rows.get(curRow).append(c);  
        if (curRow == 0 || curRow == numRows - 1) goingDown = !goingDown; //遍历到两端，改变方向  
        curRow += goingDown ? 1 : -1;  
    }  
  
    StringBuilder ret = new StringBuilder();  
    for (StringBuilder row : rows) ret.append(row);  
    return ret.toString();  
}
```

时间复杂度：O (n) ， n 是字符串的长度。

空间复杂度：O (n) ， 保存每个字符需要的空间。

解法二

找出按 Z 形排列后字符的规律，然后直接保存起来。



我们可以看到，图形其实是有周期的，0, 1, 2 ... 7 总过 8 个，然后就又开始重复相同的路径。周期的计算就是 $\text{cycleLen} = 2 \times \text{numRows} - 2 = 2 \times 5 - 2 = 8$ 个。

我们发现第 0 行和最后一行一个周期内有一个字符，所以第一个字符下标是 0，第二个字符下标是 $0 + \text{cycleLen} = 8$ ，第三个字符下标是 $8 + \text{cycleLen} = 16$ 。

其他行都是两个字符。

第 1 个字符和第 0 行的规律是一样的。

第 2 个字符其实就是下一个周期的第 0 行的下标减去当前行。什么意思呢？

我们求一下第 1 行第 1 个周期内的第 2 个字符，下一个周期的第 0 行的下标是 8，减去当前行 1，就是 7 了。

我们求一下第 1 行第 2 个而周期内的第 2 个字符，下一个周期的第 0 行的下标是 16，减去当前行 1，就是 15 了。

我们求一下第 2 行第 1 个周期内的第 2 个字符，下一个周期的第 0 行的下标是 8，减去当前行 2，就是 6 了。

当然期间一定要保证下标小于 n，防止越界。

可以写代码了。

```
public String convert(String s, int numRows) {
```

```

if (numRows == 1)
    return s;

StringBuilder ret = new StringBuilder();
int n = s.length();
int cycleLen = 2 * numRows - 2;

for (int i = 0; i < numRows; i++) {
    for (int j = 0; j + i < n; j += cycleLen) { //每次加一个周期
        ret.append(s.charAt(j + i));
        if (i != 0 && i != numRows - 1 && j + cycleLen - i < n) //除去第 0 行和最后一
行
            ret.append(s.charAt(j + cycleLen - i));
    }
}
return ret.toString();
}

```

时间复杂度： $O(n)$ ，虽然是两层循环，但第二次循环每次加的是 `cycleLen`，无非是把每个字符遍历了 1 次，所以两层循环内执行的次数肯定是字符串的长度。

空间复杂度： $O(n)$ ，保存字符串。

总结

这次算是总结起来最轻松的了，这道题有些找规律的意思。解法一顺着排列的方式遍历，解法二直接从答案入口找出下标的规律。

07、题目描述（简单难度）

Given a 32-bit signed integer, reverse digits of an integer.

Example 1:

Input: 123
Output: 321

Example 2:

Input: -123
Output: -321

Example 3:

Input: 120
Output: 21

Note:

Assume we are dealing with an environment which could only store integers within the 32-bit signed integer range: $[-2^{31}, 2^{31} - 1]$. For the purpose of this problem, assume that your function returns 0 when the reversed integer overflows.

很简单，就是输入整数，输出它的倒置。

第一反应就是，取余得到个位数，然后除以 10 去掉个位数，然后用一个变量保存倒置的数。

```
public int reverse(int x) {  
    int rev = 0;  
    while (x != 0) {  
        int pop = x % 10;  
        x /= 10;  
        rev = rev * 10 + pop;  
    }  
    return rev;  
}
```

然后似乎不是那么理想。

Submission Result: **Wrong Answer**

[More Details >](#)

>_

Input: 1534236469

Output: 1056389759

Expected: 0

为什么呢？倒置过来不应该是 9646324351 吗。其实题目里讲了，int 的范围是 $[-2^{31}, 2^{31}-1]$ 也就是 $[-2147483648, 2147483647]$ 。明显 9646324351 超出了范围，造成了溢出。所以我们需要在输出前，判断是否溢出。

问题的关键就是下边的一句了。

```
rev = rev * 10 + pop;
```

为了区分两个 rev，更好的说明，我们引入 temp。

```
temp = rev * 10 + pop;
```

```
rev = temp;
```

我们对 $temp = rev * 10 + pop$ 进行讨论。intMAX = 2147483647, intMin = - 2147483648。

对于大于 intMax 的讨论，此时 x 一定是正数，pop 也是正数。

- 如果 $rev > intMax / 10$ ，那么没的说，此时肯定溢出了。
- 如果 $rev == intMax / 10 = 2147483647 / 10 = 214748364$ ，此时 $rev * 10$ 就是 2147483640 如果 pop 大于 7，那么就一定溢出了。但是！如果假设 pop 等于 8，那么意味着原数 x 是 8463847412 了，输入的是 int，而此时是溢出的状态，所以不可能输入，所以意味着 pop 不可能大于 7，也就意味着 $rev == intMax / 10$ 时不会造成溢出。
- 如果 $rev < intMax / 10$ ，意味着 rev 最大是 214748363， $rev * 10$ 就是 2147483630，此时再加上 pop，一定不会溢出。

对于小于 intMin 的讨论同理。

```
public int reverse(int x) {
    int rev = 0;
    while (x != 0) {
        int pop = x % 10;
        x /= 10;
        if (rev > Integer.MAX_VALUE/10 ) return 0;
        if (rev < Integer.MIN_VALUE/10 ) return 0;
        rev = rev * 10 + pop;
    }
    return rev;
}
```

时间复杂度：循环多少次呢？数字有多少位，就循环多少次，也就是 $\log_{10}(x) + 1$ 次，所以时间复杂度是 $O(\log(x))$ 。

空间复杂度： $O(1)$ 。

当然我们可以不用思考那么多，用一种偷懒的方式 AC，我们直接把 rev 定义成 long，然后输出前判断 rev 是不是在范围内，不在的话直接输出 0。

```
public int reverse(int x) {  
    long rev = 0;  
    while (x != 0) {  
        int pop = x % 10;  
        x /= 10;  
        rev = rev * 10 + pop;  
    }  
    if (rev > Integer.MAX_VALUE || rev < Integer.MIN_VALUE ) return 0;  
    return (int)rev;  
}
```

总结

比较简单的一道题，主要是在考判断是不是溢出，又是轻松的一天！

08、题目描述（中等难度）

Implement `atoi` which converts a string to an integer.

The function first discards as many whitespace characters as necessary until the first non-whitespace character is found. Then, starting from this character, takes an optional initial plus or minus sign followed by as many numerical digits as possible, and interprets them as a numerical value.

The string can contain additional characters after those that form the integral number, which are ignored and have no effect on the behavior of this function.

If the first sequence of non-whitespace characters in `str` is not a valid integral number, or if no such sequence exists because either `str` is empty or it contains only whitespace characters, no conversion is performed.

If no valid conversion could be performed, a zero value is returned.

Note:

- Only the space character ' ' is considered as whitespace character.
- Assume we are dealing with an environment which could only store integers within the 32-bit signed integer range: $[-2^{31}, 2^{31} - 1]$. If the numerical value is out of the range of representable values, `INT_MAX` ($2^{31} - 1$) or `INT_MIN` (-2^{31}) is returned.

Example 1:

```
Input: "42"  
Output: 42
```

Example 2:

```
Input: " -42"  
Output: -42  
Explanation: The first non-whitespace character is '-', which is the minus sign.  
Then take as many numerical digits as possible, which gets 42.
```

Example 3:

```
Input: "4193 with words"  
Output: 4193  
Explanation: Conversion stops at digit '3' as the next character is not a numerical digit.
```

Example 4:

```
Input: "words and 987"  
Output: 0  
Explanation: The first non-whitespace character is 'w', which is not a numerical  
digit or a +/- sign. Therefore no valid conversion could be performed.
```

Example 5:

```
Input: "-91283472332"  
Output: -2147483648  
Explanation: The number "-91283472332" is out of the range of a 32-bit signed integer.  
Therefore INT_MIN ( $-2^{31}$ ) is returned.
```

将一个字符串转为整型。

这道题，难度其实不大，和[上道题](#)有很多重合的地方。整体的思路就是遍历字符串，然后依次取出一个字符就可以了。无非是考虑一些特殊情况，还有就是理解题目意思。

经过多次试错，题目的意思是这样的。

从左遍历字符串，可以遇到空格，直到遇到 '+' 或者数字或者 '-' 就表示要转换的数字开始，如果之后遇到除了数字的其他字符（包括空格）就结束遍历，输出结果，不管后边有没有数字了，例如 " - 32332ada2323" 就输出 "- 32332"。

如果遇到空格或者 '+' 或者数字或者 '-' 之前遇到了其他字符，就直接输出 0，例如 " we1332"。

如果转换的数字超出了 int，就返回 intMax 或者 intMin。

```
public int myAtoi(String str) {
    int sign = 1;
    int ans = 0, pop = 0;
    boolean hasSign = false; //代表是否开始转换数字
    for (int i = 0; i < str.length(); i++) {
        if (str.charAt(i) == '-' && !hasSign) {
            sign = -1;
            hasSign = true;
            continue;
        }
        if (str.charAt(i) == '+' && !hasSign) {
            sign = 1;
            hasSign = true;
            continue;
        }
        if (str.charAt(i) == ' ' && !hasSign) {
            continue;
        }

        if (str.charAt(i) >= '0' && str.charAt(i) <= '9') {
            hasSign = true;
            pop = str.charAt(i) - '0';
            //和上道题判断出界一个意思只不过记得乘上 sign 。
            if (ans * sign > Integer.MAX_VALUE / 10 || (ans * sign ==
Integer.MAX_VALUE / 10 && pop * sign > 7))
                return 2147483647;
            if (ans * sign < Integer.MIN_VALUE / 10 || (ans * sign ==
Integer.MIN_VALUE / 10 && pop * sign < -8))
                return -2147483648;
            ans = ans * 10 + pop;
        } else {
            return ans * sign;
        }
    }
    return ans * sign;
}
```

时间复杂度：O (n)，n 是字符串的长度。

空间复杂度：O (1) 。

总结

这道题让自己有点感到莫名其妙，好像没有 get 到出题人的点？？？

09、题目描述（简单难度）

Determine whether an integer is a palindrome. An integer is a palindrome when it reads the same backward as forward.

Example 1:

Input: 121
Output: true

Example 2:

Input: -121
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: 10
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

Follow up:

Could you solve it without converting the integer to a string?

判断是不是回文数，负数不是回文数。

解法一

把 int 转成字符串，然后判断是否是回文串做就可以了，缺点是需要额外的空间存储字符串，当然题目也告诉了不能这样，所以 pass 。

解法二

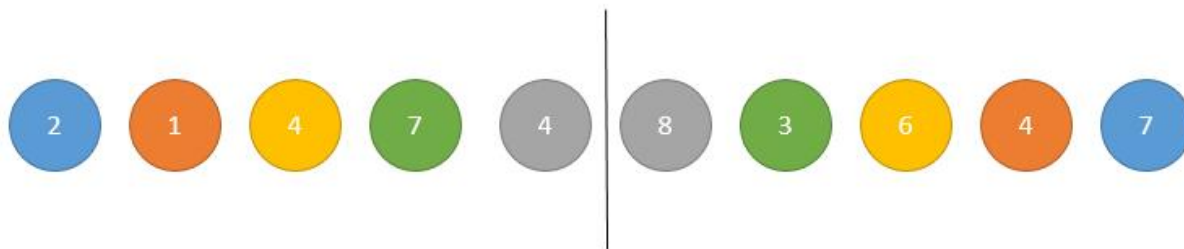
在[第 7 道题](#)我们写了倒置 int 的算法，这里当然可以用到了，只需要判断倒置前后是否相等就可以了。

记不得，当倒置后的数字超出 int 的范围时，我们返回的是 0，那么它一定不等于原数，此时一定返回 false 了，这正不正确呢？

我们只需证明，如果倒置后超出 int 的范围，那么它一定不是回文数字就好了。

反证法，我们假设存在这么一个数，倒置后是超出 int 范围的，并且它是回文数字。

int 最大为 2147483647，



让我们来讨论这个数可能是多少。

有没有可能是最高位大于 2 导致的溢出，比如最高位是 3，因为是回文串，所以最低位是 3，这就将导致转置前最高位也会是 3，所以不可能是这种情况。

有没有可能是第 2 高位大于 1 导致的溢出，此时保持最高位不变，假如第 2 高位是 2，因为是回文串，所以个位是 2，十位是 2，同样的会导致倒置前超出了 int 的最大值，所以也不可能是这种情况。

同理，第 3 高位，第 4，第 5，直线左边的都是上述的情况，所以不可能是前边的位数过大。

为了保证这个数是溢出的，前边 5 位必须固定不变了，因为它是回文串，所以直线后的灰色数字就一定是 4，而此时不管后边的数字取多少，都不可能是溢出的了。

综上，不存在这样一个数，所以可以安心的写代码了。

```
public int reverse(int x) {
    int rev = 0;
    while (x != 0) {
        int pop = x % 10;
        x /= 10;
        if (rev > Integer.MAX_VALUE / 10)
            return 0;
        if (rev < Integer.MIN_VALUE / 10)
            return 0;
        rev = rev * 10 + pop;
    }
    return rev;
}

public boolean isPalindrome(int x) {
    if (x < 0) {
        return false;
    }
    int rev = reverse(x);
    return x == rev;
}
```

时间复杂度：和求转置一样，x 有多少位，就循环多少次，所以是 $O(\log(x))$ 。

空间复杂度： $O(1)$ 。

解法三

其实，我们只需要将右半部分倒置然后和左半部比较就可以了。比如 1221，把 21 转置和 12 比较就行了。

```
public boolean isPalindrome(int x) {
    if (x < 0) {
        return false;
    }
    int digit = (int) (Math.log(x) / Math.log(10) + 1); //总位数
    int revert = 0;
    int pop = 0;
    //倒置右半部分
    for (int i = 0; i < digit / 2; i++) {
        pop = x % 10;
        revert = revert * 10 + pop;
        x /= 10;
    }
    if (digit % 2 == 0 && x == revert) {
        return true;
    }
    //奇数情况 x 除以 10 去除 1 位
    if (digit % 2 != 0 && x / 10 == revert) {
        return true;
    }
    return false;
}
```

时间复杂度：循环 x 的总位数的一半次，所以时间复杂度依旧是 $O(\log(x))$ 。

空间复杂度： $O(1)$ ，常数个变量。

总结

这几天都比较简单，加油加油加油！。

10、题目描述（困难难度）

Given an input string (`s`) and a pattern (`p`), implement regular expression matching with support for `.` and `*`.

```
'.' Matches any single character.  
'*' Matches zero or more of the preceding element.
```

The matching should cover the **entire** input string (not partial).

Note:

- `s` could be empty and contains only lowercase letters `a-z`.
- `p` could be empty and contains only lowercase letters `a-z`, and characters like `.` or `*`.

Example 1:

```
Input:  
s = "aa"  
p = "a"  
Output: false  
Explanation: "a" does not match the entire string "aa".
```

Example 2:

```
Input:  
s = "aa"  
p = "a*"  
Output: true  
Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".
```

Example 3:

```
Input:  
s = "ab"  
p = ".*"  
Output: true  
Explanation: ".*" means "zero or more (*) of any character (.)".
```

Example 4:

```
Input:  
s = "aab"  
p = "c*a*b"  
Output: true  
Explanation: c can be repeated 0 times, a can be repeated 1 time. Therefore it matches "aab".
```

Example 5:

```
Input:  
s = "mississippi"  
p = "mis*is*p*."  
Output: false
```

一个简单规则的匹配，「点`.`」代表任意字符，「星号`*`」代表前一个字符重复 0 次或任意次。

解法一 递归

假如没有通配符 `*`，这道题的难度就会少了很多，我们只需要一个字符，一个字符匹配就行。如果对递归不是很了解，强烈建议看下[这篇文章](#)，可以理清一下递归的思路。

- 我们假设存在这么个函数 `isMatch`，它将告诉我们 `text` 和 `pattern` 是否匹配

```
boolean isMatch ( String text, String pattern );
```

- 递归规模减小

text 和 pattern 匹配，等价于 text 和 pattern 的第一个字符匹配并且剩下的字符也匹配，而判断剩下的字符是否匹配，我们就可以调用 isMatch 函数。也就是

```
(pattern.charAt(0) == text.charAt(0) || pattern.charAt(0) ==  
'.')&&isMatch(text.substring(1), pattern.substring(1));
```

- 递归出口

随着规模的减小，当 pattern 为空时，如果 text 也为空，就返回 True，不然的话就返回 False。

```
if (pattern.isEmpty()) return text.isEmpty();
```

综上，我们的代码是

```
public boolean isMatch(String text, String pattern) {  
    if (pattern.isEmpty()) return text.isEmpty();  
  
    //判断 text 是否为空，防止越界，如果 text 为空，表达式直接判为 false，  
    text.charAt(0)就不会执行了  
    boolean first_match = (!text.isEmpty() &&  
        (pattern.charAt(0) == text.charAt(0) ||  
        pattern.charAt(0) == '.'));  
    return first_match && isMatch(text.substring(1), pattern.substring(1));  
}
```

当我们考虑了 * 呢，对于递归规模的减小，会增加对于 * 的判断，直接看代码吧。

```
public boolean isMatch(String text, String pattern) {  
    if (pattern.isEmpty()) return text.isEmpty();  
  
    boolean first_match = (!text.isEmpty() &&  
        (pattern.charAt(0) == text.charAt(0) ||  
        pattern.charAt(0) == '.'));  
    //只有长度大于 2 的时候，才考虑 *  
    if (pattern.length() >= 2 && pattern.charAt(1) == '*'){  
        //两种情况  
        //pattern 直接跳过两个字符。表示 * 前边的字符出现 0 次  
        //pattern 不变，例如 text = aa , pattern = a*，第一个 a 匹配，然后 text 的  
        第二个 a 接着和 pattern 的第一个 a 进行匹配。表示 * 用前一个字符替代。  
        return (isMatch(text, pattern.substring(2)) ||
```

```
        (first_match && isMatch(text.substring(1), pattern)));  
    } else {  
        return first_match && isMatch(text.substring(1),  
pattern.substring(1));  
    }  
}
```

时间复杂度：有点儿小复杂，待更。

空间复杂度：有点儿小复杂，待更。

解法二 动态规划

上边的递归，为了方便理解，简化下思路。

为了判断 text[0, len] 的情况，需要知道 text[1, len]

为了判断 text[1, len] 的情况，需要知道 text[2, len]

为了判断 text[2, len] 的情况，需要知道 text[3, len]

...

为了判断 text[len - 1, len] 的情况，需要知道 text[len, len]

text[len, len] 肯定好求

求出 text[len, len] 的情况，就知道了 text[len - 1, len]

求出 text[len - 1, len] 的情况，就知道了 text[len - 2, len]

...

求出 text[2, len] 的情况，就知道了 text[1, len]

求出 text[1, len] 的情况，就知道了 text[0, len]

从而知道了 text[0, len] 的情况，求得问题的解。

上边就是先压栈，然后出栈，其实我们可以直接倒过来求，可以省略压栈的过程。

我们先求 text[len, len] 的情况

利用 text[len, len] 的情况，再求 text[len - 1, len] 的情况

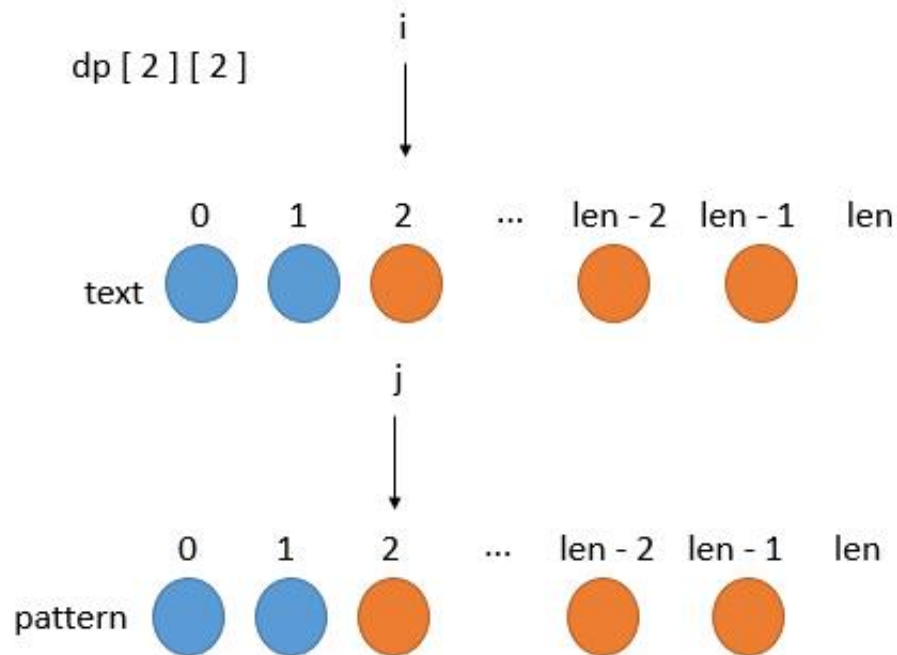
...

利用 text[2, len] 的情况，再求 text[1, len] 的情况

利用 text[1, len] 的情况，再求 text[0, len] 的情况

从而求出问题的解

我们用 $dp[i][j]$ 表示 text 从 i 开始到最后，pattern 从 j 开始到最后，此时 text 和 pattern 是否匹配。



$dp[2][2]$ 就是图中橙色的部分。

```
public boolean isMatch(String text, String pattern) {  
    // 多一维的空间，因为求  $dp[len - 1][j]$  的时候需要知道  $dp[len][j]$  的情况，  
    // 多一维的话，就可以把对  $dp[len - 1][j]$  也写进循环了  
    boolean[][] dp = new boolean[text.length() + 1][pattern.length() + 1];  
    //  $dp[len][len]$  代表两个空串是否匹配了，"" 和 ""，当然是 true 了。  
    dp[text.length()][pattern.length()] = true;  
  
    // 从 len 开始减少  
    for (int i = text.length(); i >= 0; i--) {  
        for (int j = pattern.length(); j >= 0; j--) {  
            //  $dp[text.length()][pattern.length()]$  已经进行了初始化  
            if (i == text.length() && j == pattern.length()) continue;  
  
            boolean first_match = (i < text.length() && j < pattern.length()  
                && (pattern.charAt(j) == text.charAt(i) ||  
                    pattern.charAt(j) == '.'));  
            if (j + 1 < pattern.length() && pattern.charAt(j + 1) == '*') {  
                dp[i][j] = dp[i][j + 2] || first_match && dp[i + 1][j];  
            } else {  
                dp[i][j] = first_match && dp[i + 1][j + 1];  
            }  
        }  
    }  
}
```

```
        return dp[0][0];
    }
```

时间复杂度：假设 text 的长度是 T，pattern 的长度是 P，空间复杂度就是 $O(TP)$ 。

空间复杂度：申请了 dp 空间，所以是 $O(TP)$ ，因为每次循环我们只需要知道 i 和 i + 1 时候的情况，所以我们可以向 [第 5 题](#) 一样进行优化。

```
public boolean isMatch(String text, String pattern) {
    // 多一维的空间，因为求 dp[len - 1][j] 的时候需要知道 dp[len][j] 的情况，
    // 多一维的话，就可以把对 dp[len - 1][j] 也写进循环了
    boolean[][] dp = new boolean[2][pattern.length() + 1];
    dp[text.length() % 2][pattern.length()] = true;

    // 从 len 开始减少
    for (int i = text.length(); i >= 0; i--) {
        for (int j = pattern.length(); j >= 0; j--) {
            if (i == text.length() && j == pattern.length()) continue;
            boolean first_match = (i < text.length() && j < pattern.length()
                && (pattern.charAt(j) == text.charAt(i) || pattern.charAt(j) ==
                    '.'));
            if (j + 1 < pattern.length() && pattern.charAt(j + 1) == '*') {
                dp[i % 2][j] = dp[i % 2][j + 2] || first_match && dp[(i + 1) % 2][j];
            } else {
                dp[i % 2][j] = first_match && dp[(i + 1) % 2][j + 1];
            }
        }
    }
    return dp[0][0];
}
```

时间复杂度：不变， $O(TP)$ 。

空间复杂度：主要用了两个数组进行轮换， $O(P)$ 。

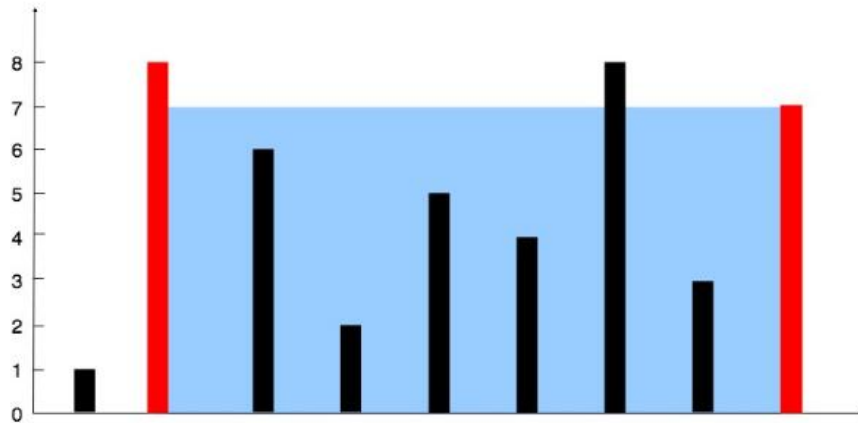
总

这道题对于递归的解法，感觉难在怎么去求时间复杂度，现在还没有什么思路，以后再来补充吧。整体来说，只要理清思路，两种算法还是比较好理解的。

11、题目描述（中等难度）

Given n non-negative integers $a_1, a_2, \dots, a_n$, where each represents a point at coordinate (i, a_i) . n vertical lines are drawn such that the two endpoints of line i is at (i, a_i) and $(i, 0)$. Find two lines, which together with x-axis forms a container, such that the container contains the most water.

Note: You may not slant the container and n is at least 2.



The above vertical lines are represented by array $[1, 8, 6, 2, 5, 4, 8, 3, 7]$. In this case, the max area of water (blue section) the container can contain is 49.

Example:

Input: $[1, 8, 6, 2, 5, 4, 8, 3, 7]$
Output: 49

每个数组代表一个高度，选两个任意的柱子往里边倒水，能最多倒多少水。

解法一 暴力解法

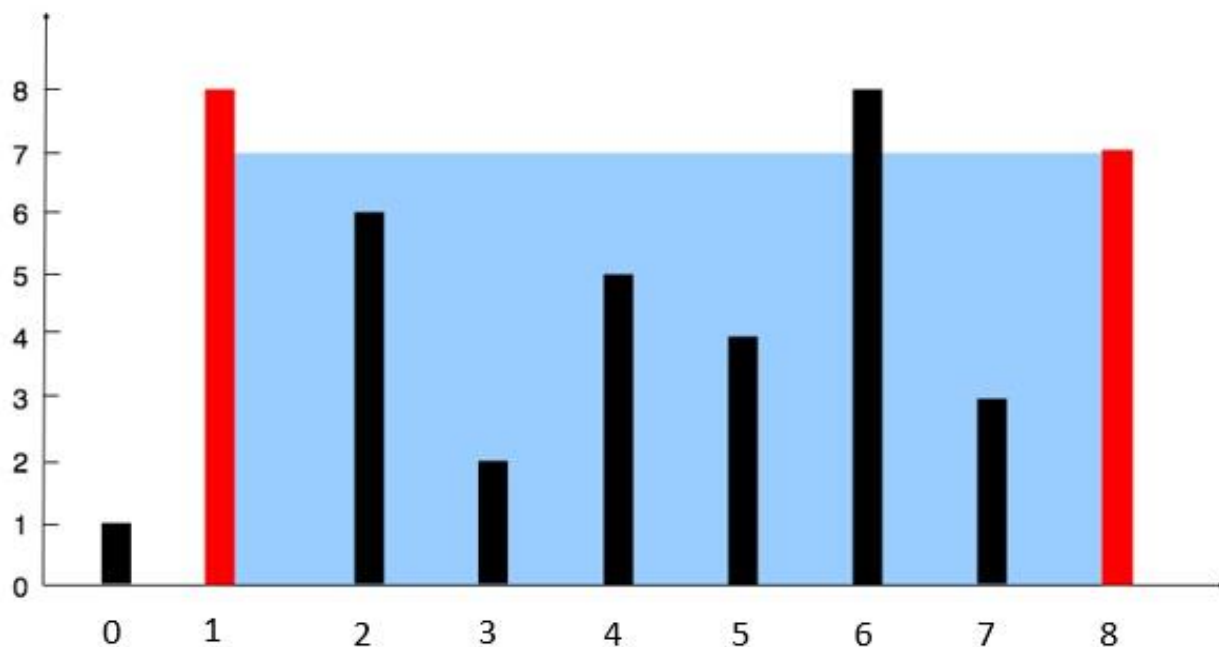
直接遍历任意两根柱子，求出能存水的大小，用一个变量保存最大的。

```
public int maxArea(int[] height) {  
    int max = 0;  
    for (int i = 0; i < height.length; i++) {  
        for (int j = i + 1; j < height.length; j++) {  
            int h = Math.min(height[i], height[j]);  
            if (h * (j - i) > max) {  
                max = h * (j - i);  
            }  
        }  
    }  
    return max;  
}
```

时间复杂度： $O(n^2)$ 。

空间复杂度： $O(1)$ 。

解法二

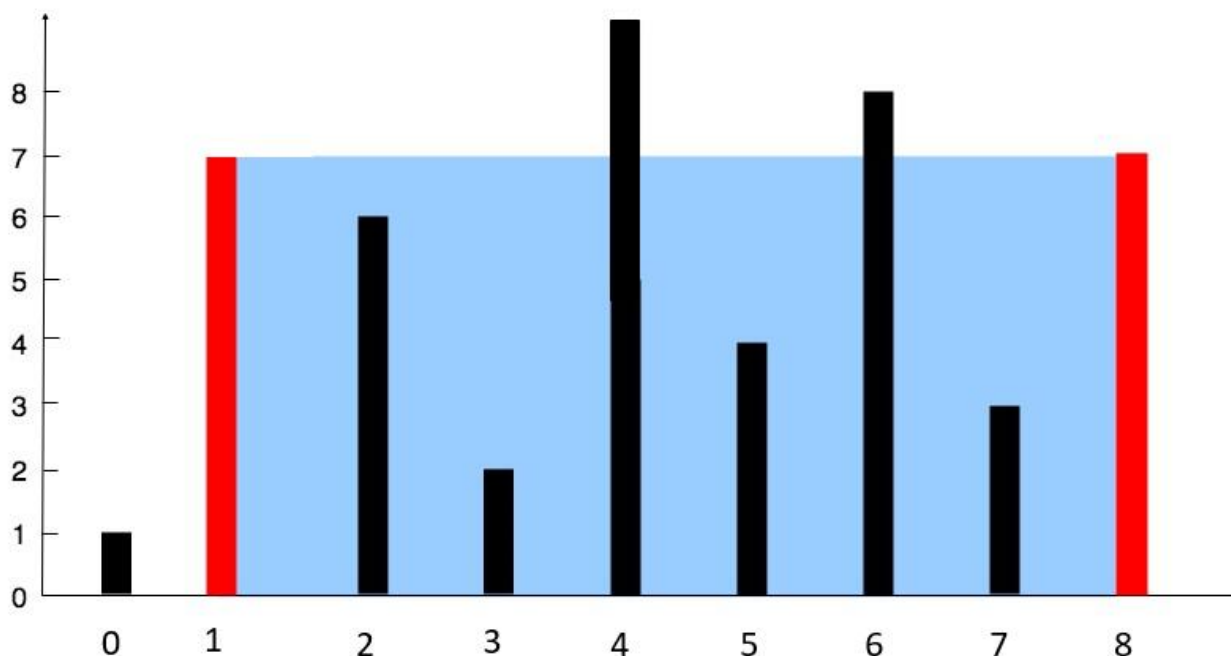


我们理一下思路，大小是由长度和高度决定，如果选 0 到 8 就保证了长度最长，此时大小是 0 号柱子的高度 1 乘以长度 8。我们如果想面积更大怎么做呢，只能减小长度，增加高度。是左边的柱子向右移动变成 1 号柱子呢？还是右边的柱子向左移动变成 7 号柱子呢？当然是哪边的柱子短就改哪边的！只有这样，高度才有可能增加。

例如我们如果把 8 号柱子变成 7 号柱子，此时长度减少了，然而高度还是 0 号柱子没有变化，所以面积就会减少。把 1 号柱子变成 2 号柱子就很好了，因为此时高度就变成了 8 号柱子的高度，面积就有可能增加。

如果左右两边柱子相等该怎么办呢？随意！

我们假设 1 号 和 8 号 柱子高度是相等的。如果他们之间的柱子只有 1 根比它俩高或者没有比它俩高的，那么最大面积就一定选取是 1 号和 8 号了，所以 1 号接着变大，或者 8 号接着减小都是无所谓的，因为答案已经确定了。



假设 1 号 和 8 号之间有 2 根或以上的柱子比它俩高，假设是 4 号和 6 号比它俩高。1 号会变到 2 号、3 号，最终为 4 号，8 号会变到 7 号，6 号，而在这个过程中产生的面积一定不会比 1 号和 8 号产生的面积大，因为过程中的柱子都比 1 号和 8 号低。所以是先变 1 号还是先变 8 号是无所谓的，无非是谁先到达更长的柱子而已。

看一下下边的算法，会更加清楚一些。

```
public int maxArea2(int[] height) {
    int maxarea = 0, l = 0, r = height.length - 1;
    while (l < r) {
        maxarea = Math.max(maxarea, Math.min(height[l], height[r]) * (r - l));
        if (height[l] < height[r])
            l++;
        else
            r--;
    }
    return maxarea;
}
```

时间复杂度： $O(n)$ 。

空间复杂度： $O(1)$ 。

总

为了减少暴力解法的时间复杂度，只能去深层次的理解题意，从而找出突破点。

12、题目描述（中等难度）

Roman numerals are represented by seven different symbols: **I**, **V**, **X**, **L**, **C**, **D** and **M**.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, two is written as **II** in Roman numeral, just two one's added together. Twelve is written as, **XII**, which is simply **X** + **II**. The number twenty seven is written as **XXVII**, which is **XX** + **V** + **II**.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not **IIII**. Instead, the number four is written as **IV**. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as **IX**. There are six instances where subtraction is used:

- I** can be placed before **V** (5) and **X** (10) to make 4 and 9.
- X** can be placed before **L** (50) and **C** (100) to make 40 and 90.
- C** can be placed before **D** (500) and **M** (1000) to make 400 and 900.

Given an integer, convert it to a roman numeral. Input is guaranteed to be within the range from 1 to 3999.

把数字转换成罗马数字，正常情况就是把每个字母相加，并且大字母在前，小字母在后，上边也介绍了像 4 和 9 那些特殊情况。

解法一

这个是自己的解法，主要思想就是每次取出一位，然后得到相应的罗马数字，然后合起来就行。

```
public String getRoman(int num,int count){ //count 表示当前的位数，个位，十位...
    char[]ten={'I','X','C','M'}; //1,10,100,1000
    char[]five={'V','L','D'}; //5,50,500
    String r="";
    if(num<=3){
        while(num!=0){
            r+=ten[count];
            num--;
        }
    }
    if(num==4){
        r=(ten[count]+"")+(five[count]+"");
    }
    if(num==5){
        r=five[count]+"";
    }
}
```

```

    }
    if(num>5&&num<9){
        r=five[count]+"";
        num-=5;
        while(num!=0){
            r+=ten[count];
            num--;
        }
    }
    if(num==9){
        r = (ten[count] + "") + (ten[count + 1] + "");
    }
    return r;
}

public String intToRoman(int num) {
    String r="";
    int count=0;
    while(num!=0){
        int pop=num%10;
        r=getRoman(pop,count)+r;
        count++;
        num/=10;
    }
    return r;
}

```

时间复杂度：num 的位数 $\log_{10}(\text{num})+1$ 所以时间复杂度是 $O(\log(n))$ 。

空间复杂度：常数个变量， $O(1)$ 。

下边在分享一些 LeetCode 讨论里的一些解法。

解法二

<https://leetcode.com/problems/integer-to-roman/discuss/6310/My-java-solution-easy-to-understand>

```

public String intToRoman(int num) {
    int[] values = {1000,900,500,400,100,90,50,40,10,9,5,4,1};
    String[] strs = {"M","CM","D","CD","C","XC","L","XL","X","IX","V","IV","I"};

    StringBuilder sb = new StringBuilder();

    for(int i=0;i<values.length;i++) {
        while(num >= values[i]) {
            num -= values[i];
            sb.append(strs[i]);
        }
    }
}

```

```

    }
}
return sb.toString();
}

```

相当简洁了，主要就是把所有的组合列出来，因为罗马数字表示的大小就是把所有字母相加，所以每次 append 那个，再把对应的值减去就行了。

时间复杂度：不是很清楚，也许是 $O(1)$ ？因为似乎和问题规模没什么关系了。

空间复杂度： $O(1)$ 。

解法三

<https://leetcode.com/problems/integer-to-roman/discuss/6376/Simple-JAVA-solution>

```

public String intToRoman(int num) {
    String M[] = {"", "M", "MM", "MMM"}; // 0, 1000, 2000, 3000
    String C[] = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC",
"CM"}; // 0, 100, 200, 300...
    String X[] = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX",
"XC"}; // 0, 10, 20, 30...
    String I[] = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII",
"IX"}; // 0, 1, 2, 3...
    return M[num/1000] + C[(num%1000)/100] + X[(num%100)/10] + I[num%10];
}

```

这就更加暴力了，把每位的情况都列出来然后直接返回，但思路清晰明了呀。

时间复杂度： $O(1)$ 或者说是 num 的位数，不是很确定。

空间复杂度： $O(1)$ 。

总

这道题感觉难度应该是 easy，没有那么难，就是理清楚题意，然后就可以往出列举就行了。

13、题目描述（简单难度）

Roman numerals are represented by seven different symbols: **I**, **V**, **X**, **L**, **C**, **D** and **M**.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, two is written as **II** in Roman numeral, just two one's added together. Twelve is written as, **XII**, which is simply **X** + **II**.

The number twenty seven is written as **XXVII**, which is **XX** + **V** + **II**.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not **IIII**. Instead, the number four is written as **IV**. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as **IX**. There are six instances where subtraction is used:

- **I** can be placed before **V** (5) and **X** (10) to make 4 and 9.
- **X** can be placed before **L** (50) and **C** (100) to make 40 and 90.
- **C** can be placed before **D** (500) and **M** (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer. Input is guaranteed to be within the range from 1 to 3999.

和上一道题相反，将罗马数字转换成阿拉伯数字。

解法一

先来一种不优雅的，也就是我开始的想发。就是遍历字符串，然后转换就可以，但同时得考虑 IV，IX 那些特殊情况。

```
public int getInt(char r) {
    int ans = 0;
    switch (r) {
        case 'I':
            ans = 1;
            break;
        case 'V':
            ans = 5;
            break;
        case 'X':
            ans = 10;
            break;
        case 'L':
            ans = 50;
            break;
        case 'C':
            ans = 100;
            break;
        case 'D':
            ans = 500;
```

```

        break;
    case 'M':
        ans = 1000;
    }
    return ans;
}

public int getInt(char r, char r_after) {
    int ans = 0;
    switch (r) {
        case 'I':
            ans = 1;
            break;
        case 'V':
            ans = 5;
            break;
        case 'X':
            ans = 10;
            break;
        case 'L':
            ans = 50;
            break;
        case 'C':
            ans = 100;
            break;
        case 'D':
            ans = 500;
            break;
        case 'M':
            ans = 1000;
            break;
    }
    if (r == 'I') {
        switch (r_after) {
            case 'V':
                ans = 4;
                break;
            case 'X':
                ans = 9;
            }
    }
    if (r == 'X') {
        switch (r_after) {
            case 'L':
                ans = 40;
                break;
        }
    }
}

```

```

        case 'C':
            ans = 90;
        }
    }
    if (r == 'C') {
        switch (r_after) {
            case 'D':
                ans = 400;
                break;
            case 'M':
                ans = 900;
            }
        }
    }
    return ans;
}

public boolean isGetTwoInt(char r, char r_after) {
    if (r == 'I') {
        switch (r_after) {
            case 'V':
                return true;
            case 'X':
                return true;
            }
        }
    }
    if (r == 'X') {
        switch (r_after) {
            case 'L':
                return true;
            case 'C':
                return true;
            }
        }
    }
    if (r == 'C') {
        switch (r_after) {
            case 'D':
                return true;
            case 'M':
                return true;
            }
        }
    }
    return false;
}

```



```

public int romanToInt(String s) {
    int ans = 0;
    for (int i = 0; i < s.length() - 1; i++) {
        ans += getInt(s.charAt(i), s.charAt(i + 1));
        //判断是否是两个字符的特殊情况
        if (isGetTwoInt(s.charAt(i), s.charAt(i + 1))) {
            i++;
        }
    }
    //将最后一个字符单独判断，如果放到上边的循环会越界
    if (!(s.length() >= 2 && isGetTwoInt(s.charAt(s.length() - 2),
s.charAt(s.length() - 1)))) {
        ans += getInt(s.charAt(s.length() - 1));
    }

    return ans;
}

```

时间复杂度：O (n) ， n 是字符串的长度。

空间复杂度：O (1) 。

下边分享一些优雅的。

解法二

<https://leetcode.com/problems/roman-to-integer/description/>

```

public int romanToInt(String s) {
    int sum=0;
    if(s.indexOf("IV")!=-1){sum-=2;}
    if(s.indexOf("IX")!=-1){sum-=2;}
    if(s.indexOf("XL")!=-1){sum-=20;}
    if(s.indexOf("XC")!=-1){sum-=20;}
    if(s.indexOf("CD")!=-1){sum-=200;}
    if(s.indexOf("CM")!=-1){sum-=200;}

    char c[]=s.toCharArray();
    int count=0;

    for(;count<=s.length()-1;count++){
        if(c[count]=='M') sum+=1000;
        if(c[count]=='D') sum+=500;
        if(c[count]=='C') sum+=100;
        if(c[count]=='L') sum+=50;
        if(c[count]=='X') sum+=10;
    }
}

```

```

        if(c[count]=='V') sum+=5;
        if(c[count]=='I') sum+=1;

    }

    return sum;

}

```

把出现的特殊情况，提前减了就可以。

时间复杂度：O(1)。

空间复杂度：O(1)。

解法三

<https://leetcode.com/problems/roman-to-integer/discuss/6509/7ms-solution-in-java.-easy-to-understand>

利用到罗马数字的规则，一般是表示数字大的字母在前，数字小的字母在后，如果不是这样，就说明出现了特殊情况，此时应该做减法。

```

private int getVal(char c){
    switch (c){
        case 'M':
            return 1000;
        case 'D':
            return 500;
        case 'C':
            return 100;
        case 'L':
            return 50;
        case 'X':
            return 10;
        case 'V':
            return 5;
        case 'I':
            return 1;
    }
    throw new IllegalArgumentException("unsupported character");
}

public int romanToInt(String s) {
    int res = 0;
    if(s.length() == 0) return res;
}

```

```
for (int i = 0; i < s.length() - 1; i++) {
    int cur = getVal(s.charAt(i));
    int nex = getVal(s.charAt(i+1));
    if (cur < nex) {
        res -= cur;
    } else {
        res += cur;
    }
}
return res + getVal(s.charAt(s.length()-1));
}
```

时间复杂度：O (1) 。

空间复杂度：O (1) 。

总

这道题也不难，自己一开始没有充分利用罗马数字的特点，而是用一些 if, switch 语句判断是否是特殊情况，看起来就很繁琐了。

14、题目描述（简单难度）

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string `""`.

Example 1:

Input: ["flower","flow","flight"]
Output: "fl"

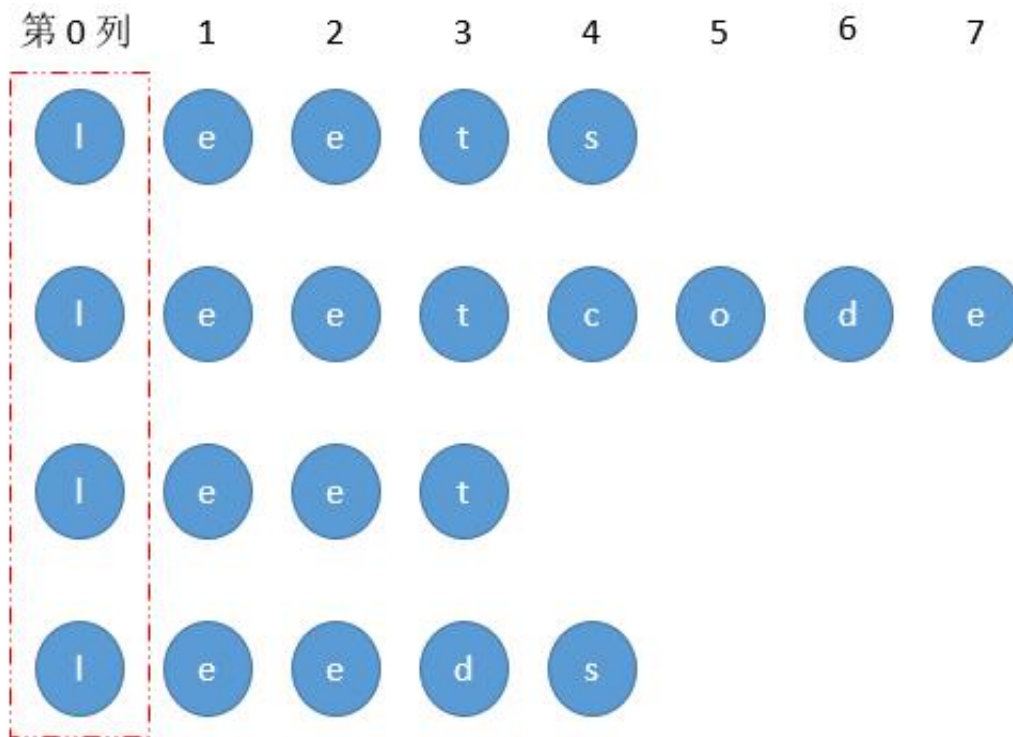
Example 2:

Input: ["dog","racecar","car"]
Output: ""
Explanation: There is no common prefix among the input strings.

Note:

All given inputs are in lowercase letters `a-z`.

解法一 垂直比较



我们把所有字符串垂直排列，然后一列一列的比较，直到某一个字符串到达结尾或者该列字符不完全相同。

下边看一下我的代码，看起来比较多

```
//这个函数判断 index 列的字符是否完全相同
public boolean isSameAtIndex(String[] strs, int index) {
    int i = 0;
    while (i < strs.length - 1) {
        if (strs[i].charAt(index) == strs[i + 1].charAt(index)) {
            i++;
        } else {
            return false;
        }
    }
    return true;
}

public String longestCommonPrefix(String[] strs) {
    if (strs.length == 0)
        return "";
    //得到最短的字符串的长度
    int minLength = Integer.MAX_VALUE;
    for (int i = 0; i < strs.length; i++) {
        if (strs[i].length() < minLength) {
            minLength = strs[i].length();
        }
    }
}
```

```

    }
}
int j = 0;
//遍历所有列
for (; j < minLength; j++) {
    //如果当前列字符不完全相同，则结束循环
    if (!isSameAtIndex(strs, j)) {
        break;
    }
}
return strs[0].substring(0, j);
}

```

下边看一下，官方的代码

```

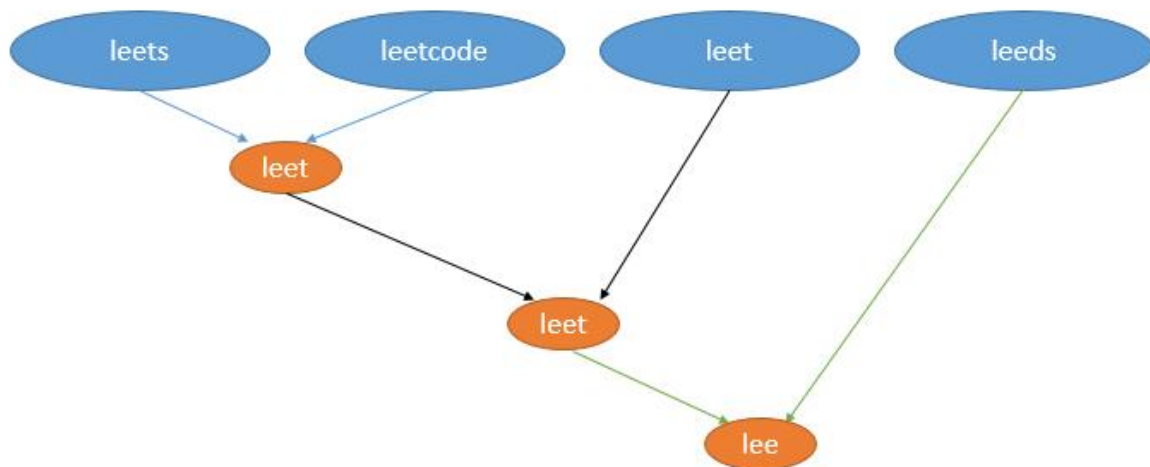
public String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";
    //遍历所有列
    for (int i = 0; i < strs[0].length() ; i++){
        char c = strs[0].charAt(i); // 保存 i 列第 0 行的字符便于后续比较
        //比较第 1,2,3... 行的字符和第 0 行是否相等
        for (int j = 1; j < strs.length; j ++) {
            /**
             * i == strs[j].length() 表明当前行已经到达末尾
             * strs[j].charAt(i) != c 当前行和第 0 行字符不相等
             * 上边两种情况返回结果
             */
            if (i == strs[j].length() || strs[j].charAt(i) != c)
                return strs[0].substring(0, i);
        }
    }
    return strs[0];
}

```

时间复杂度：最坏的情况就是 n 个 长度为 m 的完全一样的字符串，假设 S 是所有字符的和，那么 $S = m * n$ ，时间复杂度就是 $O(S)$ 。当然正常情况下并不需要比较所有字符串，最多比较 $n * \text{minLen}$ 个字符就可以了。

空间复杂度： $O(1)$ ，常数个额外空间。

解法二 水平比较



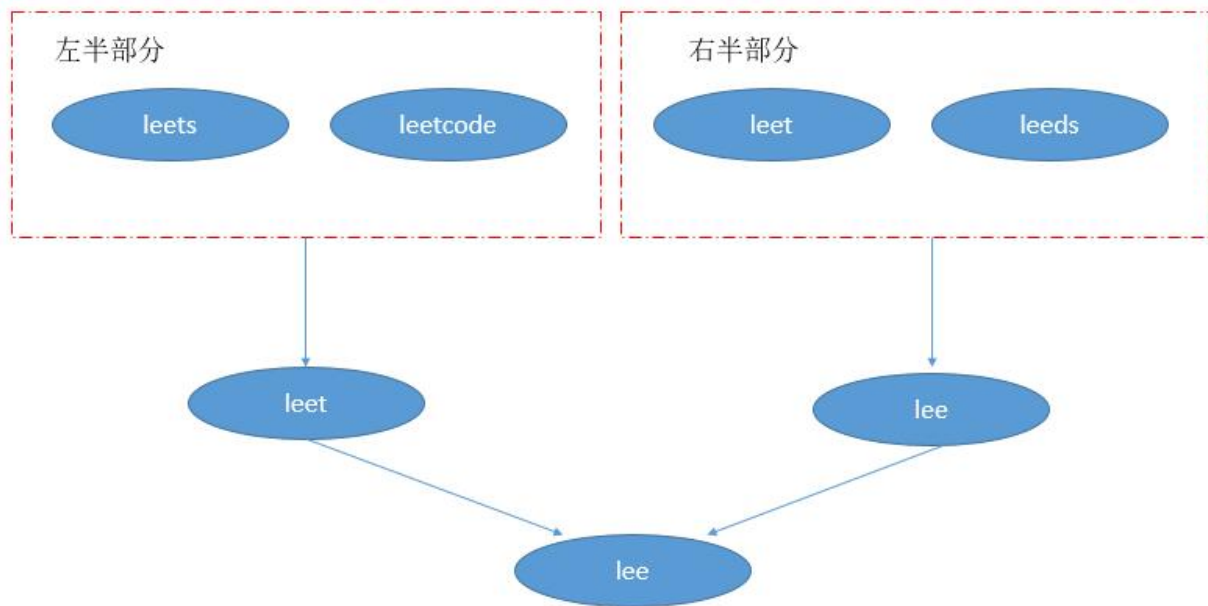
我们将字符串水平排列，第 0 个和第 1 个字符串找最长子串，结果为 leet，再把结果和第 2 个字符串比较，结果为 leet，再把结果和第 3 个字符串比较，结果为 lee，即为最终结果。

```
public String longestCommonPrefix3(String[] strs) {  
    if (strs.length == 0)  
        return "";  
    String prefix = strs[0]; // 保存结果  
    // 遍历每一个字符串  
    for (int i = 1; i < strs.length; i++) {  
        // 找到上次得到的结果 prefix 和当前字符串的最长子串  
        int minLen = Math.min(prefix.length(), strs[i].length());  
        int j = 0;  
        for (; j < minLen; j++) {  
            if (prefix.charAt(j) != strs[i].charAt(j)) {  
                break;  
            }  
        }  
        prefix = prefix.substring(0, j);  
    }  
    return prefix;  
}
```

时间复杂度：最坏情况和解法一是一样， n 个长度为 m 的完全相同的字符，就要比较所有的字符 S ， $S = n * m$ 。但对于正常情况，处于最短字符串前的字符串依旧要比较所有字符，而不是最短字符串个字符，相对于解法一较差。

空间复杂度： $O(1)$ 。

解法三 递归



我们把原来的数组分成两部分，求出左半部分的最长公共前缀，求出右半部分的最长公共前缀，然后求出的两个结果再求最长公共前缀，就是最后的结果了。

求左半部分的最长公共前缀，我们可以继续把它分成两部分，按照上边的思路接着求。然后一直分成两部分，递归下去。

直到该部分只有 1 个字符串，那么最长公共子串就是它本身了，直接返回就可以了。

```
public String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";
    return longestCommonPrefix(strs, 0, strs.length - 1);
}

//递归不断分成两部分
private String longestCommonPrefix(String[] strs, int l, int r) {
    if (l == r) {
        return strs[l];
    }
    else {
        int mid = (l + r) / 2;
        String lcpLeft = longestCommonPrefix(strs, l, mid);
        String lcpRight = longestCommonPrefix(strs, mid + 1, r);
        return commonPrefix(lcpLeft, lcpRight);
    }
}

//求两个结果的最长公共前缀
String commonPrefix(String left, String right) {
    int min = Math.min(left.length(), right.length());
    for (int i = 0; i < min; i++) {
```

```
        if ( left.charAt(i) != right.charAt(i) )
            return left.substring(0, i);
    }
    return left.substring(0, min);
}
```

时间复杂度：

空间复杂度：

每次遇到递归的情况，总是有些理不清楚，先空着吧。

总

进行了垂直比较和水平比较，又用到了递归，[solution](#) 里还介绍了二分查找，感觉这里用二分查找有些太僵硬了，反而使得时间复杂度变高了。还介绍了前缀树，这里后边遇到再总结吧。

15、题目描述（中等难度）

Given an array `nums` of n integers, are there elements a, b, c in `nums` such that $a + b + c = 0$? Find all unique triplets in the array which gives the sum of zero.

Note:

The solution set must not contain duplicate triplets.

Example:

Given array `nums = [-1, 0, 1, 2, -1, -4]`,

A solution set is:

```
[
  [-1, 0, 1],
  [-1, -1, 2]
]
```

解法一 暴力解法

无脑搜索，三层循环，遍历所有的情况。但需要注意的是，我们需要把重复的情况去除掉，就是 $[1, -1, 0]$ 和 $[0, -1, 1]$ 是属于同一种情况的。

```
public List<List<Integer>> threeSum(int[] nums) {
    List<List<Integer>> res = new ArrayList<List<Integer>>();
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++)
            for (int k = j + 1; k < nums.length; k++) {
                if (nums[i] + nums[j] + nums[k] == 0) {
                    List<Integer> temp = new ArrayList<Integer>();
```



```

        temp.add(nums[i]);
        temp.add(nums[j]);
        temp.add(nums[k]);
        //判断结果中是否已经有 temp 。
        if (isInList(res, temp)) {
            continue;
        }
        res.add(temp);
    }
}
return res;
}

public boolean isInList(List<List<Integer>> l, List<Integer> a) {
    for (int i = 0; i < l.size(); i++) {
        //判断两个 List 是否相同
        if (isSame(l.get(i), a)) {
            return true;
        }
    }
    return false;
}

public boolean isSame(List<Integer> a, List<Integer> b) {
    int count = 0;
    Collections.sort(a);
    Collections.sort(b);
    //排序后判断每个元素是否对应相等
    for (int i = 0; i < a.size(); i++) {
        if (a.get(i) != b.get(i)) {
            return false;
        }
    }
    return true;
}

```

时间复杂度：n 表示 num 的个数，三个循环 $O(n^3)$ ，而 isInList 也需要 $O(n)$ ，总共就是 $O(n^4)$ ，leetcode 复杂度到了 $O(n^3)$ 一般就报超时错误了，所以算法还得优化。

空间复杂度：最坏情况，即 $O(N)$ ，N 是指 n 个元素的排列组合个数，即 $N=C^3_n$ ，用来保存结果。

解法二

参考了[这里](#)

主要思想是，遍历数组，用 0 减去当前的数，作为 sum，然后再找两个数使得和为 sum。

这样看来遍历需要 $O(n)$ ，再找两个数需要 $O(n^2)$ 的复杂度，还是需要 $O(n^3)$ 。

巧妙之处在于怎么找另外两个数。

最最优美的地方就是，首先将给定的 num 排序。

这样我们就可以用两个指针，一个指向头，一个指向尾，去找这两个数字，这样的话，找另外两个数时间复杂度就会从 $O(n^2)$ ，降到 $O(n)$ 。

而怎么保证不加入重复的 list 呢？

要记得我们的 nums 已经有序了，所以只需要找到一组之后，当前指针要移到和当前元素不同的地方。其次在遍历数组的时候，如果和上个数字相同，也要继续后移。文字表述比较困难，可以先看下代码。

```
public List<List<Integer>> threeSum(int[] num) {
    Arrays.sort(num); //排序
    List<List<Integer>> res = new LinkedList<>();
    for (int i = 0; i < num.length-2; i++) {
        //为了保证不加入重复的 list,因为是有序的，所以如果和前一个元素相同，只需要继续后移就可以
        if (i == 0 || (i > 0 && num[i] != num[i-1])) {
            //两个指针,并且头指针从i + 1开始，防止加入重复的元素
            int lo = i+1, hi = num.length-1, sum = 0 - num[i];
            while (lo < hi) {
                if (num[lo] + num[hi] == sum) {
                    res.add(Arrays.asList(num[i], num[lo], num[hi]));
                    //元素相同要后移，防止加入重复的 list
                    while (lo < hi && num[lo] == num[lo+1]) lo++;
                    while (lo < hi && num[hi] == num[hi-1]) hi--;
                    lo++; hi--;
                } else if (num[lo] + num[hi] < sum) lo++;
                else hi--;
            }
        }
    }
    return res;
}
```

时间复杂度： $O(n^2)$ ，n 指的是 num

空间复杂度： $O(N)$ ，最坏情况，即 N 是指 n 个元素的排列组合个数，即 $N=C^3_n$ ，用来保存结果。

总

对于遍历，这里用到了从两头同时遍历，从而降低了时间复杂度，很妙！

16、题目描述（中等难度）

Given an array `nums` of n integers and an integer `target`, find three integers in `nums` such that the sum is closest to `target`. Return the sum of the three integers. You may assume that each input would have exactly one solution.

Example:

Given array `nums = [-1, 2, 1, -4]`, and `target = 1`.
The sum that is closest to the target is 2. ($-1 + 2 + 1 = 2$).

和[上一道题](#)很类似，只不过这个是给一个目标值，找三个数，使得他们的和最接近目标值。

解法一 暴力解法

遍历所有的情况，然后求出三个数的和，和目标值进行比较，选取差值最小的即可。本以为时间复杂度太大了，神奇的是，竟然 AC 了。

```
public int threeSumClosest(int[] nums, int target) {
    int sub = Integer.MAX_VALUE; //保存和 target 的差值
    int sum = 0; //保存当前最接近 target 的三个数的和
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++)
            for (int k = j + 1; k < nums.length; k++) {
                if (Math.abs((nums[i] + nums[j] + nums[k] - target)) < sub) {
                    sum = nums[i] + nums[j] + nums[k];
                    sub = Math.abs(sum - target);
                }
            }
    }
    return sum;
}
```

时间复杂度： $O(n^3)$ ，三层循环。

空间复杂度： $O(1)$ ，常数个。

解法二

受到[上一题](#)的启发，没有看的，推荐大家可以看一下。我们完全可以先将数组排序，然后先固定一个数字，然后利用头尾两个指针进行遍历，降低一个 $O(n)$ 的时间复杂度。

如果 `sum` 大于 `target` 就减小右指针，反之，就增加左指针。

```

public int threeSumClosest(int[] nums, int target) {
    Arrays.sort(nums);
    int sub=Integer.MAX_VALUE;
    int sum=0;
    for(int i=0;i<nums.length;i++){
        int lo=i+1,hi=nums.length-1;
        while(lo<hi){
            if(Math.abs((nums[lo]+nums[hi]+nums[i]-target))<sub){
                sum=nums[lo]+nums[hi]+nums[i];
                sub=Math.abs(sum-target);
            }
            if(nums[lo]+nums[hi]+nums[i]>target){
                hi--;
            }else{
                lo++;
            }
        }
    }
    return sum;
}

```

时间复杂度：如果是快速排序的 $O(\log n)$ 再加上 $O(n^2)$ ，所以就是 $O(n^2)$ 。

空间复杂度： $O(1)$ 。

总

和上一道题非常非常的相似了，先对数组排序，然后利用两头的指针，可以说是十分的优雅了。

17、题目描述（中等难度）

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent.

A mapping of digit to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example:

Input: "23"
Output: ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"].

给一串数字，每个数可以代表数字键下的几个字母，返回这些数字下的字母的所有组成可能。

解法一 定义相乘

自己想了用迭代，用递归，都理不清楚，灵机一动，想出了这个算法。

把字符串 "23" 看成 ["a","b","c"] * ["d","e","f"]，而相乘就用两个 for 循环实现即可，看代码应该就明白了。

```
public List<String> letterCombinations(String digits) {
    List<String> ans = new ArrayList<String>();
    for (int i = 0; i < digits.length(); i++) {
        ans = mul(ans, getList(digits.charAt(i) - '0'));
    }
    return ans;
}

public List<String> getList(int digit) {
    String digitLetter[] = { "", "", "abc", "def", "ghi", "jkl", "mno", "pqrs",
        "tuv", "wxyz" };
    List<String> ans = new ArrayList<String>();
    for (int i = 0; i < digitLetter[digit].length(); i++) {
        ans.add(digitLetter[digit].charAt(i) + "");
    }
    return ans;
}

//定义成两个 List 相乘
public List<String> mul(List<String> l1, List<String> l2) {
    if (l1.size() != 0 && l2.size() == 0) {
        return l1;
    }
    if (l1.size() == 0 && l2.size() != 0) {
        return l2;
    }
    List<String> ans = new ArrayList<String>();
    for (int i = 0; i < l1.size(); i++)
        for (int j = 0; j < l2.size(); j++) {
            ans.add(l1.get(i) + l2.get(j));
        }
    return ans;
}
```

解法二 队列迭代

参考[这里](#)，果然有人用迭代写了出来。主要用到了队列。

```
public List<String> letterCombinations(String digits) {
    LinkedList<String> ans = new LinkedList<String>();
    if(digits.isEmpty()) return ans;
    String[] mapping = new String[] { "0", "1", "abc", "def", "ghi", "jkl", "mno",
    "pqrs", "tuv", "wxyz" };
    ans.add("");
    for(int i =0; i<digits.length();i++){
        int x = Character.getNumericValue(digits.charAt(i));
        while(ans.peek().length()==i){ //查看队首元素
            String t = ans.remove(); //队首元素出队
            for(char s : mapping[x].toCharArray())
                ans.add(t+s);
        }
    }
    return ans;
}
```

假如是 "23"，那么

第 1 次 for 循环结束后变为 a, b, c;

第 2 次 for 循环的第 1 次 while 循环 a 出队，分别加上 d e f 然后入队，就变成 b c ad ae af

第 2 次 for 循环的第 2 次 while 循环 b 出队，分别加上 d e f 然后入队，就变成 c ad ae af bd be bf

第 2 次 for 循环的第 3 次 while 循环 c 出队，分别加上 d e f 然后入队，就变成 ad ae af bd be bf cd ce cf

这样的话队列的元素长度再也没有等于 1 的了就出了 while 循环。

解法三 递归

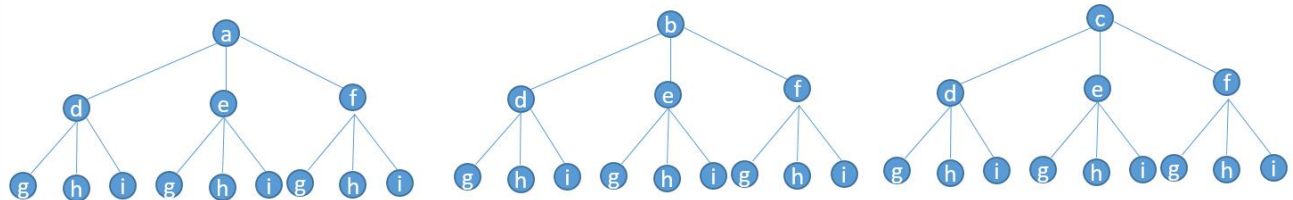
参考[这里](#)

```
private static final String[] KEYS = { "", "", "abc", "def", "ghi", "jkl", "mno",
    "pqrs", "tuv", "wxyz" };

public List<String> letterCombinations(String digits) {
    if(digits.equals("")) {
        return new ArrayList<String>();
    }
    List<String> ret = new LinkedList<String>();
    combination("", digits, 0, ret);
    return ret;
}
```

```
private void combination(String prefix, String digits, int offset, List<String>
ret) {
    //offset 代表在加哪个数字
    if (offset == digits.length()) {
        ret.add(prefix);
        return;
    }
    String letters = KEYS[(digits.charAt(offset) - '0')];
    for (int i = 0; i < letters.length(); i++) {
        combination(prefix + letters.charAt(i), digits, offset + 1, ret);
    }
}
```

digits = "234"



从 a 开始，然后递归到 d，然后 g，就把 adg 加入，然后再加入 adh，再加入 adi ... 从左到右，递归到底之后就将其加入。

总

这种题的时间复杂度和空间复杂度自己理的不太清楚就没有写了。

18、题目描述（中等难度）

Given an array `nums` of n integers and an integer `target`, are there elements a, b, c , and d in `nums` such that $a + b + c + d = target$? Find all unique quadruplets in the array which gives the sum of `target`.

Note:

The solution set must not contain duplicate quadruplets.

Example:

Given array `nums = [1, 0, -1, 0, -2, 2]`, and `target = 0`.

A solution set is:

```
[
  [-1, 0, 0, 1],
  [-2, -1, 1, 2],
  [-2, 0, 0, 2]
]
```

和3Sum类似，只不过是找四个数，使得和为 target，并且不能有重复的序列。

如果之前没有做过3Sum可以先看看，自己在上边的基础上加了一个循环而已。

```
public List<List<Integer>> fourSum(int[] num, int target) {
    Arrays.sort(num);
    List<List<Integer>> res = new LinkedList<>();
    //多加了层循环
    for (int j = 0; j < num.length - 3; j++) {
        //防止重复的
        if (j == 0 || (j > 0 && num[j] != num[j - 1]))
            for (int i = j + 1; i < num.length - 2; i++) {
                //防止重复的，不再是 i == 0，因为 i 从 j + 1 开始
                if (i == j + 1 || num[i] != num[i - 1]) {
                    int lo = i + 1, hi = num.length - 1, sum = target - num[j] -
num[i];
                    while (lo < hi) {
                        if (num[lo] + num[hi] == sum) {
                            res.add(Arrays.asList(num[j], num[i], num[lo],
num[hi]));

                            while (lo < hi && num[lo] == num[lo + 1])
                                lo++;
                            while (lo < hi && num[hi] == num[hi - 1])
                                hi--;
                            lo++;
                            hi--;
                        } else if (num[lo] + num[hi] < sum)
                            lo++;
                        else
                            hi--;
                    }
                }
            }
    }
    return res;
}
```

时间复杂度：O (n³) 。

空间复杂度：O (N) ，最坏情况，即 N 是指 n 个元素的排列组合个数，即 $N=C^4_n$ ，用来保存结果。

总

完全是按照 3Sum 的思路写的，比较好理解。

19、题目描述（中等难度）

Given a linked list, remove the n -th node from the end of list and return its head.

Example:

Given linked list: 1->2->3->4->5, and $n = 2$.

After removing the second node from the end, the linked list becomes 1->2->3->5.

Note:

Given n will always be valid.

Follow up:

Could you do this in one pass?

给定一个链表，将倒数第 n 个结点删除。

解法一

删除一个结点，无非是遍历链表找到那个结点前边的结点，然后改变下指向就好了。但由于它是链表，它的长度我们并不知道，我们得先遍历一遍得到它的长度，之后用长度减去 n 就是要删除的结点的位置，然后遍历到结点的前一个位置就好了。

```
public ListNode removeNthFromEnd(ListNode head, int n) {
    int len = 0;
    ListNode h = head;
    while (h != null) {
        h = h.next;
        len++;
    }
    //长度等于 1，再删除一个结点就为 null 了
    if (len == 1) {
        return null;
    }

    int rm_node_index = len - n;

    //如果删除的是头结点
    if (rm_node_index == 0) {
        return head.next;
    }

    //找到被删除结点的前一个结点
    h = head;
    for (int i = 0; i < rm_node_index - 1; i++) {
```

```

        h = h.next;
    }

    //改变指向
    h.next = h.next.next;
    return head;
}

```

时间复杂度：假设链表长度是 L ，那么就第一个循环是 L 次，第二个循环是 $L - n$ 次，总共 $2L - n$ 次，所以时间复杂度就是 $O(L)$ 。

空间复杂度： $O(1)$ 。

我们看到如果长度等于 1 和删除头结点的时候需要单独判断，其实我们只需要在 head 前边加一个空节点，就可以避免单独判断。

```

public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    int length = 0;
    ListNode first = head;
    while (first != null) {
        length++;
        first = first.next;
    }
    length -= n;
    first = dummy;
    while (length > 0) {
        length--;
        first = first.next;
    }
    first.next = first.next.next;
    return dummy.next;
}

```

解法二 遍历一次链表

上边我们遍历链表进行了两次，我们如何只遍历一次呢。

看了 [leetcode](#) 的讲解。

想象一下，两个人进行 100m 赛跑，假设他们的速度相同。开始的时候，第一个人就在第二个人前边 10m，这样当第一个人跑到终点的时候，第二个人相距第一个人依旧是 10m，也就是离终点 10m。

对比于链表，我们设定两个指针，先让第一个指针遍历 n 步，然后再让它俩同时开始遍历，这样的话，当第一个指针到头的时候，第二个指针就离第一个指针有 n 的距离，所以第二个指针的位置就刚好是倒数第 n 个结点。

```
public ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode first = dummy;
    ListNode second = dummy;
    //第一个指针先移动 n 步
    for (int i = 1; i <= n + 1; i++) {
        first = first.next;
    }
    //第一个指针到达终点停止遍历
    while (first != null) {
        first = first.next;
        second = second.next;
    }
    second.next = second.next.next;
    return dummy.next;
}
```

时间复杂度：

第一个指针从 0 到 n ，然后「第一个指针再从 n 到结束」和「第二个指针从 0 到倒数第 n 个结点的位置」同时进行。

而解法一无非是先从 0 到结束，然后从 0 到倒数第 n 个结点的位置。

所以其实它们语句执行的次数其实是一样的，从 0 到倒数第 n 个结点的位置都被遍历了 2 次，所以总共也是 $2L - n$ 次。只不过这个解法把解法一的两次循环合并了一下，使得第二个指针看起来是顺便遍历，想法很 nice。

所以本质上，它们其实是一样的，时间复杂度依旧是 $O(n)$ 。

空间复杂度： $O(1)$ 。

解法三

没看讲解前，和室友讨论下，如何只遍历一次链表。室友给出了一个我竟然无法反驳的观点，哈哈哈哈。

第一次遍历链表确定长度的时候，顺便把每个结点存到数组里，这样找结点的时候就不需要再遍历一次了，空间换时间？？？哈哈哈哈哈。

```
public ListNode removeNthFromEnd(ListNode head, int n) {
    List<ListNode> l = new ArrayList<ListNode>();
    ListNode h = head;
```

```
int len = 0;
while (h != null) {
    l.add(h);
    h = h.next;
    len++;
}
if (len == 1) {
    return null;
}
int remove = len - n;
if (remove == 0) {
    return head.next;
}
//直接得到，不需要再遍历了
ListNode r = l.get(remove - 1);
r.next = r.next.next;
return head;
}
```

时间复杂度： $O(L)$ 。

空间复杂度： $O(L)$ 。

总

利用两个指针先固定间隔，然后同时遍历，真的是很妙！另外室友的想法也很棒，哈哈哈哈哈。

20、题目描述（简单难度）

Given a string containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.

Note that an empty string is also considered valid.

Example 1:

Input: "()"
Output: true

Example 2:

Input: "()[]{}"
Output: true

Example 3:

Input: "]"
Output: false

Example 4:

Input: "([)]"
Output: false

Example 5:

Input: "{[]}"
Output: true

括号匹配问题。

如果只有一种括号，我们完全可以用一个计数器 count，遍历整个字符串，遇到左括号加 1，遇到右括号减 1，遍历结束后，如果 count 等于 0，则表示全部匹配。但如果有多种括号，像 ([)] 这种情况它依旧会得到 0，所以我们需要用其他的方法。

栈！

遍历整个字符串，遇到左括号就入栈，然后遇到和栈顶对应的右括号就出栈，遍历结束后，如果栈为空，就表示全部匹配。

```
public boolean isValid(String s) {  
    Stack<Character> brackets = new Stack<Character>();  
    for(int i = 0; i < s.length(); i++){  
        char c = s.charAt(i);  
        switch(c){  
            case '(':  
            case '[':  
            case '{':  
                brackets.push(c);  
                break;
```

```

        case ')':
            if(!brackets.empty()){
                if(brackets.peek() == '('){
                    brackets.pop();
                }else{
                    return false;
                }
            }else{
                return false;
            }
            break;
        case ']':
            if(!brackets.empty()){
                if(brackets.peek() == '['){
                    brackets.pop();
                }else{
                    return false;
                }
            }else{
                return false;
            }
            break;
        case '}':
            if(!brackets.empty()){
                if(brackets.peek() == '{'){
                    brackets.pop();
                }else{
                    return false;
                }
            }else{
                return false;
            }
        }
    }

    return brackets.empty();
}

```

时间复杂度：O (n) 。

空间复杂度：O (n) 。

总

如果学过数据结构，一定写过计算器，括号匹配问题一定遇到过的。

21、题目描述（简单难度）

Merge two sorted linked lists and return it as a new list. The new list should be made by splicing together the nodes of the first two lists.

Example:

Input: 1->2->4, 1->3->4
Output: 1->1->2->3->4->4

合并两个有序链表。

解法一 迭代

遍历两个链表。

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {  
    ListNode h = new ListNode(0);  
    ListNode ans=h;  
    while (l1 != null && l2 != null) {  
        if (l1.val < l2.val) {  
            h.next = l1;  
            h = h.next;  
            l1 = l1.next;  
        } else {  
            h.next = l2;  
            h = h.next;  
            l2 = l2.next;  
        }  
    }  
    if(l1==null){  
        h.next=l2;  
    }  
    if(l2==null){  
        h.next=l1;  
    }  
    return ans.next;  
}
```

时间复杂度： $O(m + n)$ 。

空间复杂度： $O(1)$ 。

解法二 递归

参考[这里](#)

```

ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    if(l1 == null) return l2;
    if(l2 == null) return l1;

    if(l1.val < l2.val) {
        l1.next = mergeTwoLists(l1.next, l2);
        return l1;
    } else {
        l2.next = mergeTwoLists(l2.next, l1);
        return l2;
    }
}

```

时间复杂度：

空间复杂度：

总

递归看起来，两个字，优雅！但是关于递归的时间复杂度，空间复杂度的求法，先留个坑吧。

22、题目描述（中等难度）

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

For example, given $n = 3$, a solution set is:

```

[
  "((()))",
  "(()())",
  "(())()",
  "()()()",
  "()(())"
]

```

给一个数字 n ，返回所有合法的括号匹配，刚好和[20题](#)相反。

自己没想出来，全部参考 LeetCode 给出的 [Solution](#)。

解法一 暴力破解

列举所有的情况，每一位有左括号和右括号两种情况，总共 $2n$ 位，所以总共 2^{2n} 种情况。

```

public List<String> generateParenthesis(int n) {
    List<String> combinations = new ArrayList();
    generateAll(new char[2 * n], 0, combinations);
    return combinations;
}

```



```

}

public void generateAll(char[] current, int pos, List<String> result) {
    if (pos == current.length) {
        if (valid(current))
            result.add(new String(current));
    } else {
        current[pos] = '(';
        generateAll(current, pos+1, result);
        current[pos] = ')';
        generateAll(current, pos+1, result);
    }
}

public boolean valid(char[] current) {
    int balance = 0;
    for (char c: current) {
        if (c == '(') balance++;
        else balance--;
        if (balance < 0) return false;
    }
    return (balance == 0);
}

```

时间复杂度：对每种情况判断是否合法需要 $O(n)$ ，所以时间复杂度是 $O(2^{2n}n)$ 。

空间复杂度： $O(2^{2n}n)$ ，乘以 n 是因为每个串的长度是 $2n$ 。此外这是假设所有情况都符合的时候，但其实不可能都符合，后边会给出更精确的情况。

解法二

解法一中，我们不停的加左括号，其实如果左括号超过 n 的时候，它肯定不是合法序列了。因为合法序列一定是 n 个左括号和 n 个右括号。

还有一种情况就是如果添加括号的过程中，如果右括号的总数量大于左括号的总数量了，后边不论再添加什么，它都不可能是合法序列了。因为每个右括号必须和之前的某个左括号匹配，如果右括号数量多于左括号，那么一定有一个右括号没有与之匹配的左括号，后边不论加多少左括号都没有用了。例如 $n = 3$ ，总共会有 6 个括号，我们加到 $()())$ 3 个括号的情况的时候，有 1 个左括号，2 个右括号，此时后边 3 个括号无论是什么，已经注定它不会是合法序列了。

基于上边的两点，我们只要避免它们，就可以保证我们生成的括号一定是合法的了。

```

public List<String> generateParenthesis(int n) {
    List<String> ans = new ArrayList();
    backtrack(ans, "", 0, 0, n);
    return ans;
}

```

```

}

public void backtrack(List<String> ans, String cur, int left, int right, int n){
    if (cur.length() == n * 2) {
        ans.add(cur);
        return;
    }
    //左括号不要超过 n
    if (left < n)
        backtrack(ans, cur+"(", left+1, right, n);
    //右括号不要超过左括号
    if (right < left)
        backtrack(ans, cur+")", left, right+1, n);
}

```

时间复杂度：

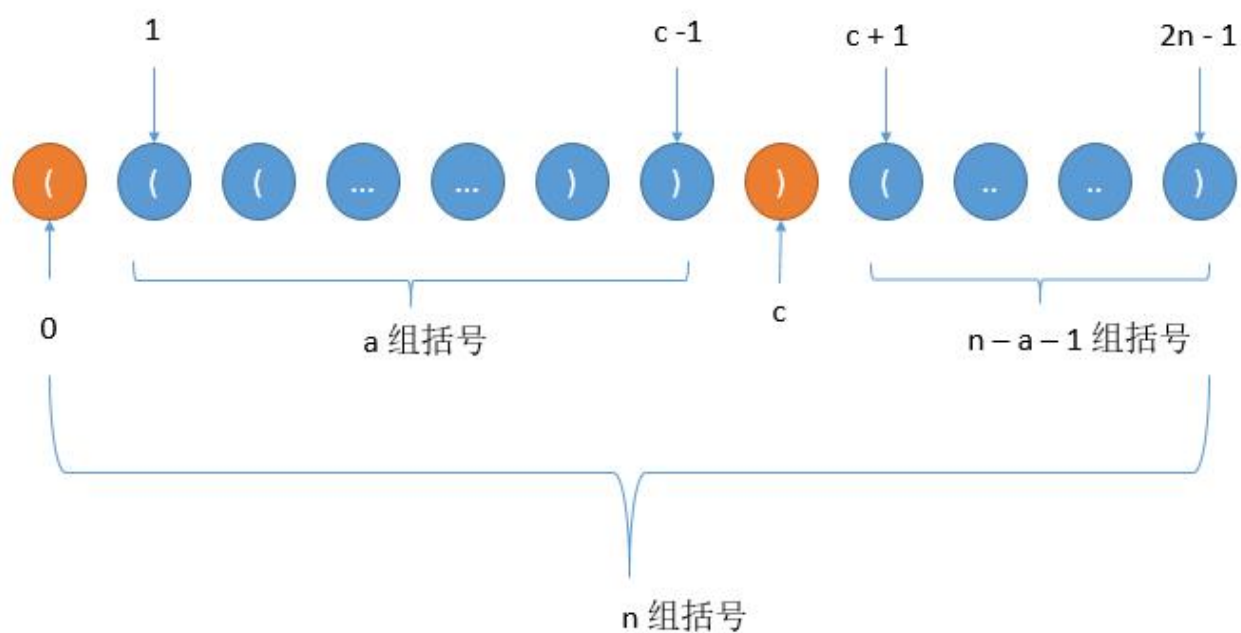
空间复杂度：

递归的复杂度分析，继续留坑 =.=。

解法三

解法二中是用列举的方法，仔细想想，我们每次用递归的时候，都是把大问题换成小问题然后去解决，这道题有没有这个思路呢？

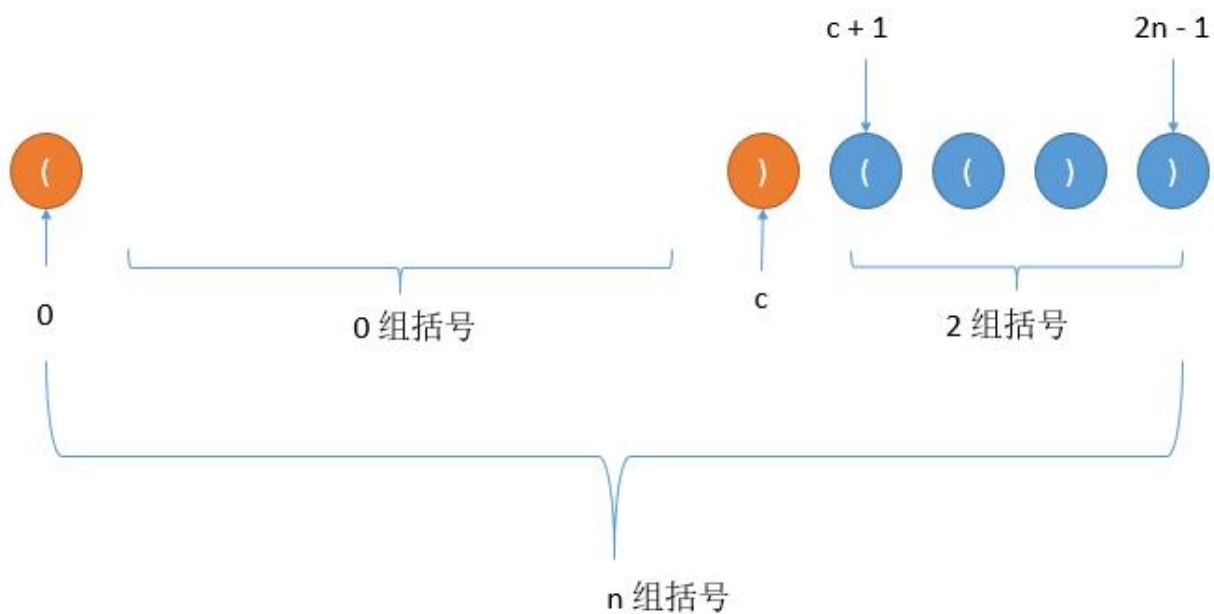
我们想一下之前的列举过程，第 0 个位置一定会是左括号，然后接着添加左括号或右括号，过程中左括号数一定大于或等于右括号数，当第一次出现左括号数等于右括号数的时候，假如此时的位置是 c 。那么位置 1 到 $c-1$ 之间一定是合法序列，此外 $c+1$ 到最后的 $2n-1$ 也是合法序列。而假设总共是 n 组括号，1 到 $c-1$ 是 a 组括号， $c+1$ 到 $2n-1$ 之间则是 $n-1-a$ 组括号，如下图

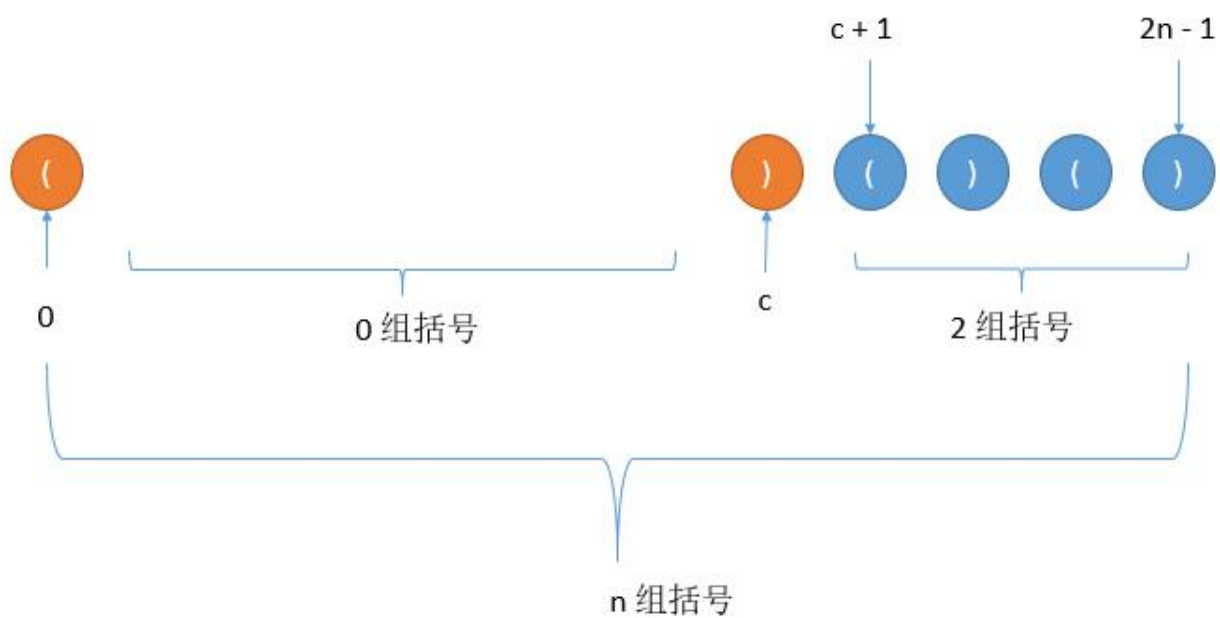


最重要的是，每一个合法序列都会有这么一个数 c ，且唯一。所以我们如果要求 n 组括号的所有序列，只需要知道 a 组括号以及 $(n-a-1)$ 组括号的所有序列，然后两两组合即可。

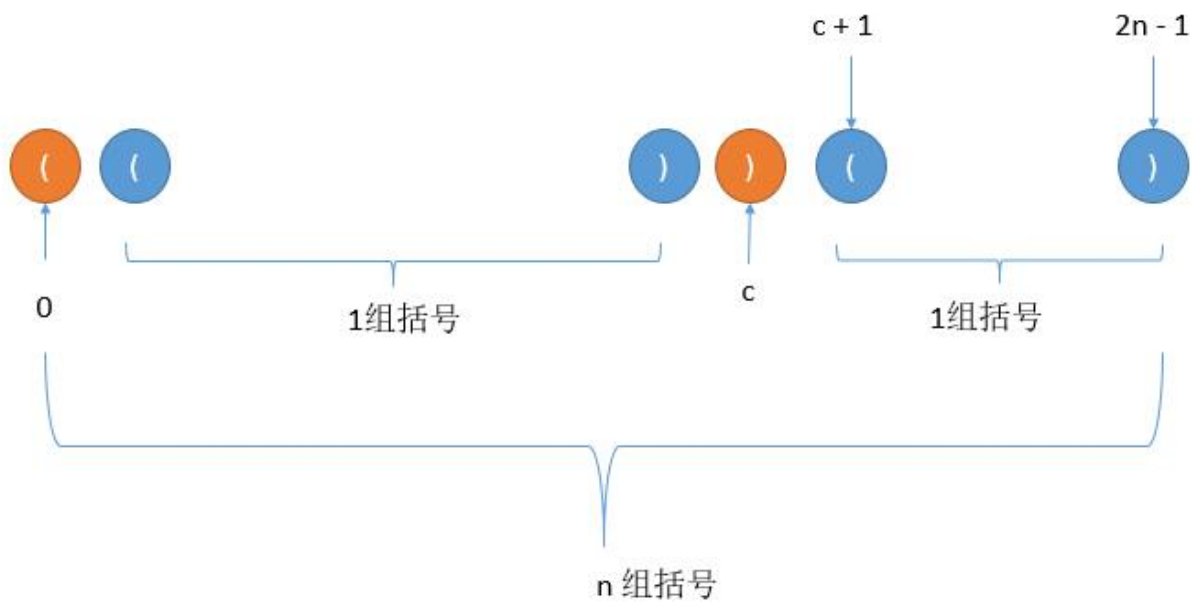
以 $n=3$ 为例，我们把 0 到 c 之间的括号数记为 a 组， $c+1$ 到最后的括号数记为 b 组，则

$a=0$, $b=2$ 对应 $() (())$ 以及 $() () ()$ 两种情况，此时 $c=1$ 。

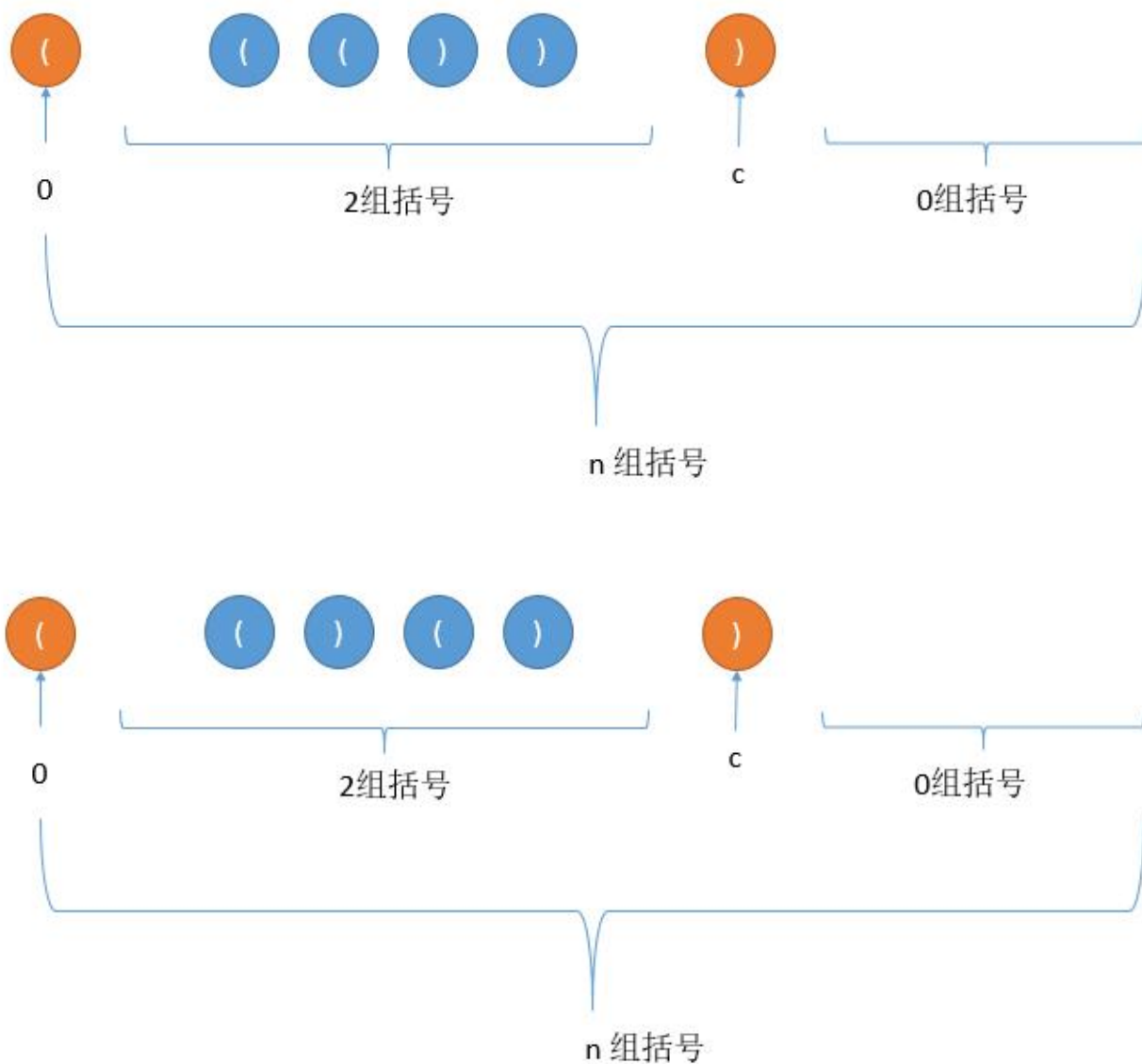




$a = 1, b = 1$, 对应 $(())()$ 一种情况, 此时 $c = 3$ 。



$a = 2, b = 0$ 对应 $((()))$, $(())()$ 两种情况, 此时 $c = 5$ 。



所以我们如果要想求 n 组括号，只需要知道 a 组和 b 组的情况，然后组合起来就可以了。

看起来我们在迭代 a ，其实本质上是在迭代 c ， $c = 2a + 1$ ，迭代 a 从 0 到 $n - 1$ ，就是迭代 c 从 1 到 $2n - 1$ 。看起来 c 都是奇数，其实是可以理解的，因为 0 到 c 间都是一组组的括号，所以 c 一定是奇数。为什么可以迭代 c ，因为上边说到每一个合法序列都对应着一个 c ，遍历 c 的话，就能得到所有的情况了，看一下代码吧。

```

public List<String> generateParenthesis(int n) {
    List<String> ans = new ArrayList();
    if (n == 0) {
        ans.add("");
    } else {
        for (int a = 0; a < n; a++)
            for (String left: generateParenthesis(a))
                for (String right: generateParenthesis(n-1-a))
                    ans.add("(" + left + ")" + right);
    }
    return ans;
}

```

时间复杂度：

空间复杂度：

留坑。

扩展 卡特兰数

如果这道题不是让你列举所有的情况，而是仅仅让你输出 n 对应下有多少种合法序列，该怎么做呢？

答案就是 $\frac{1}{n+1}C^{n+1}_{2n}$ ，也可以写成 $\frac{1}{n+1}\binom{2n}{n}$ 。怎么证明呢？我主要参考了[这里](#)，说一下。

我们假设不考虑是不是合法序列，那么就一共有 C^{n+1}_{2n} 种情况，然后我们只需要把里边的非法情况减去就可以了，一共有多少种非法情况呢？

首先我们用 C^{n+1}_{2n} 就保证了一定是有 n 个左括号， n 个右括号，那么为什么出现了非法序列？

为了方便论述，我们把左括号记为 $+1$ ，右括号记为 -1 。

ps：下边的 和 都是指两个数的和，不是你和我中的和。

我们假设非法序列的集合是 M ，而非法序列就是列举过程中右括号数比左括号数多了，也就是和小于 0 了，变成 -1 了。这种情况一旦出现，后边无论是什么括号都改变不了它是非法序列的命了。我们将第一次和等于 -1 的时候的位置记为 d 。每一个非法序列一定存在这样一个 d 。然后关键的地方到了！

此时我们把 0 到 d 所有的 -1 变成 1 ， 1 变成 -1 ，我们将每一个非法序列都这样做，就构成了一个新的集合 N ，并且这个集合 N 一定和 M 中的元素一一对应（ $N \rightarrow M$ ，在集合 N 中第一次出现和为 1 的位置也就是 d ，把 0 到 d 中所有的 -1 变成 1 ， 1 变成 -1 就回到了 M ），从而集合 M 的数量就等于集合 N 的数量。集合 N 的数量是多少呢？

我们来分析下集合 N 是什么样的，集合 N 对应的集合 M 原来的序列本来是这样的，在 0 到 d 之间和是 -1 ，也就是 -1 比 $+1$ 多一个， $d+1$ 到最后的和一定是 1 （因为 n 个 $+1$ 和 n 个 -1 的和一定是 0 ，由于 0 到 d 和是 -1 ，后边的和一定是 1 ），也就意味着 $+1$ 比 -1 多一个。而在集合 N 中，我们把 0 到 d 的 -1 变成了 $+1$ ， $+1$ 变成了 -1 ，所以也变成了 $+1$ 比 -1 多一个，所以集合 N 总共就是 $+1$ 比 -1 多 2 个的集合，也就是

$n+1$ 个 $+1$ 和 $n-1$ 个 -1 。

所以集合 N 就是 $2n$ 个位置中选 $n-1$ 个位置放 -1 ，其他位置放 $+1$ ，总共就有 C^{n-1}_{2n} ，所以集合 M 也有 C^{n-1}_{2n} 种。

所有合法序列就有 $C^n_{2n}-C^{n-1}_{2n}=\frac{1}{n+1}C^n_{2n}$ 。

将集合 M 和集合 N 建立了一一映射，从而解决了问题，神奇！！！！！！！！其实，这个数列就是卡特兰数，可以看下[维基百科](#)的定义。

卡特兰数 [编辑]

维基百科，自由的百科全书

卡特兰数是组合数学中一个常在各种计数问题中出现的数列。以比利时的数学家欧仁·查理·卡特兰（1814–1894）命名。历史上，清代数学家明安图(1692年 - 1763年)在其《割圆密率捷法》中最先发明这种计数方式，远远早于卡特兰^{[1][2][3]}。有中国学者建议将此数命名为“明安图数”或“明安图-卡特兰数”^[4]。

卡特兰数的一般项公式为 $C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$

前20项为（[OEIS](#)中的数列A000108）：1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670, 129644790, 477638700, 1767263190

而这个数列，其实除了括号匹配，还有很多类似的问题，其本质是一样的，例如，

$2n$ 个人排队买票，其中 n 个人持 50 元， n 个人持 100 元。每张票 50 元，且一人只买一张票。初始时售票处没有零钱找零。请问这 $2n$ 个人一共有多少种排队顺序，不至于使售票处找不开钱？

对于一个无限大的栈，一共 n 个元素，请问有几种合法的入栈出栈形式？

$P = a_1 * a_2 * a_3 * \dots * a_n$ ，其中 a_i 是矩阵。根据乘法结合律，不改变矩阵的相互顺序，只用括号表示成对的乘积，试问一共有几种括号化方案？

n 个结点可构造多少个不同的二叉树？

... ..

更多例子可以看[维基百科](#)和[这里](#)。

而 Solutin 给出的时间复杂度，其实就是卡特兰数。

Our complexity analysis rests on understanding how many elements there are in `generateParenthesis(n)`. This analysis is outside the scope of this article, but it turns out this is the n -th Catalan number $\frac{1}{n+1} \binom{2n}{n}$, which is bounded asymptotically by $\frac{4^n}{n\sqrt{n}}$.

- Time Complexity : $O(\frac{4^n}{\sqrt{n}})$. Each valid sequence has at most n steps during the backtracking procedure.
- Space Complexity : $O(\frac{4^n}{\sqrt{n}})$, as described above, and using $O(n)$ space to store the sequence.

[维基百科](#)的给出的性质。

卡特兰数的渐近增长为

$$C_n \sim \frac{4^n}{n^{3/2} \sqrt{\pi}}$$

它的含义是当 $n \rightarrow \infty$ 时，左式除以右式的商趋向于1。（这可以用 $n!$ 的斯特灵公式来证明。）

所有的奇卡特兰数 C_n 都满足 $n = 2^k - 1$ 。所有其他的卡特兰数都是偶数。

总

本以为这道题挺常规的，然后自己一直卡在解法三的理解上，查来查去，竟然查出了卡特兰数，虽然似乎和解法三也没什么关系，但又开阔了很多思路。解法三分析出来的迭代方法，以及用映射证明卡特兰数的求法，棒！

23、题目描述（困难难度）

Merge k sorted linked lists and return it as one sorted list. Analyze and describe its complexity.

Example:

```
Input:
[
  1->4->5,
  1->3->4,
  2->6
]
Output: 1->1->2->3->4->4->5->6
```

k 个有序链表的合并。

我们用 N 表示链表的总长度，考虑最坏情况， k 个链表的长度相等，都为 n 。

解法一 暴力破解

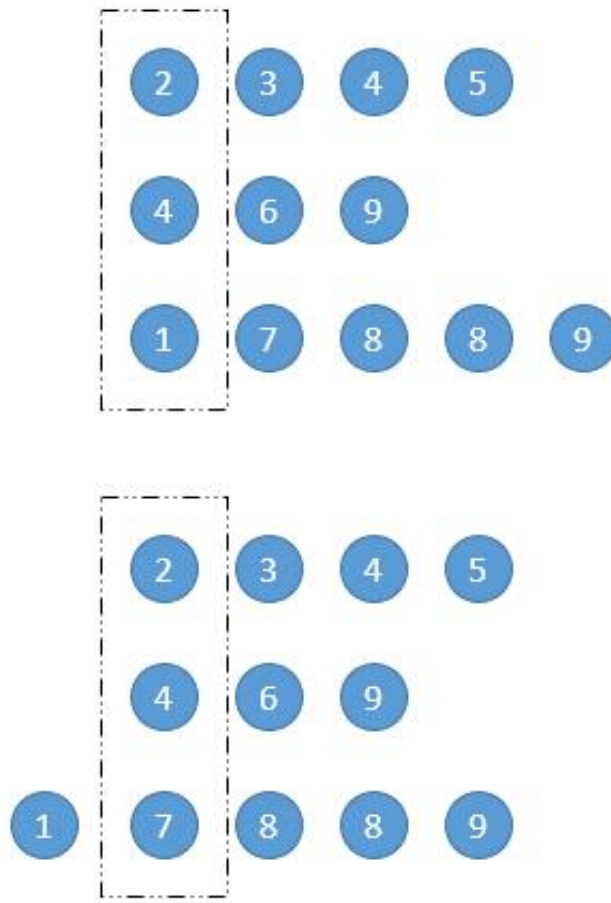
简单粗暴，遍历所有的链表，将数字存到一个数组里，然后用快速排序，最后再将排序好的数组存到一个链表里。

```
public ListNode mergeKLists(ListNode[] lists) {
    List<Integer> l = new ArrayList<Integer>();
    //存到数组
    for (ListNode ln : lists) {
        while (ln != null) {
            l.add(ln.val);
            ln = ln.next;
        }
    }
    //数组排序
    Collections.sort(l);
    //存到链表
    ListNode head = new ListNode(0);
    ListNode h = head;
    for (int i : l) {
        ListNode t = new ListNode(i);
        h.next = t;
        h = h.next;
    }
    h.next = null;
    return head.next;
}
```

时间复杂度：假设 N 是所有的数字个数，存到数组是 $O(N)$ ，排序如果是用快速排序就是 $O(N \log N)$ ，存到链表是 $O(N)$ ，所以取个最大的，就是 $O(N \log N)$ 。

空间复杂度：新建了一个链表， $O(N)$ 。

解法二 一一列比较



我们可以一列一列的比较，将最小的一个存到一个新的链表里。

```
public ListNode mergeKLists(ListNode[] lists) {  
    int min_index = 0;  
    ListNode head = new ListNode(0);  
    ListNode h = head;  
    while (true) {  
        boolean isBreak = true; // 标记是否遍历完所有链表  
        int min = Integer.MAX_VALUE;  
        for (int i = 0; i < lists.length; i++) {  
            if (lists[i] != null) {  
                // 找出最小下标  
                if (lists[i].val < min) {  
                    min_index = i;  
                    min = lists[i].val;  
                }  
                // 存在一个链表不为空，标记改完 false  
                isBreak = false;  
            }  
        }  
        if (isBreak) break;  
        h.next = lists[min_index];  
        lists[min_index] = lists[min_index].next;  
        h = h.next;  
    }  
    return head.next;  
}
```

```

        if (isBreak) {
            break;
        }
        //加到新链表中
        ListNode a = new ListNode(lists[min_index].val);
        h.next = a;
        h = h.next;
        //链表后移一个元素
        lists[min_index] = lists[min_index].next;
    }
    h.next = null;
    return head.next;
}

```

时间复杂度：假设最长的链表长度是 n ，那么 while 循环将循环 n 次。假设链表列表里有 k 个链表，for 循环执行 k 次，所以时间复杂度是 $O(kn)$ 。

空间复杂度： N 表示最终链表的长度，则为 $O(N)$ 。

其实我们不需要创建一个新链表保存，我们只需要改变得到的最小结点的指向就可以了。

```

public ListNode mergeKLists(ListNode[] lists) {
    int min_index = 0;
    ListNode head = new ListNode(0);
    ListNode h = head;
    while (true) {
        boolean isBreak = true;
        int min = Integer.MAX_VALUE;
        for (int i = 0; i < lists.length; i++) {
            if (lists[i] != null) {
                if (lists[i].val < min) {
                    min_index = i;
                    min = lists[i].val;
                }
            }
            isBreak = false;
        }

        if (isBreak) {
            break;
        }
        //最小的节点接过来
        h.next = lists[min_index];
        h = h.next;
        lists[min_index] = lists[min_index].next;
    }
}

```

```
h.next = null;
return head.next;
}
```

时间复杂度：假设最长的链表长度是 n ，那么 while 循环将循环 n 次。假设链表列表里有 k 个链表，for 循环执行 k 次，所以时间复杂度是 $O(kn)$ 。

空间复杂度： $O(1)$ 。

解法三 优先队列

解法二中，我们每次都是取出一个最小的，然后加入一个新的， $O(1)$ 的复杂度，再找最小的， $O(k)$ 的复杂度。我们完全可以用一个优先队列。

操作 [编辑]

优先队列至少需要支持下述操作：

- 插入带优先级的元素 (insert_with_priority)
- 取出具有最高优先级的元素 (pull_highest_priority_element)
- 查看最高优先级的元素 (peek)： $O(1)$ 时间复杂度

我们将优先级定义为数越小优先级越高，如果用堆实现优先队列，这样我们每次找最小不再需要 $O(k)$ ，而是 $O(\log(k))$ ，当然这样的话，我们加入新的话不再是 $O(1)$ ，也需要 $O(\log(k))$ 。可以看看[这里](#)和[这里](#)。

```
public ListNode mergeKLists(ListNode[] lists) {
    //定义优先队列的比较器
    Comparator<ListNode> cmp;
    cmp = new Comparator<ListNode>() {
        @Override
        public int compare(ListNode o1, ListNode o2) {
            // TODO Auto-generated method stub
            return o1.val-o2.val;
        }
    };

    //建立队列
    Queue<ListNode> q = new PriorityQueue<ListNode>(cmp);
    for(ListNode l : lists){
        if(l!=null){
            q.add(l);
        }
    }
}
```

```

ListNode head = new ListNode(0);
ListNode point = head;
while(!q.isEmpty()){
    //出队列
    point.next = q.poll();
    point = point.next;
    //判断当前链表是否为空，不为空就将新元素入队
    ListNode next = point.next;
    if(next!=null){
        q.add(next);
    }
}
return head.next;
}

```

时间复杂度：假如总共有 N 个节点，每个节点入队出队都需要 $\log(k)$ ，所有时间复杂度是 $O(N \log(k))$ 。

空间复杂度：优先队列需要 $O(k)$ 的复杂度。

解法四 两两合并

利用[之前](#)合并两个链表的算法，我们直接两两合并，第 0 个和第 1 个链表合并，新生成的再和第 2 个链表合并，新生成的再和第 3 个链表合并...直到全部合并完。

```

public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode h = new ListNode(0);
    ListNode ans=h;
    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) {
            h.next = l1;
            h = h.next;
            l1 = l1.next;
        } else {
            h.next = l2;
            h = h.next;
            l2 = l2.next;
        }
    }
    if(l1==null){
        h.next=l2;
    }
    if(l2==null){
        h.next=l1;
    }
}

```

```

        return ans.next;
    }
    public ListNode mergeKLists(ListNode[] lists) {
        if(lists.length==1){
            return lists[0];
        }
        if(lists.length==0){
            return null;
        }
        ListNode head = mergeTwoLists(lists[0],lists[1]);
        for (int i = 2; i < lists.length; i++) {
            head = mergeTwoLists(head,list[i]);
        }
        return head;
    }
}

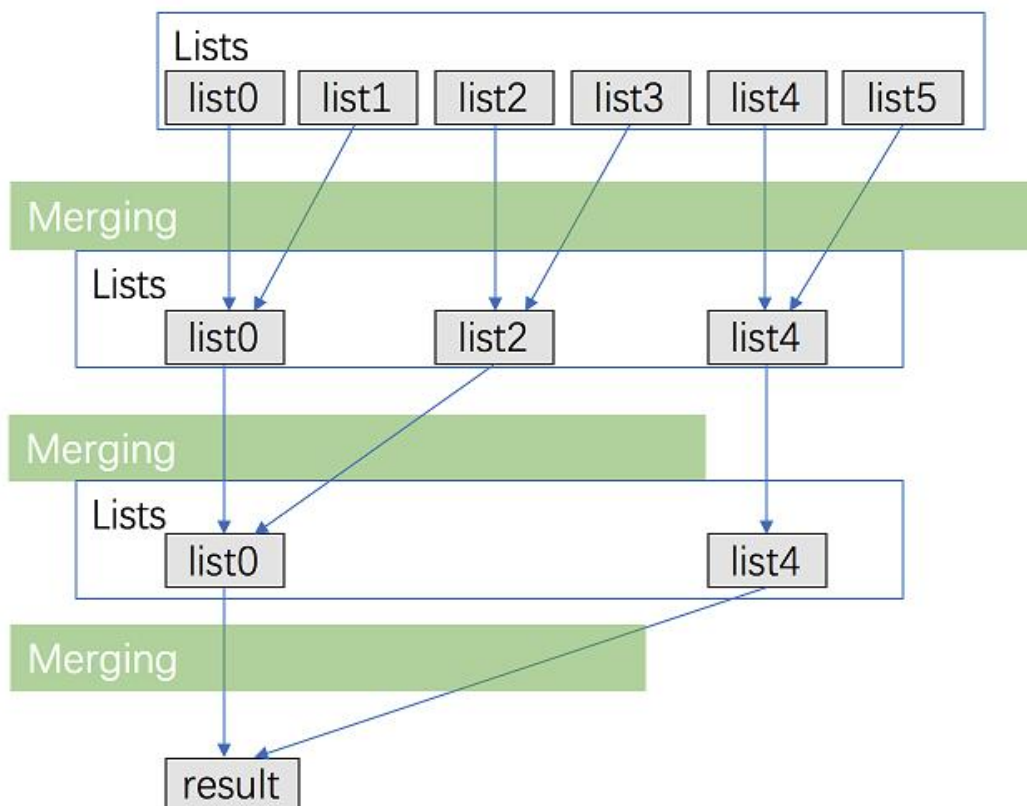
```

时间复杂度：不妨假设是 k 个链表并且长度相同，链表总长度为 N ，那么第一次合并就是 N/k 和 N/k ，第二次合并就是 $2 * N/k$ 和 N/k ，第三次合并就是 $3 * N/k$ 和 N/k ，总共进行 $n - 1$ 次合并，每次合并的时间复杂度是 $O(n)$ ，所以总时间复杂度就是 $O(\sum_{i=1}^{k-1} (i * \frac{N}{k} + \frac{N}{k})) = O(kN)$ ，可以将两项分开， N/k 其实是常数，分开的第一项是等差数列。

空间复杂度： $O(1)$ 。

#解法五 两两合并优化

依旧假设是 k 个链表，合并的过程优化下，使得只需要合并 $\log(k)$ 次。



```

public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
    ListNode h = new ListNode(0);
    ListNode ans=h;
    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) {
            h.next = l1;
            h = h.next;
            l1 = l1.next;
        } else {
            h.next = l2;
            h = h.next;
            l2 = l2.next;
        }
    }
    if(l1==null){
        h.next=l2;
    }
    if(l2==null){
        h.next=l1;
    }
    return ans.next;
}

public ListNode mergeKLists(ListNode[] lists) {

```

```

    if(lists.length==0){
        return null;
    }
    int interval = 1;
    while(interval<lists.length){
        System.out.println(lists.length);
        for (int i = 0; i + interval< lists.length; i=i+interval*2) {
            lists[i]=mergeTwoLists(lists[i],lists[i+interval]);
        }
        interval*=2;
    }

    return lists[0];
}

```

时间复杂度：假设每个链表的长度都是 n ，有 k 个链表，记总结点数是 $N = n * k$ ，那么时间复杂度就是 $O(\sum_{i=1}^{\log_2 k} N) = O(N \log k)$ 。

空间复杂度： $O(1)$ 。

总

优先队列的运用印象深刻，此外对两两链表的合并，我们仅仅改变了合并的方式就将时间复杂度降低了很多，美妙！

24、题目描述（中等难度）

Given a linked list, swap every two adjacent nodes and return its head.

Example:

Given 1->2->3->4, you should return the list as 2->1->4->3.

Note:

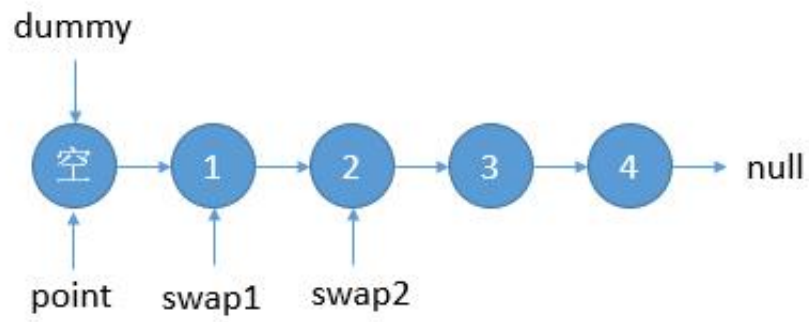
- Your algorithm should use only constant extra space.
- You may **not** modify the values in the list's nodes, only nodes itself may be changed.

给定一个链表，然后两两交换链表的位置。

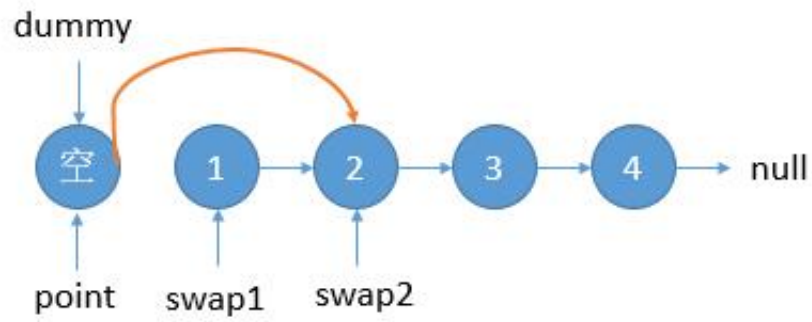
解法一 迭代

首先为了避免单独讨论头结点的情况，一般先申请一个空结点指向头结点，然后再用一个指针来遍历整个链表。

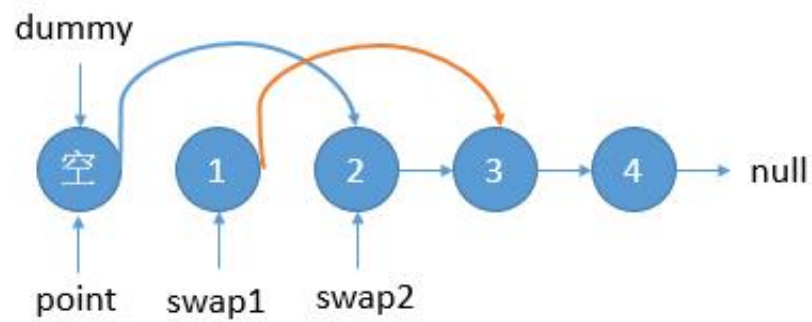
先来看一下图示：

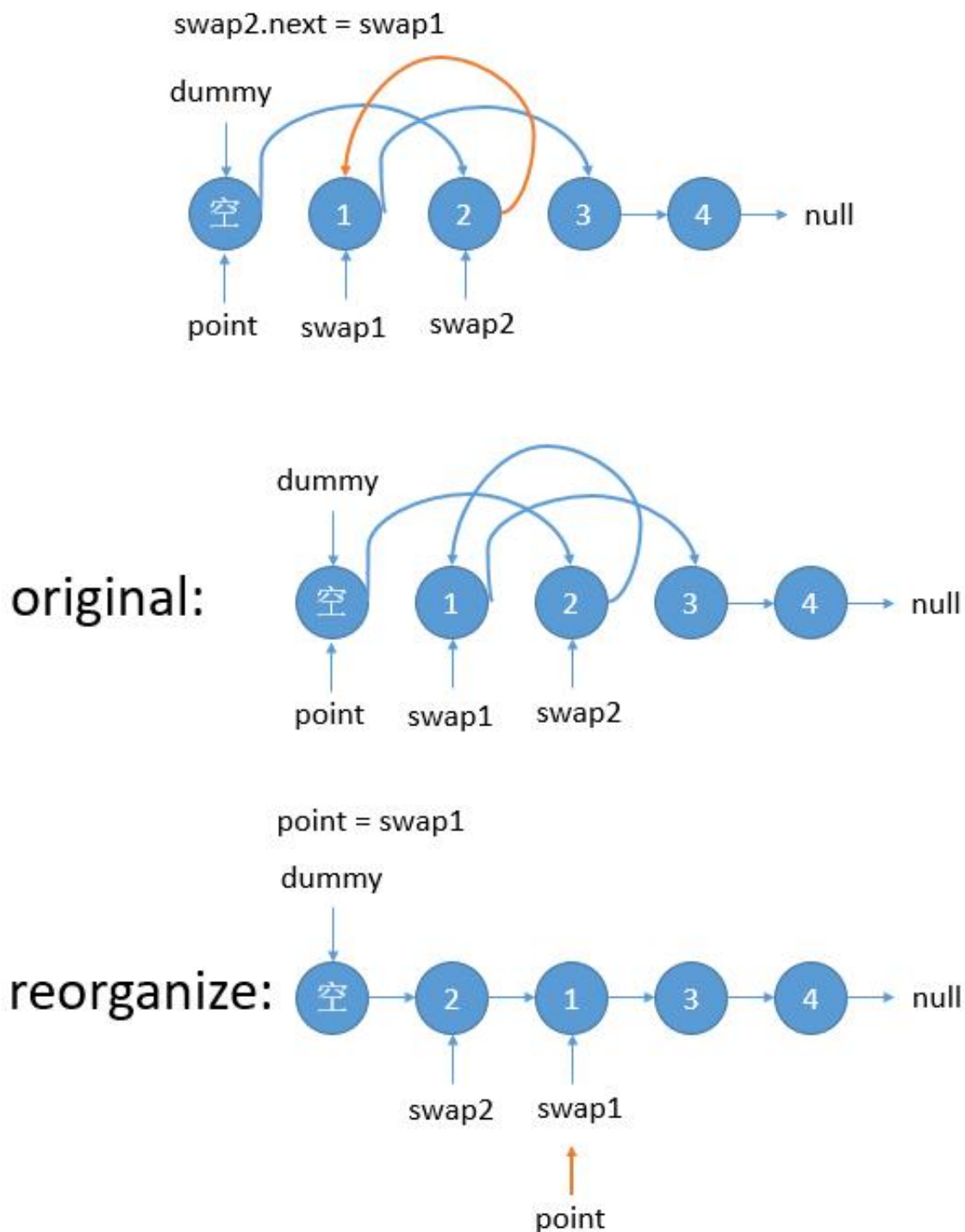


`point.next = swap2`



`swap1.next = swap2.next`





point 是两个要交换结点前边的一个位置。

```
public ListNode swapPairs(ListNode head) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode point = dummy;
    while (point.next != null && point.next.next != null) {
        ListNode swap1 = point.next;
```

```

        ListNode swap2 = point.next.next;
        point.next = swap2;
        swap1.next = swap2.next;
        swap2.next = swap1;
        point = swap1;
    }
    return dummy.next;
}

```

时间复杂度： $O(n)$ 。

空间复杂度： $O(1)$ 。

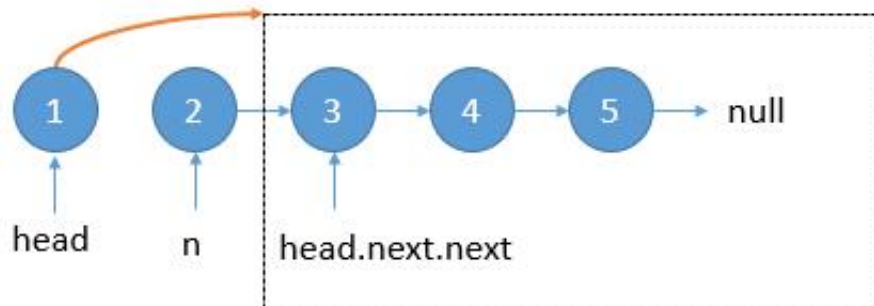
解法二 递归

参考[这里](#)。

自己画了个参考图。

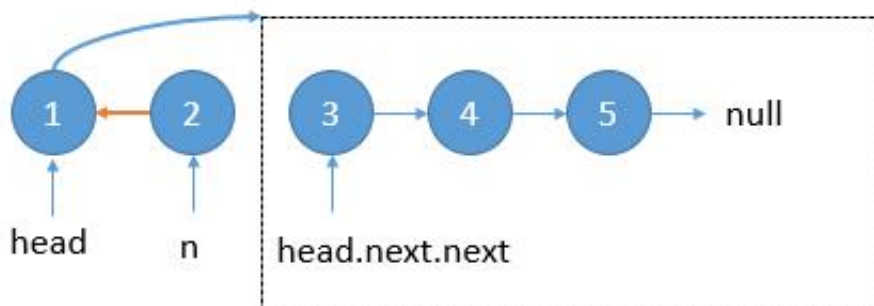
`head.next = swapPairs(head.next.next)`

进入递归，它最终返回两两交换过的链表



`n.next = head`

进入递归，它最终返回两两交换过的链表



```
public ListNode swapPairs(ListNode head) {  
    if ((head == null) || (head.next == null))  
        return head;  
    ListNode n = head.next;  
    head.next = swapPairs(head.next.next);  
    n.next = head;  
    return n;  
}
```

递归时间复杂度留坑。

总

自己开始没有想出递归的算法，每次都会被递归的简洁吸引。另外，感觉链表的一些题，只要画图打打草稿，搞清指向关系，一般不难。

25、题目描述（困难难度）

Given a linked list, reverse the nodes of a linked list k at a time and return its modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes in the end should remain as it is.

Example:

Given this linked list: 1->2->3->4->5

For $k = 2$, you should return: 2->1->4->3->5

For $k = 3$, you should return: 3->2->1->4->5

Note:

- Only constant extra memory is allowed.
- You may not alter the values in the list's nodes, only nodes itself may be changed.

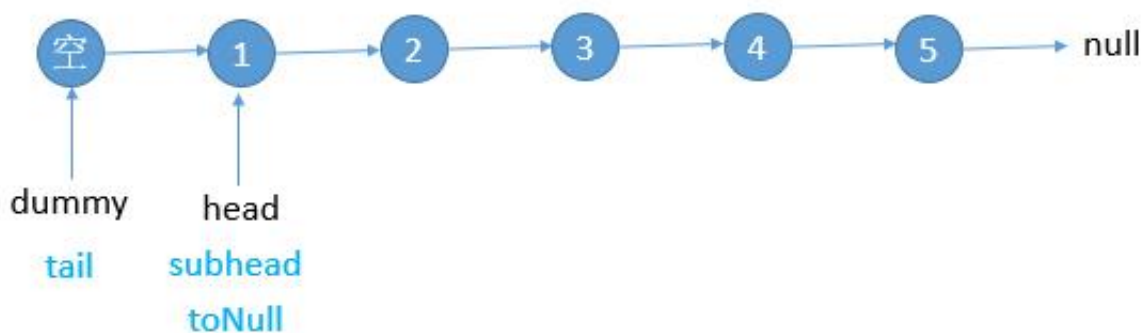
将一个链表，每 k 个倒置，最后一组不足 k 个就不倒置。

解法一 迭代

关于单链表倒置，我们在[第 2 题](#)就讨论过。有了单链表倒置，这道题无非就是用一个循环，每次将 k 个结点取下来，倒置后再接回去，然后再取 k 个，以此循环，到了最后一组如果不足 k 个，不做处理，直接返回头结点就可以了。所以关键就是，指针指来指去，大家不晕掉就好，我做了图示，大家参考一下。

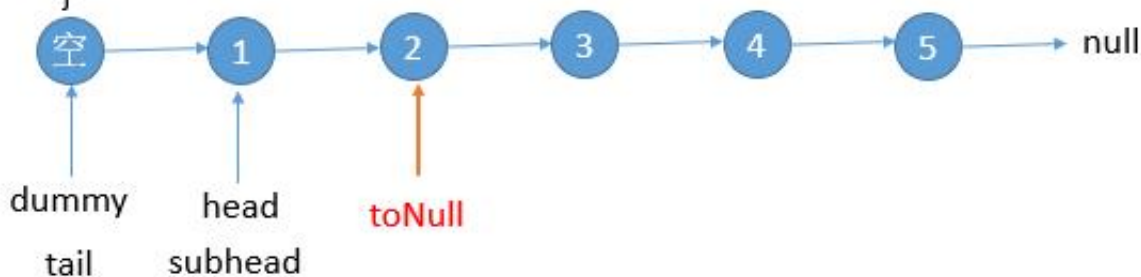
为了将头结点也一般化，我们创建一个 dummy 结点，然后整个过程主要运用三个指针，tail 指针表示已经倒置后的链表的尾部，subhead 指针表示要进行倒置的子链表，toNull 指针为了将子链表从原来链表中取下来。

```
ListNode sub_head = head;
ListNode dummy = new ListNode(0);
dummy.next = head;
ListNode tail = dummy;
ListNode toNull = head;
```



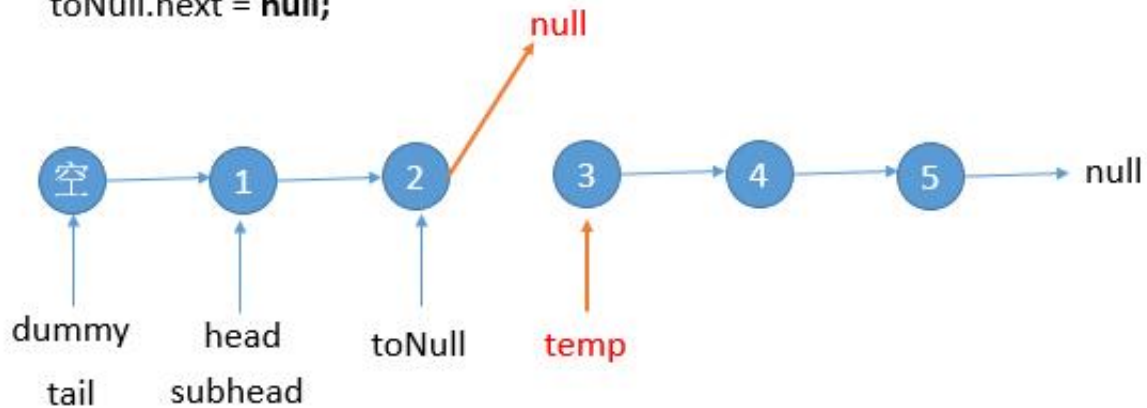
一个 while 循环，让 toNull 指针走 $k - 1$ 步使其指向子链表的尾部。中间的 if 语句就是判断当前节点数够不够 k 个了，不够的话直接返回结果就可以了。

```
int i = k;
while (i - 1 > 0) {
    toNull = toNull.next;
    if (toNull == null) {
        return dummy.next;
    }
    i--;
}
```



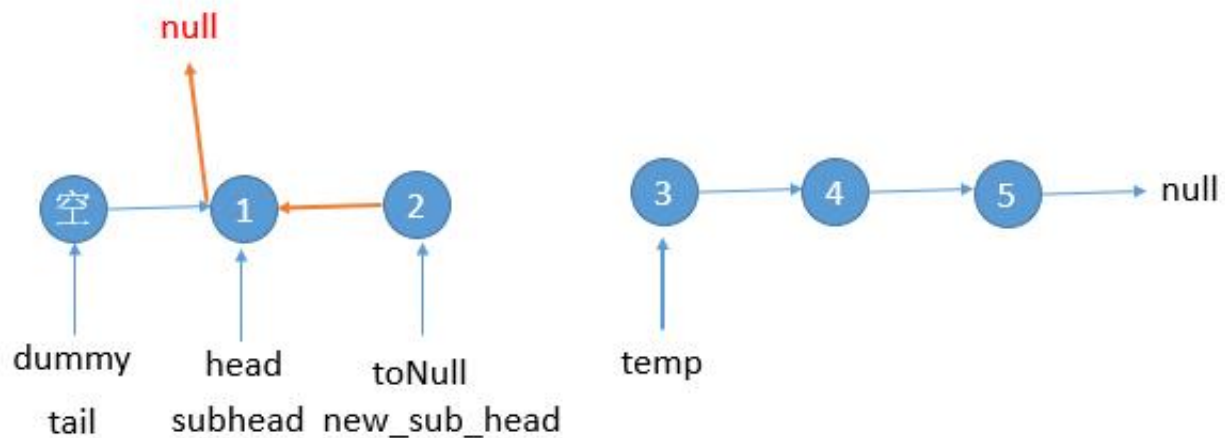
将子链表指向 null，脱离出来。并且用 temp 保存下一个结点的位置。

```
ListNode temp = toNull.next;  
toNull.next = null;
```



然后调用倒置函数，将子链表倒置。

```
ListNode new_sub_head = reverse(sub_head);
```

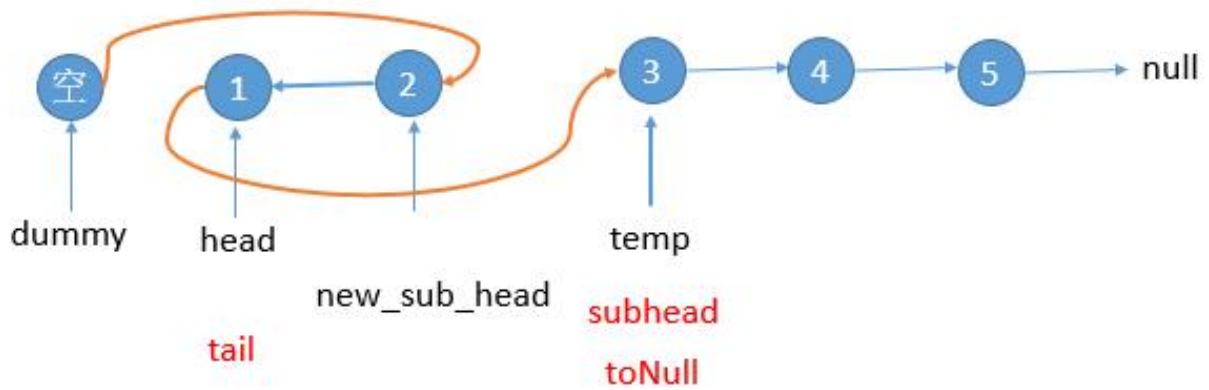


接下来四步分别是，新链表接到 `tail`（注意下边的图 `tail` 是更新后的位置，之前 `tail` 在 `dummy` 的位置）的后边；更新 `tail` 到新链表的尾部，也就是之前的 `subhead`（下图 `subhead` 也是更新后的位置，之前的位置参见上边的图）；`sub_head` 更新到 `temp` 的位置；`toNull` 到 `sub_head` 的位置；然后将新的尾部 `tail` 把之前断开的链表连起来，接到 `sub_head` 上。

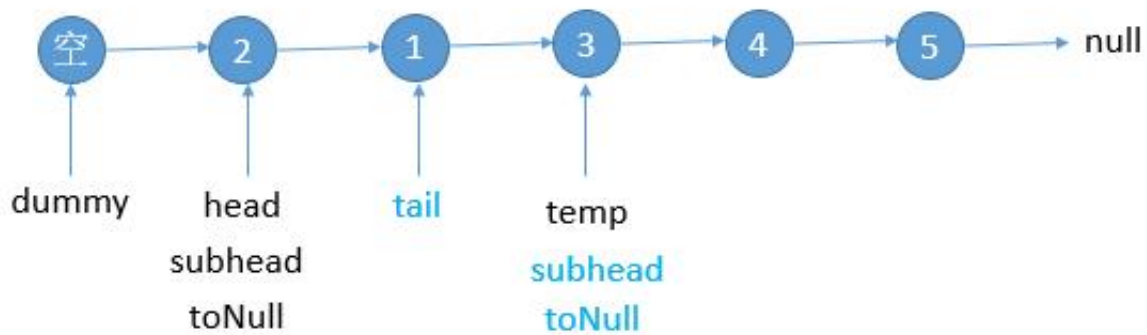
```

tail.next = new_sub_head;
tail = sub_head;
sub_head = temp;
toNull = sub_head;
tail.next = sub_head;

```



整理下其实就是下边的样子

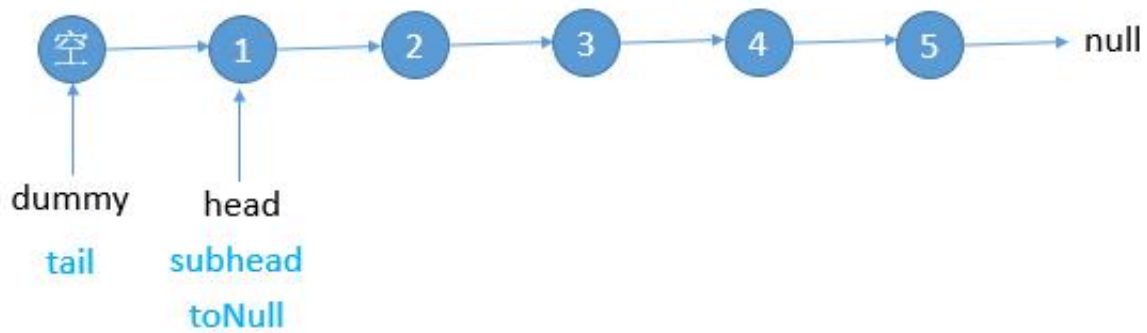


和初始的时候（下边的图）对比一下，发现 tail，subhead 和 toNull 三个指针已经就位，可以愉快的重复上边的步骤了。

```

ListNode sub_head = head;
ListNode dummy = new ListNode(0);
dummy.next = head;
ListNode tail = dummy;
ListNode toNull = head;

```



看下代码吧。

```

public ListNode reverseKGroup(ListNode head, int k) {
    if (head == null)
        return null;
    ListNode sub_head = head;
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode tail = dummy;
    ListNode toNull = head;
    while (sub_head != null) {
        int i = k;
        //找到子链表的尾部
        while (i - 1 > 0) {
            toNull = toNull.next;
            if (toNull == null) {
                return dummy.next;
            }
        }
        i--;
        ListNode temp = toNull.next;
        //将子链表断开
        toNull.next = null;
        ListNode new_sub_head = reverse(sub_head);
        //将倒置后的链表接到 tail 后边
        tail.next = new_sub_head;
        //更新 tail
        tail = sub_head; //sub_head 由于倒置其实是新链表的尾部
    }
}

```



```

        sub_head = temp;
        toNull = sub_head;
        //将后边断开的链表接回来
        tail.next = sub_head;
    }
    return dummy.next;
}

public ListNode reverse(ListNode head) {
    ListNode current_head = null;
    while (head != null) {
        ListNode next = head.next;
        head.next = current_head;
        current_head = head;
        head = next;
    }
    return current_head;
}

```

时间复杂度：while 循环中本质上我们只是将每个结点访问了一次，加上结点倒置访问的一次，所以总共加起来每个结点其实只访问了 2 次。所以时间复杂度是 $O(n)$ 。

空间复杂度： $O(1)$ 。

解法二递归

有没有被解法一的各种指针绕晕呢，我们有一个更好的选择，递归，这样看起来就会简洁很多。

```

public ListNode reverseKGroup(ListNode head, int k) {
    if (head == null)
        return null;
    ListNode point = head;
    //找到子链表的尾部
    int i = k;
    while(i - 1 > 0){
        point = point.next;
        if (point == null) {
            return head;
        }
        i--;
    }
    ListNode temp = point.next;
    //将子链表断开
    point.next = null;

    //倒置子链表，并接受新的头结点
    ListNode new_head = reverse(head);
}

```

```

//head 其实是倒置链表的尾部，然后将后边的倒置结果接过来就可以了
//temp 是链表断开后的头指针，可以参考解法一的图示
head.next = reverseKGroup(temp,k);
return new_head;
}
public ListNode reverse(ListNode head) {
    ListNode current_head = null;
    while (head != null) {
        ListNode next = head.next;
        head.next = current_head;
        current_head = head;
        head = next;
    }
    return current_head;
}

```

复杂度：递归留坑。

总

还是那句话，涉及到链表的，我们就画下图，把各个指针的移动理清楚，一般没啥问题。

26、题目描述（简单难度）

Given a sorted array `nums`, remove the duplicates **in-place** such that each element appear only *once* and return the new length.

Do not allocate extra space for another array, you must do this by **modifying the input array in-place** with $O(1)$ extra memory.

Example 1:

Given `nums = [1,1,2]`,

Your function should return `length = 2`, with the first two elements of `nums` being `1` and `2`

It doesn't matter what you leave beyond the returned length.

返回非重复数字的个数，并且把 `nums` 里重复的数字也去掉。

例如，`nums = [1, 1, 2]`，那么就返回 `2`，并且把 `nums` 变成 `[1, 2]`。

这道题，蛮简单的，但是自己写的时候多加了个 `while` 循环，但和给出的 Solution 本质还是一样的。

我写的

for 循环遍历每个数，while 循环判断当前数和它的后一个数是否相等，相等就后移一个数，并且接着判断后移的数和它后边的数是否相等，然后一直循环下去。不相等就将后一个数保存起来，并且长度加 1，然后结束循环。

```
public int removeDuplicates(int[] nums) {
    int len = 1;
    for (int i = 0; i < nums.length - 1; i++) {
        while (i < nums.length - 1) {
            if (nums[i] == nums[i + 1]) {
                i++;
            } else {
                nums[len] = nums[i + 1];
                len = len + 1;
                break;
            }
        }
    }
    return len;
}
```

时间复杂度：O (n) 。

空间复杂度：O (1) 。

Solution 给出的

利用快慢指针，i 指针从 0 开始，j 指针从 1 开始，如果 i 和 j 所指数字相等，就一直后移 j。如果不相等，i 指针后移一位用来保存当前 j 所指的数字，然后继续回到 j 的后移中去。

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int i = 0;
    for (int j = 1; j < nums.length; j++) {
        if (nums[j] != nums[i]) {
            i++;
            nums[i] = nums[j];
        }
    }
    return i + 1;
}
```

时间复杂度：O (n) 。

空间复杂度：O (1) 。

总

不同的思想，决定了写出来的代码不同，但就时间复杂度来看，它们本质还是一样的。

27、题目描述（简单难度）

Given an array *nums* and a value *val*, remove all instances of that value **in-place** and return the new length.

Do not allocate extra space for another array, you must do this by **modifying the input array in-place** with $O(1)$ extra memory.

The order of elements can be changed. It doesn't matter what you leave beyond the new length.

Example 1:

Given *nums* = [3,2,2,3], *val* = 3,

Your function should return length = 2, with the first two elements of *nums* being 2.

It doesn't matter what you leave beyond the returned length.

Example 2:

Given *nums* = [0,1,2,2,3,0,4,2], *val* = 2,

Your function should return length = 5, with the first five elements of *nums* containing 0, 1, 3, 0, and 4.

Note that the order of those five elements can be arbitrary.

It doesn't matter what values are set beyond the returned length.

和[上一题](#)类似，只不过这个是去除给定的值，看起来还更简单些。

例如给了 *nums* = [3, 2, 2, 3], *val* = 3，然后我们返回 len = 2，并且 *nums* 修改为 [2, 2]。

解法一

和上道题一样，我们利用快慢指针，此外我们还得用下反向的思维。快指针 *fast* 和慢指针 *slow*，一直移动 *fast*，如果 *fast* 指向的值不等于给定的 *val*，我们就将值赋给 *slow* 指向的位置，*slow* 后移一位。如果 *fast* 指向的值等于 *val* 了，此时 *fast* 后移一位就可以了，不做其他操作。

```
public int removeElement(int[] nums, int val) {
    int fast = 0;
    int slow = 0;
    while (fast < nums.length) {
        if (nums[fast] != val) {
            nums[slow++] = nums[fast];
        }
        fast++;
    }
    return slow;
}
```

时间复杂度： $O(n)$ 。

空间复杂度： $O(1)$ 。

解法二

参考给出的[Solution](#)。

上边的解法，我们是如果不等于 val 就赋值。但如果按题目的想法，应该是如果等于 val 就移除。我们从正方面去想，也就是等于 val 的话，我们怎么体现移除呢？

题目中有个说明我们没利用到，他告诉我们说 the order of those five elements can be arbitrary，就是说数组的顺序可以随便换，我们怎么充分利用呢？

我们可以这样，如果当前元素等于 val 了，我们就把它扔掉，然后将最后一个值赋值到当前位置，并且长度减去 1。什么意思呢？

比如 1 2 2 4 6，如果 val 等于 2。那么当移动到 2 的时候，等于 val 了。我们就把最后一个位置的 6 赋值过来，长度减去 1。就变成了 1 6 2 4。完美！达到了移除的效果。然后当又移动到新的 2 的时候，就把最后的 4 拿过来，变成 1 6 4，达到了移除的效果。看下代码吧。

```
public int removeElement(int[] nums, int val) {
    int i = 0;
    int n = nums.length;
    while (i < n) {
        if (nums[i] == val) {
            nums[i] = nums[n - 1];
            n--;
        } else {
            i++;
        }
    }
    return n;
}
```

时间复杂度：同样是 $O(n)$ ，但如果等于 val 的值比较少，解法二会更有效率些。比如 1 2 3 4，val = 2。解法一 while 循环中将调用 3 次赋值。而解法二中，仅仅当等于 val 的时候赋值 1 次。

空间复杂度： $O(1)$ 。

总

Solution 给出的想法让人耳目一新，对于移除的值少的情况，优化了不少。

28、题目描述（简单难度）

Implement `strStr()`.

Return the index of the first occurrence of needle in haystack, or **-1** if needle is not part of haystack.

Example 1:

Input: haystack = "hello", needle = "ll"

Output: 2

Example 2:

Input: haystack = "aaaaa", needle = "bba"

Output: -1

Clarification:

What should we return when `needle` is an empty string? This is a great question to ask during an interview.

For the purpose of this problem, we will return 0 when `needle` is an empty string. This is consistent to C's `strstr()` and Java's `indexOf()`.

返回一个字符串 needle 在另一个字符串 haystack 中开始的位置，如果不存在就返回 -1，如果 needle 长度是 0，就返回 0。

就是——比较就好，看下代码吧。

```
public int strStr(String haystack, String needle) {
    if (needle.length() == 0) {
        return 0;
    }
    int j = 0;
    //遍历每个字符
    for (int i = 0; i < haystack.length(); i++) {
        //相等的话计数加 1
        if (haystack.charAt(i) == needle.charAt(j)) {
            j++;
            //长度够了就返回
            if (j == needle.length()) {
                return i - j + 1;
            }
            // 不相等的话 j 清零，
            // 并且 i 回到最初的位置，之后就进入 for 循环中的 i++，从下个位置继续判断
        } else {
            i = i - j;
            j = 0;
        }
    }
    return -1;
}
```

时间复杂度：假设 haystack 和 needle 的长度分别是 n 和 k，对于每一个 i，我们最多执行 k - 1 次，总共会有 n 个 i，所以时间复杂度是 $O(kn)$ 。

空间复杂度： $O(1)$ 。

我们再看下别人的[代码](#)，用两个 for 循环。但本质其实是一样的，但可能会更好理解些吧。

```
public int strStr(String haystack, String needle) {
    for (int i = 0; ; i++) {
        for (int j = 0; ; j++) {
            if (j == needle.length()) return i;
            if (i + j == haystack.length()) return -1;
            if (needle.charAt(j) != haystack.charAt(i + j)) break;
        }
    }
}
```

总

总的来说，还是比较简单的，就是简单的遍历就实现了。

29、题目描述（中等难度）

Given two integers `dividend` and `divisor`, divide two integers without using multiplication, division and mod operator.

Return the quotient after dividing `dividend` by `divisor`.

The integer division should truncate toward zero.

Example 1:

Input: `dividend = 10, divisor = 3`

Output: 3

Example 2:

Input: `dividend = 7, divisor = -3`

Output: -2

Note:

- Both `dividend` and `divisor` will be 32-bit signed integers.
- The `divisor` will never be 0.
- Assume we are dealing with an environment which could only store integers within the 32-bit signed integer range: $[-2^{31}, 2^{31} - 1]$. For the purpose of this problem, assume that your function returns $2^{31} - 1$ when the division result overflows.

两个数相除，给出商。不能用乘法，除法和模操作。

本来觉得这道题蛮简单的，记录下自己的坎坷历程。

尝试1

先确定商的符号，然后把被除数和除数通通转为正数，然后用被除数不停的减除数，直到小于除数的时候，用一个计数遍历记录总共减了多少次，即为商了。

确定商的符号的时候，以及返回最终结果的时候，我们可能需要进行乘 -1 操作，即取相反数。而题目规定不让用乘法，所以我们需要知道计算机是怎么进行存数的。

计算机为了算减法，利用了同余的性质。

同余的定义是 $a \equiv b \pmod{m}$ ，即 $a \bmod m == b \bmod m$ ，例如 $5 \equiv 17 \pmod{12}$ 。[百度百科](#)

同余有两个性质

反身性： $a \equiv a \pmod{m}$ ；

同余式相加：若 $a \equiv b \pmod{m}$ ， $c \equiv d \pmod{m}$ ，则 $a + c \equiv b + d \pmod{m}$ ；

现在我们将模 16 的加法操作，先熟悉下下边的几个式子。

$$2 + 14 = 0$$

$$2 + (-3) = 15$$

$$5 + 15 = 4$$

重点来了！

计算 $4 - 2$ 怎么算呢？

也就是 $4 + (-2)$

$$4 \equiv 4 \pmod{16}$$

$$-2 \equiv 14 \pmod{16}$$

所以 $4 + (-2) = 4 + 14 = 2$ 。

我们利用同余的性质，把减法成功转换成了加法，所以我们只需要在计算机里边将 -2 存成 14 就行了。我们这里减去 2 就等价于加上 14。

再比如 $13 - 7$ ，也就是 $13 + (-7)$

$$13 \equiv 13 \pmod{16}$$

$$-7 \equiv 9 \pmod{16}$$

所有 $13 + (-7) = 13 + 9 = 6$

我们成功把减 7 转换成了加上 9。

减 2 转换成加 14，减 7 转换成加 9，这几组数有什么联系呢？是的 $2 + 14 = 16$ ， $7 + 9 = 16$ ，他们相加通通等于 16，也就是我们取的模。有种互补的感觉，所以我们将 14 叫做 -2 的补数，9 叫做 -7 的补数。

可以看到，我们用一些正数表示了负数，总共有 16 个数，除去 0，还剩 15 个数，不可避免的是，这 16 个数，正数和负数的个数会相差 1，我们来看看是正数多，还是负数多。

补数	0	1	2	3	4	5	6	7	8
所代表的数	0	1	2	3	4	5	6	7	等下
补数		15	14	13	12	11	10	9	
所代表的数		-1	-2	-3	-4	-5	-6	-7	

上边的列出的数，应该都没异议吧，那么正数多还是负数多呢？就取决于 8 代表多少了。

$8 + 1 = 9$ ，9 代表 -7，而 $-8 + 1 = -7$ ，所以 8 其实代表 -8。

所以 0 到 15 这 16 个数字可以表示的范围是 -8 ~ 7，-8 没有对称的正数。

我们再来看看计算机里是怎么存的，我们都知道，计算机中是以二进制的方式存储的。假设我们计算机能存储 4 位。范围就是 0000 到 1111，也就是 0 到 15。

补数	0	1	2	3	4	5	6	7	
二进制表示	0000	0001	0010	0011	0100	0101	0110	0111	
所代表的数	0	1	2	3	4	5	6	7	
补数		15	14	13	12	11	10	9	8
二进制表示		1111	1110	1101	1100	1011	1010	1001	1000
所代表的数		-1	-2	-3	-4	-5	-6	-7	-8

我们利用这个表格，求几个例子。

$2 - 3 = 2 + (-3) = 2$ 的补数 + -3 的补数 = $0010 + 1101 = 1111$

而看表格，1111 代表的数就是 -1，从而我们用加法计算出了 $2 - 3 = -1$ 。

$-3 - 2 = (-3) + (-2) = -3$ 的补数 + -2 的补数 = $1101 + 1110 = 1011$

我们可以看到 $1101 + 1110$ 本来等于 $1\ 1011$ ，因为只存储 4 位，所以最高位被丢掉了，其实这就进行了取模的操作，减去了 16。如果我们看所对应的十进制是怎么操作的，1101 表示 13，1110 表示 14， $13 + 14 = 27$ ，如果是模 16 操作下，就是 11，而 11 就是上边的结果 1011，看表格它代表的数是 -5， $-3 - 2 = -5$ ，没毛病。

而且我们发现用这种表示方式，所有的正数首位都是 0，负数的首位都是 1，我们可以这样想。

假设正数和负数的首位相同，假如首位都是 0。

那么比如 $a = 0010$ 这个正数，如果我们去找它的相反数 b ，也就是对应的负数。由假设可以知道，它的相反数的最高位也是 0。即 $0xxx$ 的形式。为了使得 a 和它的相反数相加等于 0，我们必须使得相反数 b 的第 3 位是 1，即 $0x10$ ，才能使得第 3 位的和是 0。但这样的话，第 3 位产生了进位， b 的第 2 位也得是 1，所以 b 就成了 0110，但这样虽然使得最后 3 位的和变成了 0，但是第 1 位我们假设了它是 0，由于第 2 位产生的进位，这样 a 和 b 相加不是 0 了，产生矛盾。所以假设不成立。所以正数和负数的首位一定不同，如果首位 0 代表正数，那么负数的首位一定是 1。

接下来的问题，给出一个数我们总不能查表去看它的补码吧，我们如何得出补码？

对于正数，看表格，我们直接写原码就可以了，例如 7 就是 0111。

负数呢？

我们之前讨论过，对于模 16 的话，-2 的补码是 14，也就是 $16 - 2$ 。-7 的补码是 9，也就是 $16 - 7 = 9$ 。

我们从二进制的方式看一下。

我们来求 -2 的补码，用 $16 - 2 = 1\ 0000 - 0010 = (1111 + 1) - 0010 = (1111 - 0010) + 1 = 1101 + 1 = 1110$ 。

为什么转换成 1111 减去一个数，因为用 1111 减去一个数，虽然是减法，但其实只要把这个数按位求反即可。也就是把 2，0010 按位求反变成 1101，再加上 1 就是 -2 的补码形式了，「按位取反，末位加 1」这个口诀是不是很熟悉，哈哈，这就是快速求补码的法则。但我们不要忘了它的本质，其实是用模长减去它，但是计算机并不会减法，而是巧妙的转换到了取反再加 1。

逆过程呢？如果我们知道了计算机存了个数 1110，那么它代表多少呢？首先首位是 1，它一定是一个负数，其次它是怎么得来的呢？往上翻，其实是用 $16 - 2 = 1110$ 得到的，我们现在是准备求 2，用 16 减去它就可以了，也就是 $16 - 1110 = 1\ 0000 - 1110 = (1111 + 1) - 1110 = (1111 - 1110) + 1 = 0010$ 。巧了，依旧是按位取反，末位加 1。而 0010 就是 2，所以 1110 就代表 -2。

综上，其实我们就是用原来的一部分正数（其实说它是正数也无非是我们自己定义的，想起一句话，数学就像一门宗教，你要么完全相信，要么完全不信，哈哈）表示了负数，而现在为了实现减法，我们把 1xxx 的不当做正数了，把它定义为负数，是的没有负号，但开头是 1，我们就说它是负数，再取个名字就叫补数吧（其实就是它代表的负数离它最近的一个和它同余的数，例如 -3，和它同余的最近的正数就是 13 了，所以 -3 的补数就是 13），再利用余数定理，以及计算机高位溢出等效于求模的性质，巧妙的用取反以及加法实现了减法。

说了这么多，回到开头的部分，怎么不用乘法，来实现求相反数呢？

求 x 的相反数，我们用 0 减去 x 就行。也就是 x 的相反数 $= 0 - x = 0 + (-x) = -x$ ，-x 在计算机中怎么存的呢，存的是 -x 的补码，-x 的补码怎么求？把 x 按位取反，末位加 1。Java 中就是 $\sim x + 1$ 了，此时所存的就是 x 对应的那个负数，即它的相反数了。

3 的相反数怎么求？这求什么求呀，添个负号就行了，-3 呀！但是计算机可没我们这么智能，它只存储 01，所以我们将 -3 的补码求出来存到计算机里就可以了。即把 3（0011）按位取反，末位加 1，得到 1101 就是它的补码，我们然后把 1101 存到了计算机中，我们以为它是 13，但我们给计算机重新定义了规则，它是补码，首位就代表了它是负数，计算机根据规则（按位取反，末位加 1，再添个负号）把它又还原成了我们所理解的 -3。从而我们不进行乘法，根据我们给计算机制定的规则，实现了求相反数。

```
public int divide(int dividend, int divisor) {
    int ans = 0;
    int sign = 1;
    if (dividend < 0) {
        sign = opposite(sign);
        dividend = opposite(dividend);
    }
}
```

```

    if (divisor < 0) {
        sign = opposite(sign);
        divisor = opposite(divisor);
    }
    while (divisor <= dividend) {
        ans = ans + 1;
        dividend = dividend - divisor;
    }
    return sign > 0 ? ans : opposite(ans);
}

public int opposite(int x) {
    return ~x + 1;
}

```

本来信心满满，结果 Wrong Answer。

Wrong Answer Details >

Playground Debug >

Input

-2147483648
-1

Output

0

Expected

2147483647

为什么出错了？ -2147483648 这个数有什么特殊之处吗？

我们知道 int 是用 4 个字节存储，也就是 32 位，那它表示的范围是多少呢？有多少个正数呢？除了第 1 位是 0 固定不变，其它位可以取 0 也可以取 1，所以是 2^{31} ，但这样的话还就包括了 0，所以还得减去 1 个数。也就是 $2^{31}-1=2147483647$ 。那负数有多少个呢，同理除了第 1 位是 1 固定不变，其它位可以取 0 也可以取 1，所以是 $2^{31}=2147483648$ 个负数，所以所表示的范围就是 -2147483648 到 2147483647。和之前我们讨论的是一致的，负数比正数多 1 个。

算法中，我们首先对被除数 -2147483648 取相反数，变成了多少呢？这个不好想，那我们看之前的例子，再模 16 的基础上，最小的负数 -8，取相反数变成了多少，-8 的补码 1000，按位取反，末位加 1，0111 + 1 = 1000，又回到了 1000，所以依旧是 -8。所以题目中的 -2147483648 取相反数，依旧是 -2147483648（有没有发现很神奇 -2147483648 * -1 依旧是 -2147483648）。所以上边的算法中，由于被除数依旧是个负数，所以根本没有进 while 循环，所以直接返回了 0。

尝试二

既然 -2147483648 这么特殊，那我们对它单独判断吧，如果被除数是 -2147483648，除数是 -1，我们就直接返回题目所要求的 2147483647 吧，并且如果除数是 1 就返回 -2147483648。

```

public int divide(int dividend, int divisor) {
    int ans = 0;
    int sign = 1;

```

```

    if (dividend < 0) {
        sign = opposite(sign);
        dividend = opposite(dividend);
    }
    if (divisor < 0) {
        sign = opposite(sign);
        divisor = opposite(divisor);
    }
    //单独判断一下
    if (dividend == Integer.MIN_VALUE && divisor == 1) {
        return sign > 0 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
    }
    while (divisor <= dividend) {
        ans = ans + 1;
        dividend = dividend - divisor;
    }
    return sign > 0 ? ans : opposite(ans);
}

public int opposite(int x) {
    return ~x + 1;
}

```

接着意外又发生了，这次竟然是 Time Limit Exceeded 了。

Time Limit Exceeded Details >

Playground Debug >

Last executed input

```

2147483647
2

```

尝试三

逛了逛 Discuss，由于我们每次只减 1 次除数，循环太多了，找到了[解决方案](#)。

我们每次减 1 次除数，我们其实可以每次减多次。比如 $10 / 1$ ，之前是 $10 - 1 = 9$ ，计数器加 1 变成 1，然后 $9 - 1 = 8$ ，计数器加 1 变成 2，然后 $8 - 1 = 7$ ，计数器加 1 变成 3，直至减到 $0 < 1$ ，我们结束了循环。我们其实可以翻倍减，减完 1，减 2，再减 4，在减 8，当然计数器也不能只加 1 了，减数是翻倍减的，所以计数器也会一直翻倍的加。这里肯定会遇到一个问题，比如 $10 - 1 = 9$ ， $9 - 2 = 7$ ， $7 - 4 = 3$ ， $3 - 8 = -5 < 1$ ，它就走出了 while 循环。但是 3 本来还可以减 3 次 1，所以我们只要再递归就可以了。再看 $3 / 1$ 的商，然后把之前的计数器的值加上 $3 / 1$ 的商就够了。

```

public int divide(int dividend, int divisor) {
    int ans = 1;
    int sign = 1;
    if (dividend < 0) {

```

```

        sign = opposite(sign);
        dividend = opposite(dividend);
    }
    if (divisor < 0) {
        sign = opposite(sign);
        divisor = opposite(divisor);
    }
    if (dividend == Integer.MIN_VALUE && divisor == 1) {
        return sign > 0 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
    }
    int origin_dividend = dividend;
    int origin_divisor = divisor;
    //由于 ans 初始值是 1，所以如果被除数小于除数直接返回 0
    if (dividend < divisor) {
        return 0;
    }
    dividend -= divisor;
    while (divisor <= dividend) {
        ans = ans + ans;
        divisor += divisor;
        dividend -= divisor;
    }
    int a = ans + divide(origin_dividend - divisor, origin_divisor);
    return sign > 0 ? a : opposite(a);
}
public int opposite(int x) {
    return ~x + 1;
}

```

不是超时了，神奇的错误又出现了，

```

Runtime Error Message:   Exception in thread "main" java.lang.StackOverflowError
                        at Solution.divide(Solution.java:30)
                        at Solution.divide(Solution.java:30)
                        at Solution.divide(Solution.java:30)
                        at Solution.divide(Solution.java:30)

```

```

Last executed input:      -1010369383
                        -2147483648

```

我们又看到了，-2147483648 的出现，当除数是它的时候，又出现了神奇的错误，那我们先单独判断一下除数是它，总该可以了吧，继续加上。

```
if(divisor == Integer.MIN_VALUE){
    return 0;
}
```

其他的错误又出现了

Wrong Answer Details >

Playground Debug >

Input

-2147483648
2

Output

0

Expected

-1073741824

被除数是 -2147483648，咦？我们之前不是考虑了吗，不不不，我们只考虑了除数是 1 和 -1 的时候，所以这个问题其实我们一直没有解决。我们必须修改算法了，我们的算法开始的部分是不管三七二十一，通通转换成正数，而出现 -2147483648 的时候，它无法转换成正数，我们怎么该解决呢？

解法一

虽然感觉很投机取巧，但也是最直接的方法了，既然 int 存不了，那我通通用 long 存就行了吧，最后返回的时候看看是不是 int 不能表示的 2147483648，是的话按题目要求就返回 2147483647。

```
public int divide(int dividend, int divisor) {
    long ans = divide((long)dividend, (long)(divisor));
    long m = 2147483648L;
    if(ans == m ){
        return Integer.MAX_VALUE;
    }else{
        return (int)ans;
    }
}

public long divide(long dividend, long divisor) {
    long ans = 1;
    long sign = 1;
    if (dividend < 0) {
        sign = opposite(sign);
        dividend = opposite(dividend);
    }
    if (divisor < 0) {
        sign = opposite(sign);
        divisor = opposite(divisor);
    }
    long origin_dividend = dividend;
    long origin_divisor = divisor;
```

```

    if (dividend < divisor) {
        return 0;
    }

    dividend -= divisor;
    while (divisor <= dividend) {
        ans = ans + ans;
        divisor += divisor;
        dividend -= divisor;
    }
    long a = ans + divide(origin_dividend - divisor, origin_divisor);
    return sign > 0 ? a : opposite(a);
}

public long opposite(long x) {
    return ~x + 1;
}

```

时间复杂度：最坏的情况，除数是 1，如果一次一次减除数，那么我们将减 n 次，但由于每次都翻倍了，所以总共减了 $\log(n)$ 次，所以时间复杂度是 $O(\log(n))$ 。

空间复杂度： $O(1)$ 。

解法二

上边的解法总归不够优雅，那么如何不用 long 呢？

负数比正数多一个，我们之前的思路是把负数变成正数，但由于最小的负数无法变成正数，所以出现了上边奇奇怪怪的问题。我们为什么不把思路转过来，把正数通通转为负数呢？然后很多加法会变成减法，大于号随之也会变成小于号。

```

public int divide(int dividend, int divisor) {
    int ans = -1;
    int sign = 1;
    if (dividend > 0) {
        sign = opposite(sign);
        dividend = opposite(dividend);
    }
    if (divisor > 0) {
        sign = opposite(sign);
        divisor = opposite(divisor);
    }

    int origin_dividend = dividend;
    int origin_divisor = divisor;
    if (dividend > divisor) {
        return 0;
    }
}

```

```

    }

    dividend -= divisor;
    while (divisor >= dividend) {
        ans = ans + ans;
        divisor += divisor;
        dividend -= divisor;
    }
    //此时我们传进的是两个负数，正常情况下，它就返回正数，但我们是在用负数累加，所以要取相反数
    int a = ans + opposite(divide(origin_dividend - divisor, origin_divisor));
    if(a == Integer.MIN_VALUE){
        if( sign > 0){
            return Integer.MAX_VALUE;
        }else{
            return Integer.MIN_VALUE;
        }
    }else{
        if(sign > 0){
            return opposite(a);
        }else{
            return a;
        }
    }
}

public int opposite(int x) {
    return ~x + 1;
}
}

```

时间复杂度和空间复杂度没有变化，但是我们优雅的实现了这个算法，没有借用 long。

总

这道题看起来简单，却藏了不少坑。首先，我们用一次一次减造成了超时，然后用递归实现了加倍加倍的减，接着由于 int 表示的数的范围不是对称的，最小的负数并不能转换为对应的相反数，所以我们将之前的算法思路完全逆过来，正数变负数，大于变小于，还是蛮有意思的。

30、题目描述（困难难度）

30. Substring with Concatenation of All Words

Hard  639  1023  Favorite  Share

You are given a string, **s**, and a list of words, **words**, that are all of the same length. Find all starting indices of substring(s) in **s** that is a concatenation of each word in **words** exactly once and without any intervening characters.

Example 1:

Input:

```
s = "barfoothefoobarman",  
words = ["foo","bar"]
```

Output: [0,9]

Explanation: Substrings starting at index 0 and 9 are "barfoor" and "foobar" respectively.
The output order does not matter, returning [9,0] is fine too.

Example 2:

Input:

```
s = "wordgoodgoodgoodbestword",  
words = ["word","good","best","word"]
```

Output: []

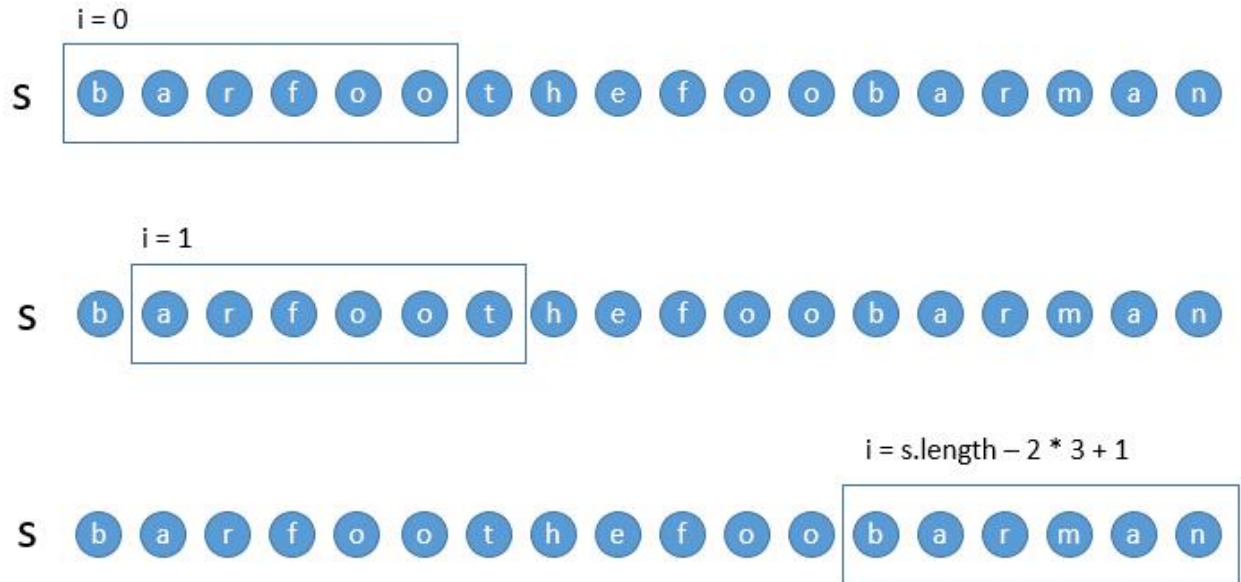
给定一个字符串 **s**，给定 **n** 个单词 **word**，找出所有子串的开始下标，使得子串包含了给定的所有单词，顺序可以不对应。如果有重复的单词，比如有 ["foo", "foo"] 那么子串也必须含有两个 "foo"，也就是说个数必须相同。

解法一

参考 leetCode 里的 [solution](#)

首先，最直接的思路，判断每个子串是否符合，符合就把下标保存起来，最后返回即可。

```
s = "barfoothefoobarman",  
words = ["foo","bar"]
```



如上图，利用循环变量 i ，依次后移，判断每个子串是否符合即可。

怎么判断子串是否符合？这也是这个题的难点了，由于子串包含的单词顺序并不需要固定，如果是两个单词 A, B，我们只需要判断子串是否是 AB 或者 BA 即可。如果是三个单词 A, B, C 也还好，只需要判断子串是否是 ABC，或者 ACB, BAC, BCA, CAB, CBA 就可以了，但如果更多单词呢？那就崩溃了。

[链接](#)的作者提出了，用两个 HashMap 来解决。首先，我们把所有的单词存到 HashMap 里，key 直接存单词，value 存单词出现的个数（因为给出的单词可能会有重复的，所以可能是 1 或 2 或者其他）。然后扫描子串的单词，如果当前扫描的单词在之前的 HashMap 中，就把该单词存到新的 HashMap 中，并判断新的 HashMap 中该单词的 value 是不是大于之前的 HashMap 该单词的 value，如果大了，就代表该子串不是我们要找的，接着判断下一个子串就可以了。如果不大于，那么我们接着判断下一个单词的情况。子串扫描结束，如果子串的全部单词都符合，那么该子串就是我们找的其中一个。看下具体的例子。

看下图，我们把 words 存到一个 HashMap 中。

words = [foo , bar , foo]

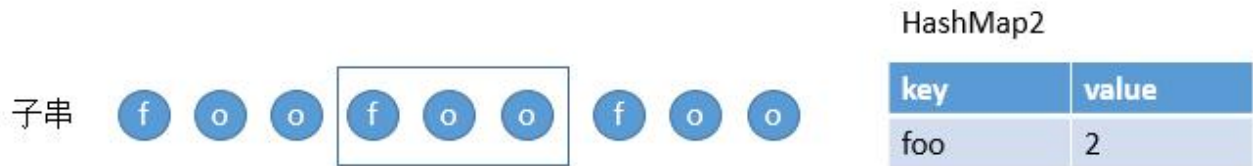
key	value
foo	2
bar	1

子串 [f o o f o o f o o]

然后遍历子串的每个单词。



第一个单词在 HashMap1 中，然后我们把 foo 存到 HashMap2 中。并且比较此时 foo 的 value 和 HashMap1 中 foo 的 value, $1 < 2$ ，所以我们继续扫描。



第二个单词也在 HashMap1 中，然后把 foo 存到 HashMap2 中，因为之前已经存过了，所以更新它的 value 为 2，然后继续比较此时 foo 的 value 和 HashMap1 中 foo 的 value, $2 \leq 2$ ，所以继续扫描下一个单词。



第三个单词也在 HashMap1 中，然后把 foo 存到 HashMap2 中，因为之前已经存过了，所以更新它的 value 为 3，然后继续比较此时 foo 的 value 和 HashMap1 中 foo 的 value, $3 > 2$ ，所以表明该字符串不符合。然后判断下个子串就好了。

当然上边的情况都是单词在 HashMap1 中，如果不在的话就更好说了，不在就表明当前子串肯定不符合了，直接判断下个子串就好了。

看一下代码吧

```
public List<Integer> findSubstring(String s, String[] words) {
    List<Integer> res = new ArrayList<Integer>();
    int wordNum = words.length;
    if (wordNum == 0) {
        return res;
    }
    int wordLen = words[0].length();
    //HashMap1 存所有单词
    HashMap<String, Integer> allWords = new HashMap<String, Integer>();
    for (String w : words) {
        int value = allWords.getOrDefault(w, 0);
        allWords.put(w, value + 1);
    }
}
```

```

    }
    //遍历所有子串
    for (int i = 0; i < s.length() - wordNum * wordLen + 1; i++) {
        //HashMap2 存当前扫描的字符串含有的单词
        HashMap<String, Integer> hasWords = new HashMap<String, Integer>();
        int num = 0;
        //判断该子串是否符合
        while (num < wordNum) {
            String word = s.substring(i + num * wordLen, i + (num + 1) *
wordLen);
            //判断该单词在 HashMap1 中
            if (allWords.containsKey(word)) {
                int value = hasWords.getOrDefault(word, 0);
                hasWords.put(word, value + 1);
                //判断当前单词的 value 和 HashMap1 中该单词的 value
                if (hasWords.get(word) > allWords.get(word)) {
                    break;
                }
            } else {
                break;
            }
            num++;
        }
        //判断是不是所有的单词都符合条件
        if (num == wordNum) {
            res.add(i);
        }
    }
    return res;
}

```

时间复杂度：假设 s 的长度是 n ， $words$ 里有 m 个单词，那么时间复杂度就是 $O(n * m)$ 。

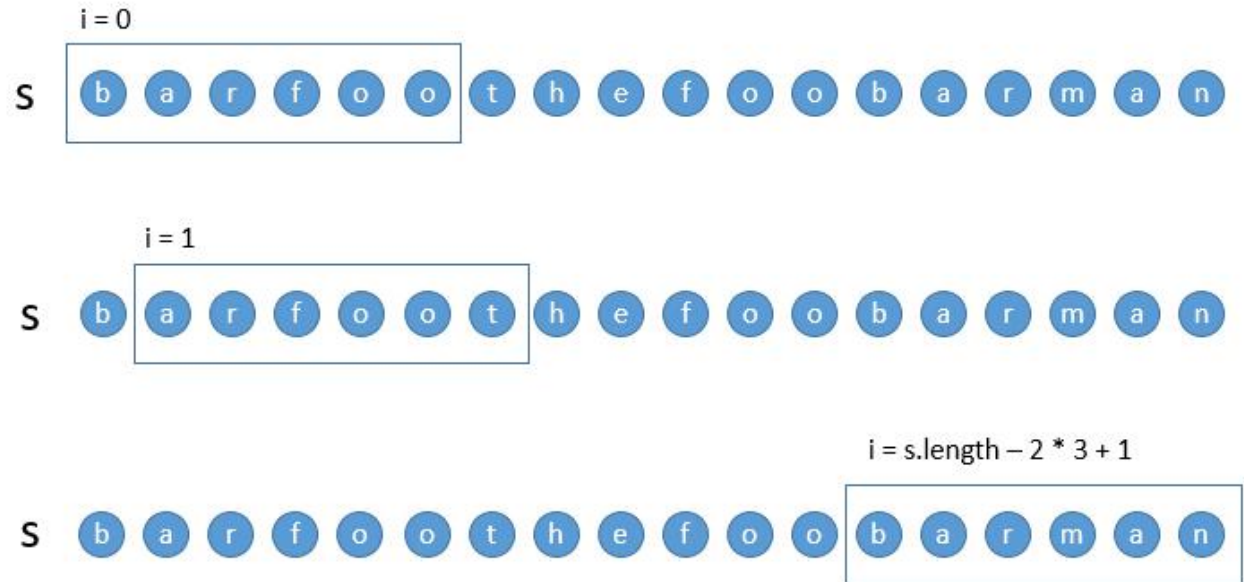
空间复杂度：两个 $HashMap$ ，假设 $words$ 里有 m 个单词，就是 $O(m)$ 。

解法二

参考 [https://leetcode.com/problems/substring-with-concatenation-of-all-words/discuss/13656/An-O\(N\)-solution-with-detailed-explanation](https://leetcode.com/problems/substring-with-concatenation-of-all-words/discuss/13656/An-O(N)-solution-with-detailed-explanation)。

我们在解法一中，每次移动一个字符。

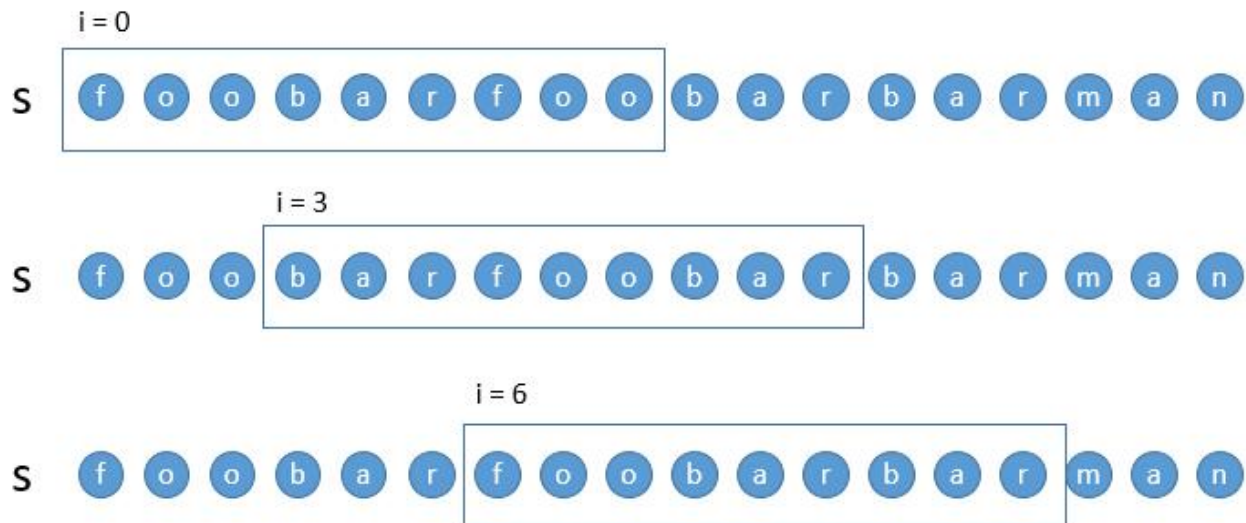
```
s = "barfoothefoobarman",  
words = ["foo","bar"]
```



现在为了方便讨论，我们每次移动一个单词的长度，也就是 3 个字符，这样所有的移动被分成了三类。

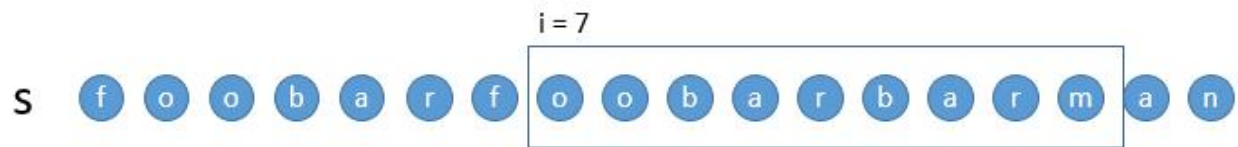
```
s = "barfoothefoobarman",  
words = ["foo","bar","foo"]
```

从 0 开始，每次移动 一个单词长度



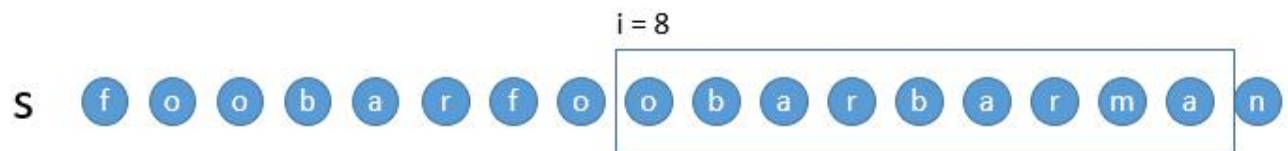
```
s = "barfoothefoobarman",  
words = ["foo","bar","foo"]
```

从 1 开始，每次移动 一个单词长度



```
s = "barfoothefoobarman",  
words = ["foo","bar","foo"]
```

从 2 开始，每次移动 一个单词长度

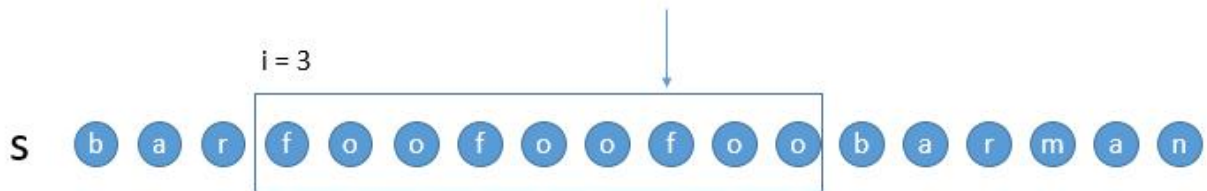


以上三类我们以第一类从 0 开始移动为例，讲一下如何对算法进行优化，有三种需要优化的情况。

- 情况一：当子串完全匹配，移动到下一个子串的时候。


```
s = "barfoothefoobarman",  
words = ["foo","bar","foo"]
```

i = 0



在解法一中，对于 $i = 3$ 的子串，我们肯定是从第一个 foo 开始判断。但其实前两个 foo 都不用判断了，因为在判断上一个 $i = 0$ 的子串的时候我们已经判断过了。所以解法一中的 HashMap2 每次并不需要清空从 0 开始，而是可以只移除之前 $i = 0$ 子串的第一个单词 bar 即可，然后直接从箭头所指的 foo 开始就可以了。

- 情况二：当判断过程中，出现不符合的单词。

```
s = "barfoothefoobarman",  
words = ["foo","bar","foo"]
```

i = 0



i = 3



i = 6

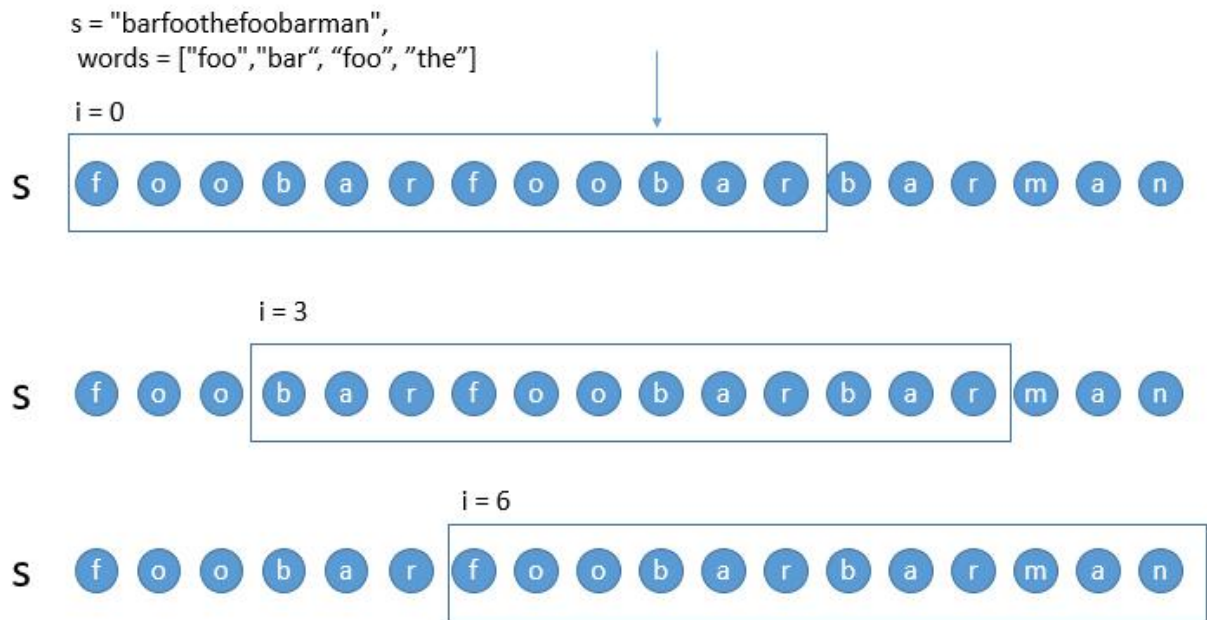


i = 9



但判断 $i = 0$ 的子串的时候，出现了 the，并不在所给的单词中。所以此时 $i = 3$ ， $i = 6$ 的子串，我们其实并不需要判断了。我们直接判断 $i = 9$ 的情况就可以了。

- 情况三：判断过程中，出现的是符合的单词，但是次数超了。



对于 $i = 0$ 的子串，此时判断的 bar 其实是在 words 中的，但是之前已经出现了一次 bar，所以 $i = 0$ 的子串是不符合要求的。此时我们只需要往后移动窗口， $i = 3$ 的子串将 foo 移除，此时子串中一定还是有两个 bar，所以该子串也一定不符合。接着往后移动，当之前的 bar 被移除后，此时 $i = 6$ 的子串，就可以接着按正常的方法判断了。

所以对于出现 $i = 0$ 的子串的情况，我们可以直接从 HashMap2 中依次移除单词，当移除了之前次数超的单词的时候，我们就可以正常判断了，直接从移除了超出了次数的单词后，也就是 $i = 6$ 开始判断就可以了。

看一下代码吧。

```
public List<Integer> findSubstring(String s, String[] words) {
    List<Integer> res = new ArrayList<Integer>();
    int wordNum = words.length;
    if (wordNum == 0) {
        return res;
    }
    int wordLen = words[0].length();
    HashMap<String, Integer> allWords = new HashMap<String, Integer>();
    for (String w : words) {
        int value = allWords.getOrDefault(w, 0);
        allWords.put(w, value + 1);
    }
    //将所有移动分成 wordLen 类情况
    for (int j = 0; j < wordLen; j++) {
        HashMap<String, Integer> hasWords = new HashMap<String, Integer>();
        int num = 0; //记录当前 HashMap2 (这里的 hasWords 变量) 中有多少个单词
        //每次移动一个单词长度
    }
}
```



```

        for (int i = j; i < s.length() - wordNum * wordLen + 1; i = i +
wordLen) {
            boolean hasRemoved = false; //防止情况三移除后, 情况一继续移除
            while (num < wordNum) {
                String word = s.substring(i + num * wordLen, i + (num + 1) *
wordLen);

                if (allWords.containsKey(word)) {
                    int value = hasWords.getOrDefault(word, 0);
                    hasWords.put(word, value + 1);
                    //出现情况三, 遇到了符合的单词, 但是次数超了
                    if (hasWords.get(word) > allWords.get(word)) {
                        // hasWords.put(word, value);
                        hasRemoved = true;
                        int removeNum = 0;
                        //一直移除单词, 直到次数符合了
                        while (hasWords.get(word) > allWords.get(word)) {
                            String firstWord = s.substring(i + removeNum *
wordLen, i + (removeNum + 1) * wordLen);
                            int v = hasWords.get(firstWord);
                            hasWords.put(firstWord, v - 1);
                            removeNum++;
                        }
                        num = num - removeNum + 1; //加 1 是因为我们把当前单词加入
到了 HashMap 2 中
                        i = i + (removeNum - 1) * wordLen; //这里依旧是考虑到了最
外层的 for 循环, 看情况二的解释
                        break;
                    }
                    //出现情况二, 遇到了不匹配的单词, 直接将 i 移动到该单词的后边 (但其实这里
                    //只是移动到了出现问题单词的地方, 因为最外层有 for 循环, i 还会移动一个
                    单词

                    //然后刚好就移动到了单词后边)
                } else {
                    hasWords.clear();
                    i = i + num * wordLen;
                    num = 0;
                    break;
                }
                num++;
            }
            if (num == wordNum) {
                res.add(i);
            }
            //出现情况一, 子串完全匹配, 我们将上一个子串的第一个单词从 HashMap2 中移除
            if (num > 0 && !hasRemoved) {

```

```

        String firstWord = s.substring(i, i + wordLen);
        int v = hasWords.get(firstWord);
        hasWords.put(firstWord, v - 1);
        num = num - 1;
    }

}

}
return res;
}

```

时间复杂度：算法中外层的两个for 循环的次数肯定是所有的子串，假设是 n 。考虑一下，最极端的情况，每个子串的判断都进了 while 循环，wordNum 等于 m 。对于解法一，因为每次都是从头判断，所以 while 循环循环了 m 次。但这里我们由于没有清空，所以每次只判断新加入的单词就可以了，只需判断一次，所以时间复杂度是 $O(n)$ 。

或者换一种理解方式，判断子串是否符合，本质上也就是判断每个单词是否符合，假设 s 的长度是 n ，那么就会大约有 n 个子串，也就是会有 n 个单词。而对于每个单词，我们只有刚开始判断不符合的时候访问一次，还有就是把它移除的时候访问一次，所以每个单词最多访问 2 次，所以时间复杂度是 $O(n)$ 。

空间复杂度：没有变化，依旧是两个 HashMap, 假设 words 里有 m 个单词，就是 $O(m)$ 。

总

这道题最大的亮点就是应用了 HashMap 了吧，使得我们不再纠结于子串包含单词的顺序。然后对于算法的优化上，还是老思路，去分析哪些判断是不必要的，然后把它除之。

31、题目描述（中等难度）

Implement **next permutation**, which rearranges numbers into the lexicographically next greater permutation of numbers.

If such arrangement is not possible, it must rearrange it as the lowest possible order (ie, sorted in ascending order).

The replacement must be **in-place** and use only constant extra memory.

Here are some examples. Inputs are in the left-hand column and its corresponding outputs are in the right-hand column.

```

1, 2, 3 → 1, 3, 2
3, 2, 1 → 1, 2, 3
1, 1, 5 → 1, 5, 1

```

这道题的难度我觉得理解题意就占了一半。题目的意思是给定一个数，然后将这些数字的位置重新排列，得到一个刚好比原数字大的一种排列。如果没有比原数字大的，就升序输出。

关键就是刚好是什么意思？比如说原数字是 A，然后将原数字的每位重新排列产生了 B C D E，然后把这 5 个数字从小到大排列，比如是 D A B E C，那么，我们要找的就是 B，就是那个刚好比 A 大的数字。

再比如 123，其他排列有 132，213，231，312，321，从小到大排列就是 **123** 132 213 231 312 321，那么我们要找的就是 132。

题目还要求空间复杂度必须是 $O(1)$ 。

#解法一

我们想几个问题。

要想使得数字变大，只要任意一位变大就可以。

要想得到刚好大于原来的数字，要变个位。

这里变大数字，只能利用交换。

如果从个位开始，从右往左进行，找一个比个位大的，交换过来，个位的数字交换到了更高位，由于个位的数字较小，所以交换过去虽然个位变大了，但数字整体变小了。例如 1 3 2，把 2 和 3 交换，变成 1 2 3，个位变大了，但整体数字变小了。

个位不行，我们再看十位，如果从十位左边找一个更大的数字交换过来，和个位的情况是一样的，数字会变小。例如 4 1 2 3，把 2 和 4 交换，2 1 4 3，数字会变小。如果从右边找一个更大的数字交换过来，由于是从低位交换过来的，所以数字满足了会变大。如 4 1 2 3，把 2 和 3 交换，变成 4 1 3 2 数字变大了。

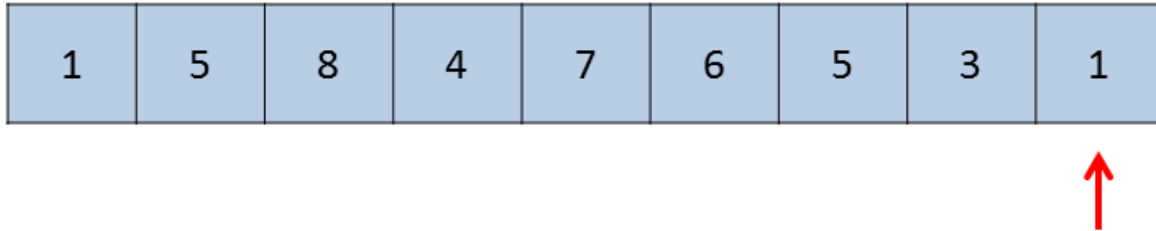
如果十位右边没有比十位数字大的，我们就左移看下一位，再看当前位右边，有没有更大的数字，没有就一直左移就可以。

还有一个问题，如果右边有不只一个大于当前位的数字选哪个？选那个刚好大于当前位的，这样会保证数字整体尽可能的小。

交换完结束了吗？并没有。因为交换完数字变大了，但并不一定是刚好大于原数字的。例如 158476531，我们从十位开始，十位右边没有大于 3 的。再看百位，百位右边没有大于 5 的。直到 4，右边出现了很多大于 4 的，选那个刚好大于 4 的，也就是 5。然后交换，变成 158**5**76**4**31，数字变大了，但并不是刚好大于 158476531，我们还需要将 5 右边的数字从小到大排列。变成158**5**13467，就可以结束了。

而最后的排序，我们其实并不需要用排序函数，因为交换的位置也就是 5 的右边的数字一定是降序的，我们只需要倒序即可了。看一下 LeetCode 提供的动图更好理解一些。

Finding first decreasing element



再看这个过程，我们其实是从右向左找到第一个数字不再递增的位置，然后从右边找到一个刚好大于当前位的数字即可。

再看下代码吧。

```
public void nextPermutation(int[] nums) {
    int i = nums.length - 2;
    //找到第一个不再递增的位置
    while (i >= 0 && nums[i + 1] <= nums[i]) {
        i--;
    }
    //如果到了最左边，就直接倒置输出
    if (i < 0) {
        reverse(nums, 0);
        return;
    }
    //找到刚好大于 nums[i]的位置
    int j = nums.length - 1;
    while (j >= 0 && nums[j] <= nums[i]) {
        j--;
    }
    //交换
    swap(nums, i, j);
    //利用倒置进行排序
    reverse(nums, i + 1);
}

private void swap(int[] nums, int i, int j) {
    int temp = nums[j];
```

```

        nums[j] = nums[i];
        nums[i] = temp;
    }

    private void reverse(int[] nums, int start) {
        int i = start, j = nums.length - 1;
        while (i < j) {
            swap(nums, i, j);
            i++;
            j--;
        }
    }
}

```

时间复杂度：最坏的情况就是遍历完所有位， $O(n)$ ，倒置也是 $O(n)$ ，所以总体依旧是 $O(n)$ 。

空间复杂度： $O(1)$ 。

总

开始看题的时候一直没理解，后来理解了题试了几种也没想出来，然后看了 Solution，理了下思路。

32、题目描述（困难难度）

Given a string containing just the characters '`'`' and '`'`'', find the length of the longest valid (well-formed) parentheses substring.

Example 1:

```

Input: "()"
Output: 2
Explanation: The longest valid parentheses substring is "()"

```

Example 2:

```

Input: ")()())"
Output: 4
Explanation: The longest valid parentheses substring is "()()"

```

给一个一堆括号的字符串，然后返回最长的合法的括号的长度。关于括号的问题，我们在 [20 题](#)和 [22 题](#)也讨论过。

解法一 暴力解法

列举所有的字符串，然后判断每个字符串是不是符合。当然这里可以做个优化就是，因为合法字符串一定是偶数个，所以可以只列举偶数长度的字符串。列举从 0 开始的，长度是 2、4、6的字符串，列举下标从 1 开始的，长度是 2、4、6的字符串，然后循环下去。当然判断字符串是否符合，利用栈来实现，在[之前](#)已经讨论过了。

```

public boolean isValid(String s) {
    Stack<Character> stack = new Stack<Character>();
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == '(') {
            stack.push('(');
        } else if (!stack.empty() && stack.peek() == '(') {
            stack.pop();
        } else {
            return false;
        }
    }
    return stack.empty();
}

public int longestValidParentheses(String s) {
    int maxlen = 0;
    for (int i = 0; i < s.length(); i++) {
        for (int j = i + 2; j <= s.length(); j+=2) {
            if (isValid(s.substring(i, j))) {
                maxlen = Math.max(maxlen, j - i);
            }
        }
    }
    return maxlen;
}

```

时间复杂度: 列举字符串是 $O(n^2)$ ，判断是否是合法序列是 $O(n)$ ，所以总共是 $O(n^3)$ 。

空间复杂度: $O(n)$ ，每次判断的时候，栈的大小。

这个算法，leetcode 会报时间超时。

解法二 暴力破解优化

解法一中，我们会做很多重复的判断，例如类似于这样的，`() () ()`，从下标 0 开始，我们先判断长度为 2 的是否是合法序列。然后再判断长度是 4 的字符串是否符合，但会从下标 0 开始判断。判断长度为 6 的字符串的时候，依旧从 0 开始，但其实之前已经确认前 4 个已经组成了合法序列，所以我们其实从下标 4 开始判断就可以了。

基于此，我们可以换一个思路，我们判断从每个位置开始的最长合法子串是多长即可。而判断是否是合法子串，我们不用栈，而是用一个变量记录当前的括号情况，遇到左括号加 1，遇到右括号减 1，如果变成 0，我们就更新下最长合法子串。

```

public int longestValidParentheses(String s) {
    int count = 0;
    int max = 0;
    for (int i = 0; i < s.length(); i++) {

```

```

count = 0;
for (int j = i; j < s.length(); j++) {
    if (s.charAt(j) == '(') {
        count++;
    } else {
        count--;
    }
    //count < 0 说明右括号多了，此时无论后边是什么，一定是非法字符串了，所以可以提前
    结束循环

    if (count < 0) {
        break;
    }
    //当前是合法序列，更新最长长度
    if (count == 0) {
        if (j - i + 1 > max) {
            max = j - i + 1;
        }
    }
}
return max;
}

```

时间复杂度： $O(n^2)$ 。

空间复杂度： $O(1)$ 。

解法三 动态规划

首先定义动态规划的数组代表什么

$dp[i]$ 代表以下标 i 结尾的合法序列的最长长度，例如下图

index	0	1	2	3	4	5	6	7
	(	)	(	)	(	(	)	)
dp	0	2	0	4	0	0	2	8

下标 1 结尾的最长合法字符串长度是 2，下标 3 结尾的最长字符串是 $str[0, 3]$ ，长度是 4。

我们分析下 dp 的规律。

首先我们初始化所有的 dp 都等于零。

以左括号结尾的字符串一定是非法序列，所以 dp 是零，不用更改。

以右括号结尾的字符串分两种情况。

- 右括号前边是 (，类似于 ()。

$dp[i] = dp[i - 2] + 2$ （前一个合法序列的长度，加上当前新增的长度 2）

类似于上图中 index = 3 的时候的情况。

$dp[3] = dp[3 - 2] + 2 = dp[1] + 2 = 2 + 2 = 4$

- 右括号前边是)，类似于))。

此时我们需要判断 $i - dp[i - 1] - 1$ （前一个合法序列的前边一个位置）是不是左括号。

例如上图的 index = 7 的时候，此时 index - 1 也是右括号，我们需要知道 $i - dp[i - 1] - 1 = 7 - dp[6] - 1 = 4$ 位置的括号的情况。

而刚好 index = 4 的位置是左括号，此时 $dp[i] = dp[i - 1] + dp[i - dp[i - 1] - 2] + 2$ （当前位置的前一个合法序列的长度，加上匹配的左括号前边的合法序列的长度，加上新增的长度 2），也就是 $dp[7] = dp[7 - 1] + dp[7 - dp[7 - 1] - 2] + 2 = dp[6] + dp[7 - 2 - 2] + 2 = 2 + 4 + 2 = 8$ 。

如果 index = 4 不是左括号，那么此时位置 7 的右括号没有匹配的左括号，所以 $dp[7] = 0$ ，不需要更新。

上边的分析可以结合图看一下，可以更好的理解，下边看下代码。

```
public int longestValidParentheses(String s) {
    int maxans = 0;
    int dp[] = new int[s.length()];
    for (int i = 1; i < s.length(); i++) {
        if (s.charAt(i) == ')') {
            //右括号前边是左括号
            if (s.charAt(i - 1) == '(') {
                dp[i] = (i >= 2 ? dp[i - 2] : 0) + 2;
            }
            //右括号前边是右括号，并且除去前边的合法序列的前边是左括号
            else if (i - dp[i - 1] > 0 && s.charAt(i - dp[i - 1] - 1) == '(') {
                dp[i] = dp[i - 1] + ((i - dp[i - 1]) >= 2 ? dp[i - dp[i - 1] - 2] : 0) + 2;
            }
            maxans = Math.max(maxans, dp[i]);
        }
    }
    return maxans;
}
```

时间复杂度：遍历了一次， $O(n)$ 。

空间复杂度： $O(n)$ 。

解法四 使用栈

从左到右扫描字符串，栈顶保存当前扫描的时候，合法序列前的一个位置位置下标是多少，啥意思嘞？

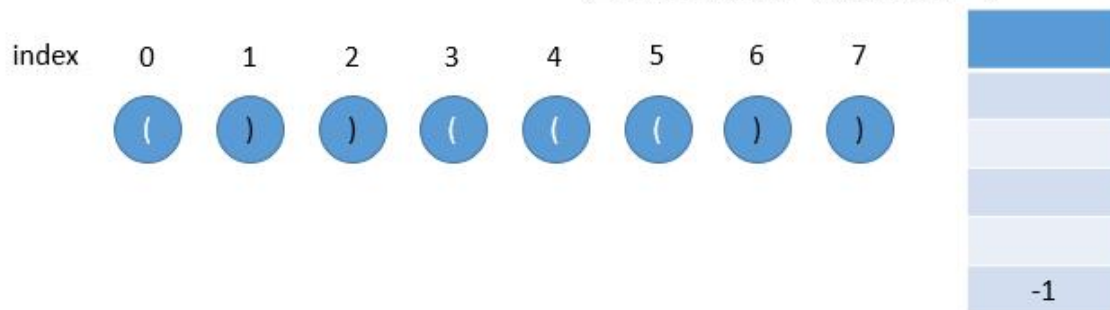
我们扫描到左括号，就将当前位置入栈。

扫描到右括号，就将栈顶出栈（代表栈顶的左括号匹配到了右括号），然后分两种情况。

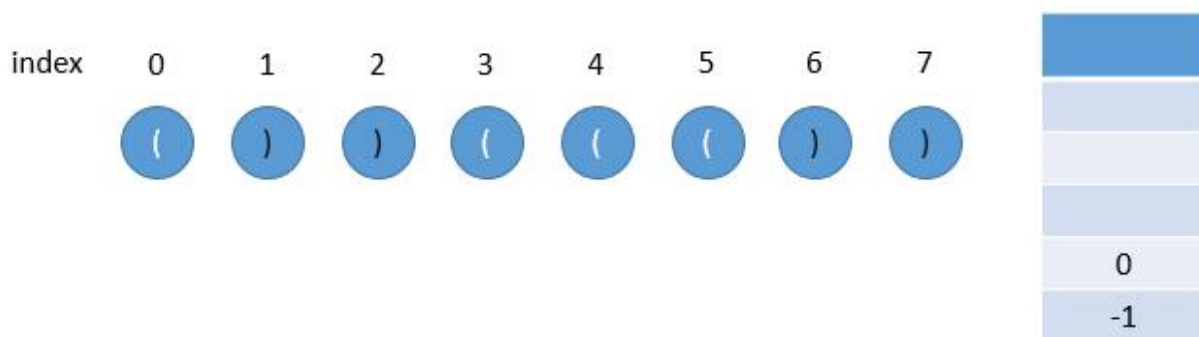
- 栈不空，那么就用当前的位置减去栈顶的存的位置，然后就得到当前合法序列的长度，然后更新一下最长长度。
- 栈是空的，说明之前没有与之匹配的左括号，那么就将当前的位置入栈。

看下图示，更好的理解一下。

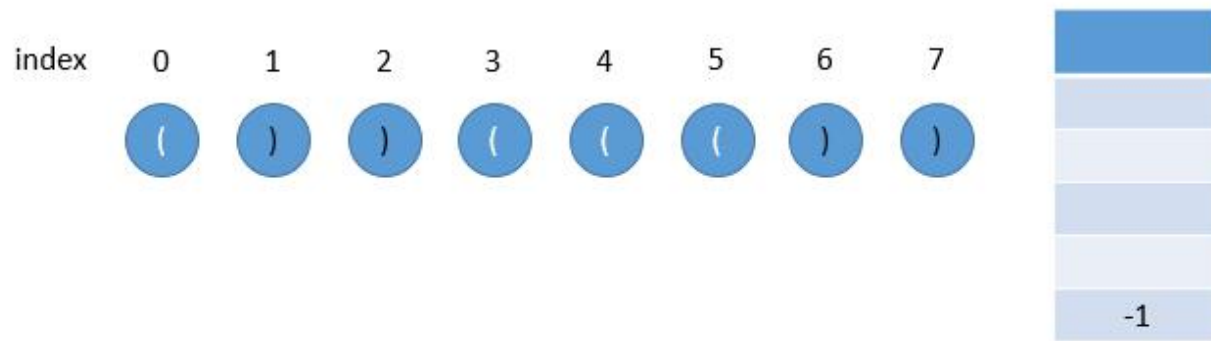
初始化栈，为了方便第一个合法序列长度的计算，
首先将 -1 入栈，初始化 $\text{max} = 0$



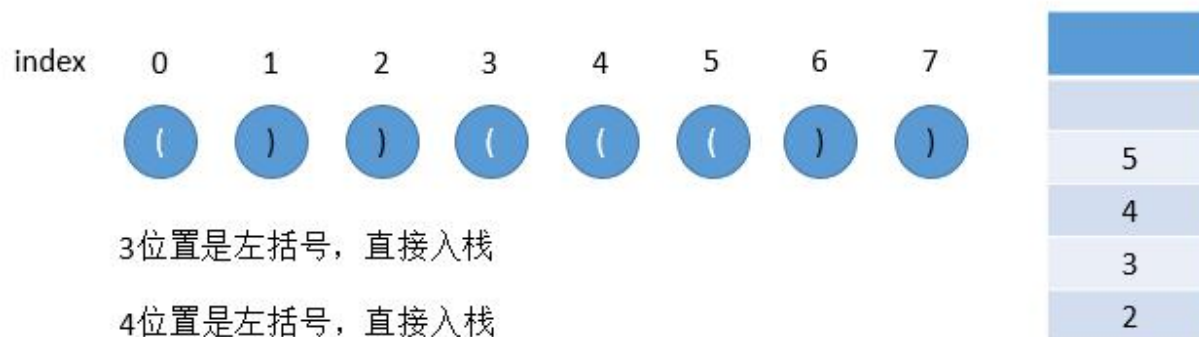
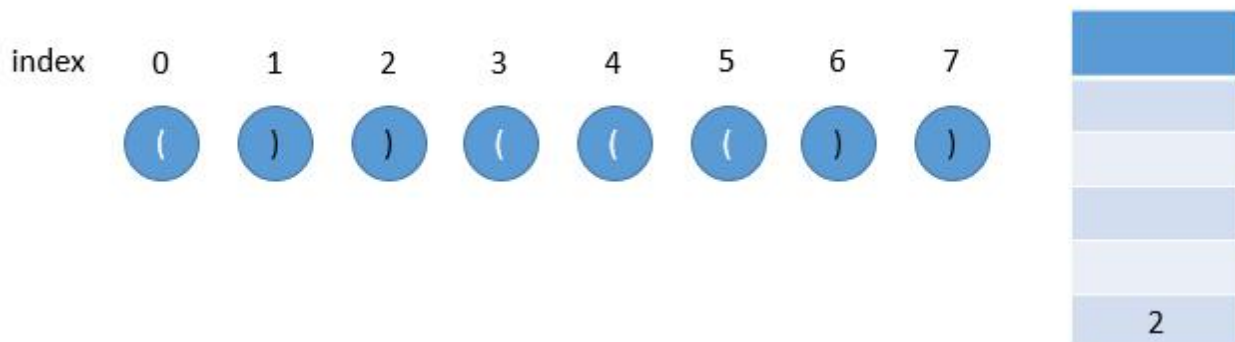
0 位置是左括号，直接入栈



1 位置是 右括号，将栈顶的 0 出栈，并且更新 $\text{max} = \text{Max}(\text{max}, 1 - (-1)) = 2$



2 位置是 右括号，将栈顶的 -1 出栈，栈顶空了，将 2 入栈

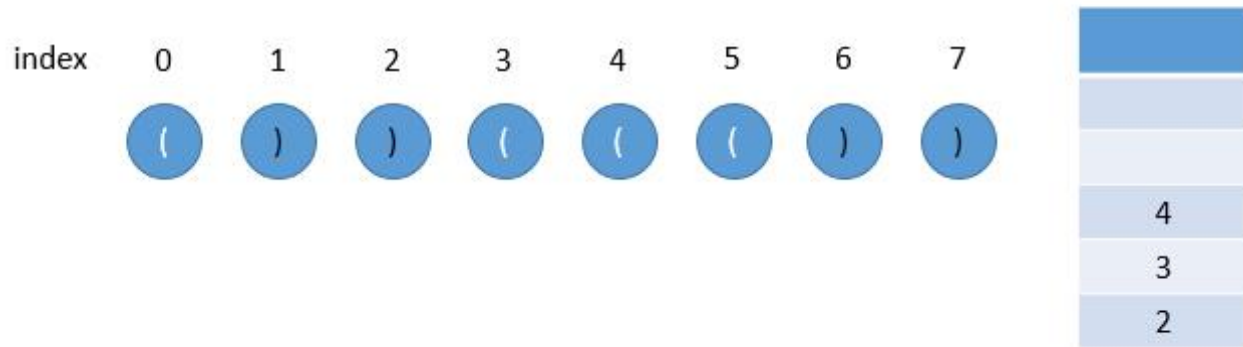


3位置是左括号，直接入栈

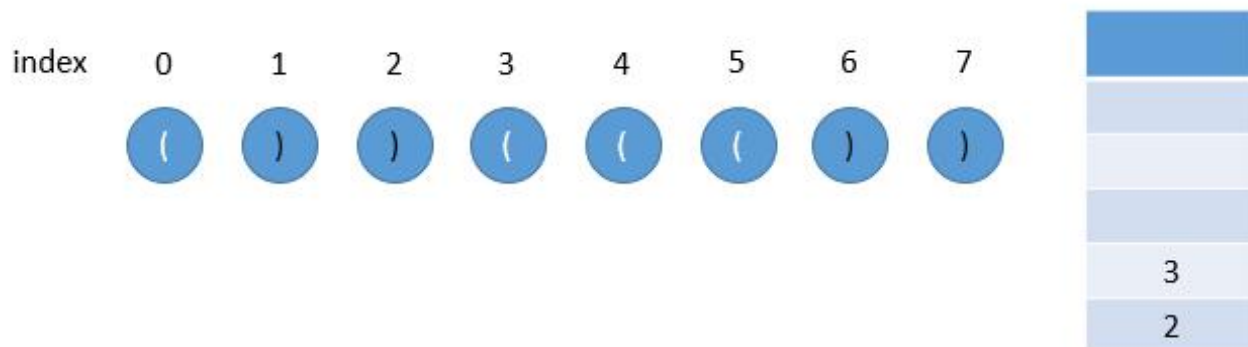
4位置是左括号，直接入栈

5位置是左括号，直接入栈

6 位置是右括号，将栈顶的 5 出栈，并且更新 $\text{max} = \text{Max}(\text{max}, 6 - 4) = 2$



7 位置是右括号，将栈顶的 4 出栈，并且更新 $\text{max} = \text{Max}(\text{max}, 7 - 3) = 4$



再看下代码

```
public int longestValidParentheses(String s) {  
    int maxans = 0;  
    Stack<Integer> stack = new Stack<>();  
    stack.push(-1);  
    for (int i = 0; i < s.length(); i++) {  
        if (s.charAt(i) == '(') {  
            stack.push(i);  
        } else {  
            stack.pop();  
            if (stack.empty()) {  
                stack.push(i);  
            } else {  
                maxans = Math.max(maxans, i - stack.peek());  
            }  
        }  
    }  
    return maxans;  
}
```

时间复杂度：O (n) 。

空间复杂度：O (n) 。

解法五 神奇解法

保持时间复杂度是 O (n) ，将空间复杂度优化到了 O (1) ，它的动机是怎么想到的没有理出来，就介绍下它的想法吧。

从左到右扫描，用两个变量 left 和 right 保存的当前的左括号和右括号的个数，都初始化为 0 。

- 如果左括号个数等于右括号个数了，那么就更新合法序列的最长长度。
- 如果左括号个数大于右括号个数了，那么就接着向右边扫描。
- 如果左括号数目小于右括号个数了，那么后边无论是什么，此时都不可能是合法序列了，此时 left 和 right 归 0，然后接着扫描。

从左到右扫描完毕后，同样的方法从右到左再来一次，因为类似这样的情况 ((()))，如果从左到右扫描到最后，left = 3, right = 2，期间不会出现 left == right。但是如果从右向左扫描，扫描到倒数第二个位置的时候，就会出现 left = 2, right = 2，就会得到一种合法序列。

```
public int longestValidParentheses(String s) {
    int left = 0, right = 0, maxlength = 0;
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == '(') {
            left++;
        } else {
            right++;
        }
        if (left == right) {
            maxlength = Math.max(maxlength, 2 * right);
        } else if (right >= left) {
            left = right = 0;
        }
    }
    left = right = 0;
    for (int i = s.length() - 1; i >= 0; i--) {
        if (s.charAt(i) == '(') {
            left++;
        } else {
            right++;
        }
        if (left == right) {
            maxlength = Math.max(maxlength, 2 * left);
        } else if (left >= right) {
            left = right = 0;
        }
    }
}
```

```
    }  
    return maxlength;  
}
```

时间复杂度： $O(n)$ 。

空间复杂度： $O(1)$ 。

总

这几种算法，暴力破解和动态规划我觉得想的话，还是能分析出来的话，最后两种算法感觉是去挖掘题的本质得到的算法，普适性不是很强。但最后一种算法，从左到右，从右到左，是真的强。

33、题目描述（中等难度）

Suppose an array sorted in ascending order is rotated at some pivot unknown to you beforehand.

(i.e., `[0, 1, 2, 4, 5, 6, 7]` might become `[4, 5, 6, 7, 0, 1, 2]`).

You are given a target value to search. If found in the array return its index, otherwise return `-1`.

You may assume no duplicate exists in the array.

Your algorithm's runtime complexity must be in the order of $O(\log n)$.

Example 1:

```
Input: nums = [4,5,6,7,0,1,2], target = 0  
Output: 4
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2], target = 3  
Output: -1
```

开始的时候想复杂了，其实就是一个排序好的数组，把前边的若干个个数，一起移动到末尾就行了。然后在 $\log(n)$ 下找到给定数字的下标。

总的来说， $\log(n)$ ，我们肯定得用二分的方法了。

解法一

参考[这里](#)首先我们想一下变化前，正常的升序。我们怎么找给定的数字。

位置:	0	1	2	3	4	5	6
值:							

我们每次只关心中间位置的值（这一点很重要），也就是上图 3 位置的数值，如果 target 小于 3 位置的值，我们就把 3 4 5 6 抛弃。然后看新的中间的位置，也就是 1 位置的数值。3 位置，1 位置的值是多少呢？我们有一个数组。

位置:	0	1	2	3	4	5	6
值:							
数组下标:	0	1	2	3	4	5	6
值:							
nums = [0, 1, 2, 4, 5, 6, 7]							

3 位置的值，刚好就是数组下标为 3 的值，1 位置的值刚好就是下标为 1 的值。

那么如果，按题目要求的，变化后，3 位置 和 1 位置的值怎么求呢？此时我们的数组变成下边这样，我们依旧把值从小到大排列。

位置:	0	1	2	3	4	5	6
值:							
数组下标:	4	5	6	0	1	2	3
值:							
nums = [4, 5, 6, 7, 0, 1, 2]							

此时 3 位置的数值对应为数组下标是 0 的值，1 位置的值对应数组下标是 5 的值。任意位置的对应规则是什么呢？ $0 \rightarrow 4, 1 \rightarrow 5, 4 \rightarrow 1$ ，就是就是 $(\text{位置} + \text{偏移}) \% \text{数组的长度}$ 。这里就是加上 4 模 7。

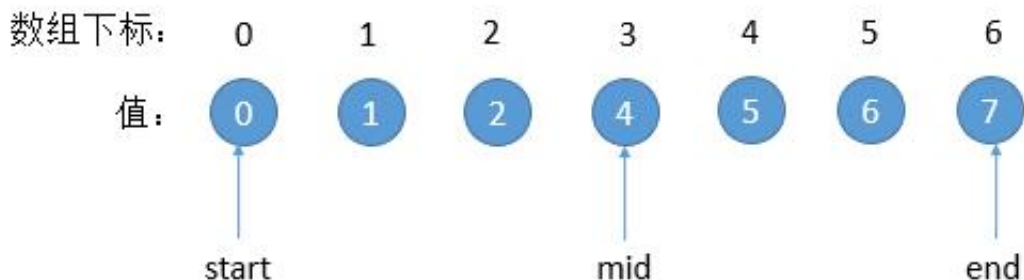
问题转换为怎么去求出这个偏移。

我们只要知道任意一个位置对应的数组下标就可以了，为了方便我们可以求位置为 0 的值对应的下标（数组中最小的数对应的下标），0 位置对应的下标就是我们要求的偏移了（0 + 偏移 = 数组下标）。这里 `nums = [4, 5, 6, 7, 0, 1, 2]`，我们就需要去求数值 0 的下标。

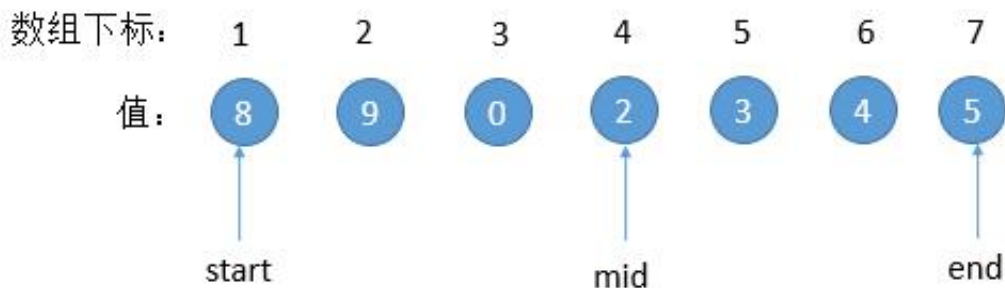
求最小值的下标，因为题目要求时间复杂度是 $O(\log(n))$ ，所以我们必须采取二分的方法去找，二分的方法就要保证每次比较后，去掉一半的元素。这里我们去比较中点和端点值的情况，那么是根据中点和起点比较，还是中点和终点比较呢？我们来分析下。

- mid 和 start 比较

mid > start : 最小值在左半部分。



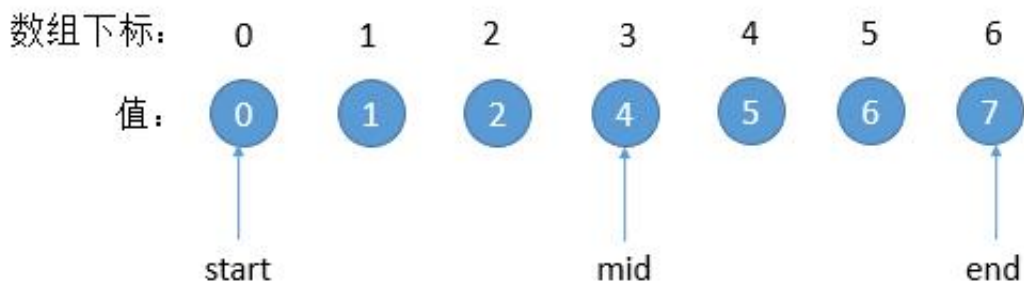
mid < start : 最小值在左半部分。

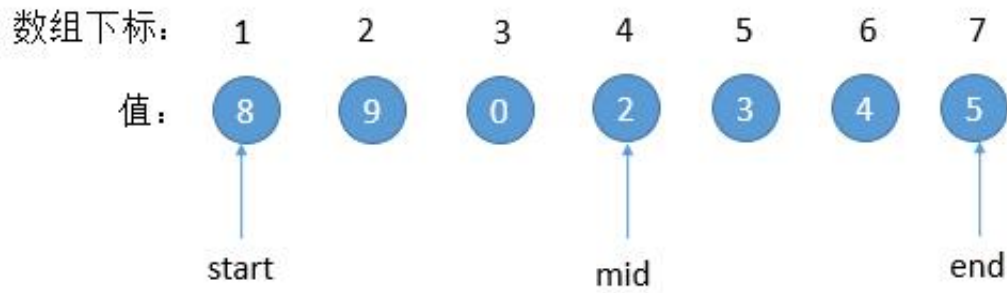


无论大于小于，最小值都在左半部分，所以 mid 和 start 比较是不可取的。

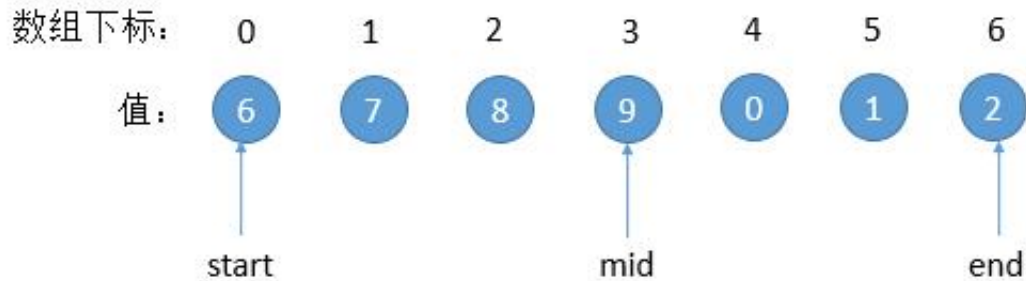
- mid 和 end 比较

mid < end : 最小值在左半部分。



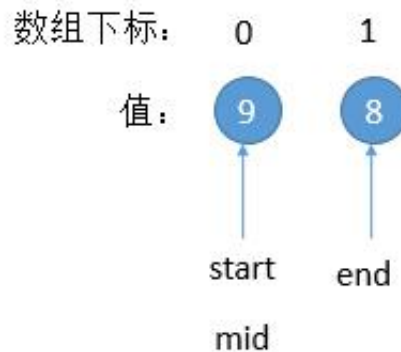


mid > end: 最小值在右半部分。



所以我们只需要把 mid 和 end 比较, mid < end 丢弃右半部分 (更新 end = mid), mid > end 丢弃左半部分 (更新 start = mid)。直到 end 等于 start 时候结束就可以了。

但这样会有一个问题的, 对于下边的例子, 就会遇到死循环了。



问题出在, 当数组剩偶数长度的时候, $mid = (start + end) / 2$, mid 取的是左端点。上图的例子, mid > end, 更新 start = mid, start 位置并不会变化。那么下一次 mid 的值也不会变, 就死循环了。所以, 我们要更新 start = mid + 1。

综上, 找最小值的下标的代码就出来了, 同时, 由于我们找的是位置 0 对应的下标, 所以偏移就是最小值的下标。


```

while (start < end) {
    int mid = (start + end) / 2;
    if (nums[mid] > nums[end]) {
        start = mid + 1 ;
    } else {
        end = mid;
    }
}
int bias = start;

```

当然，我们是找最小值对应的下标，然后求出了偏移。我们也可以找最大值的对应的下标，分析思路和之前是一样的，主要还是要注意一下边界的情况，然后就可以求出偏移。

```

while (start < end) {
    int mid = Math.round(((float)start + end) / 2);
    if (nums[mid] < nums[start]) {
        end = mid - 1;
    } else {
        start = mid;
    }
}
int n = nums.length;
bias = (start + n) - (n - 1); //此时 start 是最大值的数组下标，加上模长 n，减去最大值的位置 n - 1，就得到了偏移。因为 (位置 + 偏移) % n = 数组下标，即 (n - 1 + 偏移) % n = start, n - 1 加偏移超过了 n，所以取模理解成减 n。

```

有了偏移，我们就可以愉快的找目标值的数组下标了。

```

public int search (int[] nums, int target) {
    int start = 0;
    int end = nums.length - 1;
    //找出最小值的数组下标
    /* while (start < end) {
        int mid = (start + end) / 2;
        if (nums[mid] > nums[end]) {
            start = mid + 1 ;
        } else {
            end = mid;
        }
    }
    int bias = start;*/
    //找出最大值的数组下标
    while (start < end) {
        int mid = Math.round(((float)start + end) / 2);

```

```

        if (nums[mid] < nums[start]) {
            end = mid - 1;
        } else {
            start = mid;
        }

    }

    int n = nums.length;
    int bias = (start + n) - (n - 1); //得到偏移
    start = 0;
    end = nums.length - 1;
    while (start <= end) {
        int mid = (start + end) / 2; //中间的位置
        int mid_change = (mid + bias) % nums.length; //中间的位置对应的数组下标
        int value = nums[mid_change]; //中间位置的值
        if (target == value) {
            return mid_change;
        }
        if (target < value) {
            end = mid - 1;
        } else {
            start = mid + 1;
        }
    }
    return -1;
}

```

时间复杂度：O (log (n)) 。

空间复杂度：O (1) 。

解法二

参考[这里](#)，题目中的数组，其实是两段有序的数组。例如

[4 5 6 7 1 2 3]，[4 5 6 7]和[1 2 3]两段有序。

而对于[1 2 3 4]这种，可以看做[1 2 3 4]和[]特殊的两段有序。

而对于我们要找的 target，target 不在的那一段，所有数字可以看做无穷大，这样整个数组就可以看做有序的了，可以用正常的二分法去找 target 了，例如

[4 5 6 7 1 2 3]，如果 target = 5，那么数组可以看做[4 5 6 7 inf inf inf]。

[4 5 6 7 1 2 3]，如果 target = 2，那么数组可以看做[-inf -inf - inf -inf 1 2 3]。

和解法一一样，我们每次只关心 mid 的值，所以 mid 要么就是 nums [mid]，要么就是 inf 或者 -inf。

什么时候是 nums [mid]呢？

当 `nums[mid]` 和 `target` 在同一段里边。

- 怎么判断 `nums[mid]` 和 `target` 在同一段?

把 `nums[mid]` 和 `target` 同时与 `nums[0]` 比较, 如果它俩都大于 `nums[0]` 或者都小于 `nums[0]`, 那么就代表它俩在同一段。例如

[4 5 6 7 1 2 3], 如果 `target = 5`, 此时数组看做 [4 5 6 7 inf inf inf]。 `nums[mid] = 7`, `target > nums[0]`, `nums[mid] > nums[0]`, 所以它们在同一段 `nums[mid] = 7`, 不用变化。

- 怎么判断 `nums[mid]` 和 `target` 不在同一段?

把 `nums[mid]` 和 `target` 同时与 `nums[0]` 比较, 如果它俩一个大于 `nums[0]` 一个小于 `nums[0]`, 那么就代表它俩不在同一段。例如

[4 5 6 7 1 2 3], 如果 `target = 2`, 此时数组看做 [- inf - inf - inf - inf 1 2 3]。 `nums[mid] = 7`, `target < nums[0]`, `nums[mid] > nums[0]`, 一个大于, 一个小于, 所以它们不在同一段 `nums[mid] = - inf`, 变成了负无穷大。

看下代码吧

```
public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int num = nums[mid];
        //nums [ mid ] 和 target 在同一段
        if ((nums[mid] < nums[0]) == (target < nums[0])) {
            num = nums[mid];
        }
        //nums [ mid ] 和 target 不在同一段, 同时还要考虑下变成 -inf 还是 inf。
        } else {
            num = target < nums[0] ? Integer.MIN_VALUE : Integer.MAX_VALUE;
        }

        if (num < target)
            lo = mid + 1;
        else if (num > target)
            hi = mid - 1;
        else
            return mid;
    }
    return -1;
}
```

时间复杂度: $O(\log(n))$ 。

空间复杂度: $O(1)$ 。

解法三

参考[这里](#)，算法基于一个事实，数组从任意位置劈开后，至少有一半是有序的，什么意思呢？

比如 [4 5 6 7 1 2 3]，从 7 劈开，左边是 [4 5 6 7] 右边是 [1 2 3]，左边是有序的。

基于这个事实。

我们可以先找到哪一段是有序的 (只要判断端点即可)，然后看 target 在不在这一段里，如果在，那么就把另一半丢弃。如果不在，那么就把这一段丢弃。

```
public int search(int[] nums, int target) {
    int start = 0;
    int end = nums.length - 1;
    while (start <= end) {
        int mid = (start + end) / 2;
        if (target == nums[mid]) {
            return mid;
        }

        //左半段是有序的
        if (nums[start] <= nums[mid]) {
            //target 在这段里
            if (target >= nums[start] && target < nums[mid]) {
                end = mid - 1;
            } else {
                start = mid + 1;
            }
        }

        //右半段是有序的
    } else {
        if (target > nums[mid] && target <= nums[end]) {
            start = mid + 1;
        } else {
            end = mid - 1;
        }
    }
}

return -1;
}
```

时间复杂度： $O(\log(n))$ 。

空间复杂度： $O(1)$ 。

总

三种解法是从不同的思路去理解题意，但本质上都是找到丢弃一半的规则，从而达到 $\log(n)$ 的时间复杂度，对二分查找的本质的理解更加深刻了。

34、题目描述（中等难度）

Given an array of integers `nums` sorted in ascending order, find the starting and ending position of a given `target` value.

Your algorithm's runtime complexity must be in the order of $O(\log n)$.

If the target is not found in the array, return `[-1, -1]`.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`
Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`
Output: `[-1,-1]`

找到目标值的第一次出现和最后一次出现的位置，同样要求 $\log(n)$ 下完成。

先分享 [leetcode](https://leetcode.com/problems/search-in-sorted-array/) 提供的两个解法。

解法一 线性扫描

从左向右遍历，一旦出现等于 `target` 的值就结束，保存当前下标。如果从左到右没有找到 `target`，那么就直接返回 `[-1, -1]` 就可以了，因为从左到右没找到，那么从右到左也一定不会找到的。如果找到了，然后再从右到左遍历，一旦出现等于 `target` 的值就结束，保存当前下标。

时间复杂度是 $O(n)$ 并不满足题意，但可以了解下这个思路，从左到右，从右到左之前也遇到过。

```
public int[] searchRange(int[] nums, int target) {
    int[] targetRange = {-1, -1};

    // 从左到右扫描
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] == target) {
            targetRange[0] = i;
            break;
        }
    }

    // 如果之前没找到，直接返回 [-1, -1]
    if (targetRange[0] == -1) {
        return targetRange;
    }

    // 从右到左扫描
    for (int j = nums.length - 1; j >= 0; j--) {
        if (nums[j] == target) {
            targetRange[1] = j;
        }
    }
}
```

```
        break;
    }
}

return targetRange;
}
```

时间复杂度： $O(n)$ 。

空间复杂度： $O(1)$ 。

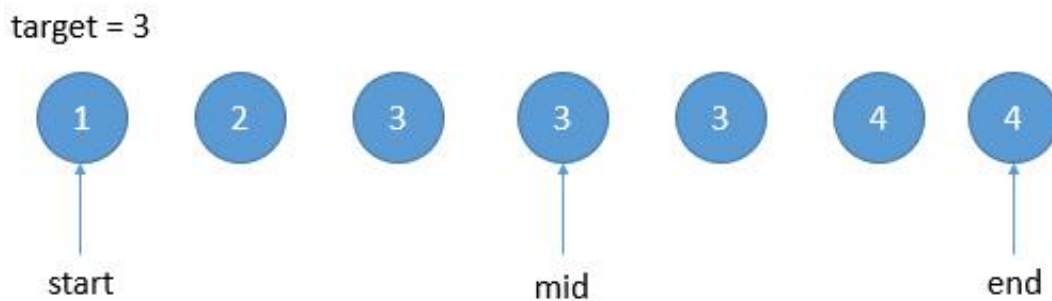
解法二 二分查找

让我们先看下正常的二分查找。

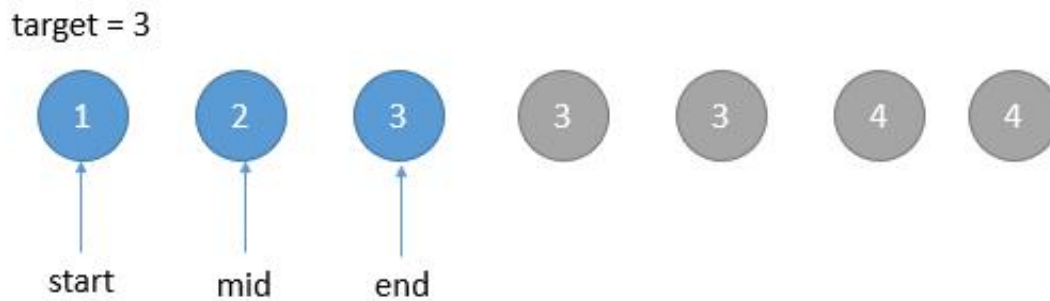
```
int start = 0;
int end = nums.length - 1;
while (start <= end) {
    int mid = (start + end) / 2;
    if (target == nums[mid]) {
        return mid;
    } else if (target < nums[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
```

二分查找中，我们找到 target 就结束了，这里我们需要修改下。

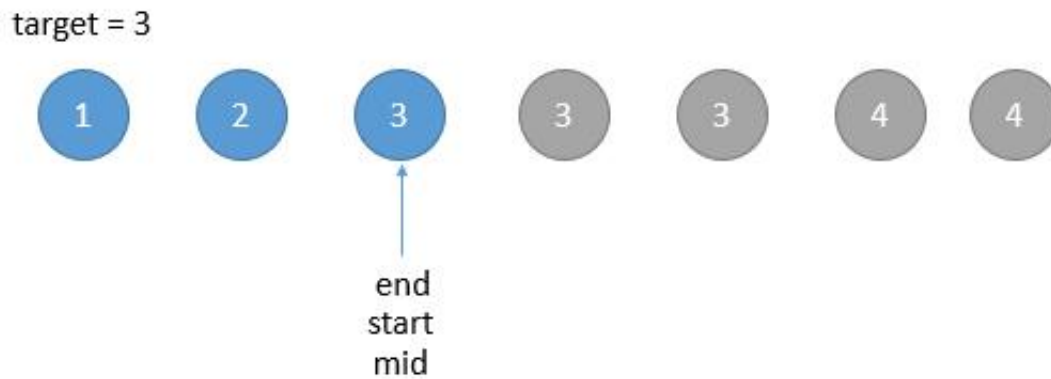
我们如果找最左边等于 target 的值，找到 target 时候并不代表我们找到了我们所需要的，例如下边的情况，



此时虽然 `mid` 指向的值等于 `target` 了，但是我们要找的其实还在左边，为了达到 $\log$ 的时间复杂度，我们依旧是丢弃一半，我们需要更新 `end = mid - 1`，图示如下。



此时 `target > nums[mid]`，更新 `start = mid + 1`。



此时 `target == nums[mid]`，但由于我们改成了 `end = mid - 1`，所以继续更新，`end` 就到了 `mid` 的左边，此时 `start > end` 了，就会走出 `while` 循环，我们要找的值刚好就是 `start` 指向的了。那么我们修改的代码如下：

```
while (start <= end) {  
    int mid = (start + end) / 2;  
    if (target == nums[mid]) {  
        end = mid - 1;  
    } else if (target < nums[mid]) {  
        end = mid - 1;  
    } else {  
        start = mid + 1;  
    }  
}
```

找右边的同样的分析思路，就是判断需要丢弃哪一边。

所以最后的代码就出来了。leetcode 中是把找左边和找右边的合并起来了，本质是一样的。

```

public int[] searchRange(int[] nums, int target) {
    int start = 0;
    int end = nums.length - 1;
    int[] ans = { -1, -1 };
    if (nums.length == 0) {
        return ans;
    }
    while (start <= end) {
        int mid = (start + end) / 2;
        if (target == nums[mid]) {
            end = mid - 1;
        } else if (target < nums[mid]) {
            end = mid - 1;
        } else {
            start = mid + 1;
        }
    }
    //考虑 target 是否存在, 判断我们要找的值是否等于 target 并且是否越界
    if (start == nums.length || nums[start] != target) {
        return ans;
    } else {
        ans[0] = start;
    }
    ans[0] = start;
    start = 0;
    end = nums.length - 1;
    while (start <= end) {
        int mid = (start + end) / 2;
        if (target == nums[mid]) {
            start = mid + 1;
        } else if (target < nums[mid]) {
            end = mid - 1;
        } else {
            start = mid + 1;
        }
    }
    ans[1] = end;
    return ans;
}

```

时间复杂度： $O(\log(n))$ 。

空间复杂度： $O(1)$ 。

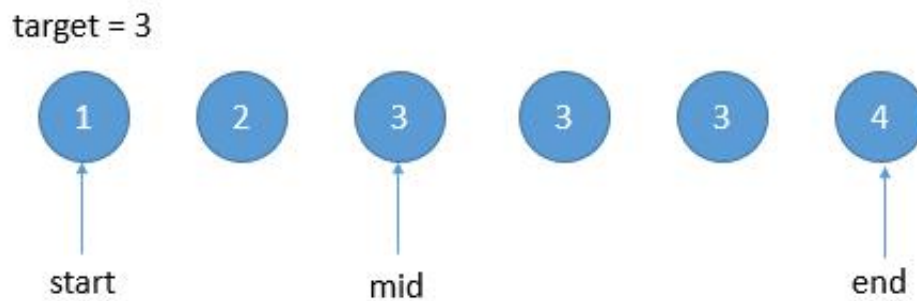
解法三

以上是 leetcode 提供的思路，我觉得不是很好，因为它所有的情况都一定是循环 $\log(n)$ 次，讲一下我最开始想到的。

相当于在解法二的基础上优化了一下，下边是解法二的代码。

```
while (start <= end) {
    int mid = (start + end) / 2;
    if (target == nums[mid]) {
        end = mid - 1;
    } else if (target < nums[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
```

考虑下边的一种情况，如果我们找最左边等于 target 的，此时 mid 的位置已经是我们要找的了，而解法二更新成了 $\text{end} = \text{mid} - 1$ ，然后继续循环了，而此时我们其实完全可以终止了。只需要判断 $\text{nums}[\text{mid} - 1]$ 是不是小于 $\text{nums}[\text{mid}]$ ，如果小于就刚好是我们要找的了。



当然，找最右边也是同样的思路，看下代码吧。

```
public int[] searchRange(int[] nums, int target) {
    int start = 0;
    int end = nums.length - 1;
    int[] ans = { -1, -1 };
    if (nums.length == 0) {
        return ans;
    }
    while (start <= end) {
        int mid = (start + end) / 2;
        if (target == nums[mid]) {
            //这里是为了处理 mid - 1 越界的问题，可以仔细想下。
            //如果 mid == 0, 那么 mid 一定是我们要找的了，而此时 mid - 1 就会越界了，
            //为了使得下边的 target > n 一定成立，我们把 n 赋成最小值
        }
    }
}
```

```

        //如果 mid > 0, 直接吧 nums[mid - 1] 赋给 n 就可以了。
        int n = mid > 0 ? nums[mid - 1] : Integer.MIN_VALUE;
        if (target > n) {
            ans[0] = mid;
            break;
        }
        end = mid - 1;
    } else if (target < nums[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
start = 0;
end = nums.length - 1;
while (start <= end) {
    int mid = (start + end) / 2;
    if (target == nums[mid]) {
        int n = mid < nums.length - 1 ? nums[mid + 1] : Integer.MAX_VALUE;
        if (target < n) {
            ans[1] = mid;
            break;
        }
        start = mid + 1;
    } else if (target < nums[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
return ans;
}

```

时间复杂度：O (log (n)) 。

空间复杂度：O (1) 。

@JZW 的提醒下，上边的虽然能 AC，但是如果要找的数字刚好就是 Integer.MIN_VALUE 的话，就会出现错误。可以修改一下。

主要是这两句，除了小于 n，还判断了当前是不是在两端。

```

if (target > n || mid == 0) {
if (target < n || mid == nums.length - 1) {

```

```

public int[] searchRange(int[] nums, int target) {

```

```

int start = 0;
int end = nums.length - 1;
int[] ans = { -1, -1 };
if (nums.length == 0) {
    return ans;
}
while (start <= end) {
    int mid = (start + end) / 2;
    if (target == nums[mid]) {
        int n = mid > 0 ? nums[mid - 1] : Integer.MIN_VALUE;
        if (target > n || mid == 0) {
            ans[0] = mid;
            break;
        }
        end = mid - 1;
    } else if (target < nums[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
start = 0;
end = nums.length - 1;
while (start <= end) {
    int mid = (start + end) / 2;
    if (target == nums[mid]) {
        int n = mid < nums.length - 1 ? nums[mid + 1] : Integer.MAX_VALUE;
        if (target < n || mid == nums.length - 1) {
            ans[1] = mid;
            break;
        }
        start = mid + 1;
    } else if (target < nums[mid]) {
        end = mid - 1;
    } else {
        start = mid + 1;
    }
}
return ans;
}

```

总体来说，这道题并不难，本质就是对二分查找的修改，以便满足我们的需求。

35、题目描述（简单难度）

Given a sorted array and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You may assume no duplicates in the array.

Example 1:

Input: [1,3,5,6], 5
Output: 2

Example 2:

Input: [1,3,5,6], 2
Output: 1

Example 3:

Input: [1,3,5,6], 7
Output: 4

Example 4:

Input: [1,3,5,6], 0
Output: 0

给定一个有序数组，依旧是二分查找，不同之处是如果没有找到指定数字，需要返回这个数字应该插入的位置。

这道题比较简单，在二分查找的基础上，只要想清楚返回啥就够了。想的话，就考虑最简单的情况如果数组只剩下 2 5，target 是 1, 3, 6 的时候，此时我们应该返回什么就行。

```
public int searchInsert(int[] nums, int target) {  
    int start = 0;  
    int end = nums.length - 1;  
    if (nums.length == 0) {  
        return 0;  
    }  
    while (start < end) {  
        int mid = (start + end) / 2;  
        if (target == nums[mid]) {  
            return mid;  
        } else if (target < nums[mid]) {
```

```

        end = mid;
    } else {
        start = mid + 1;
    }
}
//目标值在不在当前停的位置的前边还是后边
if(target>nums[start]){
    return start + 1;
}
//如果小于的话，就返回当前位置，跑步超过第二名还是第二名，所以不用减 1。
else{
    return start;
}
}

```

时间复杂度： $O(\log(n))$ 。

空间复杂度： $O(1)$ 。

这道题不难，但是对于二分查找又有了一些新认识。

首先，一定要注意，数组剩下偶数个元素的时候，中点取的是左端点。例如 1 2 3 4，中点取的是 2。正因为如此，我们更新 start 的时候不是直接取 mid，而是 mid + 1。因为剩下两个元素的时候，mid 和 start 是相同的，如果不进行加 1 会陷入死循环。

然后上边的算法，返回最终值的时候，我们进行了一个 if 的判断，那么能不能避免呢。

- 第一种思路，参考[这里](#)。

首先为了让 start 在循环的时候多加 1，我们将循环的 $start < end$ 改为 $start \leq end$ 。

这样就会出现一个问题，当 $start == end$ ，此时 mid 不仅等于了 start 还会等于 end，所以之前更新 end 是直接赋 mid，现在需要改成 $end = mid - 1$ ，防止死循环。这样就达到了目标。

```

public int searchInsert(int[] nums, int target) {
    int start = 0;
    int end = nums.length - 1;
    if (nums.length == 0) {
        return 0;
    }
    while (start <= end) {
        int mid = (start + end) / 2;
        if (target == nums[mid]) {
            return mid;
        } else if (target < nums[mid]) {
            end = mid - 1;
        } else {
            start = mid + 1;
        }
    }
}

```

```
    }

    return start;
}
```

- 第二种思路，参考[这里](#)。

我们开始更新 start 的时候，是 mid + 1，如果剩两个元素，例如 2 4，target = 6 的话，此时 mid = 0，start = mid + 1 = 1，我们返回 start + 1 = 2。如果 mid 是右端点，那么 mid = 1，start = mid + 1 = 2，这样就可以直接返回 start 了，不需要在返回的时候加 1 了。

怎么做到呢？最最开始的时候我们取 end 的时候是 end = nums.length - 1。如果我们改成 end = nums.length，这样每次取元素的时候，如果和之前对比，取到的就是右端点了。这样的话，最后返回的时候就不需要多加 1 了。

```
public int searchInsert(int[] nums, int target) {
    int start = 0;
    int end = nums.length;
    if (nums.length == 0) {
        return 0;
    }
    while (start < end) {
        int mid = (start + end) / 2;
        if (target == nums[mid]) {
            return mid;
        } else if (target < nums[mid]) {
            end = mid;
        } else {
            start = mid + 1;
        }
    }

    return start;
}
```

总

虽然题很简单，但对二分查找有了更多的理解。

36、题目描述（中等难度）

Determine if a 9x9 Sudoku board is valid. Only the filled cells need to be validated **according to the following rules**:

1. Each row must contain the digits 1-9 without repetition.
2. Each column must contain the digits 1-9 without repetition.
3. Each of the 9 3x3 sub-boxes of the grid must contain the digits 1-9 without repetition.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

A partially filled sudoku which is valid.

The Sudoku board could be partially filled, where empty cells are filled with the character '.'.

Example 1:

Input:

```
[
  ["5","3",".",".","7",".",".",".","."],
  ["6",".",".","1","9","5",".",".","."],
  [".","9","8",".",".",".","6","."],
  ["8",".",".","6",".",".","3","."],
  ["4",".",".","8",".","3",".","1"],
  ["7",".",".","2",".",".","6"],
  [".","6",".",".","2","8","."],
  [".",".","4","1","9",".","5"],
  [".",".","8",".","7","9"]
]
```

Output: true

Example 2:

Input:

```
[
  ["8","3",".",".","7",".",".","."],
  ["6",".",".","1","9","5",".",".","."],
  [".","9","8",".",".",".","6","."],
  ["8",".",".","6",".",".","3","."],
  ["4",".",".","8",".","3",".","1"],
  ["7",".",".","2",".",".","6"],
  [".","6",".",".","2","8","."],
  [".",".","4","1","9",".","5"],
  [".",".","8",".","7","9"]
]
```

Output: false

Explanation: Same as Example 1, except with the 5 in the top left corner being modified to 8. Since there are two 8's in the top left 3x3 sub-box, it is invalid.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.
- The given board contain only digits 1-9 and the character '.'.
- The given board size is always 9x9.

一个 9 * 9 的数独的棋盘。判断已经写入数字的棋盘是不是合法。需要满足下边三点，

- 每一行的数字不能重复
- 每一列的数字不能重复
- 9 个 3 * 3 的小棋盘中的数字也不能重复。

只能是 1 - 9 中的数字，不需要考虑数独最后能不能填满。

解法一 暴力解法

需要满足三条，那就一条一条判断。

```
public boolean isValidSudoku(char[][] board) {
    //判断每一行
    for (int i = 0; i < 9; i++) {
        if (!isValidRows(board[i])) {
            return false;
        }
    }
    //判断每一列
    for (int i = 0; i < 9; i++) {
        if (!isValidCols(i, board)) {
            return false;
        }
    }
    //判断每个小棋盘
    for (int i = 0; i < 9; i = i + 3) {
        for (int j = 0; j < 9; j = j + 3) {
            if (!isValidSmall(i, j, board)) {
                return false;
            }
        }
    }
    return true;
}

public boolean isValidRows(char[] board) {
    HashMap<Character, Integer> hashMap = new HashMap<>();
    for (char c : board) {
        if (c != '.') {
            if (hashMap.getOrDefault(c, 0) != 0) {
                return false;
            } else {
                hashMap.put(c, 1);
            }
        }
    }
}
```



```

    }
}
return true;
}

public boolean isValidCols(int col, char[][] board) {
    HashMap<Character, Integer> hashMap = new HashMap<>();
    for (int i = 0; i < 9; i++) {
        char c = board[i][col];
        if (c != '.') {
            if (hashMap.getOrDefault(c, 0) != 0) {
                return false;
            } else {
                hashMap.put(c, 1);
            }
        }
    }
    return true;
}

public boolean isValidSmall(int row, int col, char[][] board) {
    HashMap<Character, Integer> hashMap = new HashMap<>();
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            char c = board[row + i][col + j];
            if (c != '.') {
                if (hashMap.getOrDefault(c, 0) != 0) {
                    return false;
                } else {
                    hashMap.put(c, 1);
                }
            }
        }
    }
    return true;
}
}

```

时间复杂度：整个棋盘访问了三次，如果棋盘大小是 n ，那么就是 $3n$ 。也就是 $O(n)$ 。

空间复杂度： $O(1)$ 。

解法二

参考[这里](#)，上边的算法遍历了三遍，我们能不能只遍历一遍。

我们可以这样想一下，如果有一副纸牌，怎么看它有没有重复的？

第一种我们可以像之前一样，第一遍先看红桃，再看黑桃，再看方片，再看梅花，这样就看了四遍。我们其实可以每拿到一张牌，就把它放在一个位置，我们把一类放在同一位置。红桃放在一起，黑桃放在一起.....放的过程中如果有重复的就可以结束了。

在这里的话，我们就可以把第一行的放在一起，第二行的放在一起.....第一列的放在一起，第二列的放在一起.....第一个小棋盘的放在一起，第二个小棋盘的放在一起.....

我们用 HashSet 实现放在一起的作用，但是这样的话总共就是 9 行，9 列，9 个小棋盘，27 个 HashSet 了。我们其实可以在放的时候标志一下，例如

- 如果第 4 行有一个数字 8，我们就 (8)4，把 "(8)4" 放进去。
- 如果第 5 行有一个数字 6，我们就 5(6)，把 "5(6)" 放进去。
- 小棋盘看成一个整体，总共是 9 个，3 行 3 列，如果第 2 行第 1 列的小棋盘里有个数字 3，我们就把 "2(3)1" 放进去。

这样 1 个 HashSet 就够了。

```
public boolean isValidSudoku(char[][] board) {
    Set seen = new HashSet();
    for (int i=0; i<9; ++i) {
        for (int j=0; j<9; ++j) {
            if (board[i][j] != '.') {
                String b = "(" + board[i][j] + ")";
                if (!seen.add(b + i) || !seen.add(j + b) || !seen.add(i/3 + b + j/3))
                    return false;
            }
        }
    }
    return true;
}
```

时间复杂度：如果棋盘大小总共是 n ，那么只遍历了一次，就是 $O(n)$ 。

空间复杂度：如果棋盘大小总共是 n ，最坏的情况就是每个地方都有数字，就需要存三次， $O(n)$ 。

其实，想到了标识，其实我们可以标识的更彻底些，直接写出来。

```
public boolean isValidSudoku(char[][] board) {
    Set seen = new HashSet();
    for (int i=0; i<9; ++i) {
        for (int j=0; j<9; ++j) {
            char number = board[i][j];
            if (number != '.')
                if (!seen.add(number + " in row " + i) ||
                    !seen.add(number + " in column " + j) ||
                    !seen.add(number + " in block " + i/3 + "-" + j/3))
                    return false;
        }
    }
    return true;
}
```

```
        return false;
    }
}
return true;
}
```

总

第二种解法的作者太太聪明了！自己规定格式这种思想，很棒。

37、题目描述（困难难度）

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy **all of the following rules**:

- 1. Each of the digits 1-9 must occur exactly once in each row.
- 2. Each of the digits 1-9 must occur exactly once in each column.
- 3. Each of the the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

Empty cells are indicated by the character '.'.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

A sudoku puzzle...

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

...and its solution numbers marked in red.

Note:

- The given board contain only digits 1-9 and the character '.'.
- You may assume that the given Sudoku puzzle will have a single unique solution.
- The given board size is always 9x9.

给定一个数独棋盘，输出它的一个解。

解法一 回溯法

从上到下，从左到右遍历每个空位置。在第一个位置，随便填一个可以填的数字，再在第二个位置填一个可以填的数字，一直执行下去直到最后一个位置。期间如果出现没有数字可以填的话，就回退到上一个位置，换一下数字，再向后进行下去。

```
public void solveSudoku(char[][] board) {
    solver(board);
}
```

```

}

private boolean solver(char[][] board) {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (board[i][j] == '.') {
                char count = '1';
                while (count <= '9') {
                    if (isValid(i, j, board, count)) {
                        board[i][j] = count;
                        if (solver(board)) {
                            return true;
                        } else {
                            //下一个位置没有数字，就还原，然后当前位置尝试新的数
                            board[i][j] = '.';
                        }
                    }
                    count++;
                }
                return false;
            }
        }
    }
    return true;
}

private boolean isValid(int row, int col, char[][] board, char c) {
    for (int i = 0; i < 9; i++) {
        if (board[row][i] == c) {
            return false;
        }
    }

    for (int i = 0; i < 9; i++) {
        if (board[i][col] == c) {
            return false;
        }
    }

    int start_row = row / 3 * 3;
    int start_col = col / 3 * 3;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[start_row + i][start_col + j] == c) {
                return false;
            }
        }
    }
}

```

```
    }  
    return true;  
}
```

时间复杂度：

空间复杂度：O（1）。

总

回溯法一个很典型的应用了。

38、题目描述（简单难度）

The count-and-say sequence is the sequence of integers with the first five terms as following:

```
1.    1  
2.    11  
3.    21  
4.    1211  
5.    111221
```

1 is read off as "one 1" or 11.

11 is read off as "two 1s" or 21.

21 is read off as "one 2, then one 1" or 1211.

Given an integer n where $1 \leq n \leq 30$, generate the n^{th} term of the count-and-say sequence.

Note: Each term of the sequence of integers will be represented as a string.

Example 1:

```
Input: 1  
Output: "1"
```

Example 2:

```
Input: 4  
Output: "1211"
```

难在了题目是什么意思呢？

初始值第一行是 1。

第二行读第一行，1 个 1，去掉个字，所以第二行就是 11。

第三行读第二行，2 个 1，去掉个字，所以第三行就是 21。

第四行读第三行，1 个 2，1 个 1，去掉所有个字，所以第四行就是 1211。

第五行读第四行，1 个 1，1 个 2，2 个 1，去掉所有个字，所以第五航就是 111221。

第六行读第五行，3 个 1，2 个 2，1 个 1，去掉所以个字，所以第六行就是 312211。

然后题目要求输入 1 - 30 的任意行数，输出该行是啥。

解法一 递归

可以看出来，我们只要知道了 $n - 1$ 行，就可以写出第 n 行了，首先想到的就是递归。

第五行是 111221，求第六行的话，我们只需要知道每个字符重复的次数加上当前字符就行啦。

1 重复 3 次，就是 31，2 重复 2 次就是 22，1 重复 1 次 就是 11，所以最终结果就是 312211。

```
public String countAndSay(int n) {
    //第一行就直接输出
    if (n == 1) {
        return "1";
    }
    //得到上一行的字符串
    String last = countAndSay(n - 1);
    //输出当前行的字符串
    return getNextString(last);
}

private String getNextString(String last) {
    //长度为 0 就返回空字符串
    if (last.length() == 0) {
        return "";
    }
    //得到第 1 个字符重复的次数
    int num = getRepeatNum(last);
    // 次数 + 当前字符 + 其余的字符串的情况
    return num + " " + last.charAt(0) + getNextString(last.substring(num));
}

//得到字符 string[0] 的重复个数，例如 "111221" 返回 3
private int getRepeatNum(String string) {
    int count = 1;
    char same = string.charAt(0);
    for (int i = 1; i < string.length(); i++) {
        if (same == string.charAt(i)) {
            count++;
        } else {
            break;
        }
    }
    return count;
}
```

```
}
```

时间复杂度：

空间复杂度：O (1) 。

解法二 迭代

既然有递归，那就一定可以写出它的迭代形式。

```
public String countAndSay(int n) {
    String res = "1";
    //从第一行开始，一行一行产生
    while (n > 1) {
        String temp = "";
        for (int i = 0; i < res.length(); i++) {
            int num = getRepeatNum(res.substring(i));
            temp = temp + num + "" + res.charAt(i);
            //跳过重复的字符
            i = i + num - 1;
        }
        n--;
        //更新
        res = temp;
    }
    return res;
}

//得到字符 string[0] 的重复个数，例如 "111221" 返回 3
private int getRepeatNum(String string) {
    int count = 1;
    char same = string.charAt(0);
    for (int i = 1; i < string.length(); i++) {
        if (same == string.charAt(i)) {
            count++;
        } else {
            break;
        }
    }
    return count;
}
```

时间复杂度：

空间复杂度：O (1) 。

总

递归里边又用了个递归，我觉得这点有点意思。

39、题目描述（中等难度）

Given a **set** of candidate numbers (`candidates`) (without duplicates) and a target number (`target`), find all unique combinations in `candidates` where the candidate numbers sums to `target` .

The **same** repeated number may be chosen from `candidates` unlimited number of times.

Note:

- All numbers (including `target`) will be positive integers.
- The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [2,3,6,7], target = 7,  
A solution set is:  
[  
  [7],  
  [2,2,3]  
]
```

Example 2:

```
Input: candidates = [2,3,5], target = 8,  
A solution set is:  
[  
  [2,2,2,2],  
  [2,3,3],  
  [3,5]  
]
```

给几个数字，一个目标值，输出所有和等于目标值的组合。

解法一 回溯法

参考[这里](#)，就是先向前列举所有情况，得到一个解或者走不通的时候就回溯。和[37](#)题有异曲同工之处，也算是回溯法很典型的应用，直接看代码吧。

```
public List<List<Integer>> combinationSum(int[] nums, int target) {  
    List<List<Integer>> list = new ArrayList<>();  
    backtrack(list, new ArrayList<>(), nums, target, 0);  
    return list;  
}  
  
private void backtrack(List<List<Integer>> list, List<Integer> tempList, int []  
nums, int remain, int start){  
    if(remain < 0) return;  
    if(remain == 0) list.add(new ArrayList<>(tempList));  
    for(int i = start; i < nums.length; i++){  
        tempList.add(nums[i]);  
        backtrack(list, tempList, nums, remain - nums[i], i);  
        tempList.remove(tempList.size() - 1);  
    }  
}
```

```

else if(remain == 0) list.add(new ArrayList<>(tempList));
else{
    for(int i = start; i < nums.length; i++){
        tempList.add(nums[i]);
        backtrack(list, tempList, nums, remain - nums[i], i);
        //找到了一个解或者 remain < 0 了, 将当前数字移除, 然后继续尝试
        tempList.remove(tempList.size() - 1);
    }
}
}

```

时间复杂度:

空间复杂度:

解法二 动态规划

参考[这里](#)。动态规划的关键就是找到递进关系，看到了下边的评论想通的。



siyang2 ★ 307 🕒 November 20, 2014 2:32 AM

Hnn, I use the same idea as you did. I can help illustrate the idea.

Thinking about there is an opt[] array, each element in the array is a vector<vector>. We do DP from opt[0] to opt[target].

// base case

opt[0] = [[]]

// iteration

opt[i] =

1. add candidates[j] to each list in opt[i-candidates[j]] if i > candidates[j]
.Add each list to opt[i]
2. create candidates[j] as a new vector if i == candidates[j]
.Add this list to opt[i]

我们用一个 opt 的 list，然后依次求出 opt[0], opt[1] ... opt[target]。

opt[0], 表示和为 0 的所有情况的组合。

opt[1], 表示和为 1 的所有情况的组合。

opt[2], 表示和为 2 的所有情况的组合。

...

opt[target], 表示和为 target 的所有情况的组合, 也就是题目所要求的。

递进关系就是, sum 代表要求的和, 如果想求 opt [sum], 就遍历给定的数组 nums, 然后分两种情况。

- 如果 sum 刚好等于 nums [i], 那么就直接把 nums [i] 加到 list 里, 算作一种情况。

例如 nums = [2, 3, 6, 7], target = 7。

当求 sum = 3 的时候, 也就是求 opt [3] 的时候, 此时当遍历到 nums [1], 此时 nums [1] == sum == 3, 所以此时 opt [3] = [[3]]

- 如果 sum 大于 nums [i], 那么我们就把 opt [sum - nums [i]] 的所有情况都加上 nums [i] 然后作为 opt [sum]。

例如 nums = [1, 2, 3, 6, 7], target = 7。

当 sum 等于 5 的时候, 也就是求 opt [5] 的时候, 此时当遍历到 nums [1], 此时 nums [1] = 2 < sum。然后, 就看 opt [sum - nums [i]] = opt [5 - 2] = opt [3], 而 opt [3] 在之前已经求好了, opt [3] = [[1, 2], [3]], 然后把 opt [3] 中的每一种情况都加上 nums [1], 也就是 2, 就变成了 [[1, 2, 2], [3, 2]], 这个就是遍历到 nums [1] 时候的 opt [5] 了。

上边的想法看起来没什么问题, 但跑了下遇到一个问题。

比如求 nums = [2, 3, 6, 7], target = 7 的时候。

求 opt [5], 然后遍历到 nums [0] = 2 的时候, 就把 opt [3] = [[3]] 的所有情况加上 2, 也就是 [3 2] 加到 opt [5] 上。接着遍历到 nums [2] = 3 的时候, 就把 opt [2] = [[2]] 的所有情况加上 3, 然后 [2 3] 这种情况加到 opt [5] 上, 此时 opt [5] = [[3 2], [2 3]]。这样出现了重复的情况, 需要解决一下。

这样就相当于二维数组去重, 也就是 [[3 2], [2 3], [1]] 这样的列表去重变成 [[2 3], [1]]。最普通的想法就是两个 for 循环然后一个一个比对, 把重复的去掉。但这样实在是太麻烦了, 因为比对的时候又要比对列表是否相等, 比对列表是否相等又比较麻烦。

[这里](#)看到一个方法, 就是把每个 list 转成 string, 然后利用 HashMap 的 key 是唯一的, 把每个 list 当做 key 加入到 HashMap 中, 这样就实现了去重, 然后再把 string 还原为 list。

```
private List<List<Integer>> removeDuplicate(List<List<Integer>> list) {
    Map<String, String> ans = new HashMap<String, String>();
    for (int i = 0; i < list.size(); i++) {
        List<Integer> l = list.get(i);
        Collections.sort(l);
        String key = "";
        //[ 2 3 4] 转为 "2,3,4"
        for (int j = 0; j < l.size() - 1; j++) {
            key = key + l.get(j) + ",";
        }
        key = key + l.get(l.size() - 1);
        ans.put(key, "");
    }
    //根据逗号还原 List
    List<List<Integer>> ans_list = new ArrayList<List<Integer>>();
```

```

    for (String k : ans.keySet()) {
        String[] l = k.split(",");
        List<Integer> temp = new ArrayList<Integer>();
        for (int i = 0; i < l.length; i++) {
            int c = Integer.parseInt(l[i]);
            temp.add(c);
        }
        ans_list.add(temp);
    }
    return ans_list;
}

```

然后结合去重的方法，我们的问题就解决了。

```

public List<List<Integer>> combinationSum(int[] nums, int target) {
    List<List<List<Integer>>> ans = new ArrayList<>(); //opt 数组
    Arrays.sort(nums); // 将数组有序，这样可以提结束循环
    for (int sum = 0; sum <= target; sum++) { // 从 0 到 target 求出每一个 opt
        List<List<Integer>> ans_sum = new ArrayList<List<Integer>>();
        for (int i = 0; i < nums.length; i++) { //遍历 nums
            if (nums[i] == sum) {
                List<Integer> temp = new ArrayList<Integer>();
                temp.add(nums[i]);
                ans_sum.add(temp);
            } else if (nums[i] < sum) {
                List<List<Integer>> ans_sub = ans.get(sum - nums[i]);
                //每一个加上 nums[i]
                for (int j = 0; j < ans_sub.size(); j++) {
                    List<Integer> temp = new ArrayList<Integer>(ans_sub.get(j));
                    temp.add(nums[i]);
                    ans_sum.add(temp);
                }
            } else {
                break;
            }
        }
        ans.add(sum, ans_sum);
    }
    return removeDuplicate(ans.get(target));
}

private List<List<Integer>> removeDuplicate(List<List<Integer>> list) {
    Map<String, String> ans = new HashMap<String, String>();
    for (int i = 0; i < list.size(); i++) {
        List<Integer> l = list.get(i);
        Collections.sort(l);
    }
}

```

```

String key = "";
//[ 2 3 4 ] 转为 "2,3,4"
for (int j = 0; j < l.size() - 1; j++) {
    key = key + l.get(j) + ",";
}
key = key + l.get(l.size() - 1);
ans.put(key, "");
}
//根据逗号还原 List
List<List<Integer>> ans_list = new ArrayList<List<Integer>>();
for (String k : ans.keySet()) {
    String[] l = k.split(",");
    List<Integer> temp = new ArrayList<Integer>();
    for (int i = 0; i < l.length; i++) {
        int c = Integer.parseInt(l[i]);
        temp.add(c);
    }
    ans_list.add(temp);
}
return ans_list;
}

```

时间复杂度：

空间复杂度：

还有另一种思路可以解决重复的问题。

之前对于 $\text{nums} = [2, 3, 6, 7]$, $\text{target} = 7$ ，我们用了两层 for 循环，分别对 opt 和 nums 进行遍历。

我们先求 $\text{opt}[0]$ ，通过遍历 $\text{nums}[0]$, $\text{nums}[1]$, $\text{nums}[2]$, $\text{nums}[3]$

然后再求 $\text{opt}[1]$ ，通过遍历 $\text{nums}[0]$, $\text{nums}[1]$, $\text{nums}[2]$, $\text{nums}[3]$

然后再求 $\text{opt}[2]$ ，通过遍历 $\text{nums}[0]$, $\text{nums}[1]$, $\text{nums}[2]$, $\text{nums}[3]$

...

最后再求 $\text{opt}[7]$ ，通过遍历 $\text{nums}[0]$, $\text{nums}[1]$, $\text{nums}[2]$, $\text{nums}[3]$ 。

求 $\text{opt}[5]$ 的时候，出现了 $[2\ 3]$, $[3\ 2]$ 这样重复的情况。

我们可以把两个 for 循环的遍历颠倒一下，外层遍历 nums，内层遍历 opt。

考虑 $\text{nums}[0]$ ，求出 $\text{opt}[0]$ ，求出 $\text{opt}[1]$ ，求出 $\text{opt}[2]$ ，求出 $\text{opt}[3]$... 求出 $\text{opt}[7]$ 。

考虑 $\text{nums}[1]$ ，求出 $\text{opt}[0]$ ，求出 $\text{opt}[1]$ ，求出 $\text{opt}[2]$ ，求出 $\text{opt}[3]$... 求出 $\text{opt}[7]$ 。

考虑 $\text{nums}[2]$ ，求出 $\text{opt}[0]$ ，求出 $\text{opt}[1]$ ，求出 $\text{opt}[2]$ ，求出 $\text{opt}[3]$... 求出 $\text{opt}[7]$ 。

考虑 nums[3], 求出 opt[0], 求出 opt[1], 求出 opt[2], 求出 opt[3]... 求出 opt[7]。

这样的话, 每次循环会更新一次 opt[7], 最后次更新的 opt[7] 就是我们想要的了。

这样之前的问题, 求 opt[5] 的时候, 出现了 [2 3], [3 2] 这样重复的情况就不会出现了, 因为当考虑 nums[2] 的时候, opt[3] 里边还没有加入 [3]。

思路就是上边说的了, 但是写代码的时候遇到不少坑, 大家也可以先尝试写一下。

```
public List<List<Integer>> combinationSum(int[] nums, int target) {
    List<List<List<Integer>>> ans = new ArrayList<>();
    Arrays.sort(nums);
    if (nums[0] > target) {
        return new ArrayList<List<Integer>>();
    }
    // 先初始化 ans[0] 到 ans[target], 因为每次循环是更新 ans, 会用到 ans.get() 函数, 如果不初始化会报错
    for (int i = 0; i <= target; i++) {
        List<List<Integer>> ans_i = new ArrayList<List<Integer>>();
        ans.add(i, ans_i);
    }

    for (int i = 0; i < nums.length; i++) {
        for (int sum = nums[i]; sum <= target; sum++) {
            List<List<Integer>> ans_sum = ans.get(sum);
            List<List<Integer>> ans_sub = ans.get(sum - nums[i]);
            // 刚开始 ans_sub 的大小是 0, 所以单独考虑一下这种情况
            if (sum == nums[i]) {
                ArrayList<Integer> temp = new ArrayList<Integer>();
                temp.add(nums[i]);
                ans_sum.add(temp);
            }
            // 如果 ans.get(sum - nums[i]) 大小不等于 0, 就可以按之前的想法更新了。
            // 每个 ans_sub[j] 都加上 nums[i]
            if (ans_sub.size() > 0) {
                for (int j = 0; j < ans_sub.size(); j++) {
                    ArrayList<Integer> temp = new ArrayList<Integer>
(ans_sub.get(j));
                    temp.add(nums[i]);
                    ans_sum.add(temp);
                }
            }
        }
    }
    return ans.get(target);
}
```

总

对回溯法又有了更深的了解，一般的架构就是一个大的 for 循环，然后先 add，接着利用递归进行向前遍历，然后再 remove，继续循环。而解法二的动态规划就是一定要找到递进的规则，开始的时候就想偏了，导致迟迟想不出来。

40、题目描述（中等难度）

Given a collection of candidate numbers (`candidates`) and a target number (`target`), find all unique combinations in `candidates` where the candidate numbers sums to `target` .

Each number in `candidates` may only be used **once** in the combination.

Note:

- All numbers (including `target`) will be positive integers.
- The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8,
A solution set is:
[
  [1, 7],
  [1, 2, 5],
  [2, 6],
  [1, 1, 6]
]
```

Example 2:

```
Input: candidates = [2,5,2,1,2], target = 5,
A solution set is:
[
  [1,2,2],
  [5]
]
```

和[上一道题](#)非常像了，区别在于这里给的数组中有重复的数字，每个数字只能使用一次，然后同样是给出所有和等于 `target` 的情况。

解法一 回溯法

只需要在上题的基础上改一下就行了。直接看代码吧。

```
public List<List<Integer>> combinationSum2(int[] candidates, int target) {
    List<List<Integer>> ans = new ArrayList<>();
    getAnswer(ans, new ArrayList<>(), candidates, target, 0);
    /*****修改的地方*****/
}
```

```

// 如果是 Input: candidates = [2,5,2,1,2], target = 5,
// 输出会出现 [2 2 1] [2 1 2] 这样的情况, 所以要去重
return removeDuplicate(ans);
/*****/
}

private void getAnswer(List<List<Integer>> ans, ArrayList<Integer> temp, int[]
candidates, int target, int start) {
    if (target == 0) {
        ans.add(new ArrayList<Integer>(temp));
    } else if (target < 0) {
        return;
    } else {
        for (int i = start; i < candidates.length; i++) {
            temp.add(candidates[i]);
            /*****/修改的地方*****/
            //i -> i + 1, 因为每个数字只能用一次, 所以下次遍历的时候不从自己开始
            getAnswer(ans, temp, candidates, target - candidates[i], i + 1);
            /*****/
            temp.remove(temp.size() - 1);
        }
    }
}

private List<List<Integer>> removeDuplicate(List<List<Integer>> list) {
    Map<String, String> ans = new HashMap<String, String>();
    for (int i = 0; i < list.size(); i++) {
        List<Integer> l = list.get(i);
        Collections.sort(l);
        String key = "";
        for (int j = 0; j < l.size() - 1; j++) {
            key = key + l.get(j) + ",";
        }
        key = key + l.get(l.size() - 1);
        ans.put(key, "");
    }
    List<List<Integer>> ans_list = new ArrayList<List<Integer>>();
    for (String k : ans.keySet()) {
        String[] l = k.split(",");
        List<Integer> temp = new ArrayList<Integer>();
        for (int i = 0; i < l.length; i++) {
            int c = Integer.parseInt(l[i]);
            temp.add(c);
        }
        ans_list.add(temp);
    }
}

```



```

    }
    return ans_list;
}

```

时间复杂度：

空间复杂度：

看到[这里](#)，想法很棒，为了解决重复的情况，我们可以先把数组先排序，这样就好说了。

```

public List<List<Integer>> combinationSum2(int[] candidates, int target) {
    List<List<Integer>> ans = new ArrayList<>();
    Arrays.sort(candidates); //排序
    getAnswer(ans, new ArrayList<>(), candidates, target, 0);
    return ans;
}

private void getAnswer(List<List<Integer>> ans, ArrayList<Integer> temp, int[]
candidates, int target, int start) {
    if (target == 0) {
        ans.add(new ArrayList<Integer>(temp));
    } else if (target < 0) {
        return;
    } else {
        for (int i = start; i < candidates.length; i++) {
            //跳过重复的数字
            if(i > start && candidates[i] == candidates[i-1]) continue;
            temp.add(candidates[i]);
            /*****修改的地方*****/
            //i -> i + 1，因为每个数字只能用一次，所以下次遍历的时候不从自己开始
            getAnswer(ans, temp, candidates, target - candidates[i], i + 1);
            /*****/
            temp.remove(temp.size() - 1);
        }
    }
}

```

解法二 动态规划

怎么去更改上题的算法满足本题，暂时没想到，只想到就是再写个函数对答案再过滤一次。先记录给定的 nums 中的每个数字出现的次数，然后判断每个 list 的数字出现的次数是不是满足小于等于给定的 nums 中的每个数字出现的次数，不满足的话就剔除掉。如果大家有直接改之前算法的好办法可以告诉我，谢谢了。

此外，要注意一点就是上题中，给定的 nums 没有重复的，而这题中是有重复的。为了使得和之前一样，所以我们在算法中都得加上

```
if (i > 0 && nums[i] == nums[i - 1]) {  
    continue;  
}
```

跳过重复的数字，不然是不能 AC 的，至于原因下边分析下。

```
public List<List<Integer>> combinationSum2(int[] nums, int target) {  
    List<List<List<Integer>>> ans = new ArrayList<>(); //opt 数组  
    Arrays.sort(nums); // 将数组有序，这样可以提现结束循环  
    for (int sum = 0; sum <= target; sum++) { // 从 0 到 target 求出每一个 opt  
        List<List<Integer>> ans_sum = new ArrayList<List<Integer>>();  
        for (int i = 0; i < nums.length; i++) { //遍历 nums  
            /*****修改的地方*****/  
            if (i > 0 && nums[i] == nums[i - 1]) {  
                continue;  
            }  
            /*****/  
            if (nums[i] == sum) {  
                List<Integer> temp = new ArrayList<Integer>();  
                temp.add(nums[i]);  
                ans_sum.add(temp);  
            } else if (nums[i] < sum) {  
                List<List<Integer>> ans_sub = ans.get(sum - nums[i]);  
                //每一个加上 nums[i]  
                for (int j = 0; j < ans_sub.size(); j++) {  
                    List<Integer> temp = new ArrayList<Integer>(ans_sub.get(j));  
                    temp.add(nums[i]);  
                    ans_sum.add(temp);  
                }  
            } else {  
                break;  
            }  
        }  
        ans.add(sum, ans_sum);  
    }  
    return remove(removeDuplicate(ans.get(target)), nums);  
}  
  
private List<List<Integer>> removeDuplicate(List<List<Integer>> list) {  
    Map<String, String> ans = new HashMap<String, String>();  
    for (int i = 0; i < list.size(); i++) {  
        List<Integer> l = list.get(i);
```

```

        Collections.sort(l);
        String key = "";
        // [ 2 3 4 ] 转为 "2,3,4"
        for (int j = 0; j < l.size() - 1; j++) {
            key = key + l.get(j) + ",";
        }
        key = key + l.get(l.size() - 1);
        ans.put(key, "");
    }
    //根据逗号还原 List
    List<List<Integer>> ans_list = new ArrayList<List<Integer>>();
    for (String k : ans.keySet()) {
        String[] l = k.split(",");
        List<Integer> temp = new ArrayList<Integer>();
        for (int i = 0; i < l.length; i++) {
            int c = Integer.parseInt(l[i]);
            temp.add(c);
        }
        ans_list.add(temp);
    }
    return ans_list;
}

//过滤不满足答案的情况
private List<List<Integer>> remove(List<List<Integer>> list, int[] nums) {
    HashMap<Integer, Integer> nh = new HashMap<Integer, Integer>();
    List<List<Integer>> ans = new ArrayList<List<Integer>>(list);
    //记录每个数字出现的次数
    for (int n : nums) {
        int s = nh.getDefault(n, 0);
        nh.put(n, s + 1);
    }
    for (int i = 0; i < list.size(); i++) {
        List<Integer> l = list.get(i);
        HashMap<Integer, Integer> temp = new HashMap<Integer, Integer>();
        //记录每个 list 中数字出现的次数
        for (int n : l) {
            int s = temp.getDefault(n, 0);
            temp.put(n, s + 1);
        }
        for (int n : l) {
            //进行比较
            if (temp.get(n) > nh.get(n)) {
                ans.remove(l);
                break;
            }
        }
    }
}

```

```

    }
}
return ans;
}

```

如果不加跳过重复的数字的话，下边的样例不会通过。

Submission Result: **Memory Limit Exceeded**

[More Details >](#)

>_

Last executed input: **[1,1,1,1,1]**
9

这是因为我们求 opt 的时候每个列表的数量在以指数级增加，在上一个 opt 的基础上，每一个列表都增加 5 个列表。

opt[1] = [[1], [1], [1], [1], [1]] 数量是 5,

opt[2] = [

```

    [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ],
    [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ]
    [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ],
    [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ],
    [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], ,
    ] 数量是 5 * 5。

```

opt[3] 数量是 5 * 5 * 5。

到了 opt[9] 就是 5 的 9 次方，数量是 1953125 内存爆炸了。

另一个算法也可以改一下

```

public List<List<Integer>> combinationSum2(int[] nums, int target) {
    List<List<List<Integer>>> ans = new ArrayList<>();
    Arrays.sort(nums);
    if (nums[0] > target) {
        return new ArrayList<List<Integer>>();
    }
}

```

```

    }
    for (int i = 0; i <= target; i++) {
        List<List<Integer>> ans_i = new ArrayList<List<Integer>>();
        ans.add(i, ans_i);
    }

    for (int i = 0; i < nums.length; i++) {
        /*****修改的地方*****/
        if (i > 0 && nums[i] == nums[i - 1]) {
            continue;
        }
        /*****/
        for (int sum = nums[i]; sum <= target; sum++) {
            List<List<Integer>> ans_sum = ans.get(sum);
            List<List<Integer>> ans_sub = ans.get(sum - nums[i]);
            if (sum == nums[i]) {
                ArrayList<Integer> temp = new ArrayList<Integer>();
                temp.add(nums[i]);
                ans_sum.add(temp);
            }
            if (ans_sub.size() > 0) {
                for (int j = 0; j < ans_sub.size(); j++) {
                    ArrayList<Integer> temp = new ArrayList<Integer>
(ans_sub.get(j));
                    temp.add(nums[i]);
                    ans_sum.add(temp);
                }
            }
        }
    }
    return remove(ans.get(target), nums);
}

private List<List<Integer>> remove(List<List<Integer>> list, int[] nums) {
    HashMap<Integer, Integer> nh = new HashMap<Integer, Integer>();
    List<List<Integer>> ans = new ArrayList<List<Integer>>(list);
    for (int n : nums) {
        int s = nh.getDefault(n, 0);
        nh.put(n, s + 1);
    }
    for (int i = 0; i < list.size(); i++) {
        List<Integer> l = list.get(i);
        HashMap<Integer, Integer> temp = new HashMap<Integer, Integer>();
        for (int n : l) {

```

```

        int s = temp.getDefault(n, 0);
        temp.put(n, s + 1);
    }
    for (int n : 1) {
        if (temp.get(n) > nh.get(n)) {
            ans.remove(1);
            break;
        }
    }
}
return ans;
}

```

如果不加跳过重复的数字的话，下边的样例不会通过

Submission Result: **Wrong Answer**

[More Details >](#)

>

Input: [10,1,2,7,6,1,5]
8

Output: [[1,2,5],[1,2,5],[1,1,6],[1,1,6],[1,1,6],[2,6],[1,7],[1,7]]

Expected: [[1,1,6],[1,2,5],[1,7],[2,6]]

会发现出现了很多重复的结果，就是因为没有跳过重复的 1。在求 `opt[1]` 的时候就变成了 `[[1], [1]]` 这样子，由于后边求的时候都是直接在原来每一个列表里加数字，所有后边都是加了两次了。

总

和上题很像，基本上改一改就好了。排序的来排除重复的情况也很妙。还有就是改算法的时候，要考虑到题的要求的变化之处。

41、题目描述(困难难度)

Given an unsorted integer array, find the smallest missing positive integer.

Example 1:

```
Input: [1,2,0]
Output: 3
```

Example 2:

```
Input: [3,4,-1,1]
Output: 2
```

Example 3:

```
Input: [7,8,9,11,12]
Output: 1
```

Note:

Your algorithm should run in $O(n)$ time and uses constant extra space.

给一串数字，找出缺失的最小正数。限制了时间复杂度为 $O(n)$ ，空间复杂度为 $O(1)$ 。

解法一 交换

参考[这里](#)。

如果没限制空间复杂度，我们可以这样想。用一个等大的数组去顺序保存这些数字。

比如说，数组 `nums [3 4 -1 1 8]`，它的大小是 5。然后再创建一个等大的数组 `a`，初始化为 `[-1, -1, -1, -1, -1]`。然后我们遍历 `nums`，把数字分别存到对应的位置。1 就存到数组 `a` 的第 1 个位置 (`a [0]`)，2 就存到数组 `a` 的第 2 个位置 (`a [1]`)，3 就存到数组 `a` 的第 3 个位置 (`a [2]`) ...

`nums [0]` 等于 3，更新 `a [-1, -1, 3, -1, -1]`。

`nums [1]` 等于 4，更新 `a [-1, -1, 3, 4, -1]`。

`nums [2]` 等于 -1，不是正数，忽略。

`nums [3]` 等于 1，更新 `a [1, -1, 3, 4, -1]`。

`nums [4]` 等于 8，我们的 `a` 数组只能存 1 到 5，所以同样忽略。

最后，我们只需要遍历 `a` 数组，遇到第一次 `a [i] != i + 1`，就说明缺失了 `i + 1`。因为我们的 `a` 数组每个位置都存着比下标大 1 的数。

当然，上边都是基于有一个额外空间讲的。如果没有额外空间，怎么办呢？

我们直接把原数组当成 a 数组去用。这样的话，会出现的问题就是之前的数就会被覆盖掉。覆盖之前我们把它放回到当前数字的位置，换句话说就是交换一下位置。然后把交换回来的数字放到应该在的位置，又交换回来的新数字继续判断，直到交换回来的数字小于 0，或者大于了数组的大小，或者它就是当前位置放的数字了。接着遍历 nums 的下一个数。具体看一下。

```
nums = [3 4 -1 1 8]
```

nums[0] 等于 3，把 3 放到第 3 个位置，并且把之前第 3 个位置的 -1 放回来，更新 nums [-1, 4, 3, 1, 8]。

然后继续判断交换回来的数字，nums[0] 等于 -1，不是正数，忽略。

nums[1] 等于 4，把 4 放到第 4 个位置，并且把之前第 4 个位置的 1 放回来，更新 nums [-1, 1, 3, 4, 8]。

然后继续判断交换回来的数字，nums[1] 等于 1，把 1 放到第 1 个位置，并且把之前第 1 个位置的 -1 放回来，更新 nums [1, -1, 3, 4, 8]。

然后继续判断交换回来的数字，nums[1] 等于 -1，不是正数，忽略。

nums[2] 等于 3，刚好在第 3 个位置，不用管。

nums[3] 等于 4，刚好在第 4 个位置，不用管。

nums[4] 等于 8，我们的 nums 数组只能存 1 到 5，所以同样忽略。

最后，我们只需要遍历 nums 数组，遇到第一次 nums[i] != i + 1，就说明缺失了 i + 1。因为我们的 nums 数组每个位置都存着比下标大 1 的数。

看下代码吧，一个 for 循环，里边再 while 循环。

```
public int firstMissingPositive(int[] nums) {
    int n = nums.length;
    //遍历每个数字
    for (int i = 0; i < n; i++) {
        //判断交换回来的数字
        while (nums[i] > 0 && nums[i] <= n && nums[i] != nums[nums[i] - 1]) {
            //第 nums[i] 个位置的下标是 nums[i] - 1
            swap(nums, i, nums[i] - 1);
        }
    }
    //找出第一个 nums[i] != i + 1 的位置
    for (int i = 0; i < n; i++) {
        if (nums[i] != i + 1) {
            return i + 1;
        }
    }
    //如果之前的数都满足就返回 n + 1
    return n + 1;
}
```



```
private void swap(int[] nums, int i, int j) {  
    int temp = nums[i];  
    nums[i] = nums[j];  
    nums[j] = temp;  
}
```

时间复杂度：for 循环里边套了个 while 循环，如果粗略的讲，那时间复杂度就是 $O(n^2)$ 了。我们再从算法的逻辑上分析一下。因为每交换一次，就有一个数字放到了应该在的位置，只有 n 个数字，所以 while 里边的交换函数，最多执行 n 次。所以时间复杂度更精确的说，应该是 $O(n)$ 。

空间复杂度： $O(1)$ 。

解法二 标记

参考[这里](#)。

同样的，我们先考虑如果有额外的空间该怎么做。

还是一样，对于 `nums = [3 4 -1 1 8]`，我们创建一个等大的数组 `a`，初始化为 `[false, false, false, false, false]`。然后如果 `nums` 里有 1 就把，第一个位置 `a[0]` 改为 `true`。如果 `nums` 里有 `m`，就把 `a[m - 1]` 改为 `true`。看下具体的例子。

`nums = [3 4 -1 1 8]`

`nums[0]` 等于 3，更新 `a [false, false, true, false, false]`。

`nums[1]` 等于 4，更新 `a [false, false, true, true, false]`。

`nums[2]` 等于 -1，不是正数，忽略。

`nums[3]` 等于 1，更新 `a [true, false, true, true, false]`。

`nums[4]` 等于 8，我们的 `a` 数组只能存 1 到 5，所以同样忽略。

然后遍历数组 `a`，如果 `a[i] != true`。那么，我们就返回 `i + 1`。因为 `a[i]` 等于 `true` 就意味着 `i + 1` 存在。

问题又来了，其实我们没有额外空间，我们只能利用原来的数组 `nums`。

同样我们直接把 `nums` 用作数组 `a`。

但当我们更新的时候，如果直接把数组的数赋值成 `true`，那么原来的数字就没了。这里有个很巧妙的技巧。

考虑到我们真正关心的只有正数。开始 `a` 数组的初始化是 `false`，所以我们把正数当做 `false`，负数当成 `true`。如果我们想要把 `nums[i]` 赋值成 `true`，如果 `nums[i]` 是正数，我们直接取相反数作为标记就行，如果是负数就不用管了。这样做的好处就是，遍历数字的时候，我们只需要取绝对值，就是原来的数了。

当然这样又带来一个问题，我们取绝对值的话，之前的负数该怎么办？一取绝对值的话，就会造成干扰。简单粗暴些，我们把正数都放在前边，我们只考虑正数。负数和 0 就丢到最后，遍历的时候不去遍历就可以了。

看下具体的例子。

nums = [3 4 -1 1 8]

先把所有正数放前边，并且只考虑正数。nums = [3 4 1 8]，正数当作 false，负数当做 true。所以 nums 就可以看成 [false, false, false, false]。

nums[0] 等于 3，把第 3 个位置的数字变为负数，更新 nums [3, 4, -1, 8]，可以看做 [false, false, true, false]。

nums[1] 等于 4，把第 4 个位置的数字变为负数，更新 nums [3, 4, -1, -8]，可以看做 [false, false, true, true]。

nums[2] 等于 -1，取绝对值为 1，把第 1 个位置的数字变为负数，更新 nums [-3, 4, -1, -8]，可以看做 [true, false, true, true]。

nums[3] 等于 -8，取绝对值为 8，我们的 nums 数组只考虑 1 到 4，所以忽略。

最后再遍历 nums，如果 nums[i] 大于 0，就代表缺失了 i + 1。因为正数代表 false。

把正数移到最前边，写了两种算法，代码里注释了，大家可以参考下。

```
public int firstMissingPositive(int[] nums) {
    int n = nums.length;
    //将正数移到前边，并且得到正数的个数
    int k = positiveNumber(nums);
    for (int i = 0; i < k; i++) {
        //得到要标记的下标
        int index = Math.abs(nums[i]) - 1;
        if (index < k) {
            //判断要标记的位置的数是不是小于 0，不是小于 0 就取相反数
            int temp = Math.abs(nums[index]);
            nums[index] = temp < 0 ? temp : -temp;
        }
    }
    //找到第一个大于 0 的位置
    for (int i = 0; i < k; i++) {
        if (nums[i] > 0) {
            return i + 1;
        }
    }
    return k + 1;
}

private int positiveNumber(int[] nums) {
    //解法一 把负数和 0 全部交换到最后
    /*    int n = nums.length;
    for (int i = 0; i < n; i++) {
        while (nums[i] <= 0) {
```

```

        swap(nums, i, n - 1);
        n--;
        if (i == n) {
            break;
        }
    }
}
return n;*/

```

//解法二 用一个指针 p ，保证 p 之前的都是正数。遍历 `nums`，每遇到一个正数就把它交换到 p 指针的位置，并且 p 指针后移

```

int n = nums.length;
int p = 0;
for (int i = 0; i < n; i++) {
    if (nums[i] > 0) {
        swap(nums, i, p);
        p++;
    }
}
return p;

}

private void swap(int[] nums, int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}

```

时间复杂度： $O(n)$ 。

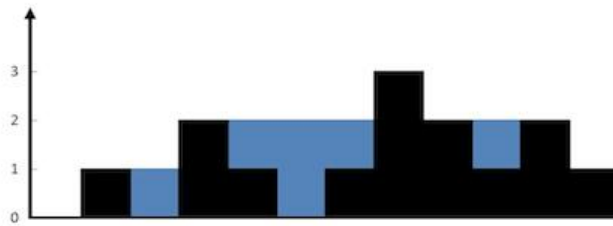
空间复杂度： $O(1)$ 。

总

对于这种要求空间复杂度的，我们可以先考虑如果有一个等大的空间，我们可以怎么做。然后再考虑如果直接用原数组怎么做，主要是要保证数组的信息不要丢失。目前遇到的，主要有两种方法就是交换和取相反数。

42、题目描述（困难难度）

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it is able to trap after raining.



The above elevation map is represented by array `[0,1,0,2,1,0,1,3,2,1,2,1]`. In this case, 6 units of rain water (blue section) are being trapped. Thanks Marcos for contributing this image!

Example:

```
Input: [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
```

黑色的看成墙，蓝色的看成水，宽度一样，给定一个数组，每个数代表从左到右墙的高度，求出能装多少单位的水。也就是图中蓝色正方形的个数。

解法一 按行求

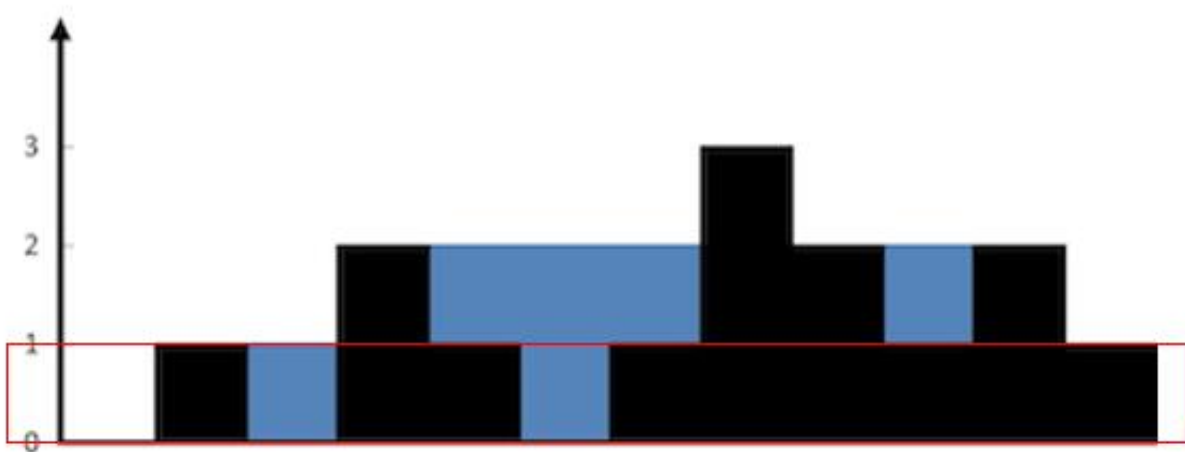
这是我最开始想到的一个解法，提交后直接 AC 了，自己都震惊了。就是先求高度为 1 的水，再求高度为 2 的水，再求高度为 3 的水。

整个思路就是，求第 i 层的水，遍历每个位置，如果当前的高度小于 i ，并且两边有高度大于等于 i 的，说明这个地方一定有水，水就可以加 1。

如果求高度为 i 的水，首先用一个变量 `temp` 保存当前累积的水，初始化为 0。从左到右遍历墙的高度，遇到高度大于等于 i 的时候，开始更新 `temp`。更新原则是遇到高度小于 i 的就把 `temp` 加 1，遇到高度大于等于 i 的，就把 `temp` 加到最终的答案 `ans` 里，并且 `temp` 置零，然后继续循环。

我们就以题目的例子讲一下。

先求第 1 行的水。



也就是红色区域中的水，数组是 $\text{height} = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]$ 。

原则是高度小于 1， $\text{temp}++$ ，高度大于等于 1， $\text{ans} = \text{ans} + \text{temp}$ ， $\text{temp} = 0$ 。

temp 初始化为 0， $\text{ans} = 0$

$\text{height}[0]$ 等于 $0 < 1$ ，不更新。

$\text{height}[1]$ 等于 $1 \geq 1$ ，开始更新 temp 。

$\text{height}[2]$ 等于 $0 < 1$ ， $\text{temp} = \text{temp} + 1 = 1$ 。

$\text{height}[3]$ 等于 $2 \geq 1$ ， $\text{ans} = \text{ans} + \text{temp} = 1$ ， $\text{temp} = 0$ 。

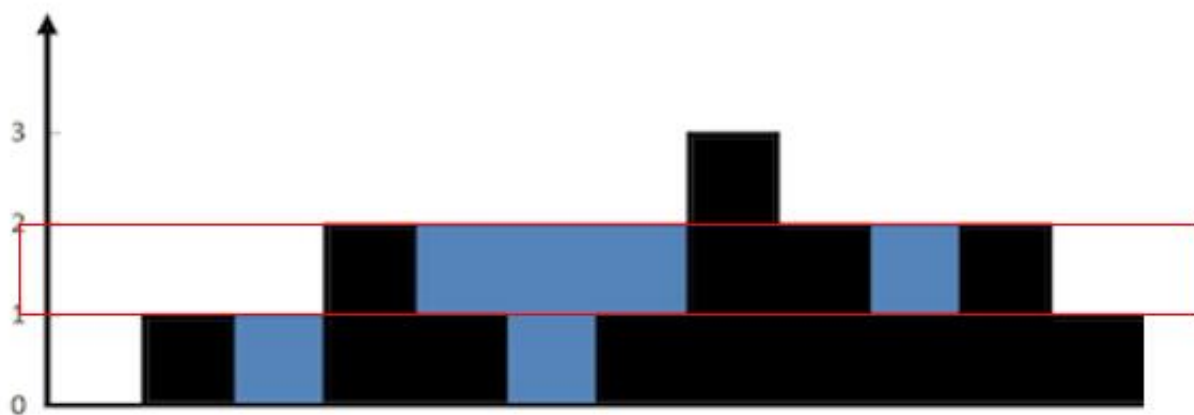
$\text{height}[4]$ 等于 $1 \geq 1$ ， $\text{ans} = \text{ans} + \text{temp} = 1$ ， $\text{temp} = 0$ 。

$\text{height}[5]$ 等于 $0 < 1$ ， $\text{temp} = \text{temp} + 1 = 1$ 。

$\text{height}[6]$ 等于 $1 \geq 1$ ， $\text{ans} = \text{ans} + \text{temp} = 2$ ， $\text{temp} = 0$ 。

剩下的 $\text{height}[7]$ 到最后，高度都大于等于 1，更新 $\text{ans} = \text{ans} + \text{temp} = 2$ ， $\text{temp} = 0$ 。而其实 temp 一直都是 0，所以 ans 没有变化。

再求第 2 行的水。



也就是红色区域中的水，

数组是 $\text{height} = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]$ 。

原则是高度小于 2， $\text{temp}++$ ，高度大于等于 2， $\text{ans} = \text{ans} + \text{temp}$ ， $\text{temp} = 0$ 。

temp 初始化为 0， ans 此时等于 2。

$\text{height}[0]$ 等于 $0 < 2$ ，不更新。

$\text{height}[1]$ 等于 $1 < 2$ ，不更新。

$\text{height}[2]$ 等于 $0 < 2$ ，不更新。

$\text{height}[3]$ 等于 $2 \geq 2$ ，开始更新

height[4] 等于 $1 < 2$, $temp = temp + 1 = 1$ 。

height[5] 等于 $0 < 2$, $temp = temp + 1 = 2$ 。

height[6] 等于 $1 < 2$, $temp = temp + 1 = 3$ 。

height[7] 等于 $3 \geq 2$, $ans = ans + temp = 5$, $temp = 0$ 。

height[8] 等于 $2 \geq 2$, $ans = ans + temp = 3$, $temp = 0$ 。

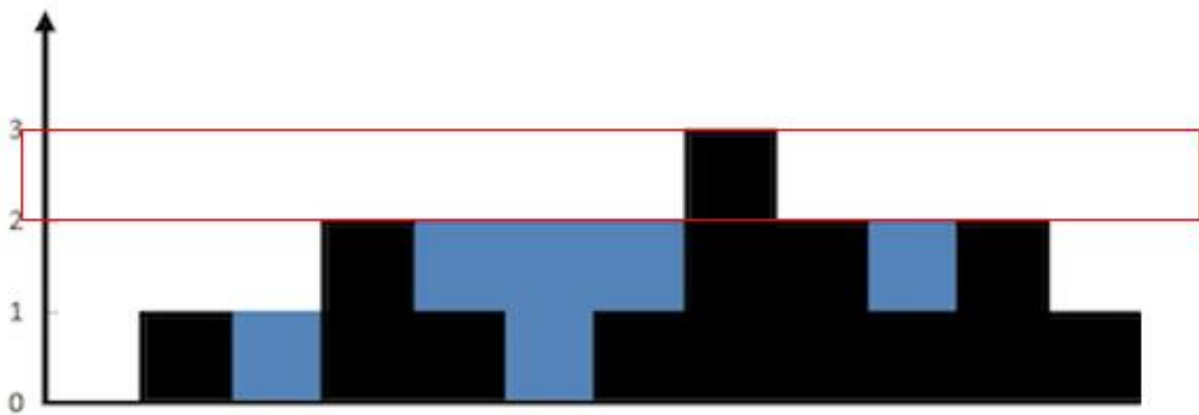
height[9] 等于 $1 < 2$, $temp = temp + 1 = 1$ 。

height[10] 等于 $2 \geq 2$, $ans = ans + temp = 6$, $temp = 0$ 。

height[11] 等于 $1 < 2$, $temp = temp + 1 = 1$ 。

然后结束循环, 此时的 ans 就是 6。

再看第 3 层。



按照之前的算法, 之前的都是小于 3 的, 不更新 temp, 然后到 height[7] 等于 3, 开始更新 temp, 但是后边没有 height 大于等于 3 了, 所以 ans 没有更新。

所以最终的 ans 就是 6。

看下代码吧。

```
public int trap(int[] height) {
    int sum = 0;
    int max = getMax(height); // 找到最大的高度, 以便遍历。
    for (int i = 1; i <= max; i++) {
        boolean isStart = false; // 标记是否开始更新 temp
        int temp_sum = 0;
        for (int j = 0; j < height.length; j++) {
            if (isStart && height[j] < i) {
                temp_sum++;
            }
            if (height[j] >= i) {
```

```

        sum = sum + temp_sum;
        temp_sum = 0;
        isStart = true;
    }
}
return sum;
}

private int getMax(int[] height) {
    int max = 0;
    for (int i = 0; i < height.length; i++) {
        if (height[i] > max) {
            max = height[i];
        }
    }
    return max;
}

```

时间复杂度：如果最大的数是 m ，个数是 n ，那么就是 $O(m * n)$ 。

空间复杂度： $O(1)$ 。

经过他人提醒，这个解法现在 AC 不了了，会报超时，但还是放在这里吧。下边讲一下，leetcode [solution](#) 提供的 4 个算法。

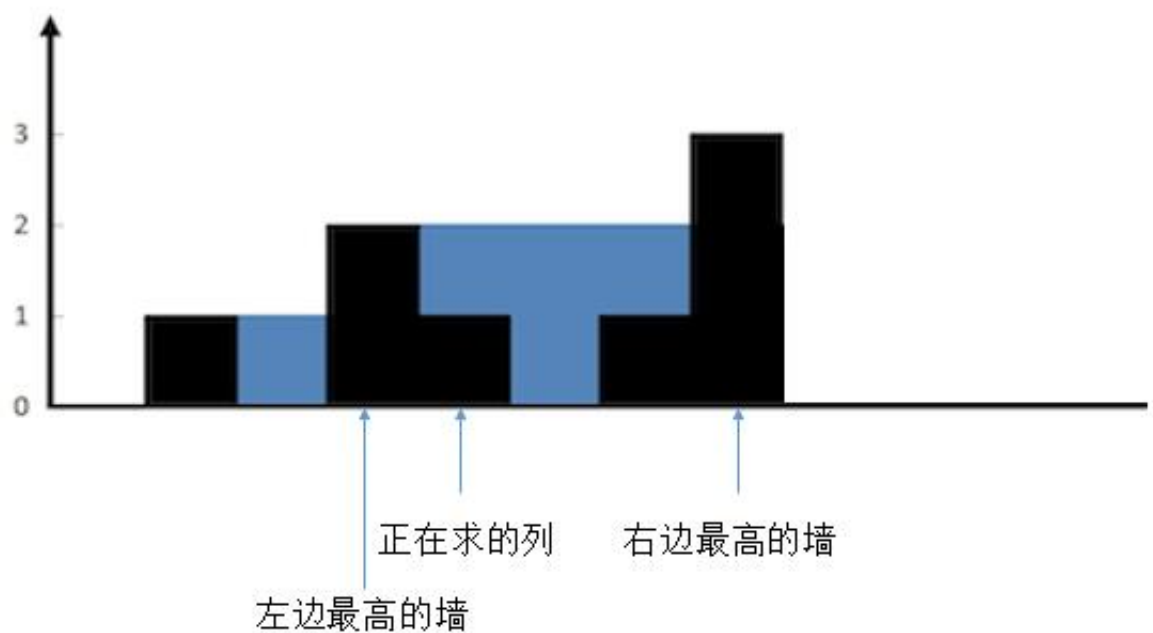
解法二 按列求

求每一列的水，我们只需要关注当前列，以及左边最高的墙，右边最高的墙就够了。

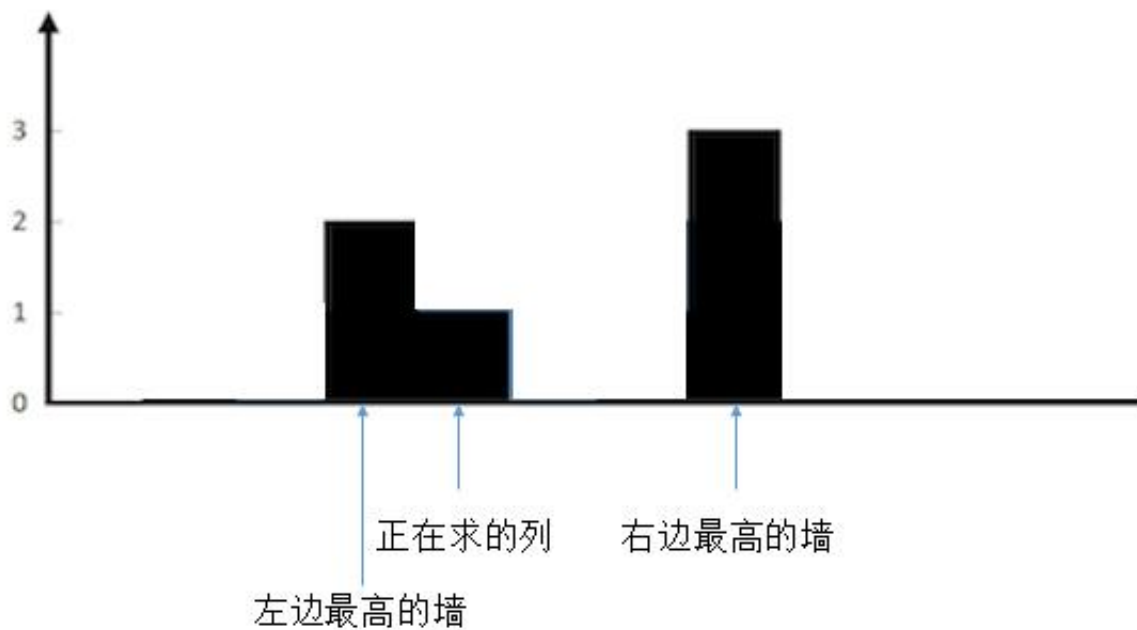
装水的多少，当然根据木桶效应，我们只需要看左边最高的墙和右边最高的墙中较矮的一个就够了。

所以，根据较矮的那个墙和当前列的墙的高度可以分为三种情况。

- 较矮的墙的高度大于当前列的墙的高度



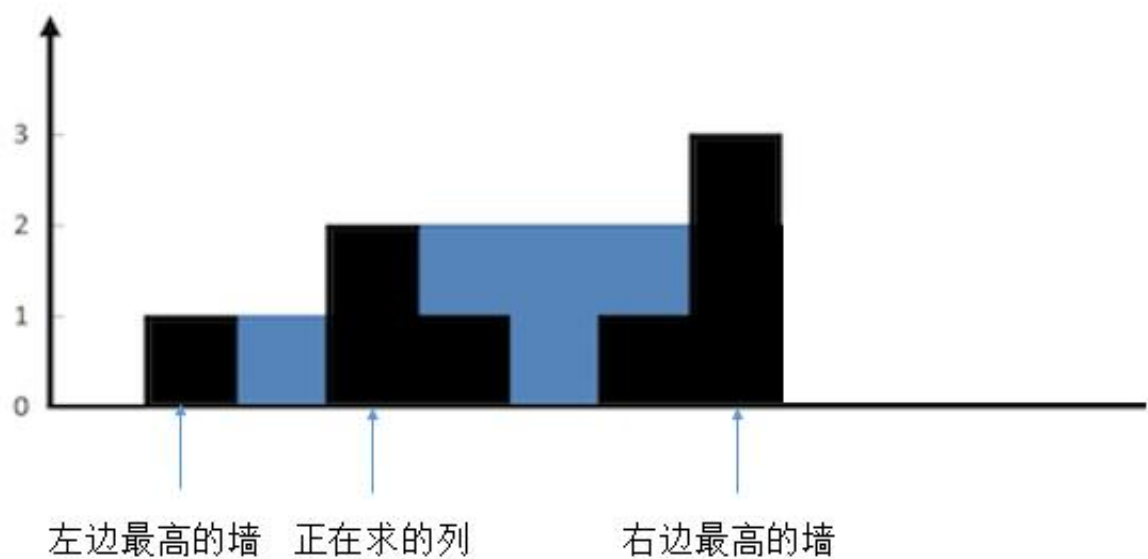
把正在求的列左边最高的墙和右边最高的墙确定后，然后为了方便理解，我们把无关的墙去掉。



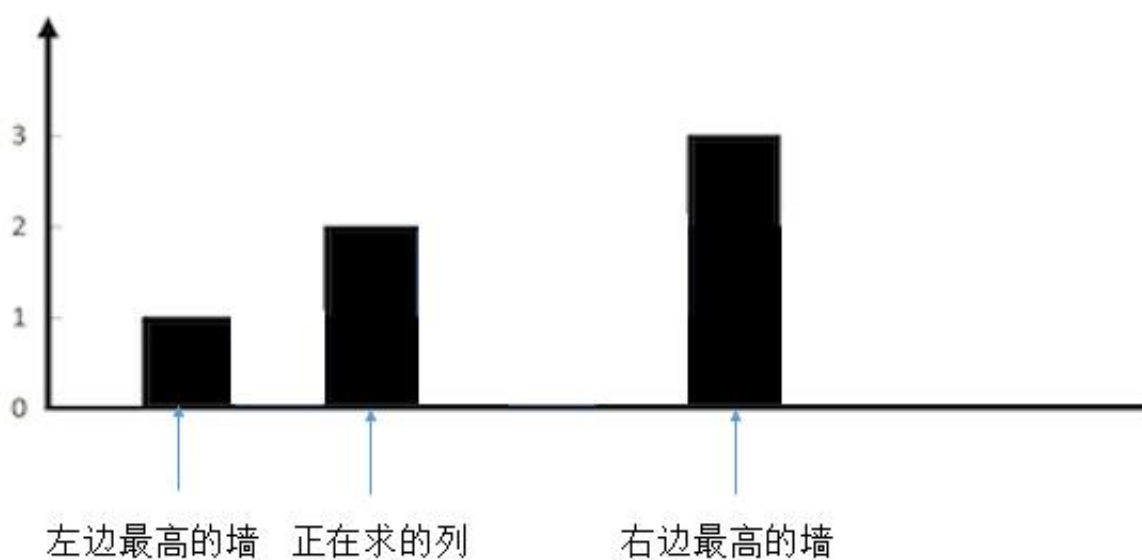
这样就很清楚了，现在想象一下，往两边最高的墙之间注水。正在求的列会有多少水？

很明显，较矮的一边，也就是左边的墙的高度，减去当前列的高度就可以了，也就是 $2 - 1 = 1$ ，可以存一个单位的水。

- 较矮的墙的高度小于当前列的墙的高度



同样的，我们把其他无关的列去掉。

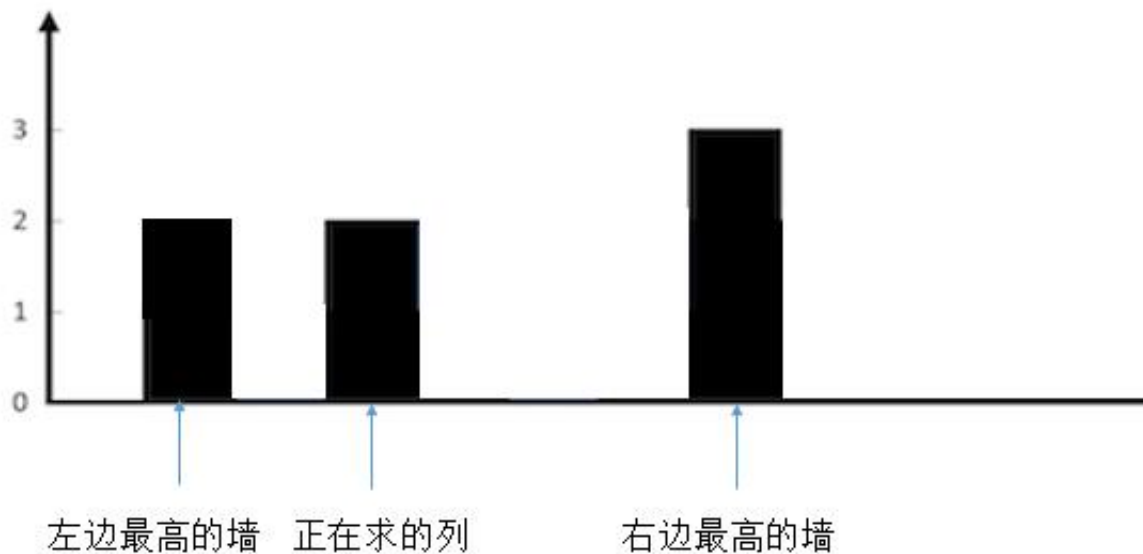


想象下，往两边最高的墙之间注水。正在求的列会有多少水？

正在求的列不会有水，因为它大于了两边较矮的墙。

- 较矮的墙的高度等于当前列的墙的高度。

和上一种情况是一样的，不会有水。



明白了这三种情况，程序就很好写了，遍历每一列，然后分别求出这一列两边最高的墙。找出较矮的一端，和当前列的高度比较，结果就是上边的三种情况。

```
public int trap(int[] height) {  
    int sum = 0;  
    //最两端的列不用考虑，因为一定不会有水。所以下标从 1 到 length - 2  
    for (int i = 1; i < height.length - 1; i++) {  
        int max_left = 0;  
        //找出左边最高  
        for (int j = i - 1; j >= 0; j--) {  
            if (height[j] > max_left) {  
                max_left = height[j];  
            }  
        }  
        int max_right = 0;  
        //找出右边最高  
        for (int j = i + 1; j < height.length; j++) {  
            if (height[j] > max_right) {  
                max_right = height[j];  
            }  
        }  
        //找出两端较小的  
        int min = Math.min(max_left, max_right);  
        //只有较小的一段大于当前列的高度才会有水，其他情况不会有水  
        if (min > height[i]) {  
            sum = sum + (min - height[i]);  
        }  
    }  
    return sum;  
}
```

```
}
```

时间复杂度： $O(n^2)$ ，遍历每一列需要 n ，找出左边最高和右边最高的墙加起来刚好又是一个 n ，所以是 n^2 。

空间复杂度： $O(1)$ 。

解法三 动态规划

我们注意到，解法二中。对于每一列，我们求它左边最高的墙和右边最高的墙，都是重新遍历一遍所有高度，这里我们可以优化一下。

首先用两个数组， $\text{max_left}[i]$ 代表第 i 列左边最高的墙的高度， $\text{max_right}[i]$ 代表第 i 列右边最高的墙的高度。（一定要注意下，第 i 列左（右）边最高的墙，是不包括自身的，和 leetcode 上边的讲的有些不同）

对于 max_left 我们其实可以这样求。

$\text{max_left}[i] = \text{Max}(\text{max_left}[i - 1], \text{height}[i - 1])$ 。它前边的墙的左边的最高高度和它前边的墙的高度选一个较大的，就是当前列左边最高的墙了。

对于 max_right 我们可以这样求。

$\text{max_right}[i] = \text{Max}(\text{max_right}[i + 1], \text{height}[i + 1])$ 。它后边的墙的右边的最高高度和它后边的墙的高度选一个较大的，就是当前列右边最高的墙了。

这样，我们再利用解法二的算法，就不用在 for 循环里每次重新遍历一次求 max_left 和 max_right 了。

```
public int trap(int[] height) {
    int sum = 0;
    int[] max_left = new int[height.length];
    int[] max_right = new int[height.length];

    for (int i = 1; i < height.length - 1; i++) {
        max_left[i] = Math.max(max_left[i - 1], height[i - 1]);
    }
    for (int i = height.length - 2; i >= 0; i--) {
        max_right[i] = Math.max(max_right[i + 1], height[i + 1]);
    }
    for (int i = 1; i < height.length - 1; i++) {
        int min = Math.min(max_left[i], max_right[i]);
        if (min > height[i]) {
            sum = sum + (min - height[i]);
        }
    }
    return sum;
}
```

时间复杂度： $O(n)$ 。

空间复杂度： $O(n)$ ，用来保存每一列左边最高的墙和右边最高的墙。

解法四 双指针

动态规划中，我们常常可以对空间复杂度进行进一步的优化。

例如这道题中，可以看到，`max_left[i]` 和 `max_right[i]` 数组中的元素我们其实只用一次，然后就再也不用到了。所以我们可以不用数组，只用一个元素就行了。我们先改造下 `max_left`。

```
public int trap(int[] height) {
    int sum = 0;
    int max_left = 0;
    int[] max_right = new int[height.length];
    for (int i = height.length - 2; i >= 0; i--) {
        max_right[i] = Math.max(max_right[i + 1], height[i + 1]);
    }
    for (int i = 1; i < height.length - 1; i++) {
        max_left = Math.max(max_left, height[i - 1]);
        int min = Math.min(max_left, max_right[i]);
        if (min > height[i]) {
            sum = sum + (min - height[i]);
        }
    }
    return sum;
}
```

我们成功将 `max_left` 数组去掉了。但是会发现我们不能同时把 `max_right` 的数组去掉，因为最后的 for 循环是从左到右遍历的，而 `max_right` 的更新是从右向左的。

所以这里要用到两个指针，`left` 和 `right`，从两个方向去遍历。

那么什么时候从左到右，什么时候从右到左呢？根据下边的代码的更新规则，我们可以知道

```
max_left = Math.max(max_left, height[i - 1]);
```

`height[left - 1]` 是可能成为 `max_left` 的变量，同理，`height[right + 1]` 是可能成为 `right_max` 的变量。

只要保证 `height[left - 1] < height[right + 1]`，那么 `max_left` 就一定小于 `max_right`。

因为 `max_left` 是由 `height[left - 1]` 更新过来的，而 `height[left - 1]` 是小于 `height[right + 1]` 的，而 `height[right + 1]` 会更新 `max_right`，所以间接的得出 `max_left` 一定小于 `max_right`。

反之，我们就从右到左更。

```
public int trap(int[] height) {
```

```

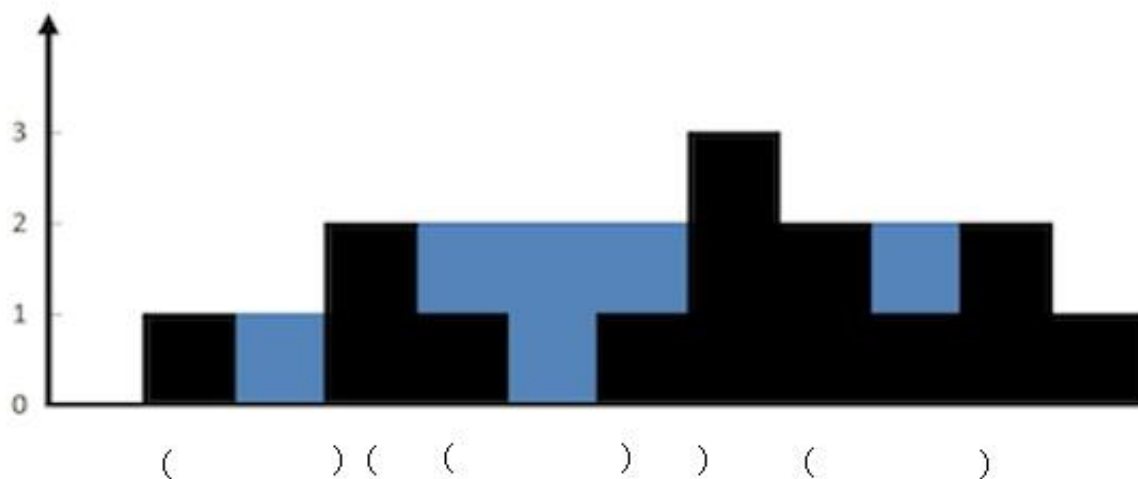
int sum = 0;
int max_left = 0;
int max_right = 0;
int left = 1;
int right = height.length - 2; // 加右指针进去
for (int i = 1; i < height.length - 1; i++) {
    //从左到右更
    if (height[left - 1] < height[right + 1]) {
        max_left = Math.max(max_left, height[left - 1]);
        int min = max_left;
        if (min > height[left]) {
            sum = sum + (min - height[left]);
        }
        left++;
    } else {
        //从右到左更
        max_right = Math.max(max_right, height[right + 1]);
        int min = max_right;
        if (min > height[right]) {
            sum = sum + (min - height[right]);
        }
        right--;
    }
}
return sum;
}

```

时间复杂度：O (n) 。

空间复杂度：O (1) 。

解法五 栈



说到栈，我们肯定会想到括号匹配了。我们仔细观察蓝色的部分，可以和括号匹配类比下。每次匹配出一对括号（找到对应的一堵墙），就计算这两堵墙中的水。

我们用栈保存每堵墙。

当遍历墙的高度的时候，如果当前高度小于栈顶的墙高度，说明这里会有积水，我们将墙的高度的下标入栈。

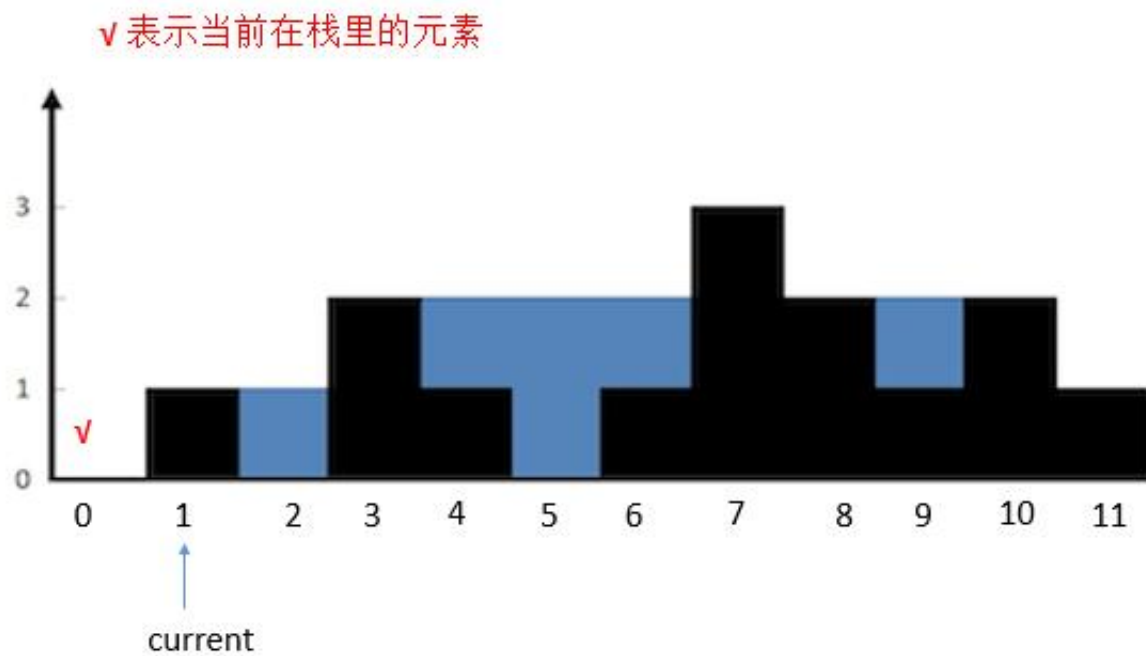
如果当前高度大于栈顶的墙的高度，说明之前的积水到这里停下，我们可以计算下有多少积水了。计算完，就把当前的墙继续入栈，作为新的积水的墙。

总体的原则就是，

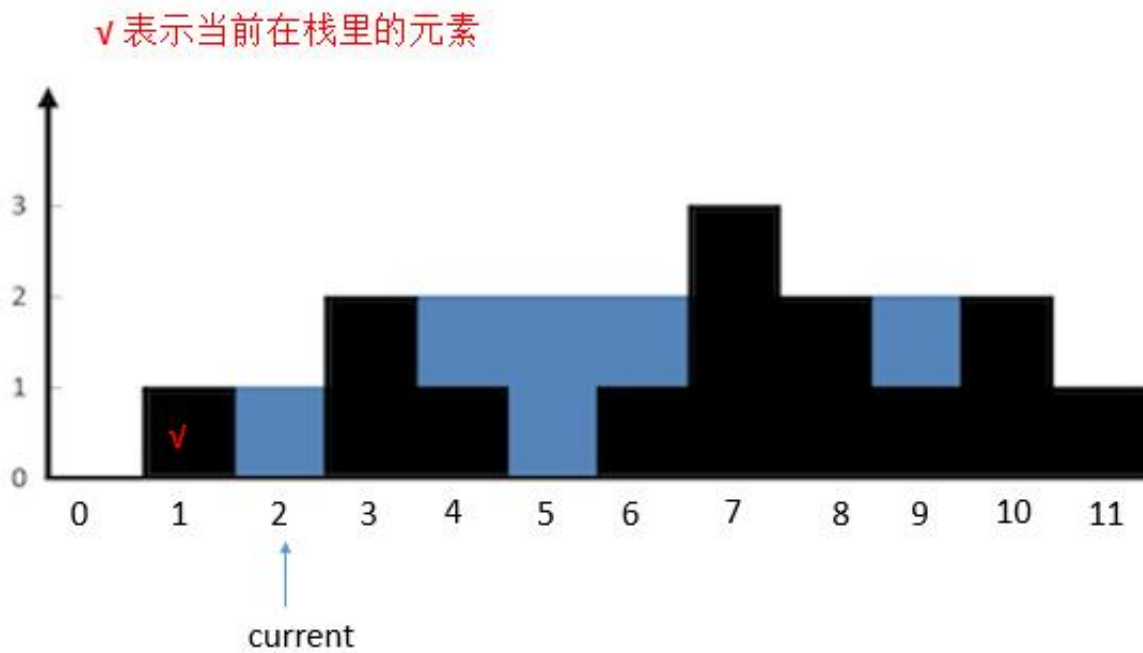
1. 当前高度小于等于栈顶高度，入栈，指针后移。
2. 当前高度大于栈顶高度，出栈，计算出当前墙和栈顶的墙之间水的多少，然后计算当前的高度和新栈的高度的关系，重复第 2 步。直到当前墙的高度不大于栈顶高度或者栈空，然后把当前墙入栈，指针后移。

我们看具体的例子。

- 首先将 `height[0]` 入栈。然后 `current` 指向的高度大于栈顶高度，所以把栈顶 `height[0]` 出栈，然后栈空了，再把 `height[1]` 入栈。`current` 后移。

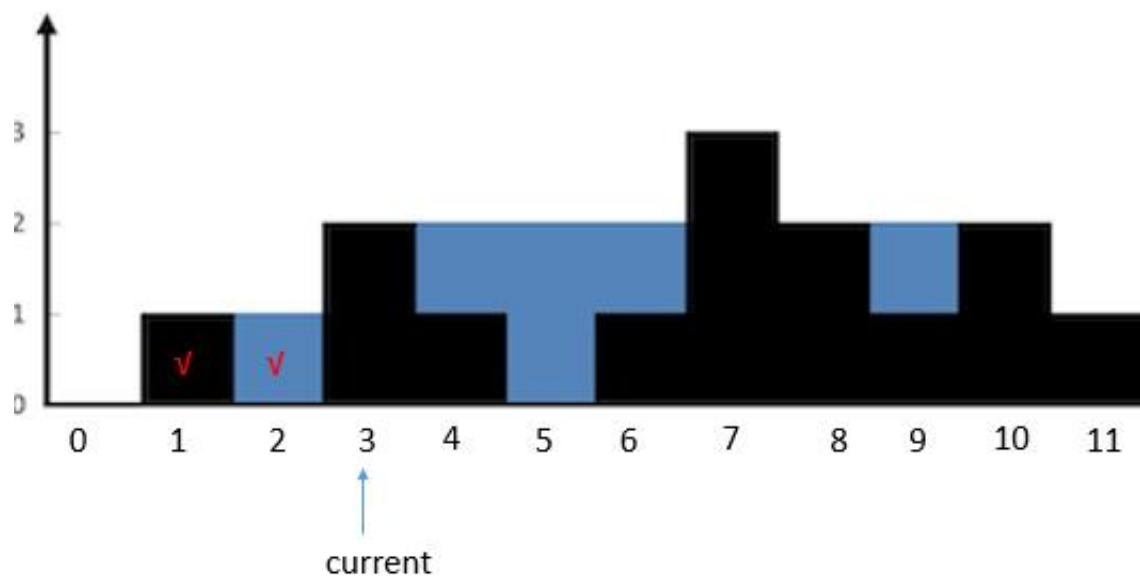


- 然后 current 指向的高度小于栈顶高度，height [2] 入栈，current 后移。



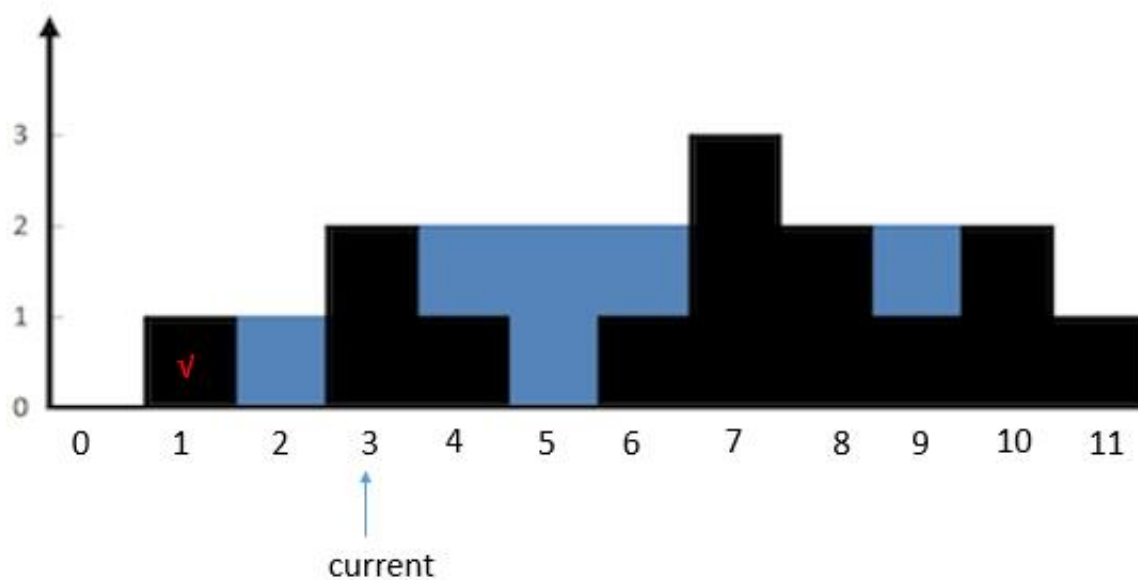
- 然后 current 指向的高度大于栈顶高度，栈顶 height [2] 出栈。计算 height [3] 和新的栈顶之间的水。计算完之后继续判断 current 和新的栈顶的关系。

√ 表示当前在栈里的元素

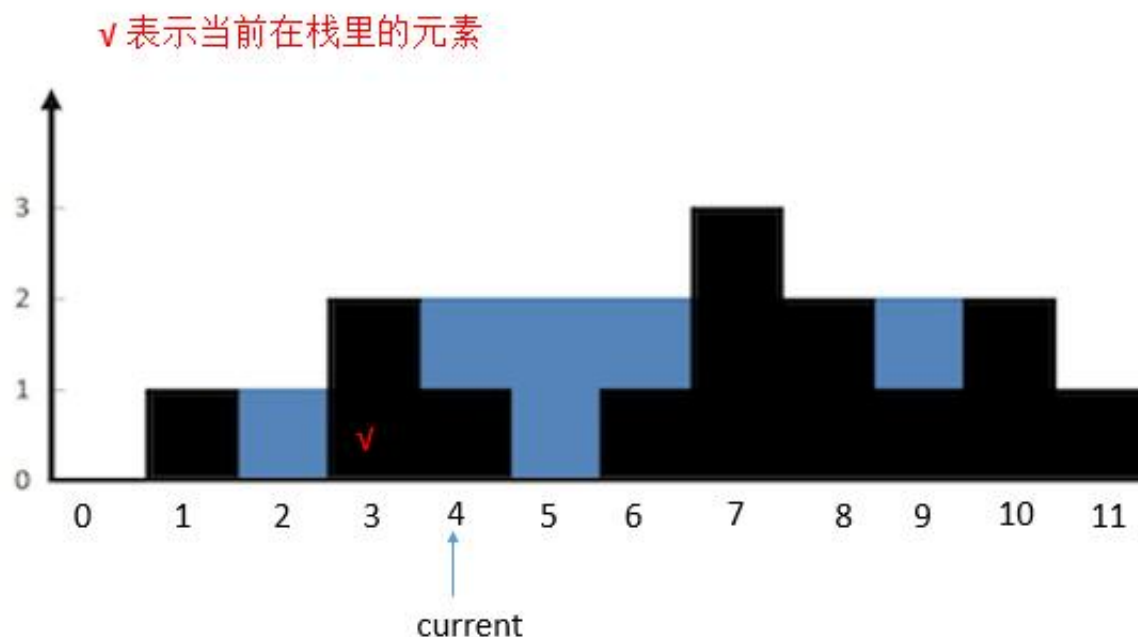


- current 指向的高度大于栈顶 height [1] 出栈，栈空。所以把 height [3] 入栈。current 后移。

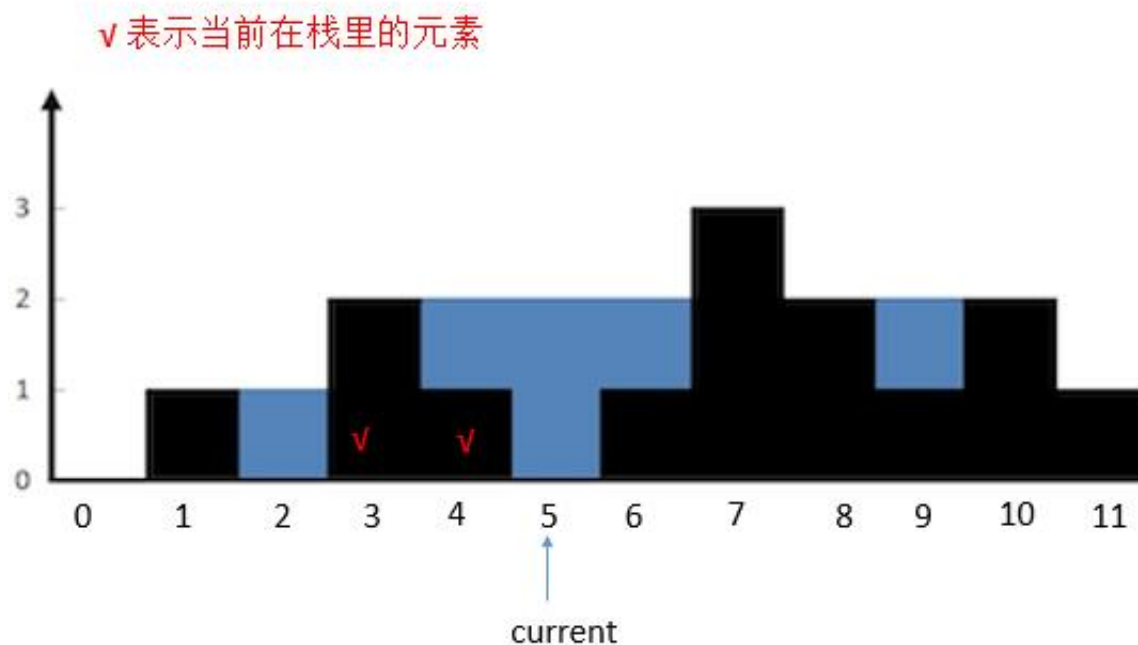
√ 表示当前在栈里的元素



- 然后 current 指向的高度小于栈顶 height [3] 的高度，height [4] 入栈。current 后移。

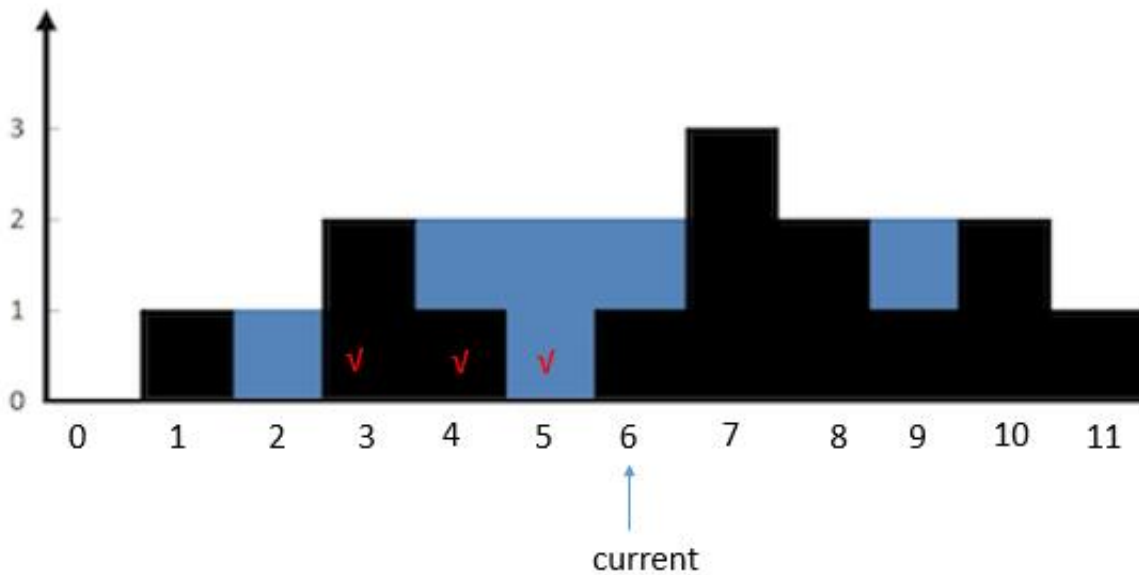


- 然后 current 指向的高度小于栈顶 height [4] 的高度，height [5] 入栈。current 后移。



- 然后 current 指向的高度大于栈顶 height [5] 的高度，将栈顶 height [5] 出栈，然后计算 current 指向的墙和新栈顶 height [4] 之间的水。计算完之后继续判断 current 的指向和新栈顶的关系。此时 height [6] 不大于栈顶 height [4]，所以将 height [6] 入栈。current 后移。

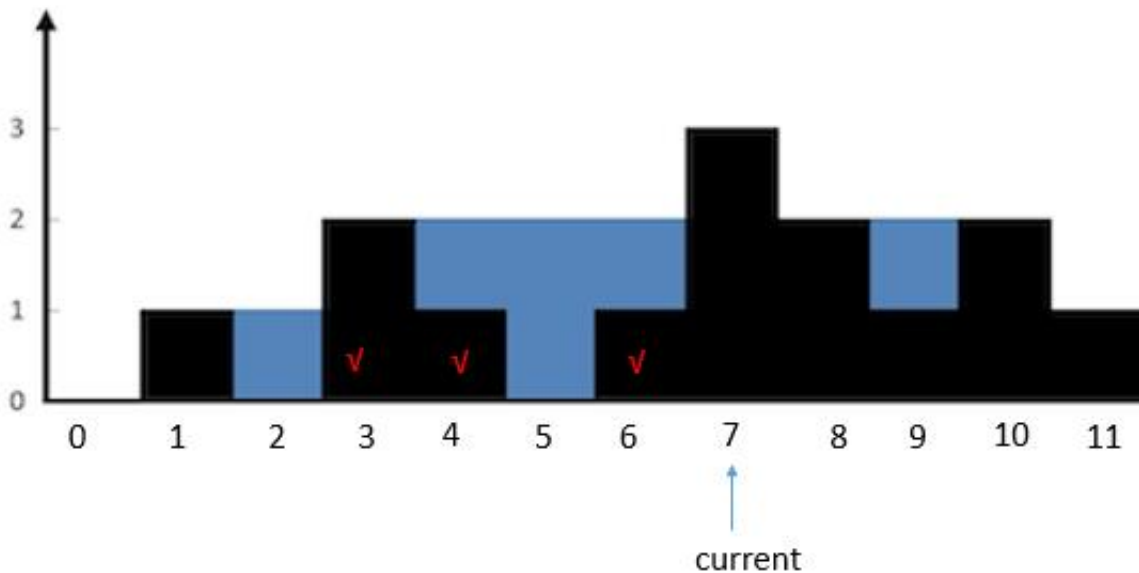
v 表示当前在栈里的元素



- 然后 current 指向的高度大于栈顶高度，将栈顶 height [6] 出栈。计算和新的栈顶 height [4] 组成两个边界中的水。然后判断 current 和新的栈顶 height [4] 的关系，依旧是大，所以把 height [4] 出栈。计算current 和 新的栈顶 height [3] 之间的水。然后判断 current 和新的栈顶 height [3] 的关系，依旧是大，所以把 height [3] 出栈，栈空。将 current 指向的 height [7] 入栈。current 后移。

其实不停的出栈，可以看做是在找与 7 匹配的墙，也就是 3。

v 表示当前在栈里的元素



而对于计算 current 指向墙和新的栈顶之间的水，根据图的关系，我们可以直接把这两个墙当做之前解法三的 max_left 和 max_right，然后之前弹出的栈顶当做每次遍历的 height [i]。水量就是 $\text{Min}(\text{max_left}, \text{max_right}) - \text{height}[i]$ ，只不过这里需要乘上两个墙之间的距离。可以看下代码继续理解下。

```

public int trap6(int[] height) {
    int sum = 0;
    Stack<Integer> stack = new Stack<>();
    int current = 0;
    while (current < height.length) {
        //如果栈不空并且当前指向的高度大于栈顶高度就一直循环
        while (!stack.empty() && height[current] > height[stack.peek()]) {
            int h = height[stack.peek()]; //取出要出栈的元素
            stack.pop(); //出栈
            if (stack.empty()) { // 栈空就出去
                break;
            }
            int distance = current - stack.peek() - 1; //两堵墙之前的距离。
            int min = Math.min(height[stack.peek()], height[current]);
            sum = sum + distance * (min - h);
        }
        stack.push(current); //当前指向的墙入栈
        current++; //指针后移
    }
    return sum;
}

```

时间复杂度：虽然 while 循环里套了一个 while 循环，但是考虑到每个元素最多访问两次，入栈一次和出栈一次，所以时间复杂度是 $O(n)$ 。

空间复杂度： $O(n)$ 。栈的空间。

总

解法二到解法三，利用动态规划，空间换时间，解法三到解法四，优化动态规划的空间，这一系列下来，让人心旷神怡。

43、题目描述（中等难度）

Given two non-negative integers `num1` and `num2` represented as strings, return the product of `num1` and `num2`, also represented as a string.

Example 1:

Input: `num1 = "2"`, `num2 = "3"`
Output: `"6"`

Example 2:

Input: `num1 = "123"`, `num2 = "456"`
Output: `"56088"`

Note:

1. The length of both `num1` and `num2` is < 110 .
2. Both `num1` and `num2` contain only digits `0-9`.
3. Both `num1` and `num2` do not contain any leading zero, except the number `0` itself.
4. You **must not use any built-in BigInteger library** or **convert the inputs to integer** directly.

Seen this question in a real interview before?



就是两个数相乘，输出结果，只不过数字很大很大，都是用 String 存储的。也就是传说中的大数相乘。

解法一

我们就模仿我们在纸上做乘法的过程写出一个算法。

$$\begin{array}{r} 1123 \\ \times 245 \\ \hline 5615 \\ 44920 \\ 224600 \\ \hline 275135 \end{array}$$

个位乘个位，得出一个数，然后个位乘十位，全部乘完以后，就再用十位乘以各个位。然后百位乘以各个位，最后将每次得出的数相加。十位的结果要补 1 个 0，百位的结果要补两个 0。相加的话我们可以直接用之前的[大数相加](#)。直接看代码吧。

```
public String multiply(String num1, String num2) {  
    if (num1.equals("0") || num2.equals("0")) {  
        return "0";  
    }  
}
```

```

String ans = "0";
int index = 0; //记录当前是哪一位, 便于后边补 0
for (int i = num2.length() - 1; i >= 0; i--) {
    int carry = 0; //保存进位
    String ans_part = ""; //直接用字符串保存每位乘出来的数
    int m = num2.charAt(i) - '0';
    //乘上每一位
    for (int j = num1.length() - 1; j >= 0; j--) {
        int n = num1.charAt(j) - '0';
        int mul = m * n + carry;
        ans_part = mul % 10 + "" + ans_part;
        carry = mul / 10;
    }
    if (carry > 0) {
        ans_part = carry + "" + ans_part;
    }
    //补 0
    for (int k = 0; k < index; k++) {
        ans_part = ans_part + "0";
    }
    index++;
    //和之前的结果相加
    ans = sumString(ans, ans_part);
}
return ans;
}
//大数相加
private String sumString(String num1, String num2) {
    int carry = 0;
    int num1_index = num1.length() - 1;
    int num2_index = num2.length() - 1;
    String ans = "";
    while (num1_index >= 0 || num2_index >= 0) {
        int n1 = num1_index >= 0 ? num1.charAt(num1_index) - '0' : 0;
        int n2 = num2_index >= 0 ? num2.charAt(num2_index) - '0' : 0;
        int sum = n1 + n2 + carry;
        carry = sum / 10;
        ans = sum % 10 + "" + ans;
        num1_index--;
        num2_index--;
    }
    if (carry > 0) {
        ans = carry + "" + ans;
    }
    return ans;
}
}

```

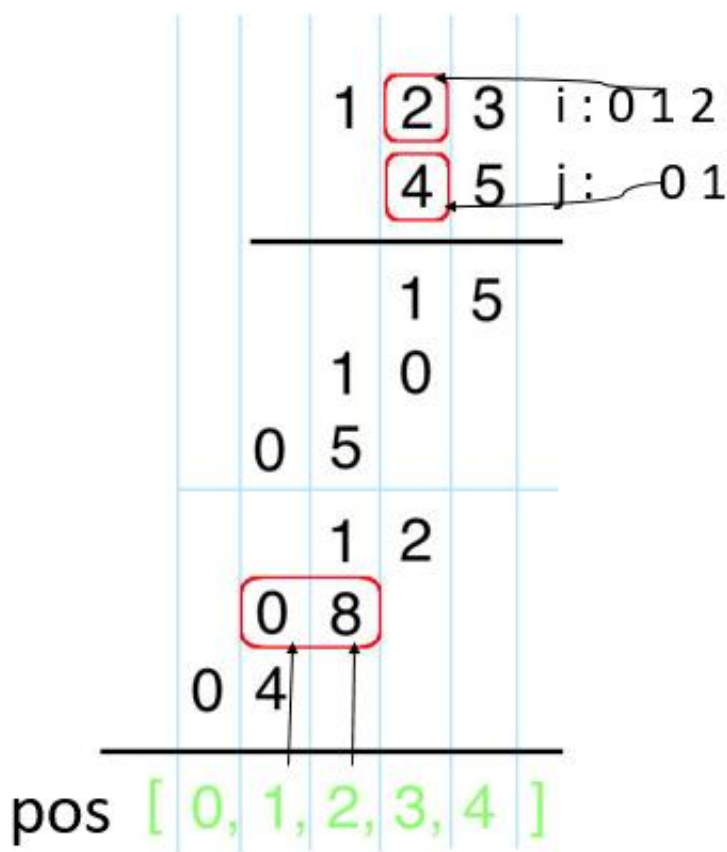
时间复杂度： $O(m * n)$ 。m, n 是两个字符串的长度。

空间复杂度： $O(1)$ 。

解法二

参考[这里](#)。

上边的解法非常简单粗暴，但是不够优雅。我们看一下从未见过的一种竖式计算。



我们把进位先不算，写到对应的位置。最后再统一更新 pos 中的每一位。

而对于运算中的每个结果，可以观察出一个结论。

num1 的第 i 位乘上 num2 的第 j 位，结果会分别对应 pos 的第 i + j 位和第 i + j + 1 位。

例如图中的红色部分，num1 的第 1 位乘上 num2 的第 0 位，结果就对应 pos 的第 $1 + 0 = 1$ 和 $1 + 0 + 1 = 2$ 位。

有了这一点，我们就可以遍历求出每一个结果，然后更新 pos 上的值就够了。

```
public String multiply(String num1, String num2) {  
    if (num1.equals("0") || num2.equals("0")) {  
        return "0";  
    }  
}
```

```

int n1 = num1.length();
int n2 = num2.length();
int[] pos = new int[n1 + n2]; //保存最后的结果
for (int i = n1 - 1; i >= 0; i--) {
    for (int j = n2 - 1; j >= 0; j--) {
        //相乘的结果
        int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0');
        //加上 pos[i+j+1] 之前已经累加的结果
        int sum = mul + pos[i + j + 1];
        //更新 pos[i + j]
        pos[i + j] += sum / 10;
        //更新 pos[i + j + 1]
        pos[i + j + 1] = sum % 10;
    }
}
StringBuilder sb = new StringBuilder();
for (int i = 0; i < pos.length; i++) {
    //判断最高位是不是 0
    if (i == 0 && pos[i] == 0) {
        continue;
    }
    sb.append(pos[i]);
}
return sb.toString();
}

```

时间复杂度： $O(m * n)$ 。m, n 是两个字符串的长度。

空间复杂度： $O(m + n)$ 。m, n 是两个字符串的长度。

总

如果按普通的思路写，这道题也不难。新的竖式的计算，让人眼前一亮，代码优雅了很多。

44、题目描述（困难难度）

Given an input string (`s`) and a pattern (`p`), implement wildcard pattern matching with support for `'?'` and `'*'`.

```
'?' Matches any single character.  
'*' Matches any sequence of characters (including the empty sequence).
```

The matching should cover the **entire** input string (not partial).

Note:

- `s` could be empty and contains only lowercase letters `a-z`.
- `p` could be empty and contains only lowercase letters `a-z`, and characters like `?` or `*`.

Example 1:

```
Input:  
s = "aa"  
p = "a"  
Output: false  
Explanation: "a" does not match the entire string "aa".
```

Example 2:

```
Input:  
s = "aa"  
p = "*"  
Output: true  
Explanation: '*' matches any sequence.
```

Example 3:

```
Input:  
s = "cb"  
p = "?a"  
Output: false  
Explanation: '?' matches 'c', but the second letter is 'a', which does not match 'b'.
```

Example 4:

```
Input:  
s = "adceb"  
p = "*a*b"  
Output: true  
Explanation: The first '*' matches the empty sequence, while the second '*' matches the substring "dce".
```

Example 5:

```
Input:  
s = "acdcab"  
p = "a*c?b"  
Output: false
```

字符串匹配，`?` 匹配单个任意字符，`*` 匹配任意长度字符串，包括空串。和[第 10 题](#)有些类似。

解法一 动态规划

直接按照之前[第 10 题](#)，修改一下就可以了。

同样是用 `dp[i][j]` 表示所有的情况，然后一层一层的根据递推关系求出来。


```

public boolean isMatch(String text, String pattern) {
    // 多一维的空间, 因为求 dp[len - 1][j] 的时候需要知道 dp[len][j] 的情况,
    // 多一维的话, 就可以把 对 dp[len - 1][j] 也写进循环了
    boolean[][] dp = new boolean[text.length() + 1][pattern.length() + 1];
    // dp[len][len] 代表两个空串是否匹配了, "" 和 "", 当然是 true 了。
    dp[text.length()][pattern.length()] = true;

    // 从 len 开始减少
    for (int i = text.length(); i >= 0; i--) {
        for (int j = pattern.length(); j >= 0; j--) {
            // dp[text.length()][pattern.length()] 已经进行了初始化
            if (i == text.length() && j == pattern.length())
                continue;
            //相比之前增加了判断是否等于 *
            boolean first_match = (i < text.length() && j < pattern.length() &&
                (pattern.charAt(j) == text.charAt(i) || pattern.charAt(j) == '?' ||
                pattern.charAt(j) == '*'));
            if (j < pattern.length() && pattern.charAt(j) == '*') {
                //将 * 跳过 和将字符匹配一个并且 pattern 不变两种情况
                dp[i][j] = dp[i][j + 1] || first_match && dp[i + 1][j];
            } else {
                dp[i][j] = first_match && dp[i + 1][j + 1];
            }
        }
    }
    return dp[0][0];
}

```

时间复杂度: text 长度是 T, pattern 长度是 P, 那么就是 $O(TP)$ 。

空间复杂度: $O(TP)$ 。

同样的, 和[第10题](#)一样, 可以优化空间复杂度。

```

public boolean isMatch(String text, String pattern) {
    // 多一维的空间, 因为求 dp[len - 1][j] 的时候需要知道 dp[len][j] 的情况,
    // 多一维的话, 就可以把 对 dp[len - 1][j] 也写进循环了
    boolean[][] dp = new boolean[2][pattern.length() + 1];
    dp[text.length() % 2][pattern.length()] = true;

    // 从 len 开始减少
    for (int i = text.length(); i >= 0; i--) {
        for (int j = pattern.length(); j >= 0; j--) {
            if (i == text.length() && j == pattern.length())
                continue;

```

```

        boolean first_match = (i < text.length() && j < pattern.length() &&
(pattern.charAt(j) == text.charAt(i)

|| pattern.charAt(j) == '?' || pattern.charAt(j) == '*'));
        if (j < pattern.length() && pattern.charAt(j) == '*') {
            dp[i % 2][j] = dp[i % 2][j + 1] || first_match && dp[(i + 1) % 2]
[j];
        } else {
            dp[i % 2][j] = first_match && dp[(i + 1) % 2][j + 1];
        }
    }
}
return dp[0][0];
}

```

时间复杂度：text 长度是 T，pattern 长度是 P，那么就是 O (TP) 。

空间复杂度：O (P) 。

解法二 迭代

参考[这里](#)，也比较好理解，利用两个指针进行遍历。

```

boolean isMatch(String str, String pattern) {
    int s = 0, p = 0, match = 0, starIdx = -1;
    //遍历整个字符串
    while (s < str.length()){
        // 一对一匹配，两指针同时后移。
        if (p < pattern.length() && (pattern.charAt(p) == '?' || str.charAt(s)
== pattern.charAt(p))){
            s++;
            p++;
        }
        // 碰到 *，假设它匹配空串，并且用 starIdx 记录 * 的位置，记录当前字符串的位置，p 后
移
        else if (p < pattern.length() && pattern.charAt(p) == '*'){
            starIdx = p;
            match = s;
            p++;
        }
        // 当前字符不匹配，并且也没有 *，回退
        // p 回到 * 的下一个位置
        // match 更新到下一个位置
        // s 回到更新后的 match
        // 这步代表用 * 匹配了一个字符
        else if (starIdx != -1){

```

```

        p = starIdx + 1;
        match++;
        s = match;
    }
    //字符不匹配, 也没有 *, 返回 false
    else return false;
}

//将末尾多余的 * 直接匹配空串 例如 text = ab, pattern = a*****
while (p < pattern.length() && pattern.charAt(p) == '*')
    p++;

return p == pattern.length();
}

```

时间复杂度：如果 str 长度是 T，pattern 长度是 P，虽然只有一个 while 循环，但是 s 并不是每次都加 1，所以最坏的时候时间复杂度会达到 $O(TP)$ ，例如 str = "bbbbbbbbbb"，pattern = "*bbbb"。每次 pattern 到最后时，又会重新开始到开头。

空间复杂度： $O(1)$ 。

递归

在[第10题](#)中还有递归的解法，但这题中如果按照第 10 题的递归的思路去解决，会导致超时，目前没想到怎么在第 10 题的基础上去改，有好的想法大家可以和我交流。

如果非要用递归的话，可以按照动态规划那个思路，先压栈，然后出栈过程其实就是动态规划那样了。所以其实不如直接动态规划。

总

动态规划的应用，理清递推的公式就可以。另外迭代的方法，也让人眼前一亮。

45、题目描述（困难难度）

Given an array of non-negative integers, you are initially positioned at the first index of the array.

Each element in the array represents your maximum jump length at that position.

Your goal is to reach the last index in the minimum number of jumps.

Example:

Input: [2,3,1,1,4]

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2.

Jump 1 step from index 0 to 1, then 3 steps to the last index.

Note:

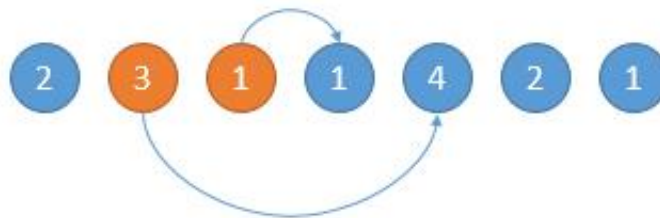
You can assume that you can always reach the last index.

从数组的第 0 个位置开始跳，跳的距离小于等于数组上对应的数。求出跳到最后个位置需要的最短步数。比如上图中的第 0 个位置是 2，那么可以跳 1 个距离，或者 2 个距离，我们选择跳 1 个距离，就跳到了第 1 个位置，也就是 3 上。然后我们可以跳 1，2，3 个距离，我们选择跳 3 个距离，就直接到最后了。所以总共需要 2 步。

解法一 顺藤摸瓜

参考[这里](#)，leetcode 讨论里，大部分都是这个思路，贪婪算法，我们每次在可跳范围内选择可以使得跳的更远的位置。

如下图，开始的位置是 2，可跳的范围是橙色的。然后因为 3 可以跳的更远，所以跳到 3 的位置。



如下图，然后现在的位置就是 3 了，能跳的范围是橙色的，然后因为 4 可以跳的更远，所以下次跳到 4 的位置。



写代码的话，我们用 end 表示当前能跳的边界，对于上边第一个图的橙色 1，第二个图中就是橙色的 4，遍历数组的时候，到了边界，我们就重新更新新的边界。

```
public int jump(int[] nums) {
```

```

int end = 0;
int maxPosition = 0;
int steps = 0;
for(int i = 0; i < nums.length - 1; i++){
    //找能跳的最远的
    maxPosition = Math.max(maxPosition, nums[i] + i);
    if( i == end){ //遇到边界，就更新边界，并且步数加一
        end = maxPosition;
        steps++;
    }
}
return steps;
}

```

时间复杂度：O (n) 。

空间复杂度：O (1) 。

这里要注意一个细节，就是 for 循环中， $i < \text{nums.length} - 1$ ，少了末尾。因为开始的时候边界是第 0 个位置，steps 已经加 1 了。如下图，如果最后一步刚好跳到了末尾，此时 steps 其实不用加 1 了。如果是 $i < \text{nums.length}$ ，i 遍历到最后的时候，会进入 if 语句中，steps 会多加 1 。



解法二 顺瓜摸藤

我们知道最终要到达最后一个位置，然后我们找前一个位置，遍历数组，找到能到达它的位置，离它最远的就是要找的位置。然后继续找上上个位置，最后到了第 0 个位置就结束了。

至于离它最远的位置，其实我们从左到右遍历数组，第一个满足的位置就是我们要找的。

```

public int jump(int[] nums) {
    int position = nums.length - 1; //要找的位置
    int steps = 0;
    while (position != 0) { //是否到了第 0 个位置
        for (int i = 0; i < position; i++) {
            if (nums[i] >= position - i) {
                position = i; //更新要找的位置
                steps++;
                break;
            }
        }
    }
}

```

```
return steps;  
}
```

时间复杂度： $O(n^2)$ ，因为最坏的情况比如 1 1 1 1 1 1，position 会从 5 更新到 0，并且每次更新都会经历一个 for 循环。

空间复杂度： $O(1)$ 。

这种想法看起来更简单了，为什么奏效呢？我们可以这样想。



从左到右跳的话，2 -> 3 -> 4 -> 1。

从右到左的话，我们找能跳到 1 的最左边的位置，我们找的只能是 4 或者是 4 左边的。

找到 4 的话，不用说，刚好完美。

如果是中间范围 3 和 4 之间的第 2 个 1 变成了 3，那么这个位置也可以跳到末尾的 1，按我们的算法我们就找到了这个 3，也就是 4 左边的位置。但其实并不影响我们的 steps，因为这个数字是 3 到 4 中间范围的数，左边界 3 也可以到这个数，所以下次找的话，会找到边界 3，或者边界 3 左边的数。会不会直接找到上个边界 2 呢？不会的，如果找到了上一个边界 2，那么意味着从 2 直接跳到 3 和 4 之间的那个数，再从这个数跳到末尾就只需 2 步了，但是其实是需要 3 步的。

总

刷这么多题，第一次遇到了贪心算法，每次找局部最优，最后达到全局最优，完美！

46、题目描述（中等难度）

Given a collection of **distinct** integers, return all possible permutations.

Example:

```
Input: [1,2,3]
Output:
[
  [1,2,3],
  [1,3,2],
  [2,1,3],
  [2,3,1],
  [3,1,2],
  [3,2,1]
]
```

描述的很简单，就是给定几个数，然后输出他们所有排列的可能。

解法一 插入

这是自己开始想到的一个方法，考虑的思路是，先考虑小问题怎么解决，然后再利用小问题去解决大问题。没错，就是递归的思路。比如说，

如果只有 1 个数字 [1]，那么很简单，直接返回 [[1]] 就 OK 了。

如果加了 1 个数字 2，[1 2] 该怎么办呢？我们只需要在上边的情况里，在 1 的空隙，也就是左边右边插入 2 就够了。变成 [[2 1], [1 2]]。

如果再加 1 个数字 3，[1 2 3] 该怎么办呢？同样的，我们只需要在上边的所有情况里的空隙里插入数字 3 就行啦。例如 [2 1] 在左边，中间，右边插入 3，变成 **3** 2 1，2 **3** 1，2 1 **3**。同理，1 2 在左边，中间，右边插入 3，变成 **3** 1 2，1 **3** 2，1 2 **3**，所以最后的结果就是 [[3 2 1], [2 3 1], [2 1 3], [3 1 2], [1 3 2], [1 2 3]]。

如果再加数字也是同样的道理，只需要在之前的情况里，数字的空隙插入新的数字就够了。

思路有了，直接看代码吧。

```
public List<List<Integer>> permute(int[] nums) {
    return permute_end(nums, nums.length-1);
}
// end 表示当前新增的数字的位置
private List<List<Integer>> permute_end(int[] nums, int end) {
    // 只有一个数字的时候
    if(end == 0){
        List<List<Integer>> all = new ArrayList<>();
        List<Integer> temp = new ArrayList<>();
        temp.add(nums[0]);
        all.add(temp);
    }
}
```

```

        return all;
    }
    //得到上次所有的结果
    List<List<Integer>> all_end = permute_end(nums,end-1);
    int current_size = all_end.size();
    //遍历每一种情况
    for (int j = 0; j < current_size; j++) {
        //在数字的缝隙插入新的数字
        for (int k = 0; k <= end; k++) {
            List<Integer> temp = new ArrayList<>(all_end.get(j));
            temp.add(k, nums[end]);
            //添加到结果中
            all_end.add(temp);
        };

    }
    //由于 all_end 此时既保存了之前的结果，和添加完的结果，所以把之前的结果要删除
    for (int j = 0; j < current_size; j++) {
        all_end.remove(0);
    }
    return all_end;
}

```

既然有递归的过程，我们也可以直接改成迭代的，可以把递归开始不停压栈的过程省略了。

```

public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> all = new ArrayList<>();
    all.add(new ArrayList<>());
    //在上边的基础上只加上最外层的 for 循环就够了，代表每次新添加的数字
    for (int i = 0; i < nums.length; i++) {
        int current_size = all.size();
        for (int j = 0; j < current_size; j++) {
            for (int k = 0; k <= i; k++) {
                List<Integer> temp = new ArrayList<>(all.get(j));
                temp.add(k, nums[i]);
                all.add(temp);
            }
        }
        for (int j = 0; j < current_size; j++) {
            all.remove(0);
        }
    }
    return all;
}

```


时间复杂度，如果只分析代码的话挺复杂的。如果从最后的结果来说，应该是 $n!$ 个结果，所以时间复杂度应该是 $O(n!)$ 。

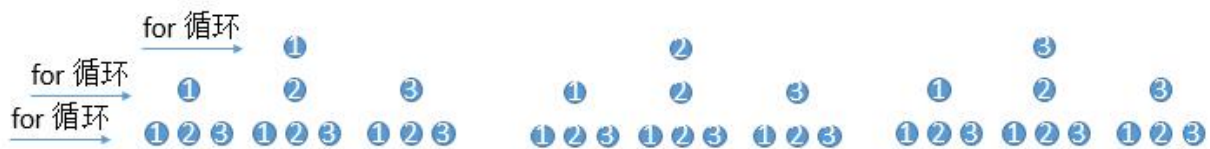
空间复杂度： $O(1)$ 。

解法二 回溯

这个开始没想到，参考[这里](#)。

其实也算是蛮典型的回溯，利用递归每次向 temp 里添加一个数字，数字添加够以后再回来进行回溯，再向后添加新的解。

可以理解成一层一层的添加，每一层都是一个 for 循环。



每调用一层就进入一个 for 循环，相当于列出了所有解，然后挑选了我们需要的。其实本质上就是深度优先遍历 DFS。

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> list = new ArrayList<>();
    backtrack(list, new ArrayList<>(), nums);
    return list;
}

private void backtrack(List<List<Integer>> list, List<Integer> tempList, int []
nums){
    if(tempList.size() == nums.length){
        list.add(new ArrayList<>(tempList));
    } else{
        for(int i = 0; i < nums.length; i++){
            if(tempList.contains(nums[i])) continue; // 已经存在的元素，跳过
            tempList.add(nums[i]); //将当前元素加入
            backtrack(list, tempList, nums); //向后继续添加
            tempList.remove(tempList.size() - 1); //将 tempList 刚添加的元素，去掉，尝试
            新的元素
        }
    }
}
```

时间复杂度：

空间复杂度：

解法三 交换

参考[这里](#)。

这个想法就很 cool 了，之前第一个解法的递归，有点儿动态规划的意思，把 1 个数字的解，2 个数字的解，3 个数字的解，一环套一环的求了出来。

假设有一个函数，可以实现题目的要求，即产生 nums 的所有的组合，并且加入到 all 数组中。不过它多了一个参数，begin，即只指定从 nums [begin] 开始的数字，前边的数字固定不变。

```
upset(int[] nums, int begin, List<List<Integer>> all)
```

如果有这样的函数，那么一切就都简单了。

如果 begin 等于 nums 的长度，那么就表示 begin 前的数字都不变，也就是全部数字不变，我们只需要把它加到 all 中就行了。

```
if (begin == nums.length) {
    ArrayList<Integer> temp = new ArrayList<Integer>();
    for (int i = 0; i < nums.length; i++) {
        temp.add(nums[i]);
    }
    all.add(new ArrayList<Integer>(temp));
    return;
}
```

如果是其它的情况，我们其实只需要用一个 for 循环，把每一个数字都放到 begin 一次，然后再变化后边的数字就够了，也就是调用 upset 函数，从 begin + 1 开始的所有组合。

```
for (int i = begin; i < nums.length; i++) {
    swap(nums, i, begin);
    upset(nums, begin + 1, all);
    swap(nums, i, begin);
}
```

总体就是这样了。

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> all = new ArrayList<>();
    //从下标 0 开始的所有组合
    upset(nums, 0, all);
    return all;
}

private void upset(int[] nums, int begin, List<List<Integer>> all) {
```

```

        if (begin == nums.length) {
            ArrayList<Integer> temp = new ArrayList<Integer>();
            for (int i = 0; i < nums.length; i++) {
                temp.add(nums[i]);
            }
            all.add(new ArrayList<Integer>(temp));
            return;
        }
        for (int i = begin; i < nums.length; i++) {
            swap(nums, i, begin);
            upset(nums, begin + 1, all);
            swap(nums, i, begin);
        }
    }

    private void swap(int[] nums, int i, int begin) {
        int temp = nums[i];
        nums[i] = nums[begin];
        nums[begin] = temp;
    }
}

```

时间复杂度：

空间复杂度：

总

这道题很经典了，用动态规划，回溯，递归各实现了一遍，当然解法一强行递归了一下，和解法三相比真是相形见绌，解法三才是原汁原味的递归，简洁优雅。

47、题目描述（中等难度）

Given a collection of numbers that might contain duplicates, return all possible unique permutations.

Example:

```

Input: [1,1,2]
Output:
[
  [1,1,2],
  [1,2,1],
  [2,1,1]
]

```

和[上一道题](#)类似，不同之处就是给定的数字中会有重复的，这样的话用之前的算法会产出重复的序列。例如，[1 1]，用之前的算法，产生的结果肯定是[[1 1], [1 1]]，也就是产生了重复的序列。但我们可以从上一题的解法中进行修改从而解决这道题。

解法一 插入

这个没想到怎么在原基础上改，可以直接了当些，在它产生的结果里，对结果去重再返回。对于去重的话，一般的方法肯定就是写两个 for 循环，然后一个一个互相比对，然后找到重复的去掉。这里，我们用[39题](#)解法二中提到的一种去重的方法。

```
public List<List<Integer>> permuteUnique(int[] nums) {
    List<List<Integer>> all = new ArrayList<>();
    List<Integer> temp = new ArrayList<>();
    temp.add(nums[0]);
    all.add(temp);
    for (int i = 1; i < nums.length; i++) {
        int current_size = all.size();
        for (int j = 0; j < current_size; j++) {
            List<Integer> last = all.get(j);
            for (int k = 0; k <= i; k++) {
                if (k < i && nums[i] == last.get(k)) {
                    continue;
                }
                temp = new ArrayList<>(last);
                temp.add(k, nums[i]);
                all.add(temp);
            }
        }
        for (int j = 0; j < current_size; j++) {
            all.remove(0);
        }
    }
    return removeDuplicate(all);
}

private List<List<Integer>> removeDuplicate(List<List<Integer>> list) {
    Map<String, String> ans = new HashMap<String, String>();
    for (int i = 0; i < list.size(); i++) {
        List<Integer> l = list.get(i);
        String key = "";
        // [ 2 3 4 ] 转为 "2,3,4"
        for (int j = 0; j < l.size() - 1; j++) {
            key = key + l.get(j) + ",";
        }
        key = key + l.get(l.size() - 1);
    }
}
```

```

        ans.put(key, "");
    }
    // 根据逗号还原 List
    List<List<Integer>> ans_list = new ArrayList<List<Integer>>();
    for (String k : ans.keySet()) {
        String[] l = k.split(",");
        List<Integer> temp = new ArrayList<Integer>();
        for (int i = 0; i < l.length; i++) {
            int c = Integer.parseInt(l[i]);
            temp.add(c);
        }
        ans_list.add(temp);
    }
    return ans_list;
}

```

解法二 回溯

看下之前的算法

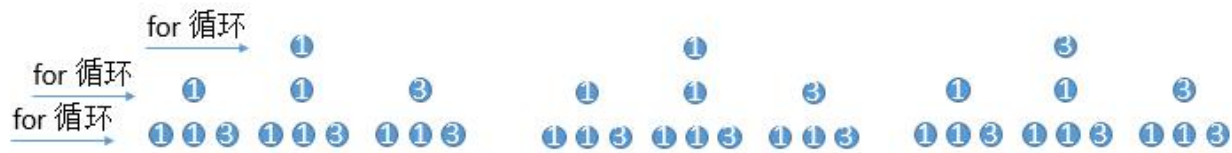
```

public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> list = new ArrayList<>();
    backtrack(list, new ArrayList<>(), nums);
    return list;
}

private void backtrack(List<List<Integer>> list, List<Integer> tempList, int []
nums){
    if(tempList.size() == nums.length){
        list.add(new ArrayList<>(tempList));
    } else{
        for(int i = 0; i < nums.length; i++){
            if(tempList.contains(nums[i])) continue; // 已经存在的元素, 跳过
            tempList.add(nums[i]); //将当前元素加入
            backtrack(list, tempList, nums); //向后继续添加
            tempList.remove(tempList.size() - 1); //将 tempList 刚添加的元素, 去掉,
            尝试新的元素
        }
    }
}

```

假如给定的数组是 [1 1 3], 我们来看一下遍历的这个图。



第一个要解决的就是这句代码

```
if(tempList.contains(nums[i])) continue; // 已经存在的元素，跳过
```

之前没有重复的元素，所以可以直接在 tempList 判断有没有当前元素，有的话就跳过。但这里的话，因为给定的有重复的元素，这个方法明显不可以了。

换个思路，我们可以再用一个 list 保存当前 tempList 中已经有的元素的下标，然后添加新元素的时候去判断下标就可以了。

第二个问题就是，可以看到有重复元素的时候，上边第 1 个图和第 2 个图产生的是完全一样的序列。所以第 2 个遍历是没有必要的。

解决的方案就是把数组首先排下顺序，然后判断一下上一个添加的元素和当前元素是不是相等，相等的话就跳过，继续下一个元素。

```
public List<List<Integer>> permuteUnique(int[] nums) {
    List<List<Integer>> list = new ArrayList<>();
    Arrays.sort(nums);
    List<Integer> old = new ArrayList<>();
    backtrack(list, new ArrayList<>(), nums, old);
    return list;
}

private void backtrack(List<List<Integer>> list, List<Integer> tempList, int[]
nums, List<Integer> old) {
    if (tempList.size() == nums.length) {
        list.add(new ArrayList<>(tempList));
    } else {
        for (int i = 0; i < nums.length; i++) {
            //解决第一个问题
            if (old.contains(i)) {
                continue;
            }
            //解决第二个问题 !old.contains(i - 1) 很关键，下边解释下
            if (i > 0 && !old.contains(i - 1) && nums[i - 1] == nums[i]) {
                continue;
            }
            old.add(i); //添加下标
            tempList.add(nums[i]); // 将当前元素加入
        }
    }
}
```

```

        backtrack(list, tempList, nums, old); // 向后继续添加
        old.remove(old.size() - 1);
        tempList.remove(tempList.size() - 1);
    }
}
}

```

解决第二个问题 `!old.contains(i - 1)` 很关键

因为上边 `old.contains(i)` 代码会使得一些元素跳过没有加到 `templist` 上，所以我们要判断 `nums[i - 1]` 是不是被跳过的那个元素，如果 `old.contains(i)` 返回 `true`，即使 `nums[i - 1] == nums[i]` 也不能跳过当前元素。因为上一个元素 `nums[i - 1]` 并没有被添加到 `templist`。可能比较绕，但是可以参照上边的图，走一下流程就懂了。如果不加 `!old.contains(i - 1)`，那么图中的第 2 行的第 2 个 1 本来应该加到 `tempList`，但是会被跳过。因为第 2 行第 1 个元素也是 1。

对于解决第一个问题，我们用了 `list` 来保存下标来解决。需要一个 $O(n)$ 的空间。有一种方法，我们可以用 $O(1)$ 的空间。不过前提是，我们需要对问题的样例了解，也就是给定的输入所包含的数字。我们需要找到一个样例中一定不包含的数字来解决我们的问题。

首先，我们假设输入的所有的数字中没有 -100 这个数字。

然后，我们就可以递归前将当前数字先保存起来，然后置为 -100 隐藏起来，递归结束后还原即可。

```

public List<List<Integer>> permuteUnique(int[] nums) {
    List<List<Integer>> list = new ArrayList<>();
    Arrays.sort(nums);
    backtrack(list, new ArrayList<>(), nums);
    return list;
}

private void backtrack(List<List<Integer>> list, List<Integer> tempList, int[]
nums) {
    if (tempList.size() == nums.length) {
        list.add(new ArrayList<>(tempList));
    } else {
        for (int i = 0; i < nums.length; i++) {
            //解决第一个问题
            if (nums[i] == -100) {
                continue;
            }
            //解决第二个问题 !old.contains(i - 1) 很关键
            if (i > 0 && nums[i-1] != -100 && nums[i - 1] == nums[i]) {
                continue;
            }
            tempList.add(nums[i]); // 将当前元素加入
            int temp = nums[i]; //保存
            nums[i] = -100; // 隐藏
        }
    }
}

```

```

        backtrack(list, tempList, nums); // 向后继续添加
        nums[i] = temp; //还原
        tempList.remove(tempList.size() - 1);
    }
}
}

```

当然这个想法局限性很大，但是如果对解决的问题很熟悉，一般是可以找到这样一个不会输入的数字，然后可以优化空间复杂度。

解法三 交换

这个改起来相对容易些，之前的想法就是在每一个位置，让每个数字轮流交换过去一下。这里的话，我们其实只要把当前位置已经有哪些数字来过保存起来，如果有重复的话，我们不让他交换，直接换下一个数字就可以了。

```

public List<List<Integer>> permuteUnique(int[] nums) {
    List<List<Integer>> all = new ArrayList<>();
    Arrays.sort(nums);
    upset(nums, 0, all);
    return all;
}

private void upset(int[] nums, int begin, List<List<Integer>> all) {
    if (begin == nums.length) {
        ArrayList<Integer> temp = new ArrayList<Integer>();
        for (int i = 0; i < nums.length; i++) {
            temp.add(nums[i]);
        }
        all.add(new ArrayList<Integer>(temp));
        return;
    }
    HashSet<Integer> set = new HashSet<>(); //保存当前要交换的位置已经有过哪些数字了
    for (int i = begin; i < nums.length; i++) {
        if (set.contains(nums[i])) { //如果存在了就跳过，不去交换
            continue;
        }
        set.add(nums[i]);
        swap(nums, i, begin);
        upset(nums, begin + 1, all);
        swap(nums, i, begin);
    }
}

```



```
}

private void swap(int[] nums, int i, int begin) {
    int temp = nums[i];
    nums[i] = nums[begin];
    nums[begin] = temp;
}
```

总

基本上都是在上道题的基础上改出来了，一些技巧也是经常遇到，比如先排序，然后判断和前一个是否重复。利用 Hash 去重的功能。利用原来的存储空间隐藏掉数据，然后再想办法还原。

48、题目描述（中等难度）

You are given an $n \times n$ 2D matrix representing an image.

Rotate the image by 90 degrees (clockwise).

Note:

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:

```
Given input matrix =  
[  
  [1,2,3],  
  [4,5,6],  
  [7,8,9]  
],  
  
rotate the input matrix in-place such that it becomes:  
[  
  [7,4,1],  
  [8,5,2],  
  [9,6,3]  
]
```

Example 2:

```
Given input matrix =  
[  
  [ 5, 1, 9,11],  
  [ 2, 4, 8,10],  
  [13, 3, 6, 7],  
  [15,14,12,16]  
],  
  
rotate the input matrix in-place such that it becomes:  
[  
  [15,13, 2, 5],  
  [14, 3, 4, 1],  
  [12, 6, 8, 9],  
  [16, 7,10,11]  
]
```

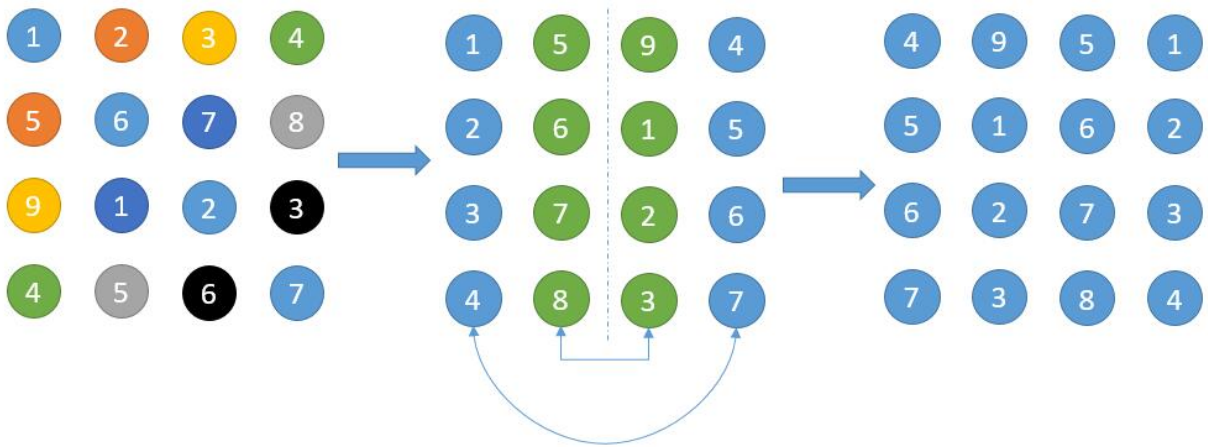
将一个矩阵顺时针旋转 90 度，并且不使用额外的空间。大概属于找规律的题，没有什么一般的思路，观察就可以了。

解法一

可以先转置，然后把每列对称交换交换一下。

以对角线为轴交换

以中轴线对称列交换



```
public void rotate(int[][] matrix) {  
    //以对角线为轴交换  
    for (int i = 0; i < matrix.length; i++) {  
        for (int j = 0; j <= i; j++) {  
            if (i == j) {  
                continue;  
            }  
            int temp = matrix[i][j];  
            matrix[i][j] = matrix[j][i];  
            matrix[j][i] = temp;  
        }  
    }  
    //交换列  
    for (int i = 0, j = matrix.length - 1; i < matrix.length / 2; i++, j--) {  
        for (int k = 0; k < matrix.length; k++) {  
            int temp = matrix[k][i];  
            matrix[k][i] = matrix[k][j];  
            matrix[k][j] = temp;  
        }  
    }  
}
```

时间复杂度： $O(n^2)$ 。

空间复杂度： $O(1)$ 。

也可以先以横向的中轴线为轴，对称的行进行交换，然后再以对角线交换。

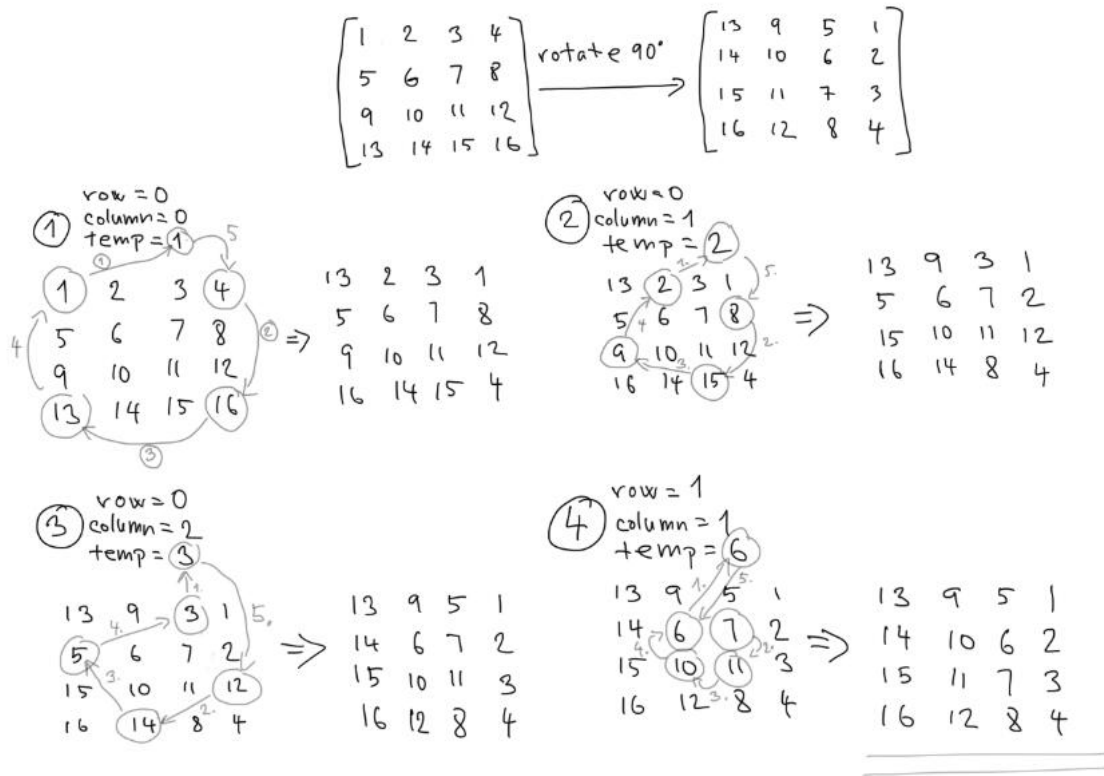
解法二

我把这个[链接](#)的思路贴过来，里边评论有张图也都顺道贴过来吧，写的很好。



ondrej-kvasnovsky ★ 31 · July 8, 2018 9:05 AM

This image could help with understanding (how the four moves are done each time the inner for is executed):



一圈一圈的循环交换，很妙！

```
public void rotate(int[][] matrix) {  
    int n=matrix.length;  
    for (int i=0; i<n/2; i++)  
        for (int j=i; j<n-i-1; j++) {  
            int tmp=matrix[i][j];  
            matrix[i][j]=matrix[n-j-1][i];  
            matrix[n-j-1][i]=matrix[n-i-1][n-j-1];  
            matrix[n-i-1][n-j-1]=matrix[j][n-i-1];  
            matrix[j][n-i-1]=tmp;  
        }  
}
```

时间复杂度：O (n²) 。

空间复杂度：O (1) 。

总

这道题就是对题目的特征进行观察就可以了。

49、题目描述（中等难度）

Given an array of strings, group anagrams together.

Example:

```
Input: ["eat", "tea", "tan", "ate", "nat", "bat"],
Output:
[
  ["ate","eat","tea"],
  ["nat","tan"],
  ["bat"]
]
```

Note:

- All inputs will be in lowercase.
- The order of your output does not matter.

给定多个字符串，然后把它们分类。只要字符串所包含的字符完全一样就算作一类，不考虑顺序。

解法一

最通用的一种解法，对于每个字符串，比较它们的每个字符出现的个数是否相等，相等的话就把它放在一个 list 中去，作为一个类别。最外层写一个 for 循环然后一一比较就可以，还可以用一个等大的布尔型数组来记录当前字符串是否已经加入的了 list 。比较两个字符串的字符出现的次数可以用一个 HashMap，具体看代码吧。

```
public List<List<String>> groupAnagrams(String[] strs) {
    List<List<String>> ans = new ArrayList<>();
    boolean[] used = new boolean[strs.length];
    for (int i = 0; i < strs.length; i++) {
        List<String> temp = null;
        if (!used[i]) {
            temp = new ArrayList<String>();
            temp.add(strs[i]);
            //两两比较判断字符串是否符合
            for (int j = i + 1; j < strs.length; j++) {
                if (equals(strs[i], strs[j])) {
                    used[j] = true;
                    temp.add(strs[j]);
                }
            }
        }
    }
}
```

```

        }
    }
    if (temp != null) {
        ans.add(temp);
    }
}
return ans;
}

private boolean equals(String string1, String string2) {
    Map<Character, Integer> hash = new HashMap<>();
    //记录第一个字符串每个字符出现的次数，进行累加
    for (int i = 0; i < string1.length(); i++) {
        if (hash.containsKey(string1.charAt(i))) {
            hash.put(string1.charAt(i), hash.get(string1.charAt(i)) + 1);
        } else {
            hash.put(string1.charAt(i), 1);
        }
    }
    //记录第二个字符串每个字符出现的次数，将之前的每次减 1
    for (int i = 0; i < string2.length(); i++) {
        if (hash.containsKey(string2.charAt(i))) {
            hash.put(string2.charAt(i), hash.get(string2.charAt(i)) - 1);
        } else {
            return false;
        }
    }
    //判断每个字符的次数是不是 0，不是的话直接返回 false
    Set<Character> set = hash.keySet();
    for (char c : set) {
        if (hash.get(c) != 0) {
            return false;
        }
    }
    return true;
}
}

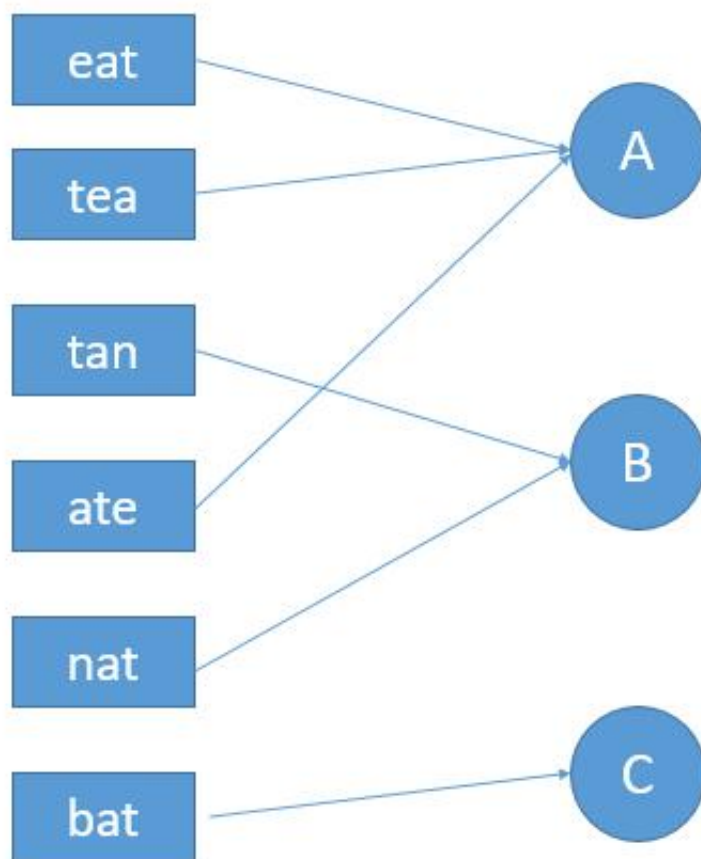
```

时间复杂度：两层 for 循环，再加上比较字符串，如果字符串最长为 K，总的时间复杂度就是 $O(n^2K)$ 。

空间复杂度： $O(NK)$ ，用来存储结果。

解法一算是比较通用的解法，不管字符串里边是大写字母，小写字母，数字，都可以用这个算法解决。这道题的话，题目告诉我们字符串中只有小写字母，针对这个限制，我们可以再用一些针对性强的算法。

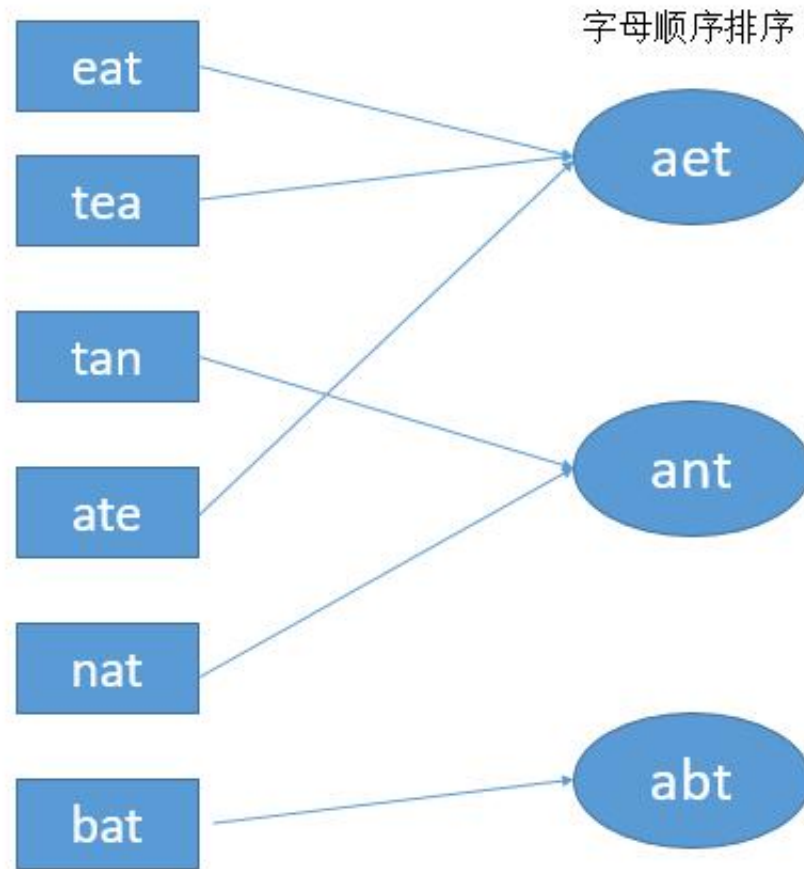
下边的算法本质是，我们只要把一类的字符串用某一种方法唯一的映射到同一个位置就可以。



解法二

参考官方给的[解法](#)。

我们将每个字符串按照字母顺序排序，这样的话就可以把 eat, tea, ate 都映射到 aet。其他的类似。



```
public List<List<String>> groupAnagrams(String[] strs) {  
    HashMap<String, List<String>> hash = new HashMap<>();  
    for (int i = 0; i < strs.length; i++) {  
        char[] s_arr = strs[i].toCharArray();  
        //排序  
        Arrays.sort(s_arr);  
        //映射到 key  
        String key = String.valueOf(s_arr);  
        //添加到对应的类中  
        if (hash.containsKey(key)) {  
            hash.get(key).add(strs[i]);  
        } else {  
            List<String> temp = new ArrayList<String>();  
            temp.add(strs[i]);  
            hash.put(key, temp);  
        }  
    }  
    return new ArrayList<List<String>>(hash.values());  
}
```


时间复杂度：排序的话算作 $O(K \log(K))$,最外层的 for 循环，所以就是 $O(n K \log(K))$ 。

空间复杂度： $O(NK)$ ， 用来存储结果。

解法三

参考[这里](#)，利用[算术基本定理](#)。

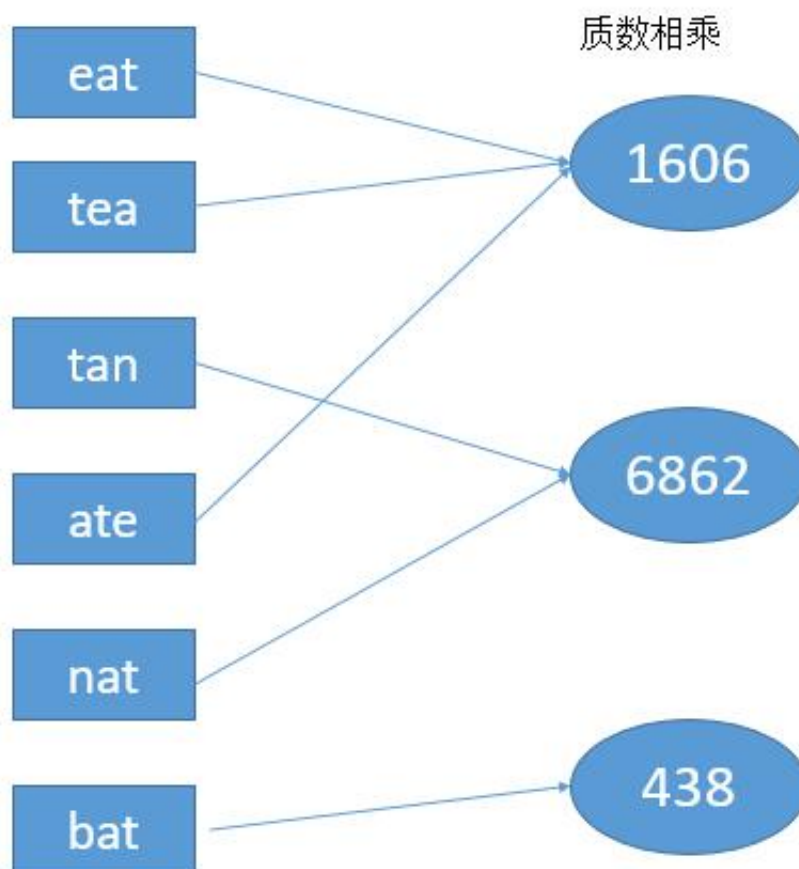
算术基本定理，又称为正整数的唯一分解定理，即：每个大于1的[自然数](#)，要么本身就是[质数](#)，要么可以写为2个以上的质数的[积](#)，而且这些质因子按大小排列之后，写法仅有一种方式。

利用这个，我们把每个字符串都映射到一个正数上。

用一个数组存储质数 $\text{prime} = \{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103\}$ 。

然后每个字符串的字符减去 'a'，然后取到 prime 中对应的质数。把它们累乘。

例如 abc，就对应 'a' - 'a'，'b' - 'a'，'c' - 'a'，即 0, 1, 2，也就是对应素数 2 3 5，然后相乘 $2 * 3 * 5 = 30$ ，就把 "abc" 映射到了 30。



```
public List<List<String>> groupAnagrams(String[] strs) {
```

```

HashMap<Integer, List<String>> hash = new HashMap<>();
//每个字母对应一个质数
int[] prime = { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 41, 43, 47, 53, 59,
61, 67, 71, 73, 79, 83, 89, 97, 101, 103 };
for (int i = 0; i < strs.length; i++) {
    int key = 1;
    //累乘得到 key
    for (int j = 0; j < strs[i].length(); j++) {
        key *= prime[strs[i].charAt(j) - 'a'];
    }
    if (hash.containsKey(key)) {
        hash.get(key).add(strs[i]);
    } else {
        List<String> temp = new ArrayList<String>();
        temp.add(strs[i]);
        hash.put(key, temp);
    }
}
return new ArrayList<List<String>>(hash.values());
}

```

时间复杂度：O (n * K) ， K 是字符串的最长长度。

空间复杂度：O (NK) ， 用来存储结果。

这个解法时间复杂度，较解法二有提升，但是有一定的局限性，因为求 key 的时候用的是累乘，可能会造成溢出，超出 int 所能表示的数字。

解法四

参考[这里](#)，记录字符串的每个字符出现的次数从而完成映射。因为有 26 个字母，不好解释，我们假设只有 5 个字母，来看一下怎么完成映射。

首先初始化 key = "0#0#0#0#0#"，数字分别代表 abcde 出现的次数，# 用来分割。

这样的话，"abb" 就映射到了 "1#2#0#0#0"。

"cdc" 就映射到了 "0#0#2#1#0"。

"dcc" 就映射到了 "0#0#2#1#0"。

```

public List<List<String>> groupAnagrams(String[] strs) {
    HashMap<String, List<String>> hash = new HashMap<>();
    for (int i = 0; i < strs.length; i++) {
        int[] num = new int[26];

```

```

//记录每个字符的次数
for (int j = 0; j < strs[i].length(); j++) {
    num[strs[i].charAt(j) - 'a']++;
}
//转成 0#2#2# 类似的形式
String key = "";
for (int j = 0; j < num.length; j++) {
    key = key + num[j] + '#';
}
if (hash.containsKey(key)) {
    hash.get(key).add(strs[i]);
} else {
    List<String> temp = new ArrayList<String>();
    temp.add(strs[i]);
    hash.put(key, temp);
}

}
return new ArrayList<List<String>>(hash.values());
}

```

时间复杂度：O (nK) 。

空间复杂度：O (NK) ，用来存储结果。

总

利用 HashMap 去记录字符的次数之前也有遇到过，很常用。解法三中利用质数相乘，是真的太强了。

50、题目描述（中等难度）

50. Pow(x, n)

Medium 755 1880 Favorite Share

Implement $\text{pow}(x, n)$, which calculates x raised to the power n (x^n).

Example 1:

Input: 2.00000, 10
Output: 1024.00000

Example 2:

Input: 2.10000, 3
Output: 9.26100

Example 3:

Input: 2.00000, -2
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Note:

- $-100.0 < x < 100.0$
- n is a 32-bit signed integer, within the range $[-2^{31}, 2^{31} - 1]$

就是求幂次方。

解法一

求幂次方，用最简单的想法，就是写一个 for 循环累乘。

至于求负幂次方，比如 2^{-10} ，可以先求出 2^{10} ，然后取倒数， $1/2^{10}$ ，就可以了。

```
double mul = 1;
if (n > 0) {
    for (int i = 0; i < n; i++) {
        mul *= x;
    }
} else {
    n = -n;
    for (int i = 0; i < n; i++) {
        mul *= x;
    }
    mul = 1 / mul;
}
```

但这样的话会出问题，之前在[29题](#)讨论过，问题出在 $n = -n$ 上，因为最小负数 -2^{31} 取相反数的话，按照计算机的规则，依旧是 -2^{31} ，所以这种情况需要单独讨论一下。

```
if (n == -2147483648) {  
    return 0;  
}
```

当然，这样做的话 -1，和 1 也需要单独讨论下，因为他们的任意次方都是 1 或者 -1。

```
if (x == -1) {  
    if ((n & 1) != 0) { //按位与不等于 0，说明是奇数  
        return -1;  
    } else {  
        return 1;  
    }  
}  
if (x == 1.0)  
    return 1;
```

综上，代码就出来了。

```
public double myPow(double x, int n) {  
    if (x == -1) {  
        if ((n & 1) != 0) {  
            return -1;  
        } else {  
            return 1;  
        }  
    }  
    if (x == 1.0)  
        return 1;  
  
    if (n == -2147483648) {  
        return 0;  
    }  
    double mul = 1;  
    if (n > 0) {  
        for (int i = 0; i < n; i++) {  
            mul *= x;  
        }  
    } else {  
        n = -n;  
        for (int i = 0; i < n; i++) {  
            mul *= x;  
        }  
        mul = 1 / mul;  
    }  
    return mul;  
}
```

```
}
```

时间复杂度：O (n) 。

空间复杂度：O (1) 。

解法二 递归

对于上边的解法，太慢了。可以优化下，类似于[29题](#)的思路。乘法的话，我们不用一次一次的相乘，得到 2 次方后，我们可以直接把 2 次方的结果相乘，就可以得到 4 次方，得到 4 次方的结果再相乘，就是 8 次方了，这样的话就会快很多了。

直接利用递归吧

对于 n 是偶数的情况， $x^n = x^{n/2} * x^{n/2}$ 。

对于 n 是奇数的情况， $x^n = x^{n/2} * x^{n/2} * x$ 。

```
public double powRecursion(double x, int n) {
    if (n == 0) {
        return 1;
    }
    //偶数的情况
    if ((n & 1) == 0) {
        double temp = powRecursion(x, n / 2);
        return temp * temp;
    } else { //奇数的情况
        double temp = powRecursion(x, n / 2);
        return temp * temp * x;
    }
}

public double myPow(double x, int n) {
    if (x == -1) {
        if ((n & 1) != 0) {
            return -1;
        } else {
            return 1;
        }
    }
    if (x == 1.0f)
        return 1;

    if (n == -2147483648) {
        return 0;
    }
    double mul = 1;
```

```

    if (n > 0) {
        mul = powRecursion(x, n);
    } else {
        n = -n;
        mul = powRecursion(x, n);
        mul = 1 / mul;
    }
    return mul;
}

```

时间复杂度：O (log (n))。

空间复杂度：

当然对于这种递归的解法的话，还有一些其他的思路，参考[这里](#)。

递归思路是下边的样子

$x^n = (x \cdot x)^{n/2}$ ，对于 n 是偶数的情况。

$x^n = (x \cdot x)^{n/2} \cdot x$ ，对于 n 是奇数的情况，

代码就很好写了。

```

public double powRecursion(double x, int n) {
    if (n == 0) {
        return 1;
    }
    //偶数的情况
    if ((n & 1) == 0) {
        return powRecursion(x * x, n / 2);
    } else { //奇数的情况
        return powRecursion(x * x, n / 2) * x;
    }
}

public double myPow(double x, int n) {
    if (x == -1) {
        if ((n & 1) != 0) {
            return -1;
        } else {
            return 1;
        }
    }
    if (x == 1.0f)
        return 1;

    if (n == -2147483648) {

```

```

        return 0;
    }
    double mul = 1;
    if (n > 0) {
        mul = powRecursion(x, n);
    } else {
        n = -n;
        mul = powRecursion(x, n);
        mul = 1 / mul;
    }
    return mul;
}

```

时间复杂度：O (log (n))。

空间复杂度：

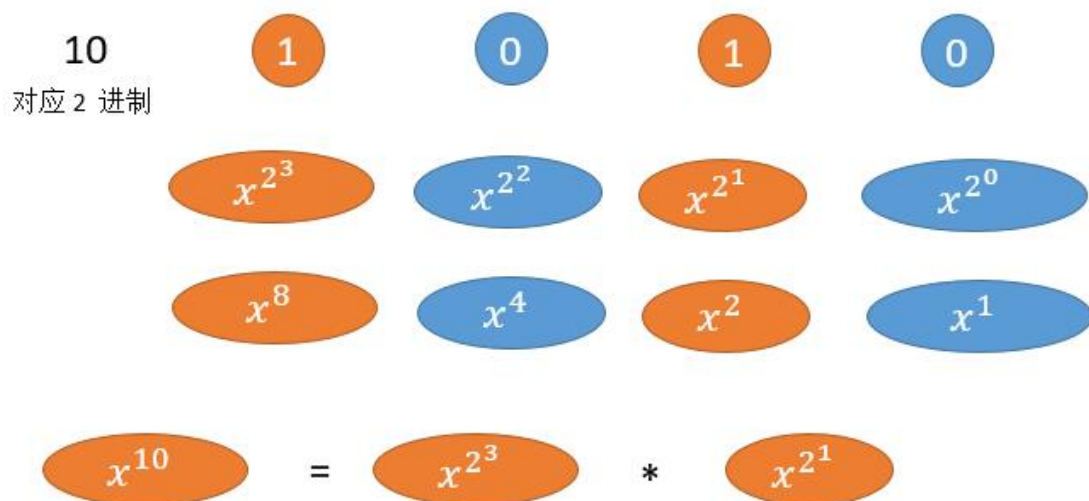
解法三 迭代

这里介绍种全新的解法，开始的时候受前边思路的影响，一直没理解。下午问同学，同学立刻想到了自己在《编程之美》看到的解法，这里分享下。

以 x 的 10 次方举例。10 的 2 进制是 1010，然后用 2 进制转 10 进制的方法把它展成 2 的幂次的和。

$$x^{10} = x^{(1010)_2} = x^{12^3 + 02^2 + 12^1 + 02^0} = x^{12^3} x^{02^2} x^{12^1} x^{02^0}$$

这样话，再看一下下边的图，它们之间的对应关系就出来了。



2 进制对应 1 0 1 0，我们把对应 1 的项进行累乘就可以了，而要进行累乘的项也是很有规律，前一项是后一项的自乘。 $x^8 = x^4 * x^4$ 。我们可以从最右边一位，开始迭代。看下代码吧。


```

public double myPow(double x, int n) {
    if (x == -1) {
        if ((n & 1) != 0) {
            return -1;
        } else {
            return 1;
        }
    }
    if (x == 1.0f)
        return 1;

    if (n == -2147483648) {
        return 0;
    }
    double mul = 1;
    if (n > 0) {
        mul = powIteration(x, n);
    } else {
        n = -n;
        mul = powIteration(x, n);
        mul = 1 / mul;
    }
    return mul;
}

public double powIteration(double x, int n) {
    double ans = 1;
    //遍历每一位
    while (n > 0) {
        //最后一位是 1, 加到累乘结果里
        if ((n & 1) == 1) {
            ans = ans * x;
        }
        //更新 x
        x = x * x;
        //n 右移一位
        n = n >> 1;
    }
    return ans;
}

```

时间复杂度： $\log(n)$ 。

空间复杂度： $O(1)$ 。

更新

2020.3.16 更新。感谢 @为爱卖小菜 指出，上边的解法虽然都能 AC，但是以上全错，少考虑了一种情况。

前边我们分析到 -2147483648 需要单独讨论。

但这样的话会出问题，之前在 29题 讨论过，问题出在 $n = -n$ 上，因为最小负数 -2^{31} 取相反数的话，按照计算机的规则，依旧是 -2^{31} ，所以这种情况需要单独讨论一下。

```
if (n == -2147483648) {  
    return 0;  
}
```

但当 $n = -2147483648$ 个时候，并不是所有的 x^n 结果都是 0。

当 x 等于 -1 或者 1 的时候结果是 1。前边的解法也考虑到了。

下边 x 等于 -1 的时候我们顺便考虑了 n 是其他数的情况，所以没直接返回 1。

```
if (x == -1) {  
    if ((n & 1) != 0) {  
        return -1;  
    } else {  
        return 1;  
    }  
}  
if (x == 1.0f)  
    return 1;
```

但其实 x 是浮点数，我们还少考虑了 -1 到 0 和 0 到 1 之间的数，此时的 x^n 的结果应该是正无穷。

此外 $x == 0$ 的话，数学上是不能算的，这里的话也输出正无穷。

综上，我们的前置条件如下

```
if (x == -1) {  
    if ((n & 1) != 0) {  
        return -1;  
    } else {  
        return 1;  
    }  
}  
if (x == 1.0f){  
    return 1;  
}
```

```

if(n == -2147483648){
    if(x > -1 && x < 1 ){
        return Double.POSITIVE_INFINITY;
    }else{
        return 0;
    }
}

```

上边就是当 `n = -2147483648` 的所有情况了。对于 `$x^n$` ，`x` 分成了四种情况。

当 `x == -1` 结果是 `1`，上边的代码我们顺便把 `n` 是其它数的情况也顺便考虑了。

当 `x == 1` 结果是 `1`。

当 `-1 < x < 1`，结果是正无穷。

当 `x < -1` 或者 `x > 1`，结果是 `0`。

[@为爱卖小菜](#) 也提供了一个新方法，可以把上边的所有情况统一起来。

因为当 `n = -2147483648` 的时候我们无法正确计算，我们可以把 `$x^{-2147483648}$` 分解成 `$x^{-2147483647} * x^{-1}$` 。这样的话两部分都可以成功求解了。

对于解法三，可以改写成下边的样子。其他解法也类似。

```

public double myPow(double x, int n) {
    double mul = 1;
    if (n > 0) {
        mul = powIteration(x, n);
    } else {
        //单独考虑 n = -2147483648
        if (n == -2147483648) {
            return myPow(x, -2147483647) * (1 / x);
        }
        n = -n;
        mul *= powIteration(x, n);
        mul = 1 / mul;
    }
    return mul;
}

public double powIteration(double x, int n) {
    double ans = 1;
    //遍历每一位
    while (n > 0) {
        //最后一位是 1，加到累乘结果里
        if ((n & 1) == 1) {

```

```
        ans = ans * x;
    }
    //更新 x
    x = x * x;
    //n 右移一位
    n = n >> 1;
}
return ans;
}
```

总

从一般的方法，到递归，最后的解法，直接从 2 进制考虑，每一个数字，都可以转换成 2 的幂次的和，从而实现了最终的解法。

结语

本来想整理至少 100 道，但到 50 道的时候电脑已经有点扛不住了，整理得也比较辛苦，所以打算后 50 道题等到下周再整理，有小伙伴亟需的话，可以微信（qing_gee）催催我。催的人多了，我就更新，哈哈。

我是爱学习爱分享的沉默王二。微信搜索「沉默王二」可以关注我的原创公众号，回复关键字「Java」就可以拉取更多离线版资料下载地址。

再贴一下作者的在线地址！

- 题解预览地址：<https://leetcode.wang>，推荐电脑端打开，手机打开的话将页面滑到最上边，左上角是菜单
- leetcode 题目地址 <https://leetcode.com/problemset/all/>
- github 项目地址：<https://github.com/wind-liang/leetcode>